



TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Sistema conversacional inteligente con arquitectura RAG

Desarrollo y evaluación de un sistema conversacional
inteligente con arquitectura RAG para el dominio académico
de la ETSIIT

Autor

Sergio Medina Muñoz (alumno)

Directores

María Luisa Rodríguez Almendros (tutor)



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Junio de 2026

Desarrollo y evaluación de un sistema conversacional inteligente con arquitectura RAG para el dominio académico de la ETSIIT

Sergio Medina Muñoz (alumno)

Palabras clave: Generación Aumentada por Recuperación (RAG), modelos de lenguaje de gran escala (LLM), recuperación de información, asistente conversacional académico, ejecución local, trazabilidad, ETSIIT, procesamiento de lenguaje natural.

Resumen

Se desarrolla un asistente conversacional académico para la Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación (ETSIIT) de la Universidad de Granada, basado en la arquitectura *Retrieval-Augmented Generation* (RAG) y ejecutado de forma íntegramente local sobre hardware de consumo y software libre. El sistema responde consultas sobre la documentación oficial del centro, calendarios, horarios, guías docentes, normativas, trámites y movilidad, con precisión y *trazabilidad*, citando siempre la fuente y absteniéndose cuando carece del dato.

A partir de un corpus propio, construido mediante un *crawler* dirigido por categorías sobre las fuentes oficiales, el sistema integra una cadena completa: indexación híbrida (densa con *embeddings* y léxica con BM25, fusionadas mediante *Reciprocal Rank Fusion*), reordenación con un *cross-encoder*, un *gate* de ámbito que rechaza las consultas ajenas al dominio, generación con un modelo de lenguaje abierto, y un conjunto de salvaguardas deterministas que garantizan la fidelidad a las fuentes y la citación verificable.

La evaluación, sobre un conjunto de preguntas, compara el sistema con modelos operando con y sin recuperación, se complementa con métricas del *framework* RAGAS y un análisis comparativo entre tres modelos. Los resultados confirman que la recuperación eleva la precisión factual enormemente, con citación verificable y una latencia inferior a 5 s en una GPU de consumo. La conclusión central es que, en un dominio cerrado y especializado, la calidad del asistente la determina la arquitectura de recuperación y sus salvaguardas de calidad, no el tamaño del modelo.

Development and evaluation of an intelligent conversational system with a RAG architecture for the academic domain of the ETSIIT

Sergio Medina Muñoz (student)

Keywords: Retrieval-Augmented Generation (RAG), large language models (LLM), information retrieval, academic conversational assistant, local deployment, traceability, ETSIIT, natural language processing.

Abstract

An academic conversational assistant is developed for the School of Computer and Telecommunication Engineering (ETSIIT) of the University of Granada, based on the *Retrieval-Augmented Generation* (RAG) architecture and run entirely locally on consumer hardware and free software. The system answers queries about the centre’s official documentation, calendars, timetables, course guides, regulations, administrative procedures and mobility, with accuracy and *traceability*, always citing the source and abstaining when it lacks the information.

From an own corpus, built by a category-driven *crawler* over the official sources, the system integrates a complete pipeline: hybrid indexing (dense, with *embeddings*, and lexical, with BM25, fused via *Reciprocal Rank Fusion*), reranking with a *cross-encoder*, an in-domain *gate* that rejects out-of-domain queries, generation with an open language model, and a set of deterministic safeguards that ensure faithfulness to the sources and verifiable citation.

The evaluation, over a set of questions, compares the system with models operating with and without retrieval, and is complemented with metrics from the RAGAS *framework* and a comparative analysis across three models. The results confirm that retrieval raises factual accuracy enormously, with verifiable citation and a latency below 5 s on a consumer GPU. The central conclusion is that, in a closed and specialized domain, the quality of the assistant is determined by the retrieval architecture and its quality safeguards, not by the size of the model.

Yo, **Sergio Medina Muñoz**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 77144014J, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Sergio Medina Muñoz

Granada a 14 de Junio de 2026 .

D. **María Luisa Rodríguez Almendros (tutor)**, Profesora del Área de Lenguajes y Sistemas Informáticos del Departamento Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Sistema conversacional inteligente con arquitectura RAG, Desarrollo y evaluación de un sistema conversacional inteligente con arquitectura RAG para el dominio académico de la ETSIIT***, ha sido realizado bajo su supervisión por **Sergio Medina Muñoz (alumno)**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 14 de Junio de 2026 .

Los Directores:

María Luisa Rodríguez Almendros (tutor)

Agradecimientos

Agradezco enormemente a mis padres, mi hermano y mi novia, por su apoyo incondicional a lo largo de la carrera y por confiar plenamente en mí.

Agradezco a mis amigos, por hacer más ligero mi paso por la carrera y por ayudarme a desconectar en los momentos más necesarios.

Agradezco a los profesores que me han impartido clase y me han ayudado a lo largo de mi carrera, y sobretodo a Marisa por su orientación y ánimo durante el desarrollo de este Trabajo Fin de Grado.

Y por último, quiero agradecer a todas las personas que, de un modo u otro, para bien o para mal, con alegrías y tristezas, han formado parte de este camino que llega a su fin.

Índice general

1. Introducción	1
1.1. Contexto y antecedentes	1
1.2. Motivación	2
1.3. Denominación del sistema	3
1.4. Objetivos	3
1.4.1. Objetivo general	3
1.4.2. Objetivos específicos	3
1.5. Hipótesis de investigación	5
1.6. Estructura de la memoria	6
2. Presupuesto y Planificación	9
2.1. Introducción	9
2.2. Metodología de Planificación	9
2.3. Fases del Proyecto y Planificación Temporal	10
2.3.1. Fase 1: Análisis del Problema y Estado del Arte . . .	10
2.3.2. Fase 2: Análisis de Requisitos y Diseño	10
2.3.3. Fase 3: Implementación del Pipeline de Ingesta	11
2.3.4. Fase 4: Implementación del Núcleo RAG	11
2.3.5. Fase 5: Evaluación Experimental	12
2.3.6. Fase 6: Redacción Final, Revisión y Defensa	12
2.3.7. Cronograma General	13
2.4. Recursos del Proyecto	13
2.4.1. Recursos Humanos	13
2.4.2. Recursos Hardware	13
2.4.3. Recursos Software	15
2.5. Presupuesto	15
2.5.1. Coste de Personal	16
2.5.2. Amortización del Hardware	16
2.5.3. Coste de Software	16
2.5.4. Otros Costes	16
2.5.5. Resumen del Presupuesto	16
2.6. Análisis de Riesgos	17
2.7. Conclusiones del Capítulo	17

3. Estado del Arte	19
3.1. Introducción	19
3.2. Modelos de Lenguaje de Gran Tamaño (LLMs)	20
3.2.1. La arquitectura Transformer	20
3.2.2. Modelos codificadores (<i>encoder-only</i>)	22
3.2.3. Modelos decodificadores (<i>decoder-only</i>)	24
3.2.4. Modelos secuencia a secuencia (<i>encoder-decoder</i>)	26
3.2.5. Comparativa: modelos comerciales frente a modelos abiertos	26
3.2.6. Leyes de escalado y tamaños de modelos	27
3.2.7. Técnicas de optimización para despliegue	28
3.2.8. Limitaciones de los LLMs	30
3.3. Recuperación Aumentada por Generación (RAG)	31
3.3.1. Fundamentos y motivación	31
3.3.2. Clasificación de arquitecturas RAG	34
3.3.3. Variantes avanzadas	36
3.4. Recuperación Vectorial y Embeddings	37
3.4.1. Fundamentos de los embeddings densos	37
3.4.2. Modelos de embedding	38
3.4.3. Bases de datos vectoriales e índices de búsqueda	39
3.5. Estrategias de Fragmentación (<i>Chunking</i>)	42
3.5.1. Estrategias principales	42
3.5.2. Parámetros clave	43
3.6. Chatbots y Asistentes Académicos	44
3.6.1. Evolución de los chatbots en educación superior	44
3.6.2. Soluciones académicas existentes	45
3.7. Evaluación de Sistemas RAG	50
3.7.1. Dimensiones de evaluación	50
3.7.2. Métricas de recuperación	51
3.7.3. Métricas de generación	52
3.7.4. Frameworks de evaluación automática	52
3.7.5. Evaluación humana	53
3.8. Herramientas y tecnologías del ecosistema RAG	53
3.8.1. Lenguajes de programación para <i>stacks</i> RAG	54
3.8.2. Servidores de inferencia local para LLMs	55
3.8.3. Frameworks de interfaz para prototipos de IA	55
3.8.4. Bibliotecas de extracción de documentos PDF	56
3.9. Posicionamiento del Trabajo Propuesto	57
3.10. Conclusiones del Capítulo	58
4. Análisis y Objetivos del Sistema	61
4.1. Introducción	61
4.2. Metodología de Análisis	62
4.3. Identificación de Actores	63

4.4. Requisitos Funcionales	63
4.4.1. Acciones del estudiante	64
4.4.2. Acciones del administrador	65
4.5. Requisitos No Funcionales	66
4.5.1. Rendimiento	67
4.5.2. Calidad de la respuesta	67
4.5.3. Usabilidad y accesibilidad	68
4.5.4. Privacidad y seguridad	69
4.5.5. Mantenibilidad y portabilidad	70
4.5.6. Restricciones técnicas y de diseño	70
4.6. Casos de Uso	71
4.6.1. Diagrama de casos de uso	71
4.6.2. Fichas detalladas	72
4.7. Matriz de Trazabilidad	76
4.8. Conclusiones del Capítulo	78
5. Diseño y Arquitectura del Sistema	79
5.1. Introducción	79
5.2. Decisiones arquitectónicas fundamentales	80
5.2.1. Pipeline RAG secuencial	80
5.2.2. Ejecución íntegramente local	81
5.2.3. Arquitectura modular por capas	81
5.3. Selección del <i>stack</i> tecnológico	82
5.3.1. Lenguaje de programación: Python	82
5.3.2. Servidor de inferencia LLM: Ollama	82
5.3.3. Índice vectorial: FAISS	83
5.3.4. Modelo de <i>embeddings</i> : familia E5 multilingüe	83
5.3.5. LLM de generación: modelos abiertos vía Ollama	84
5.3.6. Interfaz web: Streamlit	84
5.4. Arquitectura general del sistema	85
5.5. Patrones de diseño empleados	86
5.5.1. Pipe and Filter	87
5.5.2. Repository	87
5.5.3. Strategy	87
5.5.4. Facade	88
5.6. Diseño de la capa de ingesta	88
5.6.1. Estrategia de adquisición del corpus: <i>crawl</i> automático	89
5.6.2. Descomposición en subcomponentes	96
5.6.3. Diseño del recolector	97
5.6.4. Diseño del extractor	98
5.6.5. Diseño del normalizador	99
5.6.6. Diseño del segmentador	99
5.7. Diseño de la capa de indexación vectorial	101
5.7.1. Modelo de <i>embeddings</i>	101

5.7.2.	Estructura de índice: FAISS <code>IndexFlatIP</code>	102
5.7.3.	Contextualización de fragmentos (<i>Contextual Retrieval</i>)	103
5.7.4.	Indexación a nivel de registro y documentos verificados	104
5.7.5.	Persistencia y ciclo de vida del índice	105
5.8.	Diseño de la capa de recuperación	105
5.8.1.	Flujo de ejecución	106
5.8.2.	Elección de k : el número de fragmentos a recuperar .	107
5.8.3.	Umbral de similitud	107
5.8.4.	Diversificación mediante <i>Maximal Marginal Relevance</i>	108
5.8.5.	Recuperación híbrida: canal denso + canal léxico (BM25)	109
5.8.6.	Enrutamiento por categoría e inyección focalizada . .	111
5.8.7.	Consultas de listado y tamaño de contexto dinámico .	112
5.8.8.	Recuperación consciente del historial	112
5.8.9.	Estructura de datos de salida: <code>RetrievedChunk</code>	113
5.8.10.	Reranking neuronal con <i>cross-encoder</i>	113
5.8.11.	Gate de ámbito: rechazo de consultas fuera de dominio	114
5.8.12.	Filtros de coherencia: asignatura y tipo de dato	116
5.8.13.	Control de ruido: penalización por categoría no enrutada	116
5.9.	Diseño de la capa de generación	117
5.9.1.	Elección del modelo de lenguaje	117
5.9.2.	Motor de inferencia: Ollama	119
5.9.3.	Diseño del <i>prompt</i>	119
5.9.4.	Parámetros de decodificación	122
5.10.	Diseño del sistema de citación	123
5.10.1.	Problema y alternativas de diseño	123
5.10.2.	Decisión: enfoque híbrido con verificación	123
5.10.3.	Esquema del objeto <code>Citation</code>	124
5.10.4.	Post-procesado de la respuesta	124
5.11.	Diseño de la capa de interfaz	126
5.11.1.	Elección de la tecnología	126
5.11.2.	Organización visual	127
5.11.3.	Gestión del estado de conversación	128
5.11.4.	<i>Streaming</i> de respuestas	129
5.12.	Diseño del sistema de registro	129
5.12.1.	Niveles de registro	129
5.12.2.	Formato estructurado: JSONL	130
5.12.3.	Esquema de entrada de <i>log</i>	130
5.12.4.	Consideraciones de privacidad	130
5.13.	Control de calidad de la respuesta	131
5.14.	Trazabilidad del diseño a los requisitos	133
5.15.	Resumen del capítulo	136

6. Implementación	139
6.1. Introducción	139
6.2. Estructura del repositorio	140
6.2.1. Correspondencia módulo $\leftrightarrow$ capa de diseño	140
6.2.2. Gestión de dependencias y entorno	141
6.2.3. Fichero de configuración centralizado	142
6.2.4. Convenciones de calidad de código	142
6.3. Capa de ingesta	143
6.3.1. Modelo de datos del pipeline	143
6.3.2. Fetcher: descarga con reintentos y respeto a <code>robots.txt</code>	144
6.3.3. Parser: extracción de texto con patrón Strategy	144
6.3.4. Cleaner: normalización y eliminación de ruido	145
6.3.5. Chunker: fragmentación recursiva con solapamiento	146
6.3.6. Indexación a nivel de registro y documentos verificados	148
6.3.7. Integración del pipeline: <code>ingest_url()</code>	149
6.3.8. Orquestación del corpus: scripts de construcción, revisión e indexación	149
6.3.9. Trazabilidad parcial: ingesta	159
6.4. Capa de indexación	159
6.4.1. Embedder: vectorización con E5 multilingüe	159
6.4.2. VectorStore: índice FAISS con persistencia	161
6.4.3. Persistencia: separación índice/metadatos	161
6.4.4. Trazabilidad parcial: indexación	162
6.5. Capa de recuperación	162
6.5.1. Algoritmo MMR	163
6.5.2. Canal léxico: índice BM25	164
6.5.3. Fusión híbrida mediante RRF	164
6.5.4. Retriever: orquestación del flujo completo	165
6.5.5. Enrutamiento, intención y enriquecimiento de la consulta	167
6.5.6. Reranking con <i>cross-encoder</i>	168
6.5.7. Manejo de incertidumbre: el caso “sin resultados”	168
6.5.8. Capas de calidad: gate de ámbito, filtros de coherencia y control de ruido	168
6.5.9. Trazabilidad parcial: recuperación	169
6.6. Capa de generación	169
6.6.1. Cliente Ollama	170
6.6.2. Constructor de prompts	171
6.6.3. Validador de citas	172
6.6.4. Post-procesado determinista de la respuesta	173
6.6.5. Trazabilidad parcial: generación	174
6.7. Capa de interfaz conversacional	174
6.7.1. Arquitectura de la aplicación	174
6.7.2. Pipeline de consulta	175

6.7.3. Gestión de configuración	176
6.7.4. Trazabilidad parcial: interfaz	176
6.8. Registro estructurado	177
6.8.1. Formateador JSON personalizado	177
6.8.2. Configuración del logger	178
6.8.3. Identificadores de correlación	178
6.8.4. Trazabilidad parcial: registro	179
6.9. Trazabilidad global de la implementación	179
6.10. Pruebas unitarias	180
6.11. Dificultades de implementación	182
6.12. Resumen del capítulo	185
7. Evaluación, pruebas y funcionamiento	187
7.1. Introducción	187
7.2. Protocolo de evaluación	187
7.3. Conjunto de evaluación	188
7.4. Métricas	189
7.5. Resultados cuantitativos	190
7.6. Análisis comparativo: RAG frente a LLM sin contexto	192
7.7. Evaluación automática con RAGAS	193
7.8. Análisis de errores	195
7.9. Latencia y viabilidad local (H3)	196
7.10. Validación de las hipótesis	197
7.11. Demostración del funcionamiento del sistema	198
7.11.1. Interfaz conversacional	198
7.11.2. Interfaz de administración	202
7.12. Discusión de resultados	209
8. Conclusiones y trabajo futuro	211
8.1. Conclusiones generales	211
8.2. Validación de las hipótesis	211
8.3. Cumplimiento de los objetivos específicos	212
8.4. Contribuciones del trabajo	213
8.5. Limitaciones identificadas	214
8.6. Líneas de trabajo futuro	214
8.6.1. Hacia una plataforma académica integral	216
8.7. Uso de la inteligencia artificial en el proyecto	217
8.8. Valoración personal	219
8.9. Acceso al sistema	220
Bibliografía	232
Glosario	233

Índice de figuras

2.1. Diagrama de Gantt del proyecto, desglosado por fases, tareas clave e hitos de entrega, las iteraciones de refinamiento propias de la metodología adoptada se producen <i>dentro</i> de cada fase y no se representan individualmente en el diagrama por claridad visual.	14
3.1. Arquitectura del Transformer	22
3.2. Pipeline del sistema RAG	33
3.3. Imagen del asistente virtual Alhe	46
3.4. Imagen de Unimib	47
3.5. Imagen de urag	48
4.1. Diagrama UML de casos de uso del sistema GRAIA.	72
5.1. Arquitectura general de alto nivel del sistema GRAIA. Las capas inferiores operan en modo <i>batch</i> (fuera de línea) y las superiores en tiempo real ante cada consulta del usuario. . . .	86
5.2. Fases del pipeline de construcción del corpus con puerta de validación intermedia.	93
5.3. Recuperación híbrida del sistema GRAIA	106
5.4. Wireframe de la interfaz de GRAIA	128
5.5. Control de calidad de la respuesta	132
6.1. Árbol de directorios del repositorio de GRAIA.	140
6.2. Dependencias núcleo declaradas en <code>requirements.txt</code>	142
6.3. Modelo <code>Chunk</code> con identificador determinista.	143
6.4. Bucle de descarga con backoff exponencial y comprobación de <code>robots.txt</code>	144
6.5. Parser HTML: eliminación de boilerplate estructural y extracción focalizada.	145
6.6. Eliminación de boilerplate por patrones específicos del dominio UGR/ETSIIT.	146
6.7. Fragmentación recursiva por jerarquía de separadores. . . .	147
6.8. Pipeline completo de ingesta orquestado por <code>ingest_url()</code> . .	149

6.9. Filtrado multinivel de URLs: cuatro comprobaciones secuenciales.	150
6.10. Generación automática de patrones de exclusión temporal. . .	151
6.11. Crawl BFS por categoría con doble conjunto de visitados. . .	152
6.12. Heurísticas de detección de formularios y páginas índice. . .	153
6.13. Pipeline de procesamiento: filtros de calidad, asignación de título y serialización JSONL.	154
6.14. Normalización de URLs: corrección de esquema HTTP → HTTPS.	155
6.15. Clase <code>Embedder</code> : prefijos automáticos y carga diferida del modelo.	160
6.16. Método <code>search()</code> del <code>VectorStore</code> : búsqueda top- k con FAISS. . .	161
6.17. Persistencia del <code>VectorStore</code> : índice binario FAISS + metadatos JSON.	162
6.18. Implementación del algoritmo MMR con matrices precalculadas.	163
6.19. Clase <code>BM25Index</code> : construcción y búsqueda léxica.	164
6.20. Implementación de Reciprocal Rank Fusion (RRF).	165
6.21. Función <code>retrieve()</code> : flujo completo de recuperación híbrida. . .	166
6.22. Cliente Ollama: configuración de parámetros y modo batch. .	170
6.23. Construcción del bloque de contexto con markers <code>[n]</code>	171
6.24. Validación post-hoc de citas: detección y eliminación de markers alucinados.	172
6.25. Inicialización cacheada de componentes con <code>@st.cache_resource</code> . . .	174
6.26. Orquestación del pipeline en <code>handle_query()</code>	175
6.27. Formateador JSONL con campos obligatorios de GRAIA.	177
6.28. Configuración idempotente del sistema de logging.	178
7.1. Interfaz conversacional de GRAIA: identidad visual y zona de conversación.	198
7.2. Respuesta a una consulta en ámbito y a una consulta de contexto.	199
7.3. El <i>gate</i> de ámbito rechaza una consulta fuera de dominio sin generar, con la frase predeterminada de abstención.	200
7.4. Respuesta a partir de un documento verificado de tutores de movilidad, con su cita.	201
7.5. Historial de conversaciones y selección de vista de la interfaz visual de GRAIA.	202
7.6. Panel de administración de GRAIA con sus siete funciones. .	203
7.7. Reconstrucción del corpus: rastreo y descarga de las fuentes oficiales.	204
7.8. Lista de categorías para la reconstrucción del corpus.	205
7.9. Incorporación de un documento verificado al corpus.	206
7.10. Reemplazo o eliminación del documento verificado.	206

7.11. Revisión del corpus: informe resumen y búsqueda por término.	207
7.12. Limpieza del corpus: depuración de documentos irrelevantes o duplicados.	207
7.13. Adición de una fuente específica al corpus.	208
7.14. Indexación: reconstrucción de los índices FAISS y BM25. . . .	208
7.15. Evaluación del rendimiento: lanzamiento de la batería de métri- cas.	209

Índice de cuadros

2.1. Cronograma y distribución de esfuerzo del proyecto.	13
2.2. Recursos hardware empleados en el proyecto.	13
2.3. Resumen del presupuesto estimado del proyecto.	16
2.4. Análisis de riesgos del proyecto y estrategias de mitigación. .	17
3.1. Comparativa de los principales modelos codificadores basados en Transformer.	23
3.2. Comparativa entre los principales LLMs comerciales y abier- tos. MMLU (<i>Massive Multitask Language Understanding</i>) se reporta como precisión en porcentaje. “VRAM Q4” indica la VRAM aproximada necesaria para ejecutar el modelo en cuantización de 4 bits. “Español” refleja la calidad reportada en tareas multilingües en español. Ctx.: ventana de contexto en tokens.	27
3.3. Requisitos aproximados de VRAM según tamaño del modelo y nivel de cuantización. La columna de viabilidad se evalúa respecto a la GPU RTX 4070 Super (12 GB) utilizada en este proyecto.	30
3.4. Comparativa de las principales variantes arquitectónicas de RAG.	36
3.5. Comparativa de modelos de <i>embedding</i> relevantes. “Dim.”: dimensionalidad del vector de <i>embedding</i> producido por el modelo. “Max tok.”: longitud máxima de entrada en <i>tokens</i> . “Local”: si el modelo puede ejecutarse sin API externa. “Re- trieval”: si fue entrenado con un objetivo contrastivo orien- tado a recuperación. MTEB Avg.: puntuación media en el <i>benchmark</i> MTEB [1].	39
3.6. Comparativa de soluciones de almacenamiento y búsqueda vectorial. “Embebida” indica si puede ejecutarse como biblio- teca en el mismo proceso, sin servidor externo.	41
3.7. Tipos de índice FAISS y su adecuación según el volumen de la colección.	41

3.8. Comparativa de estrategias de fragmentación para sistemas RAG.	43
3.9. Comparativa de enfoques para asistentes conversacionales académicos. La fila en negrita corresponde al enfoque adoptado en este TFG.	49
3.10. Comparativa de sistemas concretos de asistentes académicos frente a la propuesta de este TFG (fila en negrita). Trazab.: trazabilidad de fuentes.	50
3.11. Comparativa de lenguajes de programación para el desarrollo de sistemas RAG.	54
3.12. Comparativa de servidores de inferencia local para LLMs. . .	55
3.13. Comparativa de <i>frameworks</i> de interfaz para prototipos de IA. .	56
3.14. Comparativa de bibliotecas de extracción de texto desde PDF en Python.	56
3.15. Comparativa entre enfoques existentes y la propuesta del presente TFG.	58
4.1. Ficha Cockburn del caso de uso CU-01.	73
4.2. Ficha Cockburn del caso de uso CU-02.	73
4.3. Ficha Cockburn del caso de uso CU-03.	74
4.4. Ficha Cockburn del caso de uso CU-04.	74
4.5. Ficha Cockburn del caso de uso CU-05.	75
4.6. Ficha Cockburn del caso de uso CU-06.	75
4.7. Ficha Cockburn del caso de uso CU-07.	76
4.8. Ficha Cockburn del caso de uso CU-08.	76
4.9. Matriz de trazabilidad entre objetivos específicos y requisitos funcionales.	77
4.10. Matriz de trazabilidad entre objetivos específicos y requisitos no funcionales.	77
5.1. Esquema del objeto Chunk producido por la capa de ingesta. .	97
5.2. Esquema del objeto RetrievedChunk devuelto por la capa de recuperación.	113
5.3. Esquema del objeto Citation (bloque de fuentes verificadas). .	124
5.4. Esquema de una entrada de <i>log</i> estructurado.	130
5.5. Correspondencia entre objetivos específicos y secciones de diseño.	134
5.6. Matriz de trazabilidad entre requisitos y elementos de diseño. .	135
6.1. Correspondencia entre subpaquetes del código y capas del diseño.	141
6.2. Distribución del corpus por categoría temática.	158
6.3. Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de ingesta.	159

6.4. Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de indexación.	162
6.5. Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de recuperación.	169
6.6. Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de generación.	174
6.7. Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de interfaz.	176
6.8. Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de registro.	179
6.9. Correspondencia entre objetivos específicos y módulos de implementación.	179
6.10. Trazabilidad global: requisitos $\leftrightarrow$ implementación.	180
6.11. Distribución de tests unitarios por capa del sistema.	180
7.1. Composición del conjunto de evaluación.	189
7.2. Resultados de GRAIA con los tres modelos de generación (74 preguntas). En negrita, el mejor valor de cada fila.	191
7.3. Cada modelo con recuperación frente a sí mismo sin contexto. La alucinación en <i>no-info</i> es el porcentaje de preguntas sin información a las que el modelo responde inventando en vez de abstenerse.	192
7.4. Métricas RAGAS (juez qwen2.5:14b-instruct) sobre las preguntas factuales respondidas por cada modelo (62 en llama; 63 en gemma y qwen; de las 74 del conjunto). En negrita, el mejor valor de cada columna. La <i>faithfulness</i> se promedia tras descartar 5 ítems por fallo de parseo del juez (57–58 ítems efectivos); la <i>answer relevancy</i> , sobre todas las respondidas.	194

Capítulo 1

Introducción

1.1. Contexto y antecedentes

Los sistemas de inteligencia artificial y las aplicaciones de procesamiento de lenguaje natural han experimentado un crecimiento exponencial en los últimos años, transformando múltiples sectores y contextos de aplicación. En particular, la emergencia de los modelos de lenguaje de gran escala (LLM, *Large Language Models*), cimentados sobre la arquitectura Transformer [2], ha redefinido las capacidades de las máquinas para comprender, generar y razonar sobre texto en lenguaje natural [3].

Sin embargo, estos modelos enfrentan desafíos significativos que limitan su aplicabilidad en dominios especializados como el académico. Entre los principales se encuentran: (i) la dependencia de datos de entrenamiento específicos de fecha, lo que genera información desactualizada; (ii) la propensión a generar respuestas plausibles pero incorrectas, fenómeno conocido como *alucinaciones* [4]; (iii) la incapacidad inherente de citar y rastrear las fuentes utilizadas en la generación de respuestas; y (iv) el elevado costo computacional de los modelos de mayor capacidad, que resulta en barreras de acceso para instituciones académicas con recursos limitados.

La arquitectura *Retrieval-Augmented Generation* (RAG) emerge como un enfoque que mitiga estos problemas al combinar dos componentes: (a) un subsistema de recuperación de información que localiza fragmentos relevantes en una base de conocimiento específica del dominio, y (b) un modelo generativo que sintetiza respuestas precisas condicionadas en ese contexto recuperado [5]. Esta aproximación ha demostrado ser particularmente efectiva en contextos empresariales y académicos, permitiendo sistemas que son simultáneamente precisos, trazables y computacionalmente eficientes en hardware de consumo.

En el contexto de la Escuela Técnica Superior de Ingenierías Informática

y de Telecomunicación (ETSIIT) de la Universidad de Granada, existe una cantidad sustancial de documentación oficial, guías docentes, normativas académicas, calendarios, procedimientos administrativos y recursos educativos. Esta documentación constituye un activo de conocimiento que, si se indexa y estructura adecuadamente, puede alimentar un sistema conversacional inteligente capaz de responder consultas académicas con precisión y trazabilidad, mitigando de esta manera la dispersión informativa.

1.2. Motivación

La motivación para este trabajo se articula en múltiples dimensiones:

Motivación académica: El desarrollo de un sistema RAG requiere integración de múltiples disciplinas: recuperación de información, procesamiento de lenguaje natural, arquitecturas vectoriales, aprendizaje automático y evaluación de sistemas. Constituye, por tanto, un caso de estudio completo que permite consolidar competencias transversales en ingeniería de software e inteligencia artificial.

Motivación institucional: La ETSIIT carece actualmente de un asistente conversacional que permita a estudiantes, personal docente y administrativo obtener respuestas precisas, instantáneas y verificables sobre procedimientos académicos, requisitos normativos, horarios y recursos. Un sistema de este tipo reduciría significativamente la carga en los servicios de atención al estudiante, mejoraría la experiencia de usuario al proporcionar acceso 24/7 a información oficial sin ningún tipo de demora y proporcionaría respuestas coherentes y objetivas a cualquier duda en relación con el ámbito informativo académico.

Motivación técnica: Aunque existen soluciones comerciales (OpenAI GPT, Gemini, Claude) que integran RAG, estas presentan limitaciones críticas en contextos académicos: privacidad de datos, costos operacionales, dependencia de proveedores externos, y falta de control sobre la arquitectura. Este trabajo quiere demostrar que es viable construir una solución equivalente utilizando modelos de lenguaje de código abierto ejecutados localmente en hardware especializado de consumo (ej: GPU RTX 4070 Super), proporcionando autonomía institucional y preservando la confidencialidad de datos académicos.

Motivación científica: La hipótesis central es que un sistema RAG bien diseñado puede reducir significativamente las alucinaciones de modelos LLM sin contexto, mientras mantiene latencias aceptables y capacidad de citación verificable. Esta hipótesis contribuye a la comprensión empírica de la efectividad de RAG en dominios especializados cerrados.

Motivación personal: Tras mi grata experiencia con cursos de IA ge-

nerativa y LLMs, me he dado cuenta que estoy ante una tecnología que me fascina con un potencial que, bien usado, puede ayudar a innumerables personas. Este TFG nace de las numerosas dudas académicas surgidas a lo largo de la carrera y de mi ambición por construir un sistema que, con un LLM como núcleo, conecte toda la información institucional de la UGR. El alcance se acota deliberadamente a la ETSIIT y al Grado en Ingeniería Informática para este TFG, sentando los cimientos de un proyecto más amplio.

1.3. Denominación del sistema

El sistema propuesto en este Trabajo Fin de Grado recibe la denominación provisional de **GRAIA**, acrónimo de *Granada Retrieval-Augmented Intelligent Academic-assistant*. Este nombre se ha elegido con tres objetivos complementarios: (i) Acreditar la procedencia geográfica del sistema, siendo Granada, sin cerrar puertas a su posible expansión geográfica; (ii) explicitar su fundamento técnico mediante la referencia directa a la arquitectura *Retrieval-Augmented Generation*, esencia del trabajo; y (iii) subrayar su naturaleza de asistente conversacional inteligente orientado al usuario académico.

Adicionalmente, la denominación evoca a las *Grayas* de la mitología griega, tres hermanas ancianas que compartían un único ojo con el que contemplaban el mundo. Esta imagen constituye una metáfora elocuente del funcionamiento de un sistema RAG: el conocimiento reside distribuido en un corpus documental y se recupera selectivamente, a través de un único canal de visión, para informar cada respuesta generada. A lo largo de esta memoria, las referencias al sistema se realizarán indistintamente mediante las expresiones *el sistema GRAIA*, *GRAIA* o *el asistente propuesto*.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar, implementar y evaluar un sistema conversacional inteligente basado en arquitectura RAG que permita responder consultas académicas sobre la ETSIIT con precisión, trazabilidad y eficiencia computacional, utilizando modelos de lenguaje de escala mediana ejecutados en hardware local.

1.4.2. Objetivos específicos

Para lograr el objetivo general, se han desagregado nueve objetivos específicos:

1. **OE1: Revisión del estado del arte.** Realizar un análisis exhaustivo del estado del arte en: (a) modelos de lenguaje de gran escala y su capacidad de razonamiento; (b) arquitecturas y metodologías RAG, incluyendo variantes contemporáneas; (c) sistemas de recuperación de información vectorial; (d) chatbots académicos y sus diseños; (e) métricas de evaluación en sistemas conversacionales; (f) estudios comparativos entre LLMs con y sin contexto aumentado. Este objetivo proporciona la fundamentación teórica y el posicionamiento de la solución propuesta respecto al estado del conocimiento.
2. **OE2: Pipeline de ingesta de documentos heterogéneos.** Desarrollar un módulo de procesamiento que sea capaz de ingerir, parsear y normalizar documentos de múltiples formatos: archivos PDF (guías docentes, normativas), documentos HTML (páginas web institucionales), bases de datos de texto estructurado (calendarios académicos), y otros formatos relevantes. Este módulo debe manejar robustamente documentos con tablas, listas, imágenes embebidas y metadatos variados.
3. **OE3: Sistema de indexación vectorial con chunking optimizado.** Implementar una estrategia de fragmentación (*chunking*) que divida documentos largos en fragmentos de longitud y semántica apropiadas, y que indexe estos fragmentos en un espacio vectorial mediante *embeddings* densos. Optimizar los parámetros de chunking (tamaño, solapamiento) para maximizar la recuperación posterior de información relevante en contextos académicos específicos.
4. **OE4: Motor de recuperación de fragmentos relevantes.** Desarrollar un subsistema de recuperación híbrida que, dada una consulta en lenguaje natural, combine un canal denso (búsqueda por similitud semántica) y un canal léxico (BM25) mediante *Reciprocal Rank Fusion* (RRF) para identificar y recuperar los fragmentos más relevantes de la base de conocimiento indexada, aplicando filtrado por umbral de similitud y diversificación mediante MMR.
5. **OE5: Integración de LLM local con contexto recuperado.** Integrar un modelo de lenguaje de código abierto ejecutándose localmente, y diseñar un mecanismo de *prompt engineering* que condicione la generación de respuestas en función del contexto recuperado en OE4, asegurando que el modelo utilice efectivamente la información recuperada para formular respuestas precisas y coherentes.
6. **OE6: Sistema de citación trazable.** Implementar un mecanismo que garantice que cada respuesta generada incluya referencias explícitas y verificables a los fragmentos y documentos originales que la sus-

tentan. Este sistema debe ser transparente para el usuario final y permitir auditoría de la trazabilidad.

7. **OE7: Interfaz web funcional.** Desarrollar una aplicación web responsiva y usable que permita a usuarios finales interactuar con el sistema conversacional. La interfaz debe presentar respuestas de forma clara, mostrar citaciones de forma evidente y registrar interacciones para análisis posterior.
8. **OE8: Dataset de evaluación con métricas cuantitativas.** Construir un conjunto de evaluación validado que contenga consultas académicas representativas con respuestas de referencia anotadas manualmente. Definir e implementar métricas cuantitativas para evaluar: (a) exactitud y relevancia de respuestas; (b) veracidad y presencia de alucinaciones; (c) calidad y cobertura de citaciones; y (d) latencia y eficiencia computacional.
9. **OE9: Evaluación comparativa RAG vs. LLM sin contexto.** Realizar un análisis comparativo sistemático entre: (i) el sistema RAG propuesto; (ii) el mismo modelo LLM operando sin recuperación de contexto. Comparar en términos de exactitud, alucinaciones, veracidad, trazabilidad y eficiencia.

1.5. Hipótesis de investigación

Este trabajo se sustenta en las siguientes hipótesis:

- **H1:** Un sistema RAG bien diseñado y optimizado para la ETSIIT alcanzará precisión superior al 80 % en respuestas a consultas académicas, y proporcionará citaciones verificables más del 90 % del tiempo.
- **H2:** El sistema RAG reducirá significativamente la tasa de alucinaciones comparado con un modelo LLM equivalente operando sin contexto recuperado, mejorando así la confiabilidad de respuestas en un dominio donde la precisión es crítica.
- **H3:** Un modelo de lenguaje de código abierto, con menor potencia que los modelos comerciales, ejecutado localmente en hardware especializado de consumo (GPU NVIDIA RTX 4070 Super) es computacionalmente suficiente para lograr respuestas de latencia aceptable (<5 segundos) y calidad comparable a modelos de mayor potencia.

1.6. Estructura de la memoria

Esta memoria se organiza en ocho capítulos, seguidos de la bibliografía y un glosario de términos. Tras el presente capítulo introductorio, la estructura es la siguiente:

Capítulo 2: Presupuesto y Planificación. Detalla la planificación temporal del proyecto, hitos, recursos utilizados y análisis de costes en términos de tiempo y recursos computacionales.

Capítulo 3: Estado del arte. Proporciona una revisión exhaustiva de la literatura y tecnologías subyacentes, incluyendo modelos de lenguaje, arquitecturas RAG, sistemas de recuperación vectorial, evaluación de chatbots académicos y las herramientas del ecosistema RAG.

Capítulo 4: Análisis y objetivos del sistema. Realiza un análisis profundo de requisitos funcionales y no funcionales del sistema propuesto, definiendo casos de uso, restricciones técnicas y especificación del comportamiento esperado.

Capítulo 5: Diseño y arquitectura. Presenta la arquitectura general del sistema, patrones de diseño utilizados, componentes principales y decisiones arquitectónicas fundamentadas en el análisis previo.

Capítulo 6: Implementación. Describe los detalles de implementación de cada componente del sistema, incluyendo tecnologías específicas, fragmentos de código relevantes y soluciones a desafíos técnicos encontrados.

Capítulo 7: Evaluación, pruebas y funcionamiento. Presenta el protocolo de evaluación, resultados cuantitativos y cualitativos, análisis comparativos y demostración del funcionamiento del sistema en escenarios representativos.

Capítulo 8: Conclusiones y trabajo futuro. Sintetiza los hallazgos principales, validación de hipótesis, contribuciones realizadas, limitaciones identificadas y propuestas para investigación y desarrollo futuro.

La memoria se completa con la bibliografía, que compila todas las referencias citadas a lo largo del documento, y un glosario de términos técnicos y acrónimos utilizados.

Cada capítulo ha sido diseñado para entenderlo sin haber leído los anteriores en la medida de lo posible, aunque existe interdependencia natural entre ellos formando parte de un mismo tema. Se recomienda lectura secuencial para una completa comprensión, aunque las personas interesadas

en aspectos técnicos específicos pueden consultar directamente los capítulos 5, 6 y 7.

Capítulo 2

Presupuesto y Planificación

2.1. Introducción

El presente capítulo detalla la planificación temporal, los recursos necesarios y la estimación presupuestaria del proyecto. Una planificación rigurosa constituye un requisito imprescindible en cualquier desarrollo de ingeniería, pues permite anticipar desviaciones, dimensionar adecuadamente los recursos y establecer un marco de referencia objetivo frente al cual medir el avance. Asimismo, el análisis de riesgos asociado, incluido al final del capítulo, forma parte integral de la planificación, ya que identifica los factores que podrían comprometer los objetivos descritos en el Capítulo 1 y establece estrategias de mitigación concretas.

La planificación que se presenta se ha elaborado bajo la hipótesis de una única mente y un único recurso humano (el autor de este proyecto) trabajando en régimen de dedicación parcial, compatible con el resto de obligaciones académicas del grado.

2.2. Metodología de Planificación

Se ha adoptado una metodología de desarrollo **iterativa e incremental** adaptada a las particularidades de un TFG: un único desarrollador, duración acotada y entregable final documental. En lugar de un único ciclo en cascada, el proyecto se organiza en seis fases secuenciales con realimentación entre ellas, permitiendo refinamientos sucesivos del sistema y de la memoria.

Cada fase se caracteriza por (i) un conjunto de tareas concretas, (ii) entregables verificables y (iii) un criterio de finalización. Se valida el avance y ajusta el plan si fuese necesario al cierre de cada fase. Este enfoque permite detectar tempranamente desviaciones técnicas o metodológicas y aplicar

correcciones antes de que se propaguen al resto del proyecto.

2.3. Fases del Proyecto y Planificación Temporal

El proyecto se desarrolla entre el 1 de marzo y el 14 de junio de 2026, con una duración total aproximada de 15 semanas. Se estima una dedicación media de 20 horas semanales, lo que se corresponde con las 300 horas de carga docente asociadas a los 12 créditos del TFG en el Grado en Ingeniería Informática de la UGR.

2.3.1. Fase 1: Análisis del Problema y Estado del Arte

Duración: semanas 1–3 (01/03/2026–21/03/2026).

Esfuerzo estimado: 55 horas.

Tareas principales:

- Definición del alcance, objetivos e hipótesis de investigación.
- Revisión bibliográfica sobre LLMs, arquitecturas RAG, recuperación vectorial y evaluación de sistemas conversacionales.
- Análisis comparativo de soluciones existentes y posicionamiento del trabajo.
- Redacción de los capítulos de introducción y estado del arte.

Entregables: capítulos de introducción y estado del arte validados.

2.3.2. Fase 2: Análisis de Requisitos y Diseño

Duración: semanas 4–5 (22/03/2026–04/04/2026).

Esfuerzo estimado: 40 horas.

Tareas principales:

- Elicitación de requisitos funcionales y no funcionales.
- Identificación de casos de uso representativos.
- Diseño de la arquitectura del sistema GRAIA (pipeline de ingesta, índice vectorial, motor de recuperación, generador, módulo de citación, interfaz de usuario).
- Elección justificada del stack tecnológico (modelo de embeddings, LLM local, motor de indexación).

- Redacción de los Capítulos 4 y 5 de la memoria.

Entregables: Especificación funcional y diseño arquitectónico; Capítulos 4 y 5 validados.

2.3.3. Fase 3: Implementación del Pipeline de Ingesta

Duración: semanas 6–7 (05/04/2026–18/04/2026).

Esfuerzo estimado: 40 horas.

Tareas principales:

- Desarrollo del módulo de scraping web para fuentes de la ETSIIT/UGR.
- Extracción de texto estructurado desde PDFs normativos.
- Limpieza, normalización y deduplicación del corpus.
- Implementación de las estrategias de *chunking* candidatas.
- Generación de *embeddings* e indexación en FAISS.

Entregables: corpus procesado, índice vectorial operativo, logs de ingesta.

2.3.4. Fase 4: Implementación del Núcleo RAG

Duración: semanas 8–10 (19/04/2026–09/05/2026).

Esfuerzo estimado: 65 horas.

Tareas principales:

- Integración del LLM local mediante Ollama.
- Implementación del módulo de recuperación con reordenamiento.
- Diseño e iteración del *prompt* de sistema de GRAIA.
- Implementación del módulo de citación y trazabilidad.
- Desarrollo de la interfaz web con Streamlit.
- Pruebas unitarias y de integración.

Entregables: sistema RAG funcional accesible vía interfaz web.

2.3.5. Fase 5: Evaluación Experimental

Duración: semanas 11–12 (10/05/2026–23/05/2026).

Esfuerzo estimado: 45 horas.

Tareas principales:

- Construcción de un *dataset* de evaluación *ad hoc* con preguntas realistas del alumnado y sus palabras clave y fuentes esperadas, verificadas manualmente.
- Ejecución de experimentos comparando tres modelos de generación (Llama 3.1, Gemma 2 y Qwen 2.5), cada uno con recuperación y como *baseline* sin contexto.
- Cálculo de métricas automáticas: precisión factual, recall de recuperación, tasa de citación verificable y abstención correcta, complementadas con las métricas RAGAS *faithfulness* y *answer relevancy*.
- Análisis manual de errores sobre las respuestas del sistema.
- Análisis comparativo y de errores.
- Redacción del Capítulo 7.

Entregables: *dataset* de evaluación, resultados experimentales, Capítulo 7 validado.

2.3.6. Fase 6: Redacción Final, Revisión y Defensa

Duración: semanas 13–15 (24/05/2026–14/06/2026).

Esfuerzo estimado: 55 horas.

Tareas principales:

- Redacción del Capítulo 8 (conclusiones, contribuciones, trabajo futuro).
- Revisión integral de la memoria (coherencia, estilo, bibliografía, figuras).
- Elaboración del material de defensa (presentación).
- Ensayos de defensa oral.

Entregables: memoria final entregada en la plataforma de la UGR; presentación de defensa.

2.3.7. Cronograma General

La Tabla 2.1 resume la distribución temporal de las fases.

Fase	Inicio	Fin	Horas
F1: Análisis y Estado del Arte	01/03/2026	21/03/2026	55
F2: Requisitos y Diseño	22/03/2026	04/04/2026	40
F3: Pipeline de Ingesta	05/04/2026	18/04/2026	40
F4: Núcleo RAG	19/04/2026	09/05/2026	65
F5: Evaluación Experimental	10/05/2026	23/05/2026	45
F6: Redacción Final y Defensa	24/05/2026	14/06/2026	55
Total			300

Cuadro 2.1: Cronograma y distribución de esfuerzo del proyecto.

2.4. Recursos del Proyecto

2.4.1. Recursos Humanos

El proyecto es desarrollado por un único recurso: el autor del proyecto, en el rol de *ingeniero junior en desarrollo de software e investigación aplicada*, bajo la supervisión académica del tutor asignado. El tutor participa en reuniones periódicas de seguimiento sin coste imputable al proyecto.

2.4.2. Recursos Hardware

El desarrollo y la ejecución local del sistema GRAIA requieren un equipo con capacidad suficiente para inferencia de LLMs cuantizados. La Tabla 2.2 detalla la configuración utilizada.

Componente	Especificación	Coste (€)
GPU	NVIDIA GeForce RTX 4070 Super (12 GB GDDR6X)	650
CPU	AMD Ryzen 7 7700 (Zen 4, 8 núcleos, AM5)	340
Memoria RAM	32 GB DDR5 a 6000 MHz	150
Almacenamiento	SSD Crucial T500 NVMe PCIe 4.0 de 1 TB	90
Placa base	Gigabyte B650M (micro-ATX, AM5)	130
Fuente de alimentación	Corsair RM650e (650 W, 80+ Gold, modular)	90
Caja	Corsair 4000D Airflow	90
Periféricos (monitor, teclado, ratón)	—	250
Total hardware		1.790

Cuadro 2.2: Recursos hardware empleados en el proyecto.

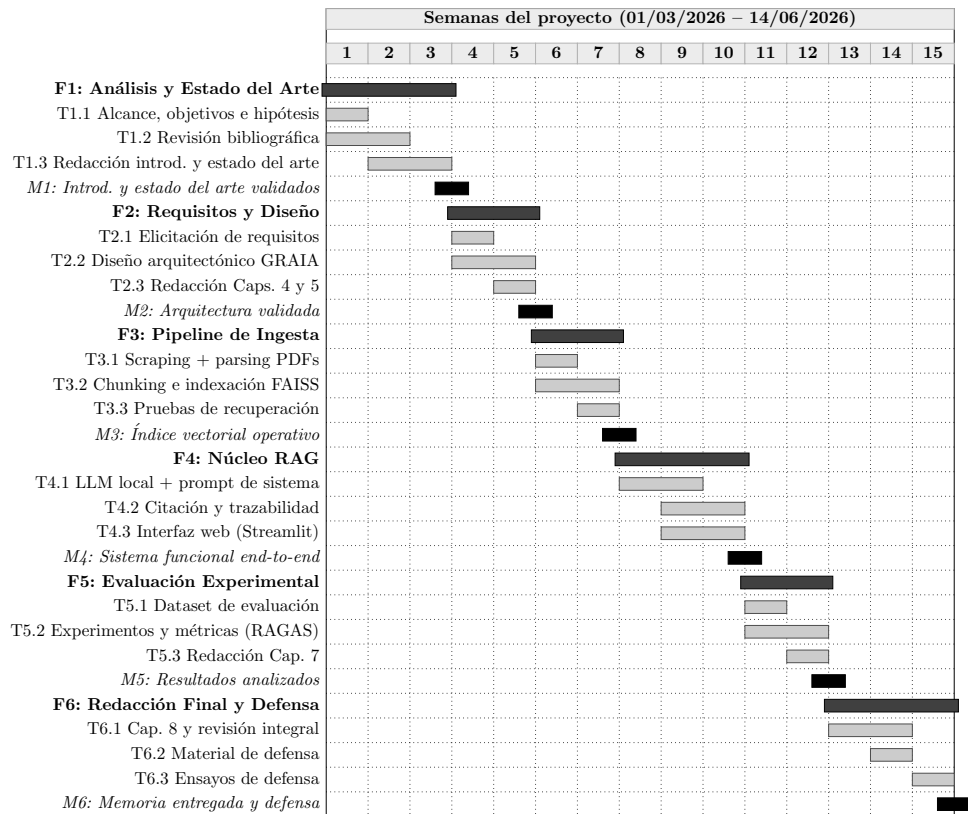


Figura 2.1: Diagrama de Gantt del proyecto, desglosado por fases, tareas clave e hitos de entrega, las iteraciones de refinamiento propias de la metodología adoptada se producen *dentro* de cada fase y no se representan individualmente en el diagrama por claridad visual.

La elección de una GPU de gama media-alta (RTX 4070 Super) se justifica por sus 12 GB de VRAM, suficientes para ejecutar modelos cuantizados a 4 bits de hasta 13B de parámetros, cumpliendo el requisito de despliegue local sin dependencia de servicios externos. El resto de componentes se ha seleccionado de forma coherente con ese perfil: una CPU Ryzen 7 7700 con 8 núcleos garantiza un rendimiento adecuado durante el preprocesado del corpus y la ingesta concurrente; 32 GB de RAM DDR5 evitan cuellos de botella al cargar índices vectoriales y varios procesos simultáneamente; y una fuente certificada 80+ Gold proporciona la eficiencia y estabilidad exigidas por cargas prolongadas de inferencia local.

2.4.3. Recursos Software

Todo el software empleado es de código abierto o de uso gratuito con fines académicos, lo que reduce el coste del proyecto y garantiza la reproducibilidad:

- **Lenguaje y entorno:** Python 3.11, Visual Studio Code.
- **Modelos y frameworks:** Ollama, sentence-transformers, FAISS, rankbm25.
- **Procesamiento de documentos:** BeautifulSoup, lxml, PyMuPDF.
- **Interfaz de usuario:** Streamlit.
- **Configuración y validación:** PyYAML, Pydantic.
- **Evaluación:** RAGAS, pandas.
- **Calidad de código y testing:** pytest, Ruff, mypy.
- **Control de versiones:** Git y GitHub.
- **Redacción:** TeXstudio, L^AT_EX, BibT_EX.
- **Gestión bibliográfica:** Zotero.

2.5. Presupuesto

El presupuesto del proyecto se estructura en tres componentes: personal, amortización del hardware y software. El software es íntegramente gratuito, por lo que su coste imputable es nulo.

2.5.1. Coste de Personal

Se aplica una tarifa de referencia de **25 €/hora** para un ingeniero informático junior en España, cifra acorde con los márgenes habituales del sector TIC en 2026 para este perfil. Con 300 horas de dedicación, el coste de personal asciende a $300 \times 25 = 7.500$ €.

2.5.2. Amortización del Hardware

El coste total del hardware (1.790 €) se amortiza linealmente a lo largo de su vida útil fiscal, establecida en **4 años (48 meses)** para equipos informáticos según la tabla de coeficientes de amortización lineal de la Ley del Impuesto sobre Sociedades (art. 12). Dado que el proyecto no consume el equipo en su totalidad, sino únicamente durante sus 3,5 meses de duración, al presupuesto solo se imputa la fracción proporcional correspondiente a dicho período: $(1.790/48) \times 3,5 \approx 131$ €.

2.5.3. Coste de Software

Todas las herramientas utilizadas son de código abierto o disponen de licencias gratuitas para uso académico. El coste imputable es **0 €**.

2.5.4. Otros Costes

Se imputa un coste menor de electricidad estimado en 40 € para el total del proyecto, asociado al consumo del equipo durante las fases de desarrollo y, particularmente, de experimentación con el LLM local.

2.5.5. Resumen del Presupuesto

Concepto	Importe (€)
Personal (300 h $\times$ 25 €/h)	7.500
Amortización hardware (3,5 meses de 48)	131
Software (código abierto)	0
Electricidad y otros	40
Subtotal	7.671
IVA (21 %)	1.611
Total presupuesto	9.282

Cuadro 2.3: Resumen del presupuesto estimado del proyecto.

El importe total estimado, impuestos incluidos, asciende a **9.282 €**. Este presupuesto debe interpretarse como una estimación académica del coste que

tendría desarrollar el mismo sistema en un contexto profesional, no como un gasto efectivamente desembolsado por el estudiante.

2.6. Análisis de Riesgos

Se han identificado los principales riesgos que podrían comprometer los objetivos del proyecto y se ha definido, para cada uno, una estrategia de mitigación concreta. La Tabla 2.4 sintetiza el análisis, empleando una escala cualitativa (Baja/Media/Alta) para probabilidad e impacto.

Riesgo	Prob.	Imp.	Mitigación
Alucinaciones del LLM en dominio académico	Alta	Alto	Diseño estricto del prompt, sistema de citación obligatoria, umbral de confianza en el retrieval, mensaje de <i>no sé</i> cuando el contexto es insuficiente.
Corpus documental insuficiente o de baja calidad	Media	Alto	Diversificación de fuentes (web + PDFs normativos), validación manual de muestras, iteración del pipeline de ingesta.
Fallo de hardware (GPU, almacenamiento)	Baja	Alto	Copias de seguridad periódicas en la nube (GitHub), código portable que permita migrar el entorno si fuese necesario.
Desviación temporal respecto al cronograma	Media	Medio	Planificación con margen, seguimiento semanal de avance frente a la Tabla 2.1, reducción controlada del alcance en tareas no críticas.
Limitaciones del LLM local (latencia, calidad)	Media	Medio	Evaluación comparativa de varios modelos (Llama 3.1, Gemma 2, Qwen 2.5), uso de cuantización 4-bit, optimización del contexto.
Pérdida de datos o código fuente	Baja	Alto	Control de versiones con Git, repositorio remoto en GitHub, <i>snapshots</i> periódicos.
Dificultad en la construcción del dataset de evaluación	Media	Alto	Comenzar la construcción en paralelo a la Fase 3, iterar con preguntas reales de estudiantes, validación cruzada de respuestas de referencia.

Cuadro 2.4: Análisis de riesgos del proyecto y estrategias de mitigación.

2.7. Conclusiones del Capítulo

La planificación presentada distribuye las 300 horas de carga docente asociadas al TFG en seis fases equilibradas, con una fase inicial de análisis robusta, un bloque central de implementación e integración del sistema GRAIA y un tramo final dedicado a evaluación, redacción y defensa. El presupuesto estimado, de 9.282 € impuestos incluidos, refleja un coste realista para un proyecto de estas características en el contexto profesional español de 2026. Finalmente, el análisis de riesgos identifica las amenazas principales (especialmente la alucinación de los LLMs y la calidad del corpus) y define estrategias de mitigación que se integrarán en el diseño y en la evaluación del sistema.

Una vez delimitado el *qué* (Capítulo 1) y planificado el *cómo* (presente capítulo), el siguiente capítulo revisa el *estado del arte* en el que se enmarca la propuesta, sentando las bases técnicas sobre las que se construirá el análisis funcional del sistema.

Capítulo 3

Estado del Arte

3.1. Introducción

El presente capítulo ofrece una revisión crítica del estado actual de las tecnologías y enfoques relevantes para el desarrollo de un sistema conversacional inteligente basado en *Retrieval-Augmented Generation* (RAG) aplicado al dominio académico. La revisión se estructura siguiendo un orden temático-metodológico: se parte de los fundamentos de los modelos de lenguaje de gran tamaño (LLMs), se introduce el paradigma RAG y sus componentes esenciales, recuperación vectorial, *embeddings* y estrategias de fragmentación (*chunking*), se analiza el panorama de los chatbots académicos institucionales, se examinan las metodologías de evaluación específicas para sistemas RAG y se revisan las herramientas y tecnologías del ecosistema RAG (lenguajes de programación, servidores de inferencia, *frameworks* de interfaz y bibliotecas de extracción documental). El capítulo concluye con el posicionamiento del presente trabajo respecto a las propuestas existentes y una síntesis de las conclusiones derivadas de la revisión.

El criterio de selección de las fuentes ha priorizado publicaciones indexadas en venues de referencia (como arXiv con alto impacto) y trabajos recientes publicados entre 2017 y 2024 que marcan hitos técnicos relevantes, se ha elegido este intervalo de tiempo debido a que el comienzo de la arquitectura Transformer fue en 2017 y las publicaciones hasta 2024 han tenido tiempo suficiente para ser contrastadas por la comunidad, reduciendo el riesgo de basar el trabajo en resultados aún no replicados, aunque cabe destacar que la evolución de este arte avanza a una gran velocidad.

3.2. Modelos de Lenguaje de Gran Tamaño (LLMs)

Los modelos de lenguaje de gran tamaño (LLMs) constituyen el núcleo generativo de los sistemas conversacionales modernos. Su evolución en los últimos años ha redefinido el procesamiento del lenguaje natural (NLP, *Natural Language Processing*), pasando de arquitecturas recurrentes con limitaciones de paralelización a modelos masivos basados en atención que exhiben capacidades emergentes a escala. Esta sección analiza en profundidad la arquitectura fundacional, las principales familias de modelos, las técnicas que permiten su despliegue eficiente y las limitaciones que motivan el uso de sistemas aumentados como RAG.

3.2.1. La arquitectura Transformer

La arquitectura *Transformer*, propuesta por Vaswani et al. [2] en el trabajo seminal “*Attention Is All You Need*”, constituye el pilar sobre el que se construyen todos los LLMs modernos. Frente a las redes recurrentes, como las RNN (*Recurrent Neural Networks*), las LSTM (*Long Short-Term Memory*) [6] y las GRU (*Gated Recurrent Units*) [7], que procesaban las secuencias de forma secuencial, limitando la paralelización y sufriendo degradación con dependencias largas, el Transformer introduce un mecanismo de **auto-atención** (*self-attention*) que permite a cada token atender simultáneamente a todos los demás tokens de la secuencia, capturando dependencias a cualquier distancia en tiempo constante.

El mecanismo de atención multi-cabeza (*Multi-Head Attention*) se define formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3.1)$$

donde Q (*queries*), K (*keys*) y V (*values*) son proyecciones lineales de las representaciones de entrada, y d_k es la dimensión de las claves, utilizada como factor de normalización para evitar que el *softmax* se sature con valores muy grandes [2].

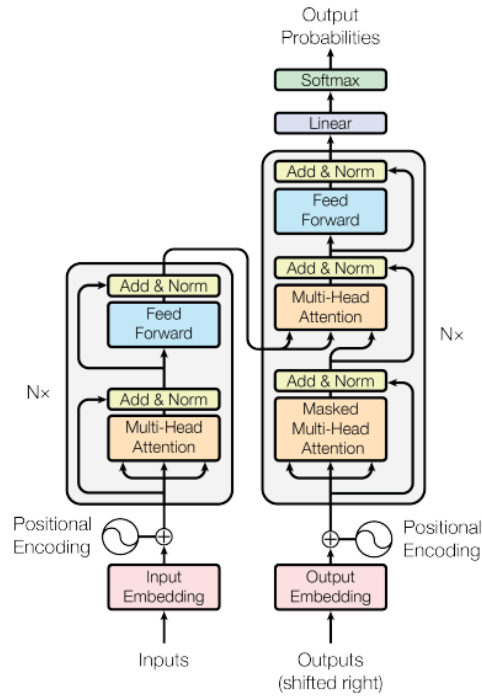
Intuitivamente, el mecanismo funciona como un sistema de consulta selectiva: cada token genera una *pregunta* (Q : “¿qué información necesito?”), una *etiqueta* (K : “esto es lo que yo ofrezco”) y un *contenido* (V : “esta es mi información útil”). Para cada token, el modelo compara su pregunta con las etiquetas de todos los demás tokens, las coincidencias más fuertes reciben mayor peso mediante la función *softmax*, y la respuesta se construye como una combinación ponderada de los contenidos V . La división por $\sqrt{d_k}$ es necesaria porque, a medida que la dimensión d_k crece, los productos escalares QK^T tienden a valores muy grandes que empujan al *softmax* hacia

distribuciones casi deterministas (con gradientes cercanos a cero), dificultando el aprendizaje. El uso de múltiples cabezas permite al modelo atender a diferentes subespacios de representación simultáneamente, capturando relaciones sintácticas, semánticas y posicionales en paralelo [2].

La arquitectura original del Transformer se compone de dos bloques principales: un **codificador** (*encoder*) que genera representaciones contextuales bidireccionales de la entrada, y un **decodificador** (*decoder*) que genera la salida de forma auto-regresiva, atendiendo tanto a sus propios tokens previos como a las representaciones del codificador mediante atención cruzada (*cross-attention*). Cada bloque apila capas de atención multi-cabeza con tres componentes adicionales: (a) redes *feed-forward* posicionales, es decir, redes neuronales densas de dos capas aplicadas independientemente a cada posición de la secuencia, que introducen no linealidad y expanden la capacidad representativa del modelo; (b) conexiones residuales [8], que suman la entrada de cada subcapa a su salida (de la forma $\text{output} = \text{subcapa}(x) + x$), facilitando el flujo del gradiente a través de redes profundas y mitigando el problema de la degradación; y (c) normalización de capa (*Layer Normalization*) [9], que normaliza las activaciones a media cero y varianza unitaria a lo largo de la dimensión de características, estabilizando y acelerando el entrenamiento.

Para codificar la posición de los tokens en la secuencia, información que la auto-atención por sí sola no captura, el diseño original emplea codificaciones posicionales sinusoidales. Trabajos posteriores han explorado alternativas como las codificaciones posicionales rotatorias RoPE (*Rotary Position Embedding*) [10], adoptadas por la mayoría de los LLMs actuales (LLaMA, Mistral, Qwen), que ofrecen mejor generalización a secuencias más largas que las vistas durante el entrenamiento.

La Figura 3.1 muestra la arquitectura general del Transformer y cómo las tres variantes principales (solo codificador, solo decodificador y secuencia a secuencia) se derivan de esta estructura base.



Familias de modelos derivadas:

Solo Encoder	Enc-Decoder	Solo Decoder
BERT, RoBERTa DeBERTa	T5, BART Flan-T5	GPT, LLaMA Mistral, Qwen

Figura 3.1: Arquitectura del Transformer original [2] y tres familias de modelos derivadas. El codificador (izquierda) emplea atención bidireccional; el decodificador (derecha) usa atención causal enmascarada y atención cruzada. Las flechas laterales representan las conexiones residuales que suman la entrada de cada subcapa a su salida, facilitando el entrenamiento de arquitecturas profundas.

3.2.2. Modelos codificadores (*encoder-only*)

Los modelos basados exclusivamente en el codificador del Transformer procesan la entrada de forma **bidireccional**: cada token tiene acceso al contexto completo de la secuencia (tanto a los tokens anteriores como a los posteriores). Esta propiedad los hace especialmente adecuados para tareas discriminativas y de comprensión del lenguaje.

BERT [11] fue el modelo pionero de esta familia. Su innovación princi-

pal fue el preentrenamiento mediante *Masked Language Modeling* (MLM), en el que se enmascara aleatoriamente el 15 % de los tokens de entrada y el modelo debe predecirlos a partir del contexto circundante. Adicionalmente, BERT emplea una tarea de *Next Sentence Prediction* (NSP) para capturar relaciones entre pares de oraciones. BERT fue preentrenado sobre Books-Corpus y Wikipedia en inglés, totalizando aproximadamente 3.300 millones de palabras, y se publicó en dos tamaños: BERT_{BASE} (110M parámetros, 12 capas) y BERT_{LARGE} (340M parámetros, 24 capas).

El éxito de BERT motivó una serie de variantes que optimizaron diferentes aspectos:

- **RoBERTa** [12]: eliminó la tarea NSP, incrementó el tamaño del *batch* y el corpus de entrenamiento, y demostró que BERT estaba significativamente subentrenado. Estos ajustes resultaron en mejoras consistentes en todos los *benchmarks*.
- **ALBERT** [13]: introdujo factorización de la matriz de *embeddings* y compartición de parámetros entre capas, reduciendo drásticamente el número de parámetros (18x menos que BERT_{LARGE}) con rendimiento competitivo.
- **DistilBERT** [14]: aplicó destilación de conocimiento para comprimir BERT a un modelo un 40 % más pequeño y un 60 % más rápido, reteniendo el 97 % del rendimiento en el *benchmark* GLUE (*General Language Understanding Evaluation*).
- **DeBERTa** [15]: mejoró el mecanismo de atención mediante la descomposición de contenido y posición (*disentangled attention*), logrando resultados estado del arte en SuperGLUE, la versión extendida y más exigente de GLUE.

La Tabla 3.1 resume las características principales de los modelos codificadores más relevantes.

Modelo	Parámetros	Capas	Año	Innovación principal
BERT _{BASE}	110M	12	2018	MLM + NSP bidireccional
BERT _{LARGE}	340M	24	2018	Mayor capacidad
RoBERTa	355M	24	2019	Entrenamiento optimizado
ALBERT	12M	12	2019	Compartición de parámetros
DistilBERT	66M	6	2019	Destilación de conocimiento
DeBERTa v3	304M	24	2021	Atención desacoplada

Cuadro 3.1: Comparativa de los principales modelos codificadores basados en Transformer.

La Tabla 3.1 evidencia dos líneas evolutivas complementarias: por un lado, la mejora de la calidad de representación incrementando capacidad o refinando el preentrenamiento (de BERT a RoBERTa y DeBERTa v3); por otro, la compresión del modelo preservando la mayor parte del rendimiento mediante destilación (DistilBERT) o compartición de parámetros (ALBERT). Esta tensión calidad–coste es la que orienta la elección posterior del codificador de *embeddings*, donde priman los modelos que maximizan la calidad de recuperación dentro de un presupuesto de memoria ejecutable en GPU de consumo.

En el contexto del presente TFG, los modelos codificadores son especialmente relevantes para la generación de *embeddings* densos utilizados en el componente de recuperación del sistema RAG (véase Sección 3.4).

3.2.3. Modelos decodificadores (*decoder-only*)

Los modelos basados exclusivamente en el decodificador emplean **atención causal**: cada token solo puede atender a los tokens previos en la secuencia, nunca a los futuros. Esta restricción, implementada mediante una máscara triangular en la matriz de atención, los convierte en generadores auto-regresivos naturales: dado un contexto, predicen el siguiente token más probable de forma iterativa. Esta familia domina el panorama actual de los LLMs de propósito general.

La familia GPT

La serie *Generative Pre-trained Transformer* (GPT) de OpenAI ha marcado los hitos principales en la evolución de los modelos decodificadores:

- **GPT-2** [16]: con 1.500 millones de parámetros, demostró que un modelo de lenguaje suficientemente grande podía realizar tareas de NLP sin entrenamiento específico (*zero-shot*), simplemente formulando la tarea como continuación de texto.
- **GPT-3** [3]: escaló a 175.000 millones de parámetros y formalizó el paradigma de *in-context learning*, donde el modelo realiza tareas proporcionándole ejemplos en el *prompt* (*few-shot learning*) sin modificar sus pesos. Este trabajo demostró empíricamente la relación entre escala y capacidades emergentes.
- **GPT-4** [17]: modelo multimodal (texto e imagen) con mejoras sustanciales en razonamiento, seguimiento de instrucciones y reducción de alucinaciones. Su arquitectura exacta no ha sido publicada oficialmente; análisis independientes sugieren que emplea una *Mixture of*

Experts (MoE), aunque el número de parámetros totales no ha sido confirmado por OpenAI.

Un avance clave en la usabilidad de estos modelos fue la introducción del *alineamiento mediante retroalimentación humana* (RLHF, *Reinforcement Learning from Human Feedback*) [18], aplicado en InstructGPT y posteriormente en ChatGPT. Esta técnica de *fine-tuning* entrena al modelo para seguir instrucciones y generar respuestas alineadas con las preferencias humanas, transformando un modelo de lenguaje general en un asistente conversacional.

Modelos abiertos

Frente a los modelos comerciales de código cerrado, la comunidad ha desarrollado alternativas abiertas de calidad competitiva que permiten el despliegue local, el ajuste fino y la inspección del modelo. Los más relevantes son:

- **LLaMA y LLaMA 2** [19]: publicados por Meta, establecieron que modelos más pequeños entrenados con más datos pueden igualar o superar a modelos mayores. LLaMA 2 introdujo variantes instruccionales (*LLaMA-2-Chat*) con alineamiento RLHF, disponibles en tamaños de 7B, 13B y 70B parámetros.
- **LLaMA 3** [20]: amplió significativamente el corpus de entrenamiento (15 billones de tokens), la ventana de contexto (hasta 128K tokens) y el rendimiento en razonamiento y código. Disponible en variantes de 8B y 70B.
- **Mistral 7B** [21]: introdujo *Grouped-Query Attention* (GQA) y *Sliding Window Attention* (SWA) para mayor eficiencia, superando a LLaMA-2 13B en la mayoría de *benchmarks* pese a tener casi la mitad de parámetros. Su variante **Mixtral 8x7B** [22] implementa una arquitectura *Mixture of Experts* (MoE) con 46.7B parámetros totales pero solo 12.9B activos por inferencia.
- **Qwen 2.5** [23]: familia de modelos desarrollada por Alibaba con excelente rendimiento multilingüe, disponible en tamaños desde 0.5B hasta 72B, con variantes especializadas en código y matemáticas.
- **Gemma 2** [24]: modelos ligeros de Google (2B y 9B) diseñados específicamente para despliegue en hardware limitado, con rendimiento competitivo frente a modelos de mayor tamaño.

3.2.4. Modelos secuencia a secuencia (*encoder-decoder*)

Los modelos secuencia a secuencia utilizan la arquitectura Transformer completa: un codificador que procesa la entrada bidireccionalmente y un decodificador que genera la salida de forma auto-regresiva, conectados mediante atención cruzada. Son especialmente adecuados para tareas de transformación de texto (traducción, resumen, reformulación).

- **T5** [25]: *Text-to-Text Transfer Transformer*, propuesto por Google, reformula todas las tareas de NLP como problemas de texto a texto. Por ejemplo, la clasificación de sentimiento se formula como “classify: *This movie is great*” → “positive”. Este enfoque unificado simplifica la interfaz de *fine-tuning* y permite la comparación justa entre tareas. T5 fue publicado en tamaños desde 60M hasta 11B parámetros.
- **BART** [26]: combina un codificador tipo BERT con un decodificador auto-regresivo, preentrenado con objetivos de *denoising* (reconstruir texto corrupto). Especialmente eficaz en tareas de generación condicional como resumen y traducción.
- **Flan-T5** [27]: versión de T5 ajustada con instrucciones (*instruction tuning*) sobre más de 1.800 tareas, demostrando que el ajuste instruccional mejora sustancialmente las capacidades *zero-shot* y *few-shot*.

Aunque los modelos decodificadores dominan actualmente el panorama de los LLMs de propósito general, los modelos encoder-decoder mantienen relevancia en aplicaciones específicas como la generación aumentada por recuperación, donde el codificador procesa los documentos recuperados y el decodificador genera la respuesta, y en tareas de transformación de texto con entrada y salida claramente diferenciadas.

3.2.5. Comparativa: modelos comerciales frente a modelos abiertos

La elección entre modelos comerciales (accesibles mediante API) y modelos abiertos (ejecutables localmente) constituye una decisión arquitectónica fundamental con implicaciones en coste, privacidad, latencia y control. La Tabla 3.2 presenta una comparación multidimensional de los modelos más relevantes.

La columna “VRAM Q4” de la Tabla 3.2 revela una restricción práctica fundamental: en una GPU (*Graphics Processing Unit*) de consumo con 12 GB de VRAM (como la RTX 4070 Super), solo los modelos de hasta 9B parámetros en cuantización de 4 bits son ejecutables, lo que acota la selección a Llama 3.1 8B, Mistral 7B, Qwen 2.5 7B, Gemma 2 9B y Phi-3 mini.

Modelo	Par.	Licencia	Ctx.	Local	VRAM Q4	Español	MMLU
<i>Modelos comerciales (API)</i>							
GPT-4o	~200B MoE	Propietaria	128K	No	N/A	Muy bueno	88.7
Claude 3.5	No pub.	Propietaria	200K	No	N/A	Muy bueno	88.7
Gemini 1.5 Pro	No pub.	Propietaria	1M	No	N/A	Muy bueno	85.9
<i>Modelos abiertos (ejecución local)</i>							
Llama 3.1 8B Inst.	8B	Meta Lic.	128K	Sí	≈5 GB	Bueno	68.4
Llama 3.1 70B	70B	Meta Lic.	128K	Sí	>40 GB	Muy bueno	82.0
Mistral 7B Inst.	7.3B	Apache 2.0	32K	Sí	≈4.5 GB	Aceptable	62.5
Mixtral 8x7B	46.7B	Apache 2.0	32K	Sí	≈26 GB	Bueno	70.6
Qwen 2.5 7B Inst.	7B	Apache 2.0	128K	Sí	≈4.5 GB	Bueno	74.2
Qwen 2.5 72B	72B	Qwen Lic.	128K	Sí	>40 GB	Muy bueno	85.3
Gemma 2 9B Inst.	9B	Gemma Lic.	8K	Sí	≈6 GB	Bueno	71.3
Phi-3 mini [28]	3.8B	MIT	128K	Sí	≈2.5 GB	Regular	69.9

Cuadro 3.2: Comparativa entre los principales LLMs comerciales y abiertos. MMLU (*Massive Multitask Language Understanding*) se reporta como precisión en porcentaje. “VRAM Q4” indica la VRAM aproximada necesaria para ejecutar el modelo en cuantización de 4 bits. “Español” refleja la calidad reportada en tareas multilingües en español. Ctx.: ventana de contexto en tokens.

Los modelos comerciales ofrecen el mejor rendimiento absoluto, pero presentan desventajas críticas: (i) dependencia de un proveedor externo, (ii) envío de datos institucionales a servidores de terceros, incompatible con los requisitos de privacidad del sistema propuesto, (iii) coste por token que escala con el uso, e (iv) imposibilidad de personalización profunda del modelo. Los modelos abiertos, por contra, permiten ejecución local con total control sobre los datos, ajuste fino sobre dominios específicos y coste fijo vinculado únicamente al hardware disponible.

3.2.6. Leyes de escalado y tamaños de modelos

Una de las observaciones más significativas en la investigación de LLMs ha sido la relación predecible entre la escala del modelo, la cantidad de datos de entrenamiento, el presupuesto computacional y el rendimiento resultante. Kaplan et al. [29] formalizaron estas **leyes de escalado** (*scaling laws*), demostrando que la pérdida del modelo sigue una ley de potencia respecto al número de parámetros, el tamaño del *dataset* y la cantidad de computación:

$$L(N) \approx \left(\frac{N_c}{N} \right)^{\alpha_N} \quad (3.2)$$

donde L es la pérdida, N el número de parámetros y $\alpha_N \approx 0.076$ un exponente empírico. Estas relaciones implican que duplicar el rendimiento requiere un incremento de aproximadamente un orden de magnitud en parámetros o datos.

El trabajo de **Chinchilla** [30] refinó estas leyes, demostrando que la mayoría de los modelos existentes estaban sobreescalados en parámetros pero subentrenados en datos. La regla de Chinchilla establece que, para un presupuesto computacional fijo, el número óptimo de tokens de entrenamiento debería ser aproximadamente 20 veces el número de parámetros. Este hallazgo influyó directamente en el diseño de modelos posteriores como LLaMA, que priorizó más datos de entrenamiento sobre más parámetros, logrando con 13B parámetros rendimiento comparable a GPT-3 (175B). Esto dio lugar a que, modelos en el rango de 7B-13B parámetros, puedan ofrecer un compromiso óptimo entre rendimiento y viabilidad de ejecución en hardware de consumo (ejemplo: GPU RTX 4070 Super con 12 GB de VRAM), especialmente cuando se combinan con técnicas de cuantización, las cuales se verán a continuación.

3.2.7. Técnicas de optimización para despliegue

La ejecución de LLMs en hardware de consumo requiere técnicas que reduzcan los requisitos de memoria y computación sin degradar excesivamente la calidad de las respuestas. Las principales técnicas son:

Cuantización

La cuantización reduce la precisión numérica de los pesos del modelo, disminuyendo el consumo de memoria y acelerando la inferencia [31]. Un modelo de 7B parámetros en punto flotante de 16 bits (FP16, *half-precision*) requiere aproximadamente 14 GB de VRAM ($7 \times 10^9 \times 2$ bytes); cuantizado a 4 bits, este requisito se reduce a unos 4 GB ($7 \times 10^9 \times 0,5$ bytes más estructuras auxiliares).

- **GPTQ** [31]: cuantización post-entrenamiento basada en el método *Optimal Brain Quantization*, que minimiza el error de reconstrucción capa por capa. Permite cuantización a 4 y 3 bits con pérdida mínima de perplexidad.
- **GGUF** (anteriormente GGML): formato de cuantización desarrollado por el proyecto `llama.cpp`¹, que permite la ejecución de modelos cuantizados tanto en CPU como en GPU. Soporta esquemas mixtos de cuantización (Q4_K_M, Q5_K_M, etc.) que asignan mayor precisión a las capas más sensibles del modelo, preservando aproximadamente el 97 % de la perplexidad original según los *benchmarks* publicados por el proyecto.

¹<https://github.com/ggerganov/llama.cpp>

- **AWQ** [32]: *Activation-Aware Weight Quantization*, que protege selectivamente los pesos más relevantes para la calidad de las activaciones, logrando mejor retención de calidad que la cuantización uniforme.
- **bitsandbytes (BnB)**: biblioteca desarrollada por Dettmers et al. que permite la cuantización dinámica a 8 y 4 bits durante la carga del modelo [33], integrada nativamente con el ecosistema HuggingFace *Transformers*. Su modo `nf4` (*NormalFloat 4-bit*) se emplea como base para QLoRA.

Ajuste fino eficiente en parámetros (PEFT)

Las técnicas PEFT (*Parameter-Efficient Fine-Tuning*) permiten adaptar un LLM a un dominio específico sin modificar la totalidad de sus pesos, reduciendo drásticamente los requisitos de memoria y computación:

- **LoRA** [34]: *Low-Rank Adaptation*, que congela los pesos originales del modelo e inyecta matrices de bajo rango entrenables en las capas de atención. Reduce el número de parámetros entrenables en un factor de $\sim 10.000\times$, permitiendo el ajuste fino de modelos de 7B en una sola GPU de consumo.
- **QLoRA** [33]: combina la cuantización a 4 bits con LoRA, habilitando el ajuste fino de modelos de 65B parámetros en una única GPU de 48 GB, o modelos de 7B–13B en GPUs de 12–16 GB.

Optimizaciones de inferencia

Además de la cuantización, existen técnicas que aceleran la generación de texto sin modificar los pesos:

- **KV-Cache**: técnica estándar en la inferencia auto-regresiva [2] que almacena las claves y valores calculados de tokens previos para evitar su recómputo en cada paso de generación, reduciendo la complejidad de $O(n^2)$ a $O(n)$ por token generado.
- **Flash Attention** [35]: reformula el cómputo de atención para maximizar el uso de la jerarquía de memoria de la GPU (SRAM, memoria estática rápida integrada en el chip, frente a HBM, *High Bandwidth Memory*, la memoria principal de mayor capacidad pero mayor latencia), logrando aceleraciones de $2\text{--}4\times$ sin cambiar el resultado numérico.
- **Grouped-Query Attention (GQA)**: técnica introducida por Ainslie et al. [36] y adoptada por Mistral [21] y LLaMA 3 [20], que com-

parte claves y valores entre múltiples cabezas de atención, reduciendo el tamaño del KV-Cache y la memoria requerida durante la inferencia.

La Tabla 3.3 ilustra los requisitos de VRAM para diferentes configuraciones de modelo y cuantización, contextualizados con el hardware disponible en este proyecto.

Modelo	FP16 (GB VRAM)	8-bit (GB VRAM)	4-bit (GB VRAM)	¿Viable? (RTX 4070S, 12 GB)
7B	~14	~7	~4	Sí (4-bit/8-bit)
13B	~26	~13	~7	Sí (4-bit)
34B	~68	~34	~18	No
70B	~140	~70	~35	No

Cuadro 3.3: Requisitos aproximados de VRAM según tamaño del modelo y nivel de cuantización. La columna de viabilidad se evalúa respecto a la GPU RTX 4070 Super (12 GB) utilizada en este proyecto.

La Tabla 3.3 traduce el tamaño del modelo en una restricción operativa concreta sobre el *hardware* disponible: en los 12 GB de una GPU RTX 4070 Super, y con cuantización de 4 bits, solo resultan viables los modelos de hasta ~8B parámetros con margen para el contexto; los de 13B quedan al límite y los de 34B o superiores son inejecutables sin descarga a CPU, que degrada drásticamente la latencia. Esta frontera de viabilidad acota el conjunto de modelos viables en una GPU de consumo a la franja 7B–8B.

3.2.8. Limitaciones de los LLMs

A pesar de sus capacidades, los LLMs presentan limitaciones bien documentadas que condicionan su uso en aplicaciones que requieren fiabilidad y verificabilidad::

1. **Alucinaciones** [4]: la generación de información factualmente incorrecta expresada con alta confianza lingüística constituye la limitación más crítica. Las alucinaciones pueden clasificarse en *intrínsecas* (contradicen la fuente) y *extrínsecas* (no verificables). En un contexto académico, donde una respuesta incorrecta sobre normativas podría perjudicar al estudiante, esta limitación es inaceptable sin mecanismos de mitigación.
2. **Corte temporal del conocimiento** (*knowledge cutoff*): los LLMs solo “saben” lo que estaba en sus datos de entrenamiento [17]. Información posterior a la fecha de corte, como cambios en normativas académicas, nuevas guías docentes o modificaciones en planes de estudio, resulta inaccesible para el modelo. Además, incluso dentro del

periodo de entrenamiento, el conocimiento sobre hechos de baja frecuencia resulta poco fiable [37].

3. **Incapacidad de acceso a información privada:** por diseño, los LLMs no pueden acceder a documentos institucionales internos, bases de datos privadas o información no publicada en Internet [38], como reglamentos específicos de la ETSIT, horarios actualizados o actas de juntas. Esta limitación es inherente a la arquitectura generativa y constituye una de las motivaciones centrales de RAG [5].
4. **Sesgo y toxicidad:** los modelos heredan y pueden amplificar sesgos presentes en los datos de entrenamiento. El alineamiento mediante RLHF [18] mitiga parcialmente este problema, pero no lo elimina por completo.
5. **Coste computacional:** el reentrenamiento de un LLM es de un costo desorbitado. Brown et al. [3] estimaron el coste de entrenamiento de GPT-3 en varios millones de dólares y miles de horas-GPU. Incluso el ajuste fino, aunque más asequible con técnicas PEFT [34], requiere recursos no despreciables y experiencia técnica.
6. **Ventana de contexto limitada:** aunque los modelos recientes han ampliado significativamente su ventana de contexto (128K–1M tokens), la calidad de la atención a información lejana en el contexto se degrada, fenómeno conocido como *“lost in the middle”* [39].

Estas limitaciones, en particular las alucinaciones, el corte temporal y la falta de acceso a información institucional, motivan directamente la adopción de la arquitectura RAG analizada en la siguiente sección, que complementa al LLM con un componente de recuperación que le proporciona contexto verificable y actualizado.

3.3. Recuperación Aumentada por Generación (RAG)

Las limitaciones expuestas en la sección anterior, alucinaciones, corte temporal del conocimiento e incapacidad de acceso a información privada, encuentran una solución elegante en el paradigma de *Retrieval-Augmented Generation* (RAG). Esta sección analiza su formulación, evolución y las variantes arquitectónicas más relevantes.

3.3.1. Fundamentos y motivación

El paradigma RAG, formalizado por Lewis et al. [5], propone combinar un componente **paramétrico** (el LLM, que almacena conocimiento en sus

pesos) con un componente **no paramétrico** (un índice de documentos externo) del que se recuperan pasajes relevantes que se inyectan en el contexto del generador antes de producir la respuesta. Esta hibridación ofrece tres ventajas fundamentales respecto al uso del LLM en solitario:

1. **Reducción de alucinaciones:** al anclar las respuestas en fuentes documentales concretas, el modelo genera contenido verificable en lugar de depender exclusivamente de su conocimiento paramétrico.
2. **Actualización sin reentrenamiento:** para incorporar nueva información basta con añadir documentos al índice y reindexarlos, sin necesidad de reentrenar o ajustar el modelo generativo.
3. **Acceso a información privada:** el índice puede contener documentos institucionales, normativos o internos que nunca formaron parte de los datos de entrenamiento del LLM.

El flujo estándar de un sistema RAG consta de tres fases: (1) **indexación** del corpus documental en representaciones vectoriales, (2) **recuperación** de los fragmentos más relevantes para la consulta del usuario, y (3) **generación** de la respuesta condicionada al contexto recuperado. La Figura 3.2 ilustra este pipeline.

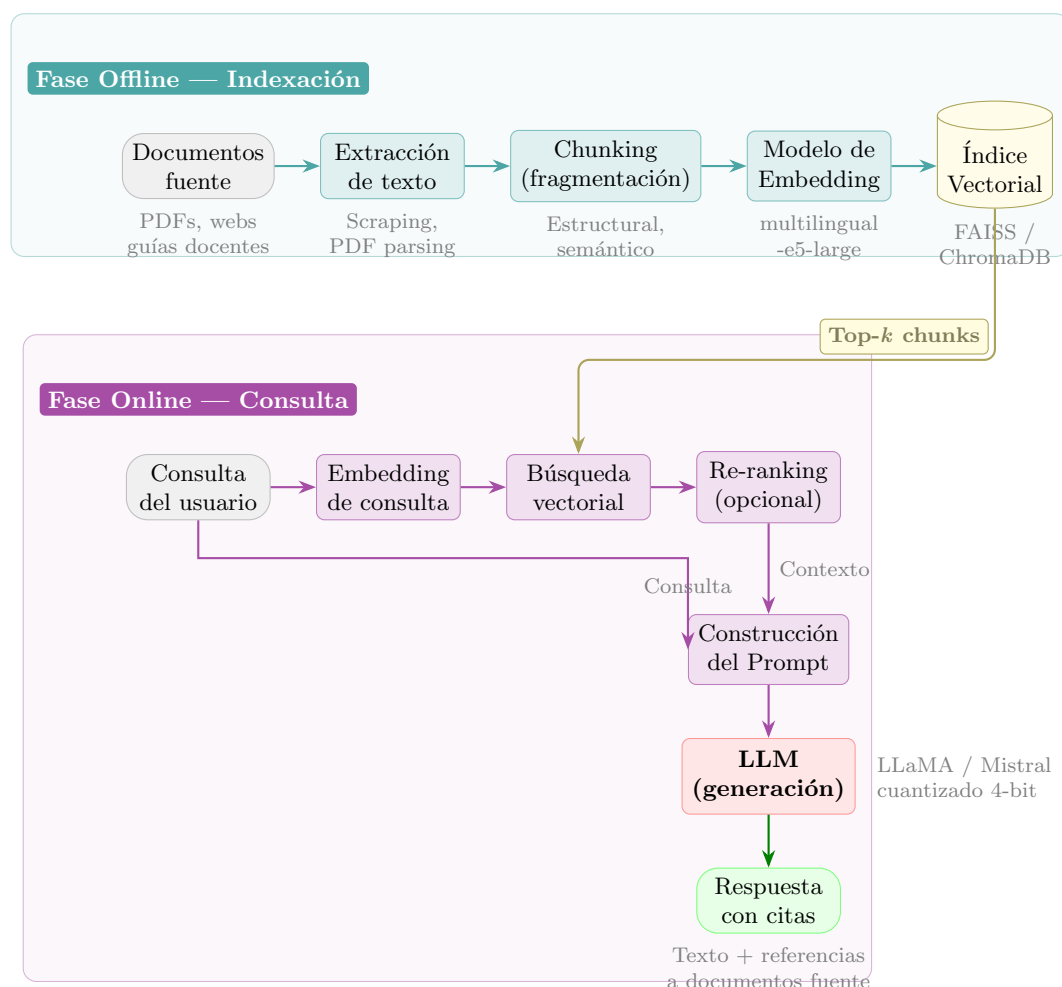


Figura 3.2: Pipeline del sistema RAG. Arriba, la fase *offline* indexa el corpus documental. Abajo, la fase *online* recupera fragmentos relevantes y genera la respuesta con citas. Esta imagen es propia, inspirada en la propuesta de Lewis et al. [5], y en la clasificación de sistemas RAG presentada por Gao et al. [40]

Trabajos previos sentaron las bases de este paradigma. REALM [41] fue pionero al integrar la recuperación de documentos directamente en el preentrenamiento del modelo. DPR (*Dense Passage Retrieval*) [42] demostró empíricamente la superioridad de los recuperadores densos basados en BERT frente a los métodos léxicos clásicos como BM25 [43] para preguntas de dominio abierto, estableciendo la búsqueda vectorial como el estándar para la recuperación semántica. FiD (*Fusion-in-Decoder*) [44] propuso procesar múltiples pasajes recuperados de forma independiente en el codificador y fusionarlos en el decodificador, mejorando significativamente la calidad de

las respuestas al aprovechar evidencia distribuida entre varios documentos.

3.3.2. Clasificación de arquitecturas RAG

Gao et al. [40] presentan una clasificación exhaustiva de las arquitecturas RAG en tres grandes familias, que reflejan la evolución del paradigma:

Naive RAG

La implementación canónica sigue un flujo lineal *indexar-recuperar-generar*:

1. Se fragmenta el corpus en *chunks* y se codifican como vectores densos.
2. Ante una consulta, se calcula su *embedding* y se recuperan los k fragmentos más similares.
3. Se construye un *prompt* que concatena la consulta con los fragmentos recuperados y se envía al LLM.

Este enfoque, aunque simple y efectivo como *baseline*, presenta limitaciones: la calidad de la respuesta depende críticamente de la calidad de la recuperación, no hay mecanismos para filtrar fragmentos irrelevantes que introducen ruido, y las consultas ambiguas o complejas a menudo no se mapean bien al espacio de *embeddings* del corpus.

Advanced RAG

Los enfoques avanzados introducen mejoras en las fases **pre-recuperación** y **post-recuperación** para abordar las limitaciones del Naive RAG:

Pre-recuperación:

- **Reescritura de consultas** (*query rewriting*): reformula la consulta del usuario para maximizar la probabilidad de recuperar documentos relevantes. Puede emplearse el propio LLM para esta reformulación [40].
- **Expansión de consultas** (*query expansion*): genera múltiples variantes o sub-consultas a partir de la consulta original, ejecuta recuperaciones independientes y fusiona los resultados [40].
- **HyDE** (*Hypothetical Document Embeddings*) [45]: genera un “documento hipotético” que respondería a la consulta y utiliza su *embedding*

para la recuperación, en lugar del *embedding* de la consulta directa. Esta técnica mitiga el *gap* semántico entre consultas cortas y documentos largos.

Post-recuperación:

- **Reordenamiento** (*re-ranking*): un modelo *cross-encoder* reordena los fragmentos recuperados por relevancia respecto a la consulta, corrigiendo errores del recuperador *bi-encoder* inicial [46].
- **Diversificación** (MMR, *Maximal Marginal Relevance*): técnica propuesta por Carbonell y Goldstein [47] que reordena los fragmentos recuperados equilibrando dos criterios: la relevancia respecto a la consulta y la diversidad respecto a los fragmentos ya seleccionados, mediante un parámetro $\lambda \in [0, 1]$. A diferencia del *re-ranking* con *cross-encoder*, cuyo objetivo es mejorar la precisión del *ranking*, MMR busca reducir la redundancia entre los fragmentos inyectados en el contexto del LLM, maximizando la cobertura informativa.
- **Compresión contextual**: elimina o resume las partes irrelevantes de los fragmentos recuperados para reducir el ruido inyectado en el *prompt* del LLM [40].
- **Selección dinámica de k** : ajusta el número de fragmentos incluidos en el contexto según la complejidad de la consulta y la confianza de la recuperación [40].

Post-generación:

- **Verificación de atribución** (*attribution verification*): comprueba que cada afirmación de la respuesta generada esté respaldada por al menos un fragmento del contexto recuperado, detectando alucinaciones extrínsecas [48]. El *framework* RARR [49] automatiza esta verificación mediante un LLM auxiliar que evalúa la fidelidad de cada oración.
- **Gestión de incertidumbre**: mecanismo por el cual el sistema detecta cuándo el contexto recuperado es insuficiente o contradictorio para responder a la consulta y, en lugar de generar una respuesta potencialmente alucinada, comunica explícitamente al usuario que no dispone de información fiable. Esta capacidad, a menudo ausente en los chatbots académicos revisados (véase Sección 3.6), resulta crítica en dominios donde una respuesta incorrecta puede perjudicar al usuario [40].

Modular RAG

Las arquitecturas modulares conciben el sistema RAG como un conjunto de componentes intercambiables organizados en un *pipeline* configurable. Cada módulo (indexación, recuperación, reordenamiento, generación, validación) puede sustituirse o combinarse independientemente. Este enfoque favorece la experimentación y la optimización iterativa.

3.3.3. Variantes avanzadas

La investigación reciente ha producido variantes que amplían el paradigma RAG base:

- **Self-RAG** [50]: el modelo aprende a generar *tokens reflexivos* que le permiten (i) decidir cuándo es necesario recuperar información, (ii) evaluar la relevancia de los fragmentos recuperados, y (iii) autocriticar la fidelidad de su propia respuesta. Esto mejora la robustez frente a consultas que no requieren recuperación y reduce la inyección de ruido.
- **CRAG** (*Corrective RAG*) [51]: introduce un evaluador de confianza que clasifica los documentos recuperados como correctos, ambiguos o incorrectos. Cuando la recuperación es insuficiente, el sistema puede recurrir a búsquedas web complementarias o reescribir la consulta.
- **Adaptive RAG** [52]: emplea un clasificador que analiza la complejidad de la consulta y selecciona dinámicamente la estrategia más adecuada: respuesta directa (sin recuperación), RAG simple o RAG multi-paso.
- **Graph RAG** [53]: construye un grafo de conocimiento a partir del corpus y utiliza resúmenes jerárquicos basados en comunidades del grafo para responder consultas que requieren síntesis de información distribuida.

La Tabla 3.4 resume las características de cada variante.

Variante	Recuperación adaptativa	Re-ranking	Auto-evaluación	Complejidad implementación
Naive RAG	No	No	No	Baja
Advanced RAG	No	Sí	No	Media
Modular RAG	No	Opcional	No	Media
Self-RAG	Sí	No	Sí	Alta
CRAG	Sí	Sí	Sí	Alta
Adaptive RAG	Sí	Opcional	No	Media-Alta
Graph RAG	No	No	No	Alta

Cuadro 3.4: Comparativa de las principales variantes arquitectónicas de RAG.

La Tabla 3.4 pone de manifiesto el compromiso fundamental entre fidelidad y coste de inferencia. El *Naive RAG* es el más simple pero el más vulnerable a recuperaciones irrelevantes; las variantes iterativas y autorreflexivas (Self-RAG, CRAG, Adaptive RAG) maximizan la fidelidad a costa de varias pasadas del LLM y, por tanto, de una latencia incompatible con un despliegue local interactivo; y Graph RAG aporta razonamiento multi-salto pero exige construir y mantener un grafo de conocimiento. El sistema propuesto se sitúa en el *Advanced RAG modular*, que incorpora mejoras de pre y post-recuperación (recuperación híbrida, *reranking*, MMR) sin renunciar a una única pasada de generación.

3.4. Recuperación Vectorial y Embeddings

El componente de recuperación constituye el puente entre la consulta del usuario y el conocimiento almacenado en el corpus. Su calidad determina directamente la calidad de la respuesta generada: un recuperador que no encuentra los fragmentos relevantes condena al generador a alucinar o producir respuestas incompletas. Esta sección analiza los fundamentos de las representaciones vectoriales, los modelos de *embedding* más relevantes y las infraestructuras de almacenamiento y búsqueda vectorial.

3.4.1. Fundamentos de los embeddings densos

Un *embedding* es una representación numérica de un fragmento de texto como un vector de dimensión fija en un espacio continuo, donde la proximidad geométrica entre vectores refleja similitud semántica [54]. Formalmente, un modelo de *embedding* f_θ , parametrizado por los pesos θ de una red neuronal, mapea una secuencia de tokens s a un vector $\mathbf{v}$ en un espacio de d dimensiones:

$$\mathbf{v} = f_\theta(s), \quad \mathbf{v} \in \mathbb{R}^d \quad (3.3)$$

donde d es la dimensionalidad del espacio de representación (por ejemplo, 768 o 1024 según el modelo), y $\mathbb{R}^d$ denota el espacio vectorial real de d dimensiones. En la práctica, el modelo f_θ procesa la secuencia de tokens a través de capas Transformer y produce un único vector resumen del significado del texto completo.

La similitud entre dos textos se calcula típicamente mediante la **similitud coseno**, que mide el coseno del ángulo formado por sus vectores de representación:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} = \frac{\sum_{i=1}^d u_i \cdot v_i}{\sqrt{\sum_{i=1}^d u_i^2} \cdot \sqrt{\sum_{i=1}^d v_i^2}} \quad (3.4)$$

donde $\mathbf{u} \cdot \mathbf{v}$ es el producto escalar de ambos vectores, $\|\mathbf{u}\|$ y $\|\mathbf{v}\|$ son sus normas euclídeas, y el resultado varía en el rango $[-1, 1]$: un valor cercano a 1 indica alta similitud semántica, 0 indica independencia y -1 indica oposición. En modelos de *embedding* normalizados, la similitud coseno equivale al producto escalar, lo que simplifica su cálculo.

Los *embeddings* densos superan a los métodos léxicos clásicos (como BM25, basado en TF-IDF) al capturar relaciones semánticas: textos como “requisitos de matrícula” y “condiciones para inscribirse en asignaturas” tendrán *embeddings* cercanos pese a no compartir palabras clave. Sin embargo, los métodos léxicos mantienen ventajas en consultas con términos técnicos muy específicos o códigos [43], lo que ha motivado el desarrollo de enfoques **híbridos** que combinan ambas señales [42].

3.4.2. Modelos de embedding

La evolución de los modelos de *embedding* ha sido rápida, impulsada por la competición en el *benchmark* MTEB (*Massive Text Embedding Benchmark*) [55], que evalúa modelos en 56 tareas agrupadas en 8 categorías (clasificación, clustering, recuperación, etc.) y múltiples idiomas.

- **Sentence-BERT (SBERT)** [56]: adaptó BERT mediante una arquitectura siamesa entrenada con pérdida contrastiva para producir *embeddings* de oraciones. Redujo el tiempo de comparación de pares de oraciones de 65 horas (cross-encoder) a 5 segundos (bi-encoder), habilitando la búsqueda semántica a gran escala.
- **E5** [57]: introdujo preentrenamiento contrastivo a gran escala con pares de texto obtenidos de forma no supervisada de la web. Su variante **multilingual-e5-large** ofrece representaciones de alta calidad en más de 100 idiomas.
- **BGE** (*BAAI General Embeddings*) [58]: familia de modelos con excelente rendimiento en MTEB. **bge-m3** destaca por ser multilingüe, multi-funcional y multi-granular, soportando tanto recuperación densa como léxica y multivectorial en un único modelo.
- **Nomic Embed** [59]: modelo abierto (Apache 2.0) con 137M parámetros que soporta secuencias de hasta 8.192 tokens, superando a OpenAI *text-embedding-3-small* en MTEB con total transparencia de datos y código.

- **OpenAI text-embedding-3 y Cohere embed-v3**: modelos comerciales con alto rendimiento, pero que requieren enviar datos a APIs externas.

La Tabla 3.5 compara los modelos más relevantes para el contexto de este trabajo.

Modelo	Dim.	Max tok.	Multilingüe	Local	Retrieval	MTEB Avg.
SBERT (all-MiniLM-L6)	384	512	No	Sí	Parcial	56.3
multilingual-e5-base	768	512	Sí	Sí	Sí	59.4
multilingual-e5-large	1024	512	Sí	Sí	Sí	61.5
bge-m3	1024	8192	Sí	Sí	Sí	64.2
nomic-embed-text-v1.5	768	8192	Sí	Sí	Sí	62.3
text-embedding-3-small	1536	8191	Sí	No	Sí	62.3
text-embedding-3-large	3072	8191	Sí	No	Sí	64.6

Cuadro 3.5: Comparativa de modelos de *embedding* relevantes. “Dim.”: dimensionalidad del vector de *embedding* producido por el modelo. “Max tok.”: longitud máxima de entrada en *tokens*. “Local”: si el modelo puede ejecutarse sin API externa. “Retrieval”: si fue entrenado con un objetivo contrastivo orientado a recuperación. MTEB Avg.: puntuación media en el *benchmark* MTEB [1].

De la Tabla 3.5 se desprende que los modelos de la familia E5, entrenados con un objetivo contrastivo orientado específicamente a recuperación, encabezan el rendimiento entre las opciones ejecutables localmente cuando se ponderan conjuntamente tres criterios: calidad de recuperación en *benchmarks* multilingües (MTEB), cobertura del español y ausencia de coste y dependencia externos. Las alternativas generalistas (mBERT, multilingual-MiniLM) ofrecen menor calidad de recuperación, mientras que las soluciones por API (OpenAI *text-embedding*) quedan descartadas por vulnerar el requisito de ejecución local.

3.4.3. Bases de datos vectoriales e índices de búsqueda

Una vez generados los *embeddings*, se necesita una infraestructura que permita almacenarlos y buscar eficientemente los vecinos más cercanos (*Approximate Nearest Neighbors*, ANN) [60]. La búsqueda exacta tiene complejidad $O(n \cdot d)$, inaceptable para colecciones grandes, lo que motiva el uso de índices aproximados que sacrifican una fracción marginal de *recall* a cambio de búsquedas sublineales.

FAISS [61], desarrollado por Meta AI, constituye la biblioteca de referencia en el ámbito, con soporte nativo para aceleración mediante CUDA (*Compute Unified Device Architecture*, la plataforma de computación paralela de NVIDIA). Implementa múltiples tipos de índices optimizados para diferentes escenarios:

- **Flat** (IndexFlatL2/IP): búsqueda exacta por fuerza bruta (sin aproximación). Útil como *baseline* para colecciones pequeñas (<100K vectores).
- **IVF** (*Inverted File Index*) [61]: particiona el espacio vectorial en *clusters* mediante k-means y busca solo en los clusters más cercanos a la consulta, reduciendo dramáticamente el espacio de búsqueda.
- **HNSW** (*Hierarchical Navigable Small World*) [62]: construye un grafo navegable multicapa que permite búsquedas aproximadas con alto *recall* y baja latencia. Es la opción preferida cuando se dispone de memoria suficiente.
- **PQ** (*Product Quantization*) [63]: comprime los vectores subdividiéndolos en sub-vectores cuantizados, reduciendo el uso de memoria hasta 64x con pérdida controlada de precisión.

Además de FAISS, han surgido **bases de datos vectoriales** completas que añaden persistencia, filtrado por metadatos, APIs REST y funcionalidades de producción:

- **ChromaDB**²: base de datos vectorial embebida con API Python, especialmente popular en prototipos RAG por su simplicidad. Soporta filtrado por metadatos y persistencia local, pero su rendimiento en búsquedas densas es inferior al de FAISS y no soporta aceleración por GPU.
- **Milvus**³: solución de nivel empresarial desarrollada por Zilliz, que requiere desplegar un servidor independiente (típicamente con Docker Compose). Ofrece alta escalabilidad y filtrado avanzado, pero su complejidad operativa es desproporcionada para sistemas mono-usuario.
- **Qdrant**⁴: base de datos vectorial en Rust con alto rendimiento, filtrado avanzado por *payloads* y soporte para vectores dispersos (híbridos denso+léxico).
- **Weaviate**⁵: añade vectorización integrada (puede generar *embeddings* automáticamente) y soporte para búsqueda híbrida (vectorial + BM25).
- **Pinecone**⁶ y **Zilliz Cloud**⁷: servicios gestionados en la nube con alta escalabilidad.

²<https://github.com/chroma-core/chroma>

³<https://github.com/milvus-io/milvus>

⁴<https://github.com/qdrant/qdrant>

⁵<https://github.com/weaviate/weaviate>

⁶<https://www.pinecone.io/>

⁷<https://zilliz.com/>

La Tabla 3.6 compara las opciones más relevantes.

Solución	Embebida (local)	Filtrado metadatos	Búsqueda híbrida	GPU	Escalabilidad	Licencia
FAISS	Sí	Manual	No	CUDA nativo	Hasta 10^9	MIT
ChromaDB	Sí	Sí	No	No	Hasta 10^6	Apache 2.0
Milvus	No (Docker)	Sí	Sí	Parcial	Hasta 10^9	Apache 2.0
Qdrant	Sí	Avanzado	Sí	No	Hasta 10^8	Apache 2.0
Weaviate	Sí	Sí	Sí	No	Hasta 10^8	BSD-3
Pinecone	No (SaaS)	Sí	Sí	N/A	Hasta 10^9	Propietaria

Cuadro 3.6: Comparativa de soluciones de almacenamiento y búsqueda vectorial. “Embebida” indica si puede ejecutarse como biblioteca en el mismo proceso, sin servidor externo.

La Tabla 3.6 permite descartar alternativas por incompatibilidad con los requisitos del sistema antes que por rendimiento: las soluciones gestionadas en la nube (Pinecone) vulneran la ejecución local y la soberanía de los datos, y las orientadas a servidor (Milvus, Weaviate) añaden una complejidad operativa injustificada para un sistema mono-usuario. Entre las opciones embebidas, FAISS ofrece el mejor rendimiento de búsqueda densa con aceleración CUDA nativa, a cambio de un filtrado por metadatos que debe gestionarse manualmente.

Dentro de FAISS, la elección del tipo de índice determina el compromiso entre precisión de la búsqueda (*recall*), velocidad y consumo de memoria. La Tabla 3.7 resume las principales familias de índices disponibles.

Tipo de índice	Mecanismo	Recall	Adecuación
<code>IndexFlatIP</code>	Búsqueda exhaustiva por producto interno	100 %	Colecciones $< 10^5$ vectores
<code>IndexIVFFlat</code>	Clustering previo + búsqueda en subconjunto	≈ 95 %	Ventaja a partir de 10^6 vectores
<code>IndexHNSWFlat</code>	Grafo de navegación jerárquico	≈ 98 %	Alto <i>recall</i> con baja latencia; mayor memoria
<code>IndexIVFPQ</code>	IVF + cuantización producto	≈ 90 %	Millones de vectores con memoria limitada

Cuadro 3.7: Tipos de índice FAISS y su adecuación según el volumen de la colección.

Para colecciones del orden de 10^3 – 10^4 fragmentos, como las típicas de un corpus académico institucional, la búsqueda exhaustiva (`IndexFlatIP`) se resuelve en menos de 20 ms en CPU y menos de 5 ms en GPU [61], lo que hace innecesario recurrir a índices aproximados que introducen pérdida de *recall* sin beneficio tangible en latencia.

3.5. Estrategias de Fragmentación (*Chunking*)

La calidad del componente de recuperación depende críticamente de cómo se segmenta el corpus documental en fragmentos (*chunks*) indexables [64]. La elección de la estrategia de fragmentación impacta directamente en dos métricas clave: la **precisión** (proporción de fragmentos recuperados que son relevantes) y la **completitud** (proporción del contenido relevante que se recupera). Un *chunk* excesivamente corto carece de contexto suficiente para responder a la consulta y produce *embeddings* poco informativos; uno demasiado largo introduce ruido, diluye la señal semántica y puede exceder la ventana de contexto del LLM cuando se concatenan múltiples fragmentos.

3.5.1. Estrategias principales

La literatura identifica cuatro familias de estrategias, cada una con ventajas e inconvenientes bien definidos [40]:

Fragmentación por tamaño fijo. Divide el texto en bloques de n tokens (o caracteres) con un solapamiento (*overlap*) opcional de m tokens entre fragmentos consecutivos. Su principal ventaja es la simplicidad de implementación y la uniformidad del tamaño de los fragmentos, que facilita la estimación de costes de contexto. Sin embargo, puede romper unidades semánticas, cortando una oración o párrafo a mitad, y los límites arbitrarios pueden separar información que el usuario necesita ver junta. El solapamiento mitiga parcialmente este problema a costa de incrementar el tamaño del índice.

Fragmentación estructural. Respeta la estructura natural del documento, utilizando delimitadores como párrafos, secciones, encabezados o marcadores HTML/Markdown para definir los límites de los fragmentos. Es especialmente adecuada para documentos normativos y académicos, como los reglamentos de la ETSIIT, las guías docentes o las normativas de la UGR, que presentan una estructura jerárquica clara (capítulos → artículos → apartados). Permite preservar metadatos de sección (título, jerarquía) que enriquecen tanto la recuperación como la citación.

Fragmentación semántica. Emplea modelos de *embedding* para calcular la similitud entre oraciones adyacentes y detectar **cambios temáticos** significativos que actúan como puntos de corte naturales. Cuando la similitud entre dos oraciones consecutivas cae por debajo de un umbral, se establece un límite de fragmento. Este enfoque produce fragmentos de longitud variable que respetan las unidades de significado, pero su calidad depende del modelo de *embedding* utilizado y del umbral elegido.

Fragmentación jerárquica (*parent-child*). Indexa el corpus a múltiples granularidades simultáneamente: fragmentos pequeños (oraciones o párra-

fos cortos) para la recuperación de alta precisión, enlazados a fragmentos mayores (secciones o documentos) que proporcionan el contexto completo al LLM. La búsqueda se realiza sobre los fragmentos pequeños, pero el contexto inyectado en el *prompt* es el fragmento padre correspondiente. Este enfoque combina precisión en la recuperación con riqueza contextual en la generación.

3.5.2. Parámetros clave

Independientemente de la estrategia elegida, dos parámetros determinan el comportamiento del fragmentador:

- **Tamaño del fragmento:** la literatura experimental sugiere tamaños entre 256 y 1.024 tokens para la mayoría de los escenarios RAG, con 512 tokens como valor comúnmente empleado como punto de partida [40].
- **Solapamiento (*overlap*):** típicamente entre el 10 % y el 20 % del tamaño del fragmento. Valores mayores mejoran la continuidad semántica pero incrementan el tamaño del índice y el coste de indexación.

Gao et al. [40] destacan que no existe una estrategia universalmente superior, por lo que es necesario evaluar el corpus correspondiente de manera empírica.

La Tabla 3.8 resume las características de cada estrategia.

Estrategia	Preserva semántica	Tamaño uniforme	Complejidad	Mejor para
Tamaño fijo	No	Sí	Baja	Prototipado rápido
Estructural	Sí	No	Media	Docs. normativos
Semántica	Sí	No	Alta	Texto libre
Jerárquica	Sí	No	Alta	Corpus heterogéneo

Cuadro 3.8: Comparativa de estrategias de fragmentación para sistemas RAG.

La Tabla 3.8 contrapone las estrategias según su capacidad para preservar la coherencia semántica frente a su complejidad: la fragmentación de tamaño fijo es la más simple pero corta unidades de significado de forma arbitraria; la fragmentación semántica respeta la estructura del texto a costa de un procesamiento más caro y dependiente del documento; y las variantes con solapamiento o recursivas buscan un punto intermedio que mantenga la continuidad entre fragmentos.).

3.6. Chatbots y Asistentes Académicos

La aplicación de sistemas conversacionales en el ámbito universitario ha experimentado una evolución significativa, pasando de chatbots basados en reglas con capacidades limitadas a sistemas inteligentes que aprovechan los avances en LLMs y RAG. Esta sección revisa esta evolución, analiza las soluciones existentes y sitúa el presente trabajo en este contexto.

3.6.1. Evolución de los chatbots en educación superior

Los primeros chatbots universitarios se basaban en enfoques de **recuperación de intenciones** (*intent-based*) [40], donde un clasificador identifica la intención del usuario entre un conjunto predefinido y devuelve una respuesta preconfigurada. Frameworks como Dialogflow⁸ (Google), Rasa⁹ y Microsoft Bot Framework¹⁰ popularizaron este paradigma. Sin embargo, estos sistemas presentan limitaciones fundamentales: (i) requieren diseñar manualmente todas las intenciones posibles, (ii) no manejan consultas fuera del dominio definido, (iii) las respuestas son estáticas y no pueden sintetizar información de múltiples fuentes, y (iv) el mantenimiento escala linealmente con el número de intenciones.

Con la llegada de los LLMs, surgió una segunda generación de asistentes que aprovecha las capacidades generativas de estos modelos. Soluciones comerciales como ChatGPT [17], Gemini o Copilot ofrecen capacidades lingüísticas avanzadas pero adolecen de tres limitaciones críticas en el contexto institucional:

1. **Desconocimiento institucional:** no pueden acceder a información específica de la universidad, normativas internas, guías docentes, procedimientos administrativos u horarios, que no forman parte de sus datos de entrenamiento [5].
2. **Riesgos de privacidad:** el envío de consultas a servidores externos implica la exposición potencial de datos sensibles del alumnado (expedientes, situaciones personales, consultas sobre convalidaciones o becas), en tensión con el RGPD¹¹.
3. **Alucinaciones normativas:** pueden generar información plausible pero falsa sobre procedimientos académicos, plazos o requisitos [4],

⁸<https://cloud.google.com/dialogflow>

⁹<https://rasa.com/>

¹⁰<https://dev.botframework.com/>

¹¹Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016.

con consecuencias potencialmente graves para el estudiante que siga estas indicaciones erróneas.

3.6.2. Soluciones académicas existentes

La adopción de asistentes conversacionales en el ámbito universitario ha seguido dos vías diferenciadas: soluciones comerciales basadas en intenciones, desplegadas por múltiples universidades españolas, y prototipos de investigación basados en RAG, todavía incipientes pero crecientes. A continuación se describen cuatro sistemas representativos, dos del contexto local de la UGR y dos internacionales con publicación científica, que permiten situar la propuesta del presente TFG.

Elvira (UGR)

Elvira [65] fue el primer asistente virtual de la Universidad de Granada, desarrollado por Virtual Solutions, empresa *spin-off* del Departamento de Ciencias de la Computación e Inteligencia Artificial de la propia UGR. Integrado en la Oficina Virtual de la universidad, Elvira emplea un enfoque basado en reconocimiento de intenciones y árboles de diálogo, acompañado de un avatar 3D. Su funcionalidad se centra en la navegación guiada por los servicios administrativos de la universidad, con respuestas predefinidas para un conjunto cerrado de consultas. Las principales limitaciones de Elvira son la rigidez de sus respuestas (no genera texto, solo selecciona entre plantillas), la incapacidad de manejar consultas fuera de su dominio predefinido y la ausencia de actualización dinámica del conocimiento.

Alhe (UGR / 1MillionBot)

Alhe [66] es el asistente virtual vigente de la UGR, desarrollado por la empresa alicantina 1MillionBot¹² y financiado con fondos europeos del programa UNIDIGITAL. Desplegado en la web principal de la UGR, el portal info/UGR y un canal de Telegram, Alhe responde sobre oferta educativa, acceso y admisión, trámites académicos, becas, movilidad, comedores, instalaciones deportivas y servicios digitales. Su arquitectura combina técnicas de NLP/NLU (*Natural Language Understanding*) para la clasificación de intenciones con un sistema de derivación a agentes humanos mediante conexión con el sistema de *tickets* de la UGR. Aunque supone un avance notable respecto a Elvira en cobertura temática y canales de acceso, Alhe comparte limitaciones fundamentales del paradigma *intent-based*: sus respuestas proceden de una base de conocimiento verificada manualmente, no genera texto

¹²<https://1millionbot.com/>

libre, no puede sintetizar información de múltiples fuentes y carece de mecanismo de citación que permita al usuario verificar la procedencia de la información proporcionada.



Figura 3.3: Imagen del asistente virtual Alhe.

Unimib Assistant (Università di Milano-Bicocca)

Dolci et al. [67] presentan **Unimib Assistant**, un chatbot basado en RAG diseñado para el estudiantado de la Università di Milano-Bicocca. El sistema se construye sobre la funcionalidad de *Custom GPTs* de OpenAI, proporcionando al modelo documentos universitarios y enlaces obtenidos de una fase previa de *need-finding* con estudiantes. Una evaluación cualitativa de usabilidad con seis estudiantes reveló que el chatbot fue valorado positivamente por su experiencia conversacional y la estructura de sus respuestas, pero presentó deficiencias críticas: imprecisiones factuales incluso cuando la información estaba presente en los documentos proporcionados, generación de enlaces no funcionales y omisión de información relevante. Desde la perspectiva de este TFG, Unimib Assistant ilustra dos riesgos del enfoque RAG con API comercial: (i) la dependencia de un proveedor externo (OpenAI) que impide el control sobre el proceso de recuperación y generación, y (ii) la ausencia de trazabilidad explícita, lo que dificulta la verificación de las respuestas por parte del usuario.



Figura 3.4: Imagen del logo definitivo de Unimib.

URAG (HCMUT, Vietnam)

Nguyen et al. [68] proponen **URAG** (*Unified RAG*), un sistema híbrido de dos niveles desplegado en la Ho Chi Minh City University of Technology (HCMUT) para el ámbito de admisiones universitarias. El primer nivel (*Tier 1*) consiste en un sistema de preguntas frecuentes (FAQ) enriquecido automáticamente que resuelve consultas recurrentes sin invocar al LLM, reduciendo la latencia y el coste computacional. Cuando la FAQ no contiene una respuesta adecuada, el segundo nivel (*Tier 2*) recupera documentos de una base aumentada y genera la respuesta mediante un LLM. Los resultados experimentales muestran que URAG supera a otros sistemas evaluados en precisión de respuestas de admisión. No obstante, el sistema depende de APIs comerciales para la generación, no aborda la trazabilidad de las respuestas (no cita fragmentos fuente) y se centra exclusivamente en el dominio de admisiones, sin cubrir el espectro completo de consultas académicas (normativas, guías docentes, procedimientos administrativos) que motiva el presente trabajo.

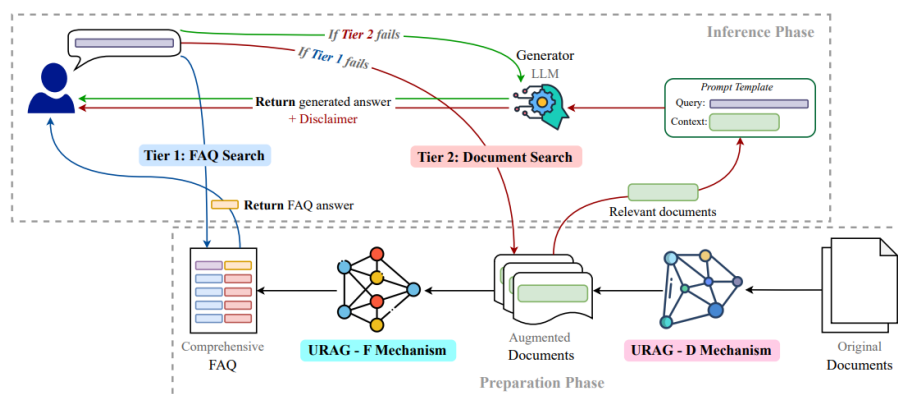


Figura 3.5: Ilustración de la arquitectura de URAG [68].

Análisis comparativo

La revisión de Gao et al. [40] documenta la proliferación de arquitecturas RAG en dominios de conocimiento cerrado, incluyendo el académico. En el contexto europeo, el cumplimiento del Reglamento General de Protección de Datos (RGPD) incentiva el despliegue de asistentes con modelos abiertos ejecutados localmente, evitando el envío de consultas estudiantiles a servidores de terceros. En el ámbito hispanohablante, las implementaciones documentadas públicamente con evaluación rigurosa son aún más escasas, lo que refuerza la originalidad de la propuesta presentada en este TFG.

Las iniciativas más rigurosas enfatizan dos requisitos que este trabajo adopta como principios de diseño: (i) la **trazabilidad**, cada respuesta debe poder verificarse contra un documento fuente, citando la sección y el documento de origen [48], y (ii) el **manejo explícito de la incertidumbre**, el sistema debe ser capaz de reconocer cuándo no dispone de información suficiente para responder y comunicarlo al usuario, en lugar de alucinar una respuesta plausible [49].

Situados unos frente a otros, los cuatro sistemas trazan una progresión que deja ver qué carece cada uno *respecto a los demás*. **Elvira** es el más limitado: al no generar texto, queda por detrás de todos en comprensión semántica y flexibilidad, restringido a un conjunto cerrado de plantillas. **Alhe** amplía notablemente la cobertura temática y los canales respecto a Elvira, pero comparte con él la barrera del paradigma *intent-based*, ni genera texto libre ni cita fuentes, de modo que frente a los sistemas RAG (Unimib, URAG y la propuesta) pierde en capacidad de síntesis y de verificación. **Unimib** y **URAG** sí incorporan generación mediante LLM y, por tanto, superan a Elvira y Alhe en comprensión y síntesis; sin embar-

go, ambos dependen de APIs comerciales, lo que los sitúa por detrás de un enfoque local en privacidad (RGPD) y reproducibilidad, y ninguno ofrece trazabilidad de fuentes, carencia que Unimib paga además con imprecisiones factuales y enlaces no funcionales. Entre ambos, URAG mitiga coste y latencia con su primer nivel de FAQ, pero estrecha su alcance al dominio de admisiones, mientras que Unimib cubre servicios más generales a costa de una evaluación apenas cualitativa ($n=6$). En suma, ninguno de los cuatro reúne simultáneamente las tres propiedades que este TFG persigue, generación con LLM, ejecución local y trazabilidad de fuentes, hueco que ocupa la propuesta.

La Tabla 3.9 resume las diferencias entre los enfoques generales, y la Tabla 3.10 compara los cuatro sistemas analizados con la propuesta de este TFG.

Tipo	Comprensión semántica	Dominio específico	Trazabilidad	Privacidad	Mantenimiento
Intent-based	Baja	Sí	N/A	Sí	Alto
LLM comercial	Alta	No	Parcial	No	Bajo
RAG con API	Alta	Sí	Sí	No	Medio
RAG local	Alta	Sí	Sí	Sí	Medio

Cuadro 3.9: Comparativa de enfoques para asistentes conversacionales académicos. La fila en negrita corresponde al enfoque adoptado en este TFG.

La Tabla 3.9 hace explícito el compromiso por dimensiones. Los enfoques *intent-based* garantizan privacidad y acotan bien el dominio, pero a costa de una comprensión semántica baja y un elevado coste de mantenimiento, pues cada intención y su respuesta se curan a mano. Los LLM comerciales invierten el balance: máxima comprensión semántica con mantenimiento mínimo, pero sin anclaje al dominio, sin privacidad, la consulta sale a la nube, y con trazabilidad solo parcial. El RAG con API recupera el anclaje al dominio y la trazabilidad, pero mantiene la dependencia de la nube y, con ella, el problema de privacidad. Solo el **RAG local** (fila en negrita) combina comprensión semántica alta, dominio acotado, trazabilidad y privacidad, aceptando como contrapartida un mantenimiento medio derivado de la curación del corpus y la gestión del modelo local. Es precisamente esa única configuración que reúne las cuatro propiedades deseables la que fundamenta el enfoque del presente trabajo.

Sistema	Arquitectura	LLM	Local	Trazab.	Dominio	Evaluación
Elvira (UGR)	Intent-based	No	Sí	No	Nav. admin.	No doc.
Alhe (UGR) ^a	Intent + NLU	No	No	No	Oferta + trám.	No doc.
Unimib Assist.	RAG (GPT)	GPT-4 (API)	No	No	Serv. grales.	Cualit. ($n=6$)
URAG (HCMUT)	FAQ + RAG	API comerc.	No	No	Admisiones	Cuantitativa
GRAIA	Adv. RAG	LLM abierto	Sí	Sí	Acad. integral	RAGAS + manual

^a Alhe se ejecuta en servidores de 1MillionBot, externos a la UGR.

Cuadro 3.10: Comparativa de sistemas concretos de asistentes académicos frente a la propuesta de este TFG (fila en negrita). Trazab.: trazabilidad de fuentes.

La Tabla 3.10 desciende de los enfoques a los sistemas reales y confirma el hueco identificado. Leída por columnas, ninguna fila salvo la última marca afirmativamente a la vez las tres propiedades decisivas, ejecución local, trazabilidad y dominio académico integral: Elvira y Alhe son locales en lo administrativo pero ni generan ni citan; Unimib y URAG generan con LLM pero son remotos (API), no citan y acotan su dominio (servicios generales y admisiones, respectivamente). La columna de evaluación añade un matiz de rigor científico: Elvira y Alhe carecen de evaluación documentada, Unimib se evalúa solo de forma cualitativa con seis usuarios y URAG aporta una evaluación cuantitativa pero circunscrita a admisiones. **GRAIA** es el único sistema que combina una arquitectura *Advanced RAG*, un LLM abierto ejecutado localmente, trazabilidad de fuentes, un dominio académico integral y una evaluación mixta (automática con RAGAS y análisis manual de errores), lo que sintetiza su aportación diferencial frente al estado del arte.

3.7. Evaluación de Sistemas RAG

La evaluación de sistemas RAG plantea dificultades específicas que la distinguen de la evaluación de LLMs aislados o de sistemas de recuperación convencionales: no basta con medir la calidad del texto generado, sino que debe evaluarse conjuntamente la calidad de la recuperación, la fidelidad del generador respecto a los documentos recuperados y la corrección factual de la respuesta final. Esta sección analiza las dimensiones de evaluación, las métricas disponibles, los *frameworks* de evaluación automática y las metodologías de evaluación humana.

3.7.1. Dimensiones de evaluación

La literatura distingue cuatro dimensiones fundamentales que deben evaluarse en un sistema RAG. El *framework* RAGAS [69] formaliza tres de ellas (relevancia del contexto, exhaustividad y fidelidad), mientras que Liu et al. [70] añaden la corrección factual de la respuesta como dimensión independiente:

1. **Relevancia del contexto recuperado** (*context relevance/precision*): grado en que los pasajes recuperados son pertinentes a la consulta del usuario. Un sistema que recupera fragmentos irrelevantes introduce ruido que degrada la calidad de la respuesta.
2. **Exhaustividad del contexto** (*context recall*): proporción de la información necesaria para responder correctamente que está presente en los fragmentos recuperados. Un *recall* bajo indica que el sistema no encuentra toda la evidencia disponible.
3. **Fidelidad** (*faithfulness/groundedness*) [69]: medida en que la respuesta generada se deriva exclusivamente del contexto recuperado, sin introducir información externa ni alucinaciones. Esta dimensión es crítica para la trazabilidad: una respuesta fiel permite verificar cada afirmación contra su fuente.
4. **Corrección de la respuesta** (*answer correctness*) [70]: grado de acierto factual de la respuesta respecto a una referencia validada (*ground truth*). Integra tanto la precisión del contenido como la completitud de la información proporcionada.

Estas dimensiones son complementarias y no sustituibles: un sistema puede tener alta fidelidad pero baja corrección (si los documentos recuperados no contienen la respuesta correcta) o alta corrección con baja fidelidad (si el LLM “sabe” la respuesta pero la genera de su conocimiento paramétrico en lugar del contexto recuperado).

3.7.2. Métricas de recuperación

El componente de recuperación se evalúa con métricas clásicas de *Information Retrieval* [71]:

- **Precision@k**: proporción de documentos relevantes entre los k recuperados.
- **Recall@k**: proporción de documentos relevantes del corpus que aparecen entre los k recuperados.
- **MRR** (*Mean Reciprocal Rank*): inverso de la posición del primer documento relevante, promediado sobre todas las consultas. Mide cuán pronto aparece el primer resultado útil.
- **nDCG@k** (*Normalized Discounted Cumulative Gain*): pondera la relevancia de los documentos recuperados por su posición, penalizando documentos relevantes en posiciones bajas.

- **Hit Rate@k**: proporción de consultas para las que al menos un documento relevante aparece entre los k recuperados.

3.7.3. Métricas de generación

Para evaluar la calidad del texto generado se dispone de métricas automáticas de diferente naturaleza:

- **ROUGE** [72]: mide el solapamiento de n -gramas entre la respuesta generada y la referencia. ROUGE-L (basado en la subsecuencia común más larga) es la variante más utilizada. Útil como indicador rápido pero correlaciona débilmente con la calidad percibida.
- **BERTScore** [73]: calcula la similitud semántica token a token usando *embeddings* de BERT, capturando paráfrasis que ROUGE ignora.
- **Evaluación LLM-as-Judge**: utiliza un LLM (típicamente GPT-4 o Claude) como evaluador automático, pidiéndole que puntúe la respuesta según criterios definidos (corrección, completitud, relevancia, claridad). Aunque introduce dependencia de un modelo externo, múltiples estudios han demostrado alta correlación con las evaluaciones humanas [74].

3.7.4. Frameworks de evaluación automática

Varios *frameworks* automatizan la evaluación integral de sistemas RAG:

- **RAGAS** [75]: el *framework* más extendido, calcula cuatro métricas principales: *faithfulness* (fidelidad al contexto), *answer relevancy* (relevancia de la respuesta a la consulta), *context precision* (proporción de contexto relevante) y *context recall* (cobertura del contexto necesario). Utiliza un LLM como juez para descomponer la respuesta en afirmaciones atómicas y verificar cada una contra el contexto.
- **DeepEval**¹³: *framework* open-source que extiende RAGAS con métricas adicionales como *hallucination score*, *toxicity* y *bias*, y permite definir umbrales de aceptación para pipelines de CI/CD (*Continuous Integration / Continuous Delivery*).
- **LangSmith**¹⁴: plataforma de LangChain para trazabilidad, evaluación y depuración de cadenas RAG, con soporte para anotación humana y evaluación automática.

¹³<https://github.com/confident-ai/deepeval>

¹⁴<https://docs.smith.langchain.com/>

- **Phoenix**¹⁵ (Arize AI): herramienta de observabilidad para LLMs que permite inspeccionar las trazas de recuperación, los *embeddings* y la calidad de las respuestas en tiempo real.

3.7.5. Evaluación humana

Las métricas automáticas, aunque eficientes, no capturan completamente la calidad percibida por el usuario final. La evaluación humana complementa las métricas automáticas mediante la valoración de aspectos como la utilidad práctica de la respuesta, la naturalidad del lenguaje, la adecuación al registro académico y la correcta atribución de fuentes [48]. Tanto el *framework* RAGAS [75] como Zheng et al. [74] coinciden en un conjunto de prácticas recomendadas para la evaluación humana:

- Construir **datasets de evaluación específicos del dominio** con preguntas realistas y respuestas de referencia verificadas.
- Emplear **escalas de Likert** (1–5) para dimensiones como corrección, utilidad, completitud y naturalidad.
- Calcular la **concordancia inter-anotadores** (Cohen’s κ [76] o Krippendorff’s α [77]) para validar la fiabilidad de las evaluaciones humanas.
- Complementar con **análisis de errores** cualitativo que identifique patrones de fallo del sistema (tipos de consultas problemáticas, fuentes de alucinación, limitaciones del corpus).

3.8. Herramientas y tecnologías del ecosistema RAG

La construcción de un sistema RAG completo requiere, más allá de los modelos y algoritmos descritos en las secciones precedentes, la selección de un *stack* tecnológico que integre componentes de muy diversa naturaleza: un lenguaje de programación, un servidor de inferencia para el LLM, un *framework* de interfaz de usuario y bibliotecas de procesamiento documental. La presente sección ofrece una revisión comparativa de las alternativas más relevantes en cada categoría, proporcionando el contexto necesario para fundamentar la selección de un *stack* tecnológico adecuado.

¹⁵<https://github.com/Arize-AI/phoenix>

3.8.1. Lenguajes de programación para *stacks* RAG

La elección del lenguaje de programación condiciona la disponibilidad de bibliotecas, la facilidad de integración entre componentes y la velocidad de prototipado [40]. La Tabla 3.11 compara los cuatro lenguajes más utilizados en proyectos de inteligencia artificial y procesamiento de lenguaje natural.

Cuadro 3.11: Comparativa de lenguajes de programación para el desarrollo de sistemas RAG.

Criterio	Python	Java	C++	JavaScript
Ecosistema NLP/ML ^b	Nativo	Parcial	Mínimo	Parcial
Soporte FAISS	Oficial	JNI ^c	Nativo	N/A
Integración Ollama	REST ^d + SDK ^e	REST	REST	REST
Frameworks RAG	LangChain, LlamaIndex	LangChain4j	—	LangChain.js
Frameworks UI rápido	Streamlit, Gradio	Spring (pesado)	—	React, Next.js
Prototipado rápido	Muy alto	Medio	Bajo	Alto
Coste de mantenimiento	Bajo	Medio	Alto	Medio

^b ML: *Machine Learning* (aprendizaje automático).

^c JNI: *Java Native Interface* (interfaz para invocar código nativo desde Java).

^d REST: *Representational State Transfer* (estilo arquitectónico de API web).

^e SDK: *Software Development Kit* (biblioteca cliente oficial).

Python domina el ecosistema de IA y NLP: la práctica totalidad de las bibliotecas clave, **sentence-transformers**, FAISS, RAGAS, Hugging Face Transformers, ofrecen *bindings* nativos o primarios en Python. La mayoría de los artículos y repositorios de referencia en RAG publican su código en Python, lo que facilita la reproducibilidad y la comparación experimental. Además, *frameworks* como Streamlit permiten construir interfaces web interactivas con un coste de desarrollo mínimo.

Java dispone de LangChain4j y un ecosistema maduro para sistemas empresariales, pero su integración con FAISS requiere JNI (*Java Native Interface*), con sobrecarga y complejidad de compilación, y carece de soporte nativo para **sentence-transformers**.

C++ es el lenguaje en el que FAISS está implementado internamente y ofrece el máximo rendimiento en la búsqueda vectorial; sin embargo, el cuello de botella de latencia en un sistema RAG es la inferencia del LLM (del orden de segundos), no la búsqueda vectorial (del orden de milisegundos), por lo que el rendimiento adicional no justifica la complejidad de desarrollo.

JavaScript/TypeScript cuenta con LangChain.js, pero el ecosistema de *embeddings* y evaluación (RAGAS) está centrado en Python, y el rendimiento de Node.js en cómputo numérico no compite con las extensiones C subyacentes a NumPy y FAISS.

3.8.2. Servidores de inferencia local para LLMs

La ejecución local de LLMs requiere un *runtime* que gestione la carga del modelo, la cuantización, la asignación de GPU y la exposición de una API de inferencia. La Tabla 3.12 compara las principales alternativas.

Cuadro 3.12: Comparativa de servidores de inferencia local para LLMs.

Criterio	Ollama	llama.cpp	vLLM	TGI
Facilidad de despliegue	Muy alta	Media	Media	Media
API compatible OpenAI	Sí	Parcial	Sí	Sí
Cuantización GGUF	Nativa	Nativa	No nativa	Parcial
Gestión de modelos	Integrada (<i>pull</i>)	Manual	Manual	Hugging Face
Soporte GPU NVIDIA	CUDA nativo	CUDA	CUDA	CUDA
<i>Streaming</i>	Sí	Sí	Sí	Sí
Optimizado para	Mono-usuario	Bajo nivel	<i>Batching</i> concurrente	Producción HF

Ollama¹⁶ es un *runtime* que envuelve `llama.cpp` y añade una capa de gestión declarativa de modelos (`ollama pull <modelo>`), una API REST estable compatible con el esquema de OpenAI (`/v1/chat/completions`), abstracción automática de la plantilla de *chat* del modelo (evitando errores manuales en *tokens* especiales) y soporte nativo para cuantizaciones en formato GGUF (4 bit, 5 bit). Su despliegue se reduce a un único binario, sin fase de compilación ni configuración de entornos virtuales.

llama.cpp¹⁷ es la biblioteca subyacente a Ollama y ofrece el máximo control sobre parámetros de inferencia, pero exige compilación manual con soporte CUDA, configuración explícita del servidor HTTP y gestión manual del formato de las peticiones.

vLLM [78] está optimizado para escenarios de alto rendimiento con *batching* continuo y paginación de atención (*PagedAttention*), ideal para servir múltiples usuarios concurrentes. Sin embargo, su soporte nativo de GGUF es limitado, lo que restringe los modelos ejecutables en GPUs con poca VRAM.

Text Generation Inference (TGI)¹⁸, de Hugging Face, ofrece una solución de producción con soporte parcial de GGUF, pero está dimensionado para despliegues empresariales y añade complejidad innecesaria para prototipos académicos.

3.8.3. Frameworks de interfaz para prototipos de IA

La capa de presentación de un sistema RAG debe proporcionar una experiencia conversacional fluida con soporte para *streaming* de tokens y

¹⁶<https://ollama.com/>

¹⁷<https://github.com/ggerganov/llama.cpp>

¹⁸<https://github.com/huggingface/text-generation-inference>

visualización de citas. La Tabla 3.13 compara las alternativas más relevantes.

Cuadro 3.13: Comparativa de *frameworks* de interfaz para prototipos de IA.

Criterio	Streamlit	Gradio	Flask + React
Complejidad de desarrollo	Muy baja	Baja	Alta
Componente chat nativo	Sí (<code>st.chat_message</code>)	Sí	Manual
<i>Streaming</i> de tokens	Sí (<code>st.write_stream</code>)	Sí	WebSocket manual
Gestión de estado	<code>st.session_state</code>	Limitada	Redux/Context
Personalización visual	Media	Baja	Total
Despliegue	Servidor integrado	Servidor integrado	Nginx + 2 procesos

Streamlit¹⁹ es un *framework* Python orientado a prototipos de ciencia de datos, con componentes reactivos integrados (incluyendo `st.chat_input` y `st.chat_message` desde la versión 1.25), *streaming* nativo mediante `st.write_stream` y despliegue autocontenido (`streamlit run app.py`). Su arquitectura reactiva, basada en la re-ejecución del *script* ante cada interacción, facilita el desarrollo iterativo.

Gradio²⁰ ofrece componentes de chat similares y está orientado específicamente a demostraciones de modelos de *machine learning*. Su principal limitación es la menor flexibilidad en la personalización del diseño y la gestión de estado más restringida para conversaciones con historial.

Flask [79] + **React** [80] (o FastAPI + React) proporciona la máxima flexibilidad, pero el coste de desarrollo es significativamente mayor: configuración de rutas, *middleware* CORS (*Cross-Origin Resource Sharing*), implementación manual de WebSocket para *streaming*, proceso de *build* del *frontend* y mantenimiento de dos bases de código (Python + TypeScript).

3.8.4. Bibliotecas de extracción de documentos PDF

La ingesta de documentos en formato PDF requiere bibliotecas especializadas capaces de extraer texto preservando el orden de lectura, especialmente en documentos con columnas múltiples como normativas universitarias. La Tabla 3.14 compara las opciones más utilizadas en Python.

Cuadro 3.14: Comparativa de bibliotecas de extracción de texto desde PDF en Python.

Biblioteca	Velocidad	Orden de lectura	Tablas	API de metadatos
PyMuPDF (<i>fitz</i>)	Alta	Bueno	Básico	Completa
pypdf	Baja	Aceptable	No	Básica
pdfplumber	Baja	Bueno	Excelente	Limitada
pdfminer.six	Media	Bueno	No	Básica

¹⁹<https://streamlit.io/>

²⁰<https://gradio.app/>

PyMuPDF²¹ (`fitz`) destaca por su velocidad de extracción, significativamente superior a las demás alternativas, su buena preservación del orden de lectura en documentos multi-columna y su API completa para acceder a metadatos estructurales del documento (índice, anotaciones, fuentes).

pdfplumber²² es superior en la extracción de tablas, pero es considerablemente más lento para texto corriente, que constituye la mayor parte de los corpus académicos típicos.

pypdf²³ es la opción más simple pero también la más lenta, con un orden de lectura que puede degradarse en documentos complejos.

pdfminer.six²⁴ ofrece un compromiso intermedio en velocidad, pero sin ventajas decisivas sobre PyMuPDF en ningún criterio relevante para la extracción masiva de texto.

3.9. Posicionamiento del Trabajo Propuesto

La revisión realizada permite identificar el hueco que el presente TFG pretende cubrir. Los sistemas comerciales generales ofrecen capacidades lingüísticas avanzadas pero carecen de conocimiento institucional y plantean problemas de privacidad. Los chatbots académicos tradicionales basados en reglas resultan rígidos y poco escalables. Las arquitecturas RAG recientes proporcionan el marco adecuado, pero su aplicación concreta al dominio académico universitario, y en particular a la ETSIIT, con sus normativas específicas, no se encuentra documentada públicamente de forma rigurosa.

El trabajo propuesto se posiciona, por tanto, como una aplicación original del paradigma RAG a un dominio académico concreto, con las siguientes aportaciones diferenciales respecto al estado del arte revisado:

- **Despliegue íntegramente local** mediante LLMs cuantizados ejecutados sobre GPU de consumo, garantizando privacidad y eliminando dependencias de servicios externos.
- **Pipeline de ingesta específico** para fuentes heterogéneas de la ETSIIT/UGR (scraping web + extracción de PDFs), con manejo explícito de la estructura jerárquica de documentos normativos.
- **Sistema de citación y trazabilidad** que permite verificar cada respuesta contra su fuente original, requisito crítico en el contexto académico.

²¹<https://pymupdf.readthedocs.io/>

²²<https://github.com/jsvine/pdfplumber>

²³<https://github.com/py-pdf/pypdf>

²⁴<https://github.com/pdfminer/pdfminer.six>

- **Evaluación experimental rigurosa** sobre un *dataset* construido *ad hoc* con preguntas realistas del alumnado, empleando métricas automáticas (RAGAS), comparación con un *baseline* sin recuperación y análisis manual de errores.
- **Comparación empírica** entre al menos tres modelos de generación para justificar las decisiones de diseño finales, fundamentando la elección del modelo por defecto.

En la Tabla 3.15 se resume la comparación entre las principales alternativas identificadas y la propuesta del presente trabajo.

Enfoque	Local	Dominio académico	Trazabilidad	Evaluación rigurosa
ChatGPT/Gemini (general)	No	No	Parcial	—
Chatbots basados en reglas	Sí	Sí	N/A	Limitada
RAG genérico (LangChain demo)	Variable	No	Parcial	Limitada
Propuesta TFG	Sí	Sí	Sí	Sí

Cuadro 3.15: Comparativa entre enfoques existentes y la propuesta del presente TFG.

La Tabla 3.15 condensa el posicionamiento del trabajo: cada enfoque previo satisface solo un subconjunto de los cuatro criterios diferenciales. Los sistemas comerciales generales sacrifican la ejecución local y el conocimiento del dominio; los chatbots basados en reglas son locales y específicos pero carecen de trazabilidad real y de evaluación rigurosa; y las demostraciones de RAG genérico no se especializan en el dominio ni documentan una evaluación sistemática. La propuesta de este TFG es la única que reúne simultáneamente ejecución íntegramente local, especialización en el dominio académico, trazabilidad verificable y evaluación experimental rigurosa, combinación que delimita su aportación frente al estado del arte.

3.10. Conclusiones del Capítulo

El análisis del estado del arte confirma que el paradigma RAG constituye la aproximación técnicamente más sólida para abordar el problema planteado en este TFG. Los fundamentos teóricos (Transformers, LLMs, embeddings densos) están suficientemente maduros, existen implementaciones abiertas de calidad industrial (FAISS, modelos LLaMA/Mistral, sentence-transformers) y la literatura reciente ofrece arquitecturas avanzadas (Self-RAG, HyDE) así como metodologías de evaluación específicas (RAGAS).

Al mismo tiempo, la revisión revela un espacio claro de contribución: no existe, hasta donde se ha podido verificar, un trabajo público que aplique

estas técnicas al dominio académico de la ETSIIT con los requisitos específicos de privacidad, trazabilidad y evaluación rigurosa que se plantean en el Capítulo 1. El siguiente capítulo aborda el análisis funcional del sistema, traduciendo esta fundamentación técnica en requisitos y casos de uso.

Capítulo 4

Análisis y Objetivos del Sistema

4.1. Introducción

El presente capítulo especifica formalmente el sistema GRAIA desde la perspectiva del análisis de requisitos. Mientras que el Capítulo 1 estableció los objetivos generales y específicos del trabajo en términos de *qué* se pretende construir, este capítulo responde a la pregunta complementaria de *qué debe hacer exactamente* el sistema para satisfacer dichos objetivos, y bajo qué restricciones. Constituye, por tanto, la bisagra entre la motivación del proyecto y su diseño arquitectónico, que se aborda en el Capítulo 5.

El análisis se ha realizado siguiendo los principios de la ingeniería de requisitos clásica, con una adaptación ligera del estándar IEEE 830 [81] para la especificación de requisitos funcionales y no funcionales, y empleando la notación UML junto con fichas estructuradas de tipo Cockburn [82] para los casos de uso. Esta combinación persigue asegurar la trazabilidad entre objetivos, requisitos, casos de uso y componentes del diseño, y facilitar la validación cualitativa del sistema.

Las secciones que componen este capítulo son las siguientes. La Sección 4.2 describe la metodología de análisis empleada. La Sección 4.3 identifica los actores que interactúan directamente con el sistema. La Sección 4.4 presenta los requisitos funcionales, organizados según el actor que ejecuta cada acción, y la Sección 4.5 presenta los requisitos no funcionales del sistema, incluyendo las restricciones técnicas y de diseño que condicionan el desarrollo. La Sección 4.6 detalla los casos de uso mediante diagrama UML y fichas Cockburn. La Sección 4.7 presenta la matriz de trazabilidad entre objetivos específicos y requisitos. Finalmente, la Sección 4.8 cierra el capítulo y anticipa el bloque de diseño.

4.2. Metodología de Análisis

Dada la naturaleza del proyecto, un trabajo de investigación aplicada desarrollado por un único recurso y sin un cliente externo que defina requisitos de forma canónica, la elicitación de requisitos no puede basarse en entrevistas formales con usuarios finales. En su lugar, se ha adoptado un enfoque **iterativo e incremental** inspirado en la ingeniería de requisitos ágil, apoyado en cuatro fuentes complementarias:

1. **Análisis del dominio.** Inspección sistemática de la documentación oficial de la ETSIT y de la Universidad de Granada (guías docentes, normativas, calendarios, procedimientos académicos) con el objetivo de caracterizar los tipos de consultas que un estudiante puede formular a un asistente conversacional académico.
2. **Revisión de literatura sobre chatbots académicos y sistemas RAG.** Las conclusiones del estado del arte (Capítulo 3) proporcionan un catálogo de requisitos reiteradamente identificados en trabajos previos: trazabilidad de respuestas, gestión de incertidumbre, rechazo cortés de consultas fuera del ámbito, baja latencia, etc.
3. **Reuniones de seguimiento con el tutor del TFG.** Actúan como mecanismo principal de validación externa. Las reuniones periódicas permiten refinar el alcance y descartar requisitos no defendibles o fuera de tiempo para los plazos del proyecto (Capítulo 2).
4. **Inspección de sistemas análogos.** Observación del comportamiento de asistentes conversacionales generalistas (ChatGPT, Gemini) y académicos (chatbots institucionales de diversas universidades) para identificar patrones de interacción esperados por los usuarios y limitaciones conocidas.

A partir de estas fuentes se construyó un catálogo inicial de requisitos candidatos, que fue refinado en tres iteraciones. En cada iteración, los requisitos fueron **clasificados** (funcionales frente a no funcionales), **priorizados** según el esquema MoSCoW [83], *Must have*, *Should have*, *Could have*, *Won't have*, y **validados** en la reunión de seguimiento correspondiente. El resultado final, presentado en las Secciones 4.4 y 4.5, contiene únicamente los requisitos cuya implementación es realista en el marco temporal del TFG y cuya verificación es posible con los medios disponibles.

El formato elegido para los requisitos es una adaptación ligera del estándar **IEEE 830-1998** [81], que especifica cada requisito mediante una ficha estructurada con identificador, descripción, prioridad, fuente y objetivo específico asociado. Esta elección se justifica por tres razones: (i) es un estándar

reconocido en la docencia de ingeniería del software, (ii) introduce la trazabilidad entre objetivos y requisitos como un campo explícito en la ficha, lo que refuerza la auditabilidad del análisis, y (iii) es lo suficientemente ligero como para no saturar la memoria con documentación superflua, objeción habitual a otros estándares más pesados como IEEE 29148 [84].

4.3. Identificación de Actores

Se han identificado dos actores que interactúan directamente con GRAIA:

Actor: Estudiante. Usuario final principal del sistema. Se trata de un estudiante matriculado en la ETSIT que plantea consultas en lenguaje natural sobre aspectos académicos del centro: asignaturas, guías docentes, normativas de evaluación, calendarios, procedimientos administrativos, requisitos de TFG, horarios, entre otros. Se asume que posee un nivel tecnológico equivalente al de cualquier estudiante universitario promedio, junto al uso habitual de chatbots y buscadores, pero no necesariamente conocimiento técnico sobre el funcionamiento interno del sistema. Su interacción con el sistema se limita a tres acciones: iniciar una nueva conversación, enviar una consulta y consultar las fuentes citadas en la respuesta. En particular, el estudiante *no* decide ni interviene en la naturaleza de la respuesta (válida, abstención o rechazo por estar fuera de ámbito): esa determinación la realiza el sistema de forma autónoma.

Actor: Administrador del sistema. Rol técnico interno encargado del mantenimiento del sistema. No interactúa con el sistema mediante consultas, sino que opera sobre él a través de procedimientos de mantenimiento. Sus acciones consisten en reconstruir y gestionar el corpus documental (incorporación de documentos verificados, revisión, limpieza y adición de fuentes específicas), reindexar el corpus cuando procede y evaluar el rendimiento del sistema sobre un conjunto de preguntas de referencia.

4.4. Requisitos Funcionales

Los requisitos funcionales describen las acciones que cada actor puede realizar sobre el sistema para alcanzar sus objetivos. Por ello se presentan agrupados según el actor que las ejecuta: el estudiante, que interactúa a través de la interfaz conversacional, y el administrador, que opera el sistema mediante procedimientos de mantenimiento. Cada requisito se especifica mediante una ficha con los siguientes campos, conforme al formato IEEE 830 adaptado:

- **Identificador:** código único con prefijo RF- seguido de dos dígitos.
- **Nombre:** título breve y descriptivo de la acción.
- **Actor:** actor que ejecuta la acción (estudiante o administrador).
- **Descripción:** especificación de la acción en lenguaje natural, formulada en términos observables.
- **Prioridad:** clasificación MoSCoW. *M* = *Must have*, *S* = *Should have*, *C* = *Could have*.
- **Fuente:** origen del requisito (análisis del dominio, literatura, tutor, sistema análogo).
- **OE asociado:** objetivo específico del Capítulo 1 al que da cobertura.

A continuación se detallan los diez requisitos funcionales identificados, agrupados por actor.

4.4.1. Acciones del estudiante

Identificador	RF-01
Nombre	Iniciar una nueva conversación
Actor	Estudiante
Descripción	El estudiante puede iniciar una nueva conversación, descartando el contexto de la conversación previa y comenzando una sesión limpia a la espera de una nueva consulta.
Prioridad	M
Fuente	Análisis del dominio
OE asociado	OE7

Identificador	RF-02
Nombre	Enviar una consulta
Actor	Estudiante
Descripción	El estudiante puede redactar y enviar una consulta académica en lenguaje natural, a espera de recibir una respuesta del sistema.
Prioridad	M
Fuente	Análisis del dominio
OE asociado	OE4, OE5

Identificador	RF-03
Nombre	Consultar las fuentes
Actor	Estudiante
Descripción	El estudiante puede acceder a las fuentes que respaldan la respuesta generada por el sistema, de modo que pueda contrastar por sí mismo la información u obtener más información relevante.
Prioridad	M
Fuente	Tutor
OE asociado	OE6

4.4.2. Acciones del administrador

El administrador opera el sistema a través de procedimientos de mantenimiento. (Capítulo 6).

Identificador	RF-04
Nombre	Reconstruir el corpus documental
Actor	Administrador
Descripción	El administrador puede reconstruir por completo el corpus documental, se rastrean las fuentes institucionales declaradas por categoría, se descarga y extrae los documentos relevantes y produce el corpus depurado y deduplicado.
Prioridad	M
Fuente	Análisis del dominio
OE asociado	OE2

Identificador	RF-05
Nombre	Incorporar documentos verificados a una URL concreta
Actor	Administrador
Descripción	El administrador puede incorporar o sustituir el contenido asociado a una URL concreta con un documento limpio, conservando la URL oficial para las citas. Está pensado para las fuentes difíciles de extraer de forma fiable (tablas como el horario, imágenes como el calendario).
Prioridad	S
Fuente	Análisis del dominio
OE asociado	OE2

Identificador	RF-06
Nombre	Revisar el corpus
Actor	Administrador
Descripción	El administrador puede revisar el contenido y la calidad del corpus, obteniendo un informe resumen, vista por categoría, búsqueda en títulos y texto, y exportación a CSV.
Prioridad	S
Fuente	Análisis del dominio
OE asociado	OE2

Identificador	RF-07
Nombre	Limpiar el corpus
Actor	Administrador
Descripción	El administrador puede eliminar documentos del corpus según diferentes criterios: por URL exacta, por tamaño mínimo (documentos por debajo de un número dado de caracteres) o por término contenido en el documento.
Prioridad	S
Fuente	Análisis del dominio
OE asociado	OE2

Identificador	RF-08
Nombre	Añadir fuentes específicas
Actor	Administrador
Descripción	El administrador puede añadir fuentes concretas al corpus incorporando documentos individuales identificados por su URL sin necesidad de reconstruir todo el corpus.
Prioridad	C
Fuente	Análisis del dominio
OE asociado	OE2

Identificador	RF-09
Nombre	Reindexar el corpus
Actor	Administrador
Descripción	El administrador puede regenerar el índice vectorial, que fragmenta el corpus depurado, calcula los <i>embeddings</i> con el modelo E5 y construye el índice FAISS persistente sobre el que opera la recuperación.
Prioridad	M
Fuente	Literatura
OE asociado	OE3

Identificador	RF-10
Nombre	Evaluar el rendimiento del sistema
Actor	Administrador
Descripción	El administrador puede evaluar el rendimiento del sistema mediante reproduciendo el <i>pipeline</i> de recuperación y generación sobre un conjunto de preguntas con respuesta o fuente esperada y calcula métricas objetivas (recall de recuperación, acierto de respuesta y tasa de rechazo correcto/incorrecto) para determinar su rendimiento.
Prioridad	M
Fuente	Literatura
OE asociado	OE8, OE9

4.5. Requisitos No Funcionales

Los requisitos no funcionales especifican *cómo* debe comportarse el sistema en términos de calidad: rendimiento, fiabilidad, usabilidad, privacidad

y demás atributos que condicionan la aceptabilidad del sistema sin describir funcionalidades observables. Se emplea el mismo formato de ficha que en la Sección 4.4, con el prefijo RNF- y un campo adicional de **métrica o método de verificación** que establece cómo se comprobará el cumplimiento del requisito: experimentalmente en el Capítulo 7 para los requisitos de rendimiento y calidad, y mediante inspección del código o análisis cualitativo para el resto.

4.5.1. Rendimiento

Identificador	RNF-01
Nombre	Latencia de respuesta
Descripción	El tiempo transcurrido entre la emisión de una consulta por parte del estudiante y la visualización completa de la respuesta generada debe ser inferior a 5 segundos en condiciones normales de uso, sobre el hardware especificado en el Capítulo 2.
Prioridad	M
Métrica	Latencia media y percentil 95 sobre el <i>dataset</i> de evaluación.
OE asociado	OE5, OE8

Identificador	RNF-02
Nombre	Eficiencia de la recuperación
Descripción	La búsqueda vectorial de los k fragmentos más similares en el índice FAISS debe completarse en menos de 200 ms para un corpus de hasta 10^5 fragmentos, de modo que la latencia total (RNF-01) quede dominada por el coste de inferencia del LLM y de la reordenación neuronal, no por la búsqueda vectorial.
Prioridad	S
Métrica	Tiempo de la búsqueda vectorial en FAISS, sub-milisegundo para el corpus actual; el coste de la reordenación con <i>cross-encoder</i> se imputa a la latencia total (RNF-01).
OE asociado	OE4

4.5.2. Calidad de la respuesta

Identificador	RNF-03
Nombre	Precisión mínima
Descripción	Sobre el <i>dataset</i> de evaluación construido, el sistema debe alcanzar una precisión mínima del 80 % en las consultas con respuesta disponible en el corpus, conforme a la hipótesis H1 del Capítulo 1.
Prioridad	M
Métrica	Precisión sobre <i>ground truth</i> anotado manualmente.
OE asociado	OE8, OE9

Identificador	RNF-04
Nombre	Reducción de alucinaciones frente a LLM sin contexto
Descripción	La tasa de respuestas alucinadas del sistema RAG debe ser significativamente inferior a la del mismo LLM operando sin recuperación de contexto, medida sobre el mismo <i>dataset</i> de evaluación, conforme a la hipótesis H2.
Prioridad	M
Métrica	<i>Faithfulness</i> de RAGAS [75] y comparación con el mismo modelo sin recuperación (<i>baseline</i>), complementadas con análisis manual de errores.
OE asociado	OE9

Identificador	RNF-05
Nombre	Cobertura de citación
Descripción	Al menos el 90 % de las respuestas generadas para consultas con información disponible en el corpus deben incluir al menos una cita verificable a un fragmento del corpus, conforme a la hipótesis H1.
Prioridad	M
Métrica	Proporción de respuestas con al menos una cita válida.
OE asociado	OE6

4.5.3. Usabilidad y accesibilidad

Identificador	RNF-06
Nombre	Usabilidad de la interfaz
Descripción	La interfaz conversacional debe ser utilizable sin formación previa por un estudiante con nivel tecnológico medio. Debe cumplir los principios básicos de diseño de chatbots: mensaje de bienvenida explícito, <i>feedback</i> visual durante la generación, presentación clara de las citas y manejo de errores comprensible.
Prioridad	S
Métrica	Verificación por inspección de que la interfaz cumple los principios básicos de un <i>chatbot</i> usable: mensaje de bienvenida, <i>feedback</i> visual durante la generación (<i>streaming</i>), presentación clara de las citas y manejo comprensible de errores. La evaluación formal de usabilidad (p. ej. cuestionario SUS) se plantea como trabajo futuro (Capítulo 8).
OE asociado	OE7

Identificador	RNF-07
Nombre	Soporte lingüístico principal (español) con apertura multilingüe
Descripción	El sistema debe operar <i>principalmente</i> en español, puesto que es la lengua mayoritaria de la comunidad ETSIIT: las consultas del usuario, las respuestas generadas, los mensajes de la interfaz y los mensajes de error deben estar disponibles en español con total corrección. Como funcionalidad complementaria, el sistema debe aceptar consultas formuladas en otras lenguas europeas mayoritarias (inglés, francés, italiano, alemán, portugués) para dar servicio al alumnado Erasmus, devolviendo la respuesta en la lengua de la consulta siempre que sea posible. Para habilitar ambos escenarios, el modelo de <i>embeddings</i> y el LLM deben disponer de soporte contrastado al menos para español y, preferiblemente, tener naturaleza multilingüe. Si el calendario del proyecto impidiera validar adecuadamente el soporte multilingüe, su desarrollo completo se pospondrá a las líneas de trabajo futuro del Capítulo 8, manteniendo el español como requisito obligatorio.
Prioridad	M (español), C (multilingüe)
Métrica	Verificación obligatoria mediante pruebas con consultas en español sobre el <i>dataset</i> de evaluación; verificación deseable complementaria con un subconjunto de consultas en inglés y otra lengua europea representativa del alumnado Erasmus.
OE asociado	OE5, OE7

4.5.4. Privacidad y seguridad

Identificador	RNF-08
Nombre	Privacidad de los datos del usuario
Descripción	El sistema no debe almacenar información personal identificable del estudiante. Las interacciones se registran de forma anónima, sin correlación posible con una identidad concreta, y no se transmiten a terceros.
Prioridad	M
Métrica	Inspección del código y del esquema de almacenamiento.
OE asociado	OE6, OE8

Identificador	RNF-09
Nombre	Ejecución íntegramente local
Descripción	La inferencia del LLM y la recuperación vectorial deben ejecutarse exclusivamente en el hardware local del proyecto. El sistema no debe depender en tiempo de ejecución de servicios en la nube ni de APIs externas de pago, garantizando la confidencialidad del corpus y la independencia tecnológica.
Prioridad	M
Métrica	Inspección del código y verificación de ausencia de tráfico externo.
OE asociado	OE5

4.5.5. Mantenibilidad y portabilidad

Identificador	RNF-10
Nombre	Mantenibilidad del código
Descripción	El código fuente debe estar organizado de forma modular siguiendo una arquitectura por capas, con separación clara entre ingesta, indexación, recuperación, generación e interfaz. Debe incluir documentación en el repositorio y pruebas unitarias mínimas sobre los componentes críticos.
Prioridad	S
Métrica	Revisión estructural y cobertura de tests unitarios.
OE asociado	OE2, OE3, OE4, OE5

Identificador	RNF-11
Nombre	Portabilidad a otros dominios documentales
Descripción	La arquitectura del sistema debe ser transferible a otros dominios documentales cerrados (otras escuelas, otras universidades, otros corpora) sin cambios estructurales, modificando únicamente la configuración de ingesta y el corpus de entrada.
Prioridad	C
Métrica	Análisis cualitativo de acoplamiento entre componentes.
OE asociado	—

4.5.6. Restricciones técnicas y de diseño

Identificador	RNF-12
Nombre	Restricción de hardware de ejecución
Descripción	La ejecución debe producirse sobre el equipo descrito en el Capítulo 2, con GPU NVIDIA RTX 4070 Super de 12 GB de VRAM. Esta restricción impone un límite superior práctico al tamaño del LLM utilizable, modelos de hasta 13B parámetros cuantizados a 4 bits, y condiciona la elección del modelo de <i>embeddings</i> .
Prioridad	M
Métrica	Verificación de que el modelo seleccionado se ejecuta dentro del presupuesto de VRAM disponible.
OE asociado	OE5

Identificador	RNF-13
Nombre	Uso de software de código abierto
Descripción	Todo el software empleado debe ser de código abierto o disponer de licencia gratuita para uso académico, garantizando la reproducibilidad y evitando dependencias comerciales. Se excluyen explícitamente las APIs de pago de proveedores externos (OpenAI, Google) como motor de inferencia.
Prioridad	M
Métrica	Inspección del inventario de dependencias y de sus licencias.
OE asociado	OE5

4.6. Casos de Uso

Los casos de uso concretan cómo los actores identificados en la Sección 4.3 interactúan con el sistema para alcanzar sus objetivos. El estudiante dispone de un único caso de uso que articula sus tres acciones (iniciar conversación, enviar consulta y consultar fuentes) en un flujo natural de interacción. El administrador, en cambio, ejecuta cada una de sus acciones de mantenimiento de forma independiente, por lo que se modela un caso de uso por cada requisito funcional del administrador (RF-04 a RF-10): reconstruir el corpus, incorporar un documento verificado, revisar el corpus, limpiar el corpus, añadir una fuente específica, reindexar el corpus y evaluar el rendimiento del sistema. Se presentan, en primer lugar, el diagrama UML global y, a continuación, las fichas detalladas en formato Cockburn.

4.6.1. Diagrama de casos de uso

La Figura 4.1 muestra el diagrama UML de casos de uso del sistema GRAIA. El actor *Estudiante* concentra la interacción conversacional, mientras que el *Administrador del sistema* asume los casos de uso de operación.

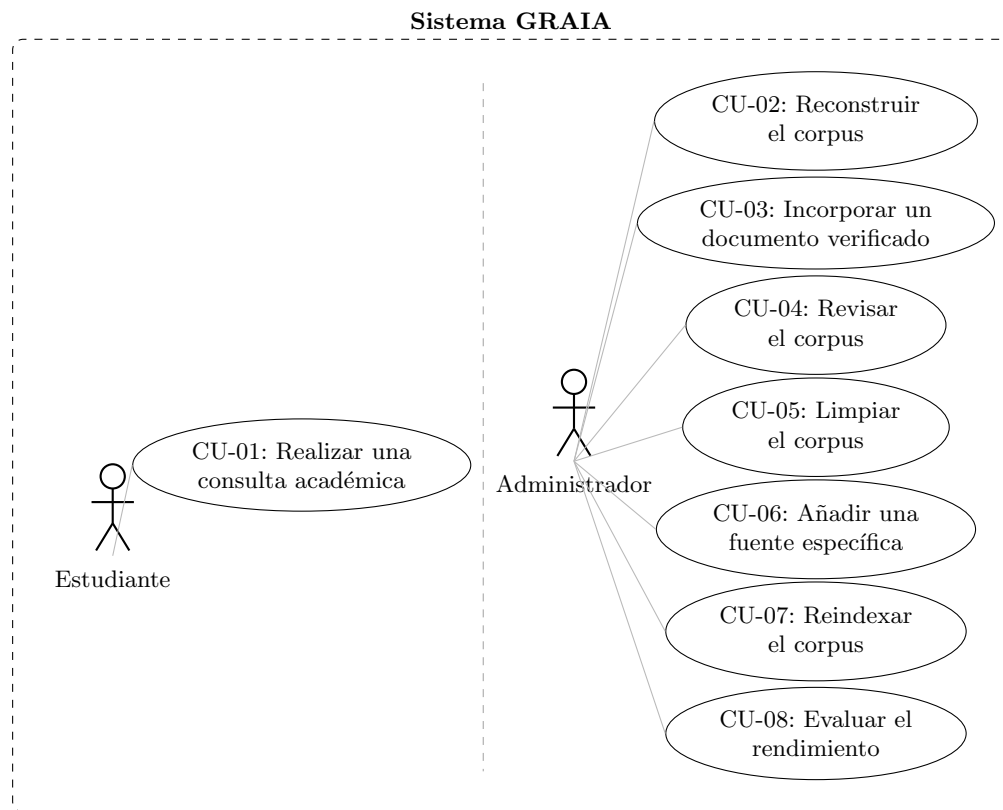


Figura 4.1: Diagrama UML de casos de uso del sistema GRAIA.

4.6.2. Fichas detalladas

Se presentan las fichas Cockburn de los ocho casos de uso del sistema: CU-01 (la interacción completa del estudiante) y los siete casos de operación del administrador, CU-02 a CU-08, uno por cada requisito funcional del administrador.

Identificador	CU-01
Nombre	Realizar una consulta académica
Actor primario	Estudiante
Precondiciones	El sistema está operativo y el corpus está indexado y disponible para la consulta.
Postcondición de éxito	El estudiante ha recibido la respuesta a su consulta y, si lo desea, ha accedido a las fuentes que la respaldan.
Flujo principal	1. El estudiante inicia una nueva conversación. 2. El estudiante formula su consulta académica en lenguaje natural. 3. El estudiante envía la consulta al sistema. 4. El sistema devuelve una respuesta acompañada de las fuentes que la sustentan, que el estudiante recibe. 5. El estudiante, opcionalmente, accede a las fuentes citadas para contrastar la información.
Flujos alternativos	4a. Información insuficiente: el sistema, de forma autónoma, devuelve un mensaje de abstención cuando el corpus no contiene información fiable; el estudiante lo recibe y puede reformular la consulta. 4b. Consulta fuera de ámbito: el sistema rechaza cortésmente la consulta y redirige al estudiante al ámbito académico. En ambos casos el estudiante no interviene en la determinación.
Excepciones	3a. Si el sistema no logra elaborar la respuesta, comunica el error y el estudiante puede reintentar el envío.
Requisitos cubiertos	RF-01, RF-02, RF-03

Cuadro 4.1: Ficha Cockburn del caso de uso CU-01.

Identificador	CU-02
Nombre	Reconstruir el corpus
Actor primario	Administrador del sistema
Precondiciones	El administrador dispone de acceso al entorno de ejecución y de las fuentes institucionales declaradas en la configuración.
Postcondición de éxito	El corpus documental queda reconstruido, depurado y deduplicado, listo para su revisión e indexación.
Flujo principal	1. El administrador solicita la reconstrucción completa del corpus. 2. El sistema rastrea las fuentes institucionales declaradas por categoría. 3. El sistema descarga y extrae el contenido textual de los documentos relevantes. 4. El sistema depura y deduplica el corpus y lo almacena. 5. El sistema informa de las estadísticas del corpus resultante.
Flujos alternativos	3a. Si algún documento no puede obtenerse o procesarse, el sistema registra el error y continúa con el resto.
Excepciones	—
Requisitos cubiertos	RF-04

Cuadro 4.2: Ficha Cockburn del caso de uso CU-02.

Identificador	CU-03
Nombre	Incorporar un documento verificado
Actor primario	Administrador del sistema
Precondiciones	Existe un corpus y una URL ya presente en él cuyo contenido se desea sustituir por una versión verificada a mano.
Postcondición de éxito	El contenido asociado a esa URL queda sustituido por el documento verificado, conservando la URL para las citas.
Flujo principal	1. El administrador proporciona el documento verificado y la URL oficial a la que asociarlo. 2. El sistema sustituye en el corpus el contenido asociado a esa URL por el documento verificado. 3. El sistema lo etiqueta como documento verificado y autocontenido. 4. El sistema confirma la incorporación.
Flujos alternativos	—
Excepciones	2a. Si la URL no existe en el corpus, el sistema lo notifica y no realiza cambios.
Requisitos cubiertos	RF-05

Cuadro 4.3: Ficha Cockburn del caso de uso CU-03.

Identificador	CU-04
Nombre	Revisar el corpus
Actor primario	Administrador del sistema
Precondiciones	Existe un corpus documental.
Postcondición de éxito	El administrador obtiene una visión del contenido y la calidad del corpus que le permite decidir acciones de mantenimiento.
Flujo principal	1. El administrador solicita una revisión del corpus. 2. El sistema presenta un informe con el número de documentos, su distribución por categoría y la posibilidad de buscar en títulos y texto. 3. El administrador inspecciona el informe.
Flujos alternativos	—
Excepciones	—
Requisitos cubiertos	RF-06

Cuadro 4.4: Ficha Cockburn del caso de uso CU-04.

Identificador	CU-05
Nombre	Limpiar el corpus
Actor primario	Administrador del sistema
Precondiciones	Existe un corpus documental que puede contener documentos no deseados.
Postcondición de éxito	Los documentos que cumplen el criterio indicado han sido eliminados del corpus.
Flujo principal	1. El administrador indica el criterio de eliminación: por URL exacta, por tamaño mínimo (documentos por debajo de un número dado de caracteres) o por término contenido en el documento. 2. El sistema identifica los documentos que cumplen el criterio. 3. El sistema elimina los documentos seleccionados y actualiza el corpus.
Flujos alternativos	2a. En modo simulación, el sistema muestra los documentos que se eliminarían sin aplicar cambios.
Excepciones	—
Requisitos cubiertos	RF-07

Cuadro 4.5: Ficha Cockburn del caso de uso CU-05.

Identificador	CU-06
Nombre	Añadir una fuente específica
Actor primario	Administrador del sistema
Precondiciones	Existe un corpus al que incorporar una fuente puntual.
Postcondición de éxito	El documento identificado por su URL queda incorporado al corpus.
Flujo principal	1. El administrador indica la URL de la fuente y su categoría. 2. El sistema descarga y procesa el documento. 3. El sistema lo incorpora al corpus sin necesidad de reconstruirlo por completo. 4. El sistema confirma la incorporación.
Flujos alternativos	—
Excepciones	2a. Si la fuente no puede obtenerse o procesarse, el sistema lo notifica y no modifica el corpus.
Requisitos cubiertos	RF-08

Cuadro 4.6: Ficha Cockburn del caso de uso CU-06.

Identificador	CU-07
Nombre	Reindexar el corpus
Actor primario	Administrador del sistema
Precondiciones	Existe un corpus depurado y validado.
Postcondición de éxito	El índice vectorial refleja el corpus actual y las consultas posteriores pueden recuperar sus documentos.
Flujo principal	1. El administrador solicita la (re)indexación del corpus. 2. El sistema fragmenta el corpus en unidades de tamaño acotado. 3. El sistema genera la representación vectorial (<i>embeddings</i>) de cada fragmento. 4. El sistema construye el índice vectorial persistente. 5. El sistema confirma el resultado informando de las estadísticas del índice.
Flujos alternativos	1a. El administrador puede solicitar una simulación que muestre las estadísticas de fragmentación sin construir el índice.
Excepciones	3a. Si la generación de la representación vectorial falla por falta de recursos, la operación se aborta y se conserva el índice anterior.
Requisitos cubiertos	RF-09

Cuadro 4.7: Ficha Cockburn del caso de uso CU-07.

Identificador	CU-08
Nombre	Evaluar el rendimiento del sistema
Actor primario	Administrador del sistema
Precondiciones	Existe un corpus indexado y un conjunto de preguntas de referencia con respuesta o fuente esperada.
Postcondición de éxito	El administrador obtiene un informe de métricas objetivas que permite comparar configuraciones del sistema.
Flujo principal	1. El administrador solicita la evaluación del rendimiento del sistema. 2. El sistema reproduce el proceso de recuperación y generación sobre cada pregunta del conjunto de referencia. 3. El sistema calcula las métricas: recall de recuperación, acierto de respuesta y tasa de rechazo correcto/incorrecto. 4. El sistema presenta el informe de resultados al administrador.
Flujos alternativos	—
Excepciones	—
Requisitos cubiertos	RF-10

Cuadro 4.8: Ficha Cockburn del caso de uso CU-08.

4.7. Matriz de Trazabilidad

La trazabilidad, entre los objetivos específicos del Capítulo 1 y los requisitos del presente capítulo, es una herramienta clave para verificar formalmente que el análisis no ha dejado ningún objetivo sin cobertura. La Tabla 4.9

presenta la matriz de trazabilidad entre objetivos específicos (OE1–OE9) y requisitos funcionales. La Tabla 4.10 presenta lo mismo pero con los requisitos no funcionales.

	OE1	OE2	OE3	OE4	OE5	OE6	OE7	OE8	OE9
RF-01							•		
RF-02				•	•				
RF-03						•			
RF-04		•							
RF-05		•							
RF-06		•							
RF-07		•							
RF-08		•							
RF-09			•						
RF-10								•	•

Cuadro 4.9: Matriz de trazabilidad entre objetivos específicos y requisitos funcionales.

	OE1	OE2	OE3	OE4	OE5	OE6	OE7	OE8	OE9
RNF-01					•			•	
RNF-02				•					
RNF-03								•	•
RNF-04									•
RNF-05						•			
RNF-06							•		
RNF-07					•		•		
RNF-08						•		•	
RNF-09					•				
RNF-10		•	•	•	•				
RNF-11									
RNF-12					•				
RNF-13					•				

Cuadro 4.10: Matriz de trazabilidad entre objetivos específicos y requisitos no funcionales.

El objetivo específico OE1 (*revisión del estado del arte*) no aparece cubierto por ningún requisito, lo cual es esperable: constituye una actividad de investigación bibliográfica previa al diseño del sistema, no una acción ejecutable sobre él; su cumplimiento se verifica con la propia existencia del Capítulo 3. El resto de objetivos específicos (OE2–OE9) quedan cubiertos por al menos un requisito funcional, lo que valida formalmente que el análisis es completo. El requisito RNF-11 (portabilidad) no está asociado directamente a un OE porque se corresponde con un atributo transversal del sistema, defendible como buena práctica de ingeniería y como habilitador de las líneas de trabajo futuro anticipadas en el Capítulo 8.

4.8. Conclusiones del Capítulo

El presente capítulo ha especificado formalmente el sistema GRAIA mediante diez requisitos funcionales expresados como acciones de los actores y agrupados según quién las ejecuta (tres del estudiante y siete del administrador), trece requisitos no funcionales organizados en seis categorías de calidad (rendimiento, calidad de la respuesta, usabilidad, privacidad, mantenibilidad y restricciones técnicas), ocho casos de uso representativos del comportamiento esperado del sistema (uno del estudiante y siete del administrador, uno por cada requisito funcional del administrador, con ficha detallada en formato Cockburn para cada uno) y una matriz de trazabilidad explícita que demuestra la cobertura de los objetivos específicos del Capítulo 1 por parte de los requisitos definidos.

El análisis realizado cumple tres propiedades deseables en una especificación rigurosa: es **completo**, en el sentido de que cada objetivo específico con naturaleza funcional está cubierto por al menos un requisito; es **trazable**, gracias a la matriz explícita y a los campos de trazabilidad integrados en cada ficha; y es **verificable**, puesto que cada requisito no funcional lleva asociado un método de comprobación concreto: los de rendimiento y calidad se evalúan experimentalmente en el Capítulo 7, y los demás se verifican por inspección del código o análisis cualitativo.

Con esta especificación en mano, el siguiente capítulo aborda el *diseño y la arquitectura* del sistema GRAIA: cómo se descomponen las acciones descritas en componentes concretos, cómo se comunican entre sí y qué patrones arquitectónicos se emplean para satisfacer los requisitos no funcionales (especialmente los de rendimiento, mantenibilidad y portabilidad) sin comprometer la trazabilidad exigida en los requisitos funcionales.

Capítulo 5

Diseño y Arquitectura del Sistema

5.1. Introducción

El Capítulo 4 ha establecido *qué* debe hacer el sistema GRAIA (catorce requisitos funcionales, doce no funcionales y cuatro casos de uso críticos). El presente capítulo aborda el *cómo*: de qué manera se descomponen esas capacidades en componentes de software concretos, cómo se comunican entre sí y qué patrones arquitectónicos y tecnologías se emplean para satisfacer simultáneamente los requisitos funcionales y las restricciones de calidad (rendimiento, mantenibilidad, portabilidad y privacidad), apoyándose en los catálogos de patrones de diseño de Gamma et al. [85] y de patrones arquitectónicos de Buschmann et al. [86].

La transición del análisis al diseño exige tomar decisiones que no se derivan mecánicamente de los requisitos, sino que implican compromisos de ingeniería. A lo largo de este capítulo se explicitan esas decisiones, las alternativas consideradas y las razones por las que se descartan, conforme al principio de *justificación obligatoria* que guía la memoria en su conjunto.

El capítulo se organiza de la siguiente forma. La Sección 5.2 presenta las decisiones arquitectónicas fundamentales que condicionan el resto del diseño. La Sección 5.3 justifica la selección de cada tecnología del *stack*, comparándola con las alternativas disponibles. La Sección 5.4 describe la arquitectura general del sistema mediante un diagrama de alto nivel. La Sección 5.5 identifica los patrones de diseño empleados y su razón de ser. Las Secciones 5.6 a 5.12 detallan el diseño interno de cada subsistema. La Sección 5.14 cierra el capítulo con la matriz de trazabilidad entre requisitos y componentes de diseño.

5.2. Decisiones arquitectónicas fundamentales

Antes de detallar componentes individuales, conviene explicitar las tres decisiones de nivel arquitectónico que condicionan todo el sistema. Cada una responde a un *por qué* raíz:

5.2.1. Pipeline RAG secuencial

La primera decisión es adoptar la arquitectura *Retrieval-Augmented Generation* (RAG) en su variante secuencial, recuperación seguida de generación, frente a alternativas como el *fine-tuning* del LLM sobre el corpus o la generación puramente paramétrica.

- **¿Por qué RAG y no *fine-tuning*?** El *fine-tuning* sobre un corpus de dominio exige un volumen de datos supervisados del que no se dispone (no existen pares pregunta-respuesta anotados sobre la normativa ETSIT), requiere reentrenamiento ante cada actualización del corpus y dificulta la trazabilidad de las fuentes [5]. RAG, en cambio, separa la base de conocimiento (corpus indexado) del modelo generativo, lo que permite actualizar la primera sin tocar el segundo y, crucialmente, citar las fuentes exactas que sustentan cada respuesta.
- **¿Por qué secuencial y no iterativo?** Variantes como Self-RAG [50] introducen ciclos de reflexión que mejoran la fidelidad a costa de multiplicar las llamadas de inferencia. Con un LLM local de 7–8 B de parámetros en una GPU de consumo (RTX 4070 Super, 12 GB VRAM), cada iteración adicional penaliza la latencia de forma crítica para el RNF-01 (< 5 s). Se opta, por tanto, por un pipeline de pasada única (*single-pass*): una sola recuperación seguida de una sola generación, sin ciclos de retroalimentación, con un *prompt* cuidadosamente diseñado para maximizar la fidelidad en una sola generación. Si el sistema de evaluación (Capítulo 7) revelara problemas sistemáticos de fidelidad, la migración a un enfoque iterativo se contempla como línea de trabajo futuro (Capítulo 8).

En términos de la taxonomía presentada en la Sección 3.3.2 del Capítulo 3 [40], GRAIA se clasifica como un **Advanced RAG con diseño modular**. Supera el paradigma Naive RAG al incorporar técnicas de post-recuperación, reordenación neuronal de los candidatos mediante un *cross-encoder* [46] y diversificación de resultados mediante MMR [47] para reducir la redundancia entre fragmentos, y de post-generación, gestión explícita de la incertidumbre y verificación de atribución para detectar respuestas sin respaldo documental. Además, su arquitectura por capas desacopladas (Sección 5.5) permite la sustitución independiente de cada componente, rasgo

definitorio del paradigma Modular RAG. Se descartan explícitamente las variantes más complejas del espectro RAG:

- **Self-RAG** [50]: introduce múltiples ciclos de inferencia (generación, autoevaluación, regeneración) que multiplican la latencia total. En una GPU de consumo (RTX 4070 Super, 12 GB VRAM), cada pase adicional del LLM añade del orden de 1–3 s, lo que hace inviable cumplir el RNF-01 (< 5 s por respuesta).
- **CRAG** (*Corrective RAG*) [51]: además del mismo problema de latencia por ciclos de corrección, su mecanismo de *fallback* recurre a búsquedas web externas cuando la recuperación local se considera insuficiente. Esto entra en conflicto directo con el RNF-09 (ejecución íntegramente local e independencia de servicios externos).
- **Adaptive RAG** [52]: requiere un clasificador de complejidad entrenado *ad hoc* que decide dinámicamente la estrategia de recuperación. Ese clasificador exige datos de entrenamiento supervisados específicos del dominio ETSIIT, de los que no se dispone, además de introducir un componente adicional que mantener y evaluar.
- **Graph RAG** [53]: presupone la construcción previa de un grafo de conocimiento a partir del corpus. Para el volumen estimado de GRAIA (10^3 – 10^4 fragmentos), la complejidad de construcción y mantenimiento del grafo resulta desproporcionada frente al beneficio esperable, y la representación tabular y textual del corpus ETSIIT no se presta naturalmente a una estructura de grafo.

5.2.2. Ejecución íntegramente local

La segunda decisión es ejecutar *toda* la cadena, embeddings, índice vectorial e inferencia del LLM, en el hardware local del proyecto, sin depender de APIs comerciales en la nube. Esta decisión viene impuesta por los requisitos RNF-08 (privacidad), RNF-09 (independencia tecnológica) y la reproducibilidad, y se analiza en detalle en la Sección 5.9.1, donde se contrastan las alternativas local y remota. Se acepta conscientemente la penalización en calidad bruta del LLM respecto a modelos propietarios de frontera, mitigándola con modelos abiertos optimizados y evaluación empírica (RNF-03, RNF-04).

5.2.3. Arquitectura modular por capas

La tercera decisión es organizar el sistema en capas desacopladas con interfaces bien definidas, de modo que cada componente pueda sustituirse

de forma independiente. Esta decisión responde directamente a los requisitos RNF-10 (mantenibilidad) y RNF-11 (portabilidad a otros dominios): si en el futuro se desea emplear otro modelo de *embeddings*, otro LLM o aplicar el sistema a otra universidad, basta con sustituir el módulo correspondiente sin alterar el resto de la cadena.

5.3. Selección del *stack* tecnológico

Cada componente del *stack* se ha elegido tras evaluar las alternativas más relevantes, cuya comparativa detallada se presenta en la Sección 3.8 del Capítulo 3. Se exponen a continuación las decisiones tomadas y su justificación vinculada a los requisitos del sistema.

5.3.1. Lenguaje de programación: Python

Se elige **Python 3.11+** como lenguaje principal del proyecto (véase la comparativa en la Tabla 3.11). Esta decisión responde a cuatro razones convergentes:

1. **Ecosistema nativo.** Todas las bibliotecas clave del *stack* ofrecen *bindings* nativos en Python: **sentence-transformers** [56], FAISS [61], Ollama SDK y RAGAS [75].
2. **Rapidez de prototipado.** El calendario acotado del TFG (véase el Capítulo 2) exige ciclos de desarrollo cortos; Python permite iterar sin fase de compilación.
3. **Frameworks de interfaz ligeros.** Streamlit permite construir la interfaz web con un coste de desarrollo mínimo frente a las alternativas (véase Sección 3.8.3).
4. **Coherencia con la comunidad investigadora.** Los repositorios de referencia en RAG (Lewis et al. [5], Gao et al. [40], Asai et al. [50]) publican su código en Python, facilitando la reproducibilidad.

5.3.2. Servidor de inferencia LLM: Ollama

Se elige **Ollama** [87] como servidor de inferencia local (véase la comparativa en la Tabla 3.12). La elección se justifica por su alineación directa con los requisitos del sistema:

1. **Despliegue *zero-config*.** La instalación se reduce a un único binario y la descarga de modelos a `ollama pull <modelo>`, satisfaciendo la simplicidad operativa necesaria para un TFG.

2. **API compatible con OpenAI.** Ollama expone un *endpoint* REST (/v1/chat/completions) que hace el código agnóstico al proveedor: migrar a un servicio en la nube requeriría únicamente cambiar la URL base.
3. **Soporte nativo de cuantización GGUF.** Los modelos cuantizados en formato GGUF (4 bit, 5 bit) son esenciales para los 12 GB de VRAM disponibles (véase el Capítulo 2). Ollama gestiona automáticamente la carga en GPU.
4. **Gestión integrada de modelos.** El registro local con versionado facilita la reproducibilidad experimental.

5.3.3. Índice vectorial: FAISS

Se elige **FAISS** (*Facebook AI Similarity Search*) [61] como motor de indexación vectorial y búsqueda de vecinos aproximados (véase la comparativa en las Tablas 3.6 y 3.7). La elección responde a cuatro razones vinculadas a los requisitos:

1. **Rendimiento y escalabilidad (RNF-02).** FAISS garantiza holgadamente los < 200 ms de recuperación exigidos, con soporte nativo para GPU, y escala sin cambios si el corpus crece.
2. **Ejecución local sin servidor (RNF-09).** FAISS se importa como una biblioteca de Python, sin requerir un servidor externo (como Milvus con Docker) ni enviar datos a la nube (como Pinecone).
3. **Persistencia nativa (RF-09).** Los índices se serializan con `faiss.write_index()` y se recargan sin recalcular los *embeddings*.
4. **Referencia estándar en la literatura.** FAISS es el índice vectorial empleado por los principales trabajos de RAG y *retrieval* denso (Lewis et al. [5], Karpukhin et al. [42], Gao et al. [40]), lo que facilita la comparación con el estado del arte.

5.3.4. Modelo de *embeddings*: familia E5 multilingüe

Se elige la familia **E5 multilingüe** [57] (`multilingual-e5-base` o `multilingual-e5-large`) como modelo de *embeddings* (véase la comparativa en la Tabla 3.5). La elección responde a cuatro criterios alineados con los requisitos:

1. **Entrenamiento específico para recuperación (RNF-03, RNF-04).** A diferencia de modelos generalistas como mBERT [11], los mo-

delos E5 se entrenaron con un objetivo contrastivo orientado explícitamente a tareas de *retrieval* [57], situándolos en los primeros puestos del *benchmark* MTEB para español.

2. **Soporte multilingüe nativo (RNF-07).** El modelo se entrenó sobre textos en más de cien lenguas con representación sustancial del español.
3. **Ejecución local eficiente (RNF-09).** Con 278 M de parámetros (*base*) o 560 M (*large*), genera *embeddings* en milisegundos por fragmento en la RTX 4070 Super, sin competir significativamente por VRAM con el LLM.
4. **Licencia abierta (MIT).** Permite su uso académico y comercial sin restricciones.

5.3.5. LLM de generación: modelos abiertos vía Ollama

El modelo de lenguaje para la generación de respuestas debe cumplir simultáneamente los requisitos de calidad (RNF-03, RNF-04), latencia (RNF-01), soporte de español (RNF-07) y ejecutabilidad local en 12 GB de VRAM (RNF-09). Como muestra la Tabla 3.2, esto restringe la selección a modelos abiertos ejecutables en 12 GB de VRAM, lo que en cuantización de 4 bits (Q4.K_M) abarca desde unos 8 B hasta unos 14 B de parámetros (ocupando aproximadamente entre 5 y 9 GB). Los candidatos evaluados experimentalmente en el Capítulo 7 son Llama 3.1 8B [88], Gemma 2 9B y Qwen 2.5 14B, que cubren tres familias distintas y dos rangos de tamaño (8–9 B frente a 14 B).

La selección definitiva del modelo se realiza en función de los resultados experimentales del Capítulo 7, donde se comparan la precisión factual, la tasa de citación verificable, la abstención correcta y la latencia sobre el *dataset* de evaluación.

5.3.6. Interfaz web: Streamlit

Se elige **Streamlit** como *framework* de interfaz web (véase la comparativa en la Tabla 3.13). La decisión se fundamenta en cuatro razones alineadas con los requisitos:

1. **Componentes conversacionales nativos (RF-02).** Streamlit ofrece `st.chat_input` y `st.chat_message` desde la versión 1.25, proporcionando una experiencia de chat completa sin HTML ni JavaScript personalizados.

2. **Streaming integrado (RNF-01).** La función `st.write_stream` permite mostrar la respuesta del LLM token a token, mitigando la percepción de latencia.
3. **Gestión de estado sencilla (RF-02).** `st.session.state` mantiene el historial de conversación en memoria sin base de datos de sesiones.
4. **Despliegue autocontenido.** El servidor integrado reduce el despliegue a `streamlit run app.py`, alineándose con la simplicidad operativa del sistema.

5.4. Arquitectura general del sistema

La Figura 5.1 presenta la arquitectura de alto nivel de GRAIA. El sistema se organiza en cinco capas funcionales dispuestas en un flujo de datos que va desde las fuentes documentales hasta la respuesta al usuario:

1. **Capa de ingesta:** encargada de la adquisición, limpieza y fragmentación del corpus documental. Opera en modo *batch* (fuera de línea) y produce los fragmentos y metadatos que alimentan la capa siguiente. Cubre los requisitos RF-04 y RF-09.
2. **Capa de indexación:** transforma los fragmentos textuales en vectores densos mediante el modelo de *embeddings* y los almacena en el índice FAISS junto con los metadatos asociados. Opera también en modo *batch*. Cubre RF-09.
3. **Capa de recuperación:** dada una consulta del usuario, la busca simultáneamente en el canal denso (FAISS) y el canal léxico (BM25), fusiona ambos rankings mediante RRF, aplica un umbral de relevancia, reordena los candidatos con un *reranker* neuronal (*cross-encoder*) y diversifica con MMR. Opera en tiempo real. Cubre RF-02 y RNF-02.
4. **Capa de generación:** construye el *prompt* con los fragmentos recuperados, invoca el LLM vía Ollama, extrae las citas de la respuesta y gestiona los casos de incertidumbre. Opera en tiempo real. Cubre RF-02 y RF-03.
5. **Capa de presentación:** la interfaz Streamlit gestiona la entrada del usuario, muestra la respuesta con las citas formateadas y mantiene el historial de sesión. Opera en tiempo real. Cubre RF-01, RF-02 y RF-03. El registro de interacciones se delega en el sistema transversal de registro (Sección 5.12), que sostiene la evaluación del sistema (RF-10).

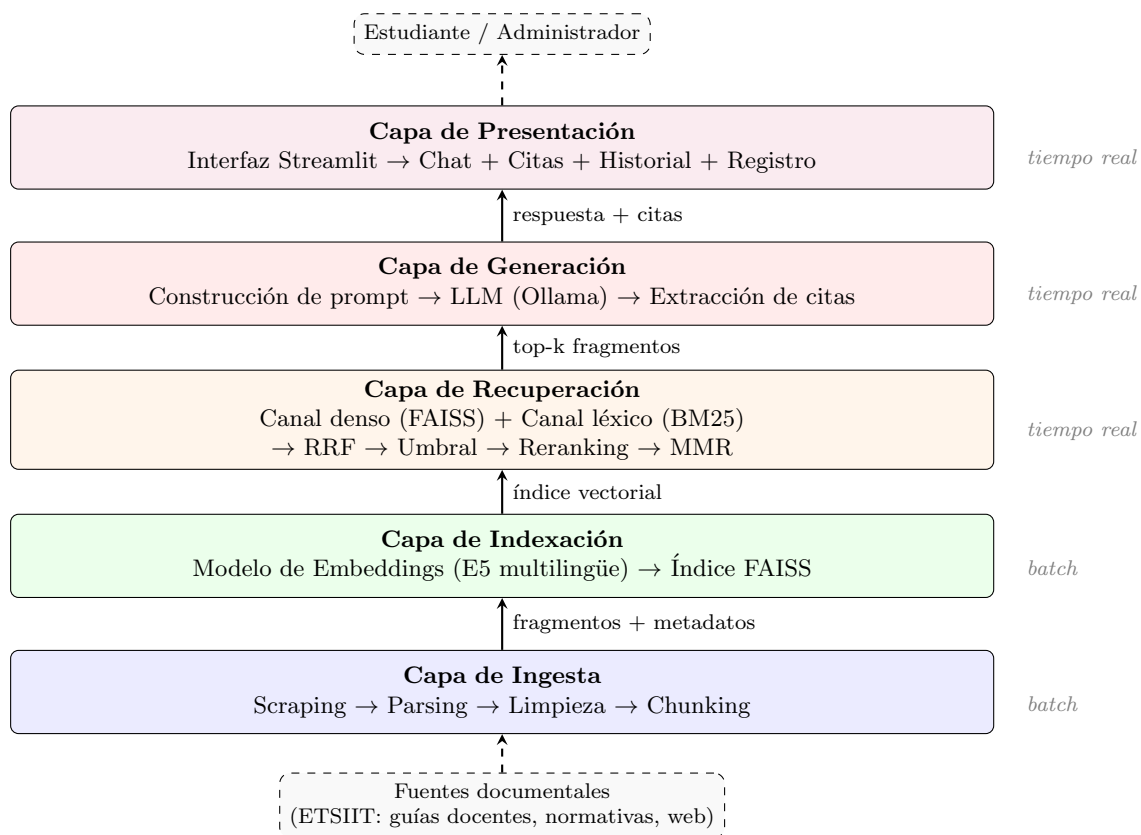


Figura 5.1: Arquitectura general de alto nivel del sistema GRAIA. Las capas inferiores operan en modo *batch* (fuera de línea) y las superiores en tiempo real ante cada consulta del usuario.

La separación en capas garantiza que las decisiones de diseño de un subsistema no se propagan al resto. Por ejemplo, sustituir el modelo de *embeddings* (capa de indexación) solo requiere regenerar el índice, sin modificar ni la capa de recuperación ni la de generación. Análogamente, cambiar el LLM (capa de generación) no afecta al índice vectorial ni a la interfaz. Esta propiedad de bajo acoplamiento es esencial para la portabilidad a otros dominios (RNF-11) y para la experimentación comparativa del Capítulo 7.

5.5. Patrones de diseño empleados

El diseño del sistema se apoya en cuatro patrones de diseño bien establecidos en la ingeniería del software. Se identifica a continuación cada patrón, dónde se aplica en la arquitectura y por qué se ha elegido frente a alternativas.

5.5.1. Pipe and Filter

El patrón *Pipe and Filter* [86] estructura un procesamiento como una cadena de etapas (filtros) conectadas por canales (pipes), donde la salida de cada etapa alimenta la entrada de la siguiente. Se aplica tanto en la **capa de ingesta** (scraping → parsing → limpieza → chunking) como en la **cadena de recuperación-generación** (codificación de consulta → búsqueda → filtrado → prompt → LLM → extracción de citas).

¿Por qué este patrón? Porque cada etapa opera sobre una interfaz bien definida (entrada textual o vectorial, salida textual o vectorial), lo que permite: (a) testear cada filtro de forma aislada con datos de prueba, (b) insertar o retirar etapas sin afectar al resto (por ejemplo, añadir un *re-ranker* entre la búsqueda y el prompt), y (c) paralelizar etapas independientes en la ingesta. Estas tres propiedades responden directamente al RNF-10 (mantenibilidad) y a la reproducibilidad (al poder fijar la salida de cada filtro).

5.5.2. Repository

El patrón *Repository* [85] centraliza el acceso a los datos persistentes detrás de una interfaz abstracta, desacoplando la lógica de negocio del mecanismo de almacenamiento concreto. Se aplica en dos puntos:

- **Repositorio del índice vectorial:** encapsula las operaciones de FAISS (`add`, `search`, `write_index`, `read_index`) detrás de una interfaz que podría sustituirse por ChromaDB u otro motor sin modificar la capa de recuperación. Esto soporta el RNF-11 (portabilidad).
- **Repositorio de metadatos:** almacena la correspondencia entre los identificadores de los vectores y los metadatos del fragmento (título del documento, URL, sección). Esta información es imprescindible para la citación (RF-03) y se mantiene separada del índice FAISS, que solo almacena vectores numéricos.

5.5.3. Strategy

El patrón *Strategy* [85] encapsula una familia de algoritmos intercambiables detrás de una interfaz común, permitiendo seleccionar el algoritmo en tiempo de configuración. Se aplica en tres puntos del sistema:

- **Estrategia de chunking:** el usuario administrador puede elegir entre fragmentación de tamaño fijo con solapamiento o fragmentación semántica basada en la jerarquía del documento (RF-09).

- **Estrategia de embeddings:** el modelo de *embeddings* se especifica en la configuración; cambiar de `multilingual-e5-base` a `multilingual-e5-large` no requiere modificar código.
- **Estrategia de LLM:** el modelo de generación se selecciona en la configuración de Ollama; la capa de generación es agnóstica al modelo concreto (RF-02, RNF-10).

¿Por qué Strategy y no simple configuración? Porque no se trata solo de cambiar un parámetro, sino de intercambiar algoritmos con lógica diferente (el *chunking* semántico tiene una implementación distinta al de tamaño fijo). El patrón Strategy formaliza esta variabilidad y la hace explícita en el diseño.

5.5.4. Facade

El patrón *Facade* [85] expone una interfaz simplificada sobre un subsistema complejo. Se aplica en la **capa de generación**, donde una función orquestadora (`handle_query` en el módulo de interfaz) coordina la secuencia completa: recibe la consulta del usuario, invoca la recuperación, construye el prompt, llama al LLM, extrae las citas y devuelve la respuesta formateada. La capa de presentación solo necesita llamar a esta función, sin conocer la complejidad interna.

¿Por qué? Porque la interfaz Streamlit no debe contener lógica de recuperación ni de prompt engineering; su responsabilidad es exclusivamente la presentación. Facade impone esta separación de responsabilidades, alineándose con el principio de responsabilidad única y con el RNF-10.

5.6. Diseño de la capa de ingesta

La capa de ingesta es responsable de transformar las fuentes documentales heterogéneas de la ETSIIT en una colección homogénea de fragmentos textuales indexables, materializando el **OE2** (*Pipeline de ingesta de documentos heterogéneos*). Se ejecuta en modo *batch*: no interviene durante la interacción con el usuario, sino que se lanza de forma periódica o bajo demanda para construir o actualizar la base de conocimiento. Este diseño satisface los requisitos RF-04 a RF-09 (adquisición, extracción, normalización y segmentación del corpus) y condiciona de manera determinante la calidad final del sistema RAG, pues como señala [40] la calidad de la recuperación está acotada superiormente por la calidad de los fragmentos indexados.

5.6.1. Estrategia de adquisición del corpus: *crawl* automático

Una decisión de diseño previa a la descomposición en subcomponentes es *cómo* se identifica y recopila el conjunto de fuentes documentales que alimentará el sistema. Se han evaluado dos alternativas:

1. **Recopilación manual:** un operador humano (el administrador del sistema o, en el contexto de este TFG, el propio autor) selecciona, descarga y organiza individualmente cada documento (PDFs de normativas, páginas HTML de guías docentes, etc.) en un directorio local.
2. ***Crawl* recursivo automatizado:** un rastreador web parte de un conjunto reducido de páginas semilla (*seed URLs*) dentro del dominio `etsiit.ugr.es`, sigue recursivamente todos los hipervínculos y descarga tanto páginas HTML como documentos PDF enlazados, aplicando filtros de dominio y de relevancia temática.

Se ha optado por la segunda alternativa por cuatro razones fundamentales:

Cobertura. La web de la ETSIIT es un sitio dinámico gestionado por un CMS Drupal con centenares de páginas interconectadas. Un proceso manual dependería del conocimiento del operador sobre la estructura del sitio y omitiría inevitablemente páginas descubiertas solo a través de enlaces internos profundos (subpáginas de departamentos, PDFs enlazados desde normativas, preguntas frecuentes de movilidad, etc.). El *crawl* recursivo garantiza que toda página alcanzable desde las semillas se evalúa como candidata.

Reproducibilidad. El catálogo de fuentes se declara en un fichero de configuración versionado (`sources.yaml`) que contiene las URLs semilla, los dominios permitidos, los patrones de exclusión y los parámetros del rastreo (profundidad máxima, retardo de cortesía, límite de páginas). Cualquier investigador puede reproducir exactamente la misma ingesta ejecutando un único comando, sin depender de instrucciones verbales ni capturas de pantalla.

Escalabilidad. Si en el futuro se ampliase el alcance del sistema a otras escuelas o facultades de la Universidad de Granada (línea futura señalada en el Capítulo 8), bastaría con añadir nuevos dominios y semillas al fichero de configuración; el pipeline de ingesta se ejecutaría sin modificar código. En un enfoque manual, cada nueva escuela multiplicaría linealmente el esfuerzo

humano requerido, con el riesgo añadido de inconsistencias en la estructura de los datos recopilados.

Actualización periódica. La información académica cambia cada curso: se actualizan guías docentes, calendarios y normativas. El rastreador puede re-ejecutarse al inicio de cada curso académico modificando únicamente el parámetro `current_academic_year` del fichero de configuración; los filtros temporales se regeneran automáticamente y el nuevo rastreo descarga solo los documentos del curso vigente. Gracias al manifiesto de ingesta (descrito en la Sección 5.6.3), el proceso de actualización se reduce a un único cambio de configuración seguido de un comando de ejecución, frente a las horas que exigiría una revisión manual documento por documento.

Diseño del catálogo de fuentes: lista blanca por categoría

El rastreador adopta un enfoque de **lista blanca por categoría**: en lugar de descargar todo lo alcanzable y filtrar a posteriori, el operador declara explícitamente qué contenido quiere mediante un fichero de configuración declarativo (`sources.yaml`). Esta decisión responde a una característica del dominio: el CMS Drupal de la ETSIIT genera múltiples variantes de URL para un mismo documento (sufijos `_N` por versionado interno) y aloja ficheros históricos de cursos anteriores con formatos de nombre heterogéneos (`((XX-YY), YYYY_YYYY, MesYYYY)`). Un enfoque reactivo que intentase excluir todo lo irrelevante sería frágil e insostenible ante esta variabilidad; la lista blanca invierte la carga de la prueba y produce un corpus predecible.

El fichero `sources.yaml` se estructura en tres bloques jerárquicos, cada uno con una responsabilidad diferenciada:

- **Bloque global:** parámetros compartidos por todas las categorías. Incluye los dominios permitidos (`etsiit.ugr.es`, `grados.ugr.es`, `secretariageneral.ugr.es`, `www.ugr.es`), las extensiones de fichero descargables (`.html`, `.htm`, `.pdf`), patrones de exclusión estáticos (versiones en inglés, secciones institucionales genéricas), activación de deduplicación por *hash* de contenido y parámetros operativos del rastreo (retardo de cortesía, límite global de seguridad). Dos parámetros merecen atención especial:
 - `current_academic_year`: declara el curso vigente (p. ej., "25-26"). A partir de este único valor, el rastreador genera *automáticamente* patrones de exclusión temporal para todos los cursos anteriores, cubriendo las variantes de formato observadas en las URLs de la ETSIIT: `((XX-YY), XX-YY, 20XX-YY, YYYY_YYYY, MesYYYY,`

Mes YYYY y abreviaturas (Nov21, nov24). Para documentos temporales, Mes 2025 se interpreta correctamente como perteneciente al curso 24-25 (segundo semestre), no al 25-26.

- **deduplicate**: activa la eliminación de documentos con contenido idéntico (mismo *hash* SHA-256), neutralizando el versionado Drupal que genera copias del mismo PDF con sufijos `_0`, `_1`, `...`, `_N`.
- **Bloque categorías**: define un **perfil de descarga** independiente para cada tipo de contenido del dominio académico. Cada perfil contiene:
 - URLs semilla propias, desde las que parte el descubrimiento BFS.
 - Una lista de **include_patterns** (expresiones regulares que determinan qué enlaces descubiertos se descargan).
 - Una lista opcional de **exclude_patterns** por categoría, para filtrar URLs irrelevantes específicas de esa sección (p. ej., fichas de protección de datos en trámites).
 - Un indicador **temporal** que activa o desactiva el filtrado por curso académico.
 - Profundidad máxima de descubrimiento y límite de páginas propio.

Las ocho categorías definidas son: *guías docentes*, *calendario* (calendarios, horarios y asignaciones de aulas), *normativa*, *trámites*, *movilidad*, *TFG*, *estudiantes* y *profesorado*. Un enlace descubierta que no case con ningún **include_pattern** de la categoría activa simplemente no se descarga.

- **Bloque extra_urls**: lista un conjunto reducido de URLs externas al dominio **etsiit.ugr.es** que contienen información normativa esencial publicada en otros servidores de la UGR (**secretariageneral.ugr.es**, **grados.ugr.es**). Cada entrada incluye un campo **label** con el título legible del documento y la categoría a la que pertenece. Estos documentos se descargan directamente sin aplicar descubrimiento recursivo.

Desacoplamiento y extensibilidad. Un aspecto central de este diseño es el **desacoplamiento entre la configuración y el código de rastreo**. Todo el conocimiento sobre la estructura del sitio web, los patrones de URL y las categorías temáticas reside en **sources.yaml**; el código de **build_corpus.py** es un motor genérico que interpreta esa configuración sin contener ninguna referencia a URLs, dominios o categorías concretas. Esto tiene dos consecuencias prácticas relevantes:

1. **Adaptación a otras universidades.** Si en el futuro se ampliase el alcance del sistema a otras escuelas o facultades de la Universidad de Granada, o incluso a otras universidades, (línea futura señalada en el Capítulo 8), el esfuerzo de adaptación se reduciría a añadir o sustituir en `sources.yaml` los dominios, semillas y patrones de la nueva institución. El código de rastreo, procesado e indexación no requeriría modificación alguna.
2. **Modificación independiente de filtros.** Un operador puede añadir, eliminar o ajustar cualquier patrón de inclusión o exclusión sin alterar el flujo de ejecución del rastreador. Los filtros son *datos declarativos*, no lógica de programa, lo que reduce drásticamente el riesgo de introducir regresiones al ajustar la configuración.

Arquitectura del pipeline de construcción

El proceso completo de construcción del corpus se articula en tres fases secuenciales con una puerta de validación humana intermedia (Figura 5.2):

1. **Fase de rastreo** (`build_corpus.py`): el rastreador parte de las semillas de cada categoría, ejecuta un recorrido BFS independiente, descarga páginas HTML y PDFs, aplica el pipeline *fetch* $\rightarrow$ *parse* $\rightarrow$ *clean* con filtros de calidad y almacena los documentos procesados en formato JSONL junto con un manifiesto e informe de rastreo.
2. **Fase de revisión** (`review_corpus.py`): el operador (administrador del sistema o, en este TFG, el propio autor) inspecciona estadísticas por categoría, previsualizaciones de contenido y documentos atípicos (excesivamente cortos o largos) para validar la calidad del corpus antes de proceder. Además de la inspección, la herramienta permite al operador **editar el corpus manualmente**: puede eliminar documentos que considere irrelevantes o de baja calidad, e insertar nuevos documentos proporcionando una URL y una categoría, de modo que el corpus final refleje el criterio experto del administrador y no dependa exclusivamente del rastreo automático.
3. **Fase de indexación** (`index_corpus.py`): los documentos validados se fragmentan, se vectorizan con el modelo E5 y se almacenan en el índice FAISS.

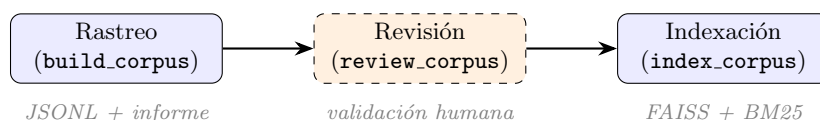


Figura 5.2: Fases del pipeline de construcción del corpus con puerta de validación intermedia.

Esta separación en fases con revisión intermedia es una decisión de diseño intencionada: permite detectar problemas de cobertura o calidad del corpus *antes* de invertir tiempo de GPU en la generación de *embeddings*, lo que reduce el coste de iteración durante el desarrollo y la evaluación del sistema.

Diseño del flujo de rastreo por categoría

El rastreador ejecuta un **recorrido BFS independiente por cada categoría** definida en `sources.yaml`. Cada categoría parte de sus propias semillas y mantiene un conjunto de URLs visitadas local que controla el descubrimiento dentro de esa categoría; un segundo conjunto de URLs visitadas global impide descargar el mismo documento dos veces a través de peticiones HTTP. Las semillas de cada categoría se encolan siempre, independientemente de que hayan sido descubiertas previamente por otra categoría, puesto que son documentos que el operador ha declarado explícitamente como relevantes.

Cada URL candidata descubierta por el BFS atraviesa una **cascada de filtros** antes de ser descargada:

1. **Filtro de dominio:** verifica que el dominio pertenece a la lista de dominios permitidos.
2. **Filtro de extensión:** solo acepta `.html`, `.htm`, `.pdf` y URLs sin extensión (páginas dinámicas del CMS).
3. **Filtro de exclusión estática:** aplica los patrones *regex* globales (versiones en inglés, noticias, posgrados, etc.) y los patrones de exclusión propios de la categoría.
4. **Filtro temporal:** si la categoría tiene `temporal: true`, aplica los patrones de exclusión de cursos anteriores generados automáticamente.
5. **Filtro de inclusión:** verifica que la URL decodificada case con algún `include_pattern` de la categoría activa.

Una propiedad importante de esta cascada es que **cada filtro es una función pura e independiente**. Añadir un nuevo patrón de exclusión

a una categoría no requiere modificar ningún otro filtro, ni el código del recorrido BFS, ni los parámetros globales. Esta independencia facilita la depuración: cuando un documento no deseado aparece en el corpus, basta con añadir un patrón de exclusión a la categoría correspondiente en `sources.yaml` y re-ejecutar el rastreo.

El bloque `extra_urls` se procesa tras las categorías: cada URL se descarga directamente sin aplicar el filtro de inclusión ni el descubrimiento BFS, puesto que se trata de documentos que el operador ha identificado manualmente como necesarios. Estas URLs se descargan siempre, independientemente de que el BFS de alguna categoría las haya visitado previamente, para garantizar que la configuración explícita del operador prevalezca sobre el comportamiento automático del rastreador.

Heurísticas de calidad del contenido

El filtrado por URL descrito anteriormente opera *antes* de la descarga: descarta URLs que, por su estructura, no pueden contener contenido relevante. Sin embargo, determinados documentos superan todos los filtros de URL y, una vez descargados y procesados, resultan igualmente inútiles para el sistema RAG. Estos falsos positivos se agrupan en tres familias:

1. **Páginas índice.** El CMS Drupal de la ETSIIT genera páginas de nivel intermedio que solo contienen un listado de hipervínculos a subpáginas, sin aportar contenido informativo propio. Si estos documentos se indexan, producen fragmentos con escasa señal semántica que contaminan los resultados de recuperación.
2. **Formularios y solicitudes para rellenar.** La sección de secretaría de la ETSIIT publica impresos administrativos en formato PDF (solicitudes de reconocimiento de créditos, formularios de matrícula). El texto extraído de estos documentos es una retahíla de campos vacíos (“*D.N.I.*”, “*Apellido 1*”, “*Firma del solicitante*”), sin valor informativo para un asistente conversacional.
3. **Documentos con título ausente o genérico.** Muchos PDFs institucionales carecen de metadatos de título, o contienen un título genérico insertado por la herramienta de generación del PDF. La ausencia de título dificulta el sistema de citación (Sección 5.10), que necesita un título legible para presentar las fuentes al usuario.

Para abordar estos casos se diseñan tres **heurísticas de post-descarga** que se aplican durante la fase de procesado, justo antes de serializar cada documento en el corpus JSONL:

Detección de páginas índice. Se define una heurística basada en dos indicadores complementarios: (i) la longitud del texto limpio, y (ii) la *densidad de líneas cortas*. Las páginas HTML con menos de 600 caracteres de contenido limpio se descartan automáticamente, puesto que la experiencia empírica muestra que ninguna página informativa de la ETSIIT produce tan poco texto tras la eliminación de *boilerplate*. Para páginas de entre 600 y 1 500 caracteres, se calcula la proporción de líneas con menos de 60 caracteres: si supera el 70 %, se clasifica como índice, ya que este patrón es característico de listas de enlaces. Los PDFs se excluyen de esta heurística porque, por su naturaleza, siempre contienen contenido propio.

Detección de formularios. Se compila un conjunto de indicadores léxicos propios de impresos administrativos (“D.N.I.”, “Apellido”, “Firma”, “N.º de expediente”, “Sello del registro”) y se cuentan las coincidencias en los primeros 500 caracteres del texto limpio. Si aparecen dos o más indicadores, el documento se clasifica como formulario y se descarta. El umbral de dos indicadores evita falsos positivos: un único término como “Firma” podría aparecer legítimamente en una normativa que describe un procedimiento de firma.

Inferencia de títulos. Se aplica una estrategia de *fallback* en tres niveles de prioridad: (1) si la URL pertenece a una entrada `extra_urls` con un campo `label` en `sources.yaml`, se usa esa etiqueta como título, la configuración explícita del operador siempre prevalece; (2) si el parser ha extraído un título válido (más de un carácter) del documento, se utiliza ese título; (3) en caso contrario, se deriva el título del último segmento de la ruta de la URL, reemplazando guiones y guiones bajos por espacios y eliminando la extensión del fichero (p. ej., `Reglamento-permanencia-UGR.pdf` → “*Reglamento permanencia UGR*”). Este mecanismo garantiza que todo documento del corpus tenga un título legible para el sistema de citación.

Estas tres heurísticas se diseñan como **funciones puras sin estado compartido**, lo que facilita su prueba unitaria aislada y permite desactivarlas o sustituirlas individualmente si un corpus diferente presentase características distintas.

Normalización de URLs y deduplicación

El CMS Drupal de la ETSIIT genera dos fuentes sistemáticas de duplicados que, sin tratamiento explícito, inflarían el corpus e introducirían fragmentos redundantes en el índice vectorial:

- **Duplicados por esquema HTTP/HTTPS.** Numerosos enlaces in-

ternos del sitio utilizan `http://` aunque el servidor redirige transparentemente a `https://`. Sin normalización, el rastreador trataría ambas variantes como documentos distintos.

- **Duplicados por versionado Drupal.** El CMS genera copias de documentos PDF con sufijos numéricos incrementales (`documento.pdf`, `documento_0.pdf`, `documento_1.pdf`), todas con idéntico contenido.

Para el primer caso, se aplica una **normalización de URL** que convierte `http://` a `https://` para todos los dominios `ugr.es` y elimina barras finales superfluas. Esta normalización se ejecuta tanto al encolar las URLs descubiertas como al registrar las URLs ya visitadas, garantizando que el conjunto de visitados actúe correctamente como filtro de duplicados.

Para el segundo caso, se aplica una **deduplicación por hash** del contenido: tras procesar cada documento (*parse* $\rightarrow$ *clean*), se calcula un *hash* SHA-256 truncado a 16 caracteres del texto limpio. Si el hash coincide con el de un documento ya procesado, el duplicado se descarta y se registra en el *log* con la URL del original. Este enfoque es agnóstico a la URL y detecta cualquier par de documentos con contenido idéntico, independientemente del mecanismo que haya generado la copia.

5.6.2. Descomposición en subcomponentes

Siguiendo el patrón *Pipe and Filter* introducido en la Sección 5.5, la capa de ingesta se descompone en cuatro subcomponentes encadenados, cada uno con una responsabilidad única:

1. **Recolector (*Fetcher*):** adquiere los documentos brutos de sus fuentes originales mediante *scraping* (páginas web ETSIT, archivos PDF institucionales, guías docentes en formato HTML). Produce una colección de archivos con formato y codificación originales.
2. **Extractor (*Parser*):** convierte cada documento bruto a texto plano Unicode, preservando la estructura lógica esencial (títulos, secciones) y descartando elementos no informativos (menús de navegación, pies de página repetidos, anuncios).
3. **Normalizador (*Cleaner*):** aplica transformaciones textuales deterministas para eliminar ruido y homogeneizar la representación (normalización Unicode, colapso de espacios, eliminación de caracteres de control).
4. **Segmentador (*Chunker*):** divide el texto normalizado en fragmentos de tamaño acotado y adjunta los metadatos necesarios para la posterior trazabilidad de citas (RF-03).

La salida del pipeline es una lista de objetos **Chunk** serializable en formato JSONL, con el esquema descrito en la Tabla 5.1.

Campo	Tipo	Descripción
<code>chunk_id</code>	<code>string</code>	Identificador único determinista (<i>hash</i> SHA-256 truncao del texto). Permite detectar duplicados y actualizaciones incrementales.
<code>text</code>	<code>string</code>	Contenido textual del fragmento.
<code>source_url</code>	<code>string</code>	URL original de la que procede el documento, base para la cita (RF-03).
<code>doc_title</code>	<code>string</code>	Título del documento fuente, empleado en la presentación de citas.
<code>doc_type</code>	<code>enum</code>	Tipo de fuente: <code>guia_docente</code> , <code>normativa</code> , <code>web_etsiit</code> , <code>otros</code> .
<code>section_path</code>	<code>list[string]</code>	Ruta jerárquica de la sección (p. ej. ["Grado", "Competencias"]).
<code>char_start</code> , <code>char_end</code>	<code>int</code>	Offsets en el documento original, útiles para depuración y evaluación.
<code>ingested_at</code>	<code>datetime</code>	Marca temporal de la ingesta, para detectar obsolescencia.

Cuadro 5.1: Esquema del objeto **Chunk** producido por la capa de ingesta.

La inclusión de `source_url`, `doc_title` y `section_path` en el propio fragmento, en lugar de mantenerlos en una tabla externa, responde a un principio de *localidad*: toda la información necesaria para citar un fragmento viaja con él a lo largo del pipeline, lo que simplifica la implementación del sistema de citas (Sección 5.10) y reduce la probabilidad de inconsistencias.

5.6.3. Diseño del recolector

El recolector se diseña como una jerarquía de clases concretas bajo una interfaz común **SourceFetcher**, con una implementación específica por tipo de fuente:

- **WebFetcher**: descarga páginas HTML estáticas mediante la biblioteca `requests`, acompañada de `BeautifulSoup` para el *parsing* posterior. Se opta por `requests` frente a `Scrapy` por simplicidad: el volumen documental de la ETSIIT, del orden de cientos, no miles, de páginas, no justifica la complejidad de un *framework* de *crawling* completo. Se opta por `requests` frente a `Selenium` porque el contenido relevante de la web ETSIIT es HTML estático renderizado en servidor, sin dependencias de JavaScript dinámico que exijan un navegador *headless*.
- **PDFFetcher**: descarga archivos PDF (normativas, reglamentos) y los almacena localmente.

- **LocalFetcher**: lee archivos ya presentes en el sistema de ficheros, útil para fuentes aportadas manualmente.

Cada recolector respeta el archivo `robots.txt` del dominio y aplica un *rate limiting* explícito (1 petición por segundo por defecto) para evitar sobrecargar los servidores institucionales. Los errores HTTP se gestionan con una política de reintentos exponenciales acotada (máximo tres intentos), y los documentos inaccesibles se registran en el *log* (RF-04) pero no interrumpen el pipeline.

La trazabilidad completa de los documentos recolectados se persiste en un manifiesto `manifest.jsonl` con la URL, el hash del contenido y la marca temporal. Este manifiesto habilita la *ingesta incremental*: en ejecuciones posteriores, solo los documentos cuyo hash haya cambiado se reprocesan, lo que responde al requisito de eficiencia operativa sin formar parte explícita de los RNF.

5.6.4. Diseño del extractor

El extractor convierte documentos brutos a texto plano. Su diseño se bifurca según el formato de origen:

Para PDF. Se emplea PyMuPDF (`fitz`) [89] como biblioteca principal. La comparativa detallada con las alternativas (`pypdf`, `pdfplumber`, `pdfminer.six`) se presenta en la Tab. 3.14 del Capítulo 3. La elección se fundamenta en tres factores: PyMuPDF es significativamente más rápido que `pypdf` en tareas de extracción de texto, preserva mejor el orden de lectura en documentos con columnas múltiples, común en normativas universitarias, y ofrece un API estable para acceder a metadatos estructurales. Se descarta `pdfplumber` porque, aunque superior en extracción de tablas, es significativamente más lento para texto corriente, que constituye la mayor parte del corpus ETSIIT.

Para HTML. Se emplea BeautifulSoup [90] con el *parser* `lxml` (más rápido y tolerante a HTML mal formado que el *parser* estándar de Python). El extractor HTML aplica una *lista negra* de selectores CSS correspondientes a elementos no informativos (`nav`, `footer`, `aside`, cajas de cookies, menús laterales) antes de extraer el texto. Los *headings* `h1-h6` se preservan como marcadores estructurales para la posterior segmentación por secciones.

Una decisión importante es *no* convertir el HTML a Markdown intermedio (como hace LangChain por defecto). La conversión a Markdown introduce artefactos (asteriscos residuales, enlaces expandidos) que perjudican la

calidad de los *embeddings*; se prefiere extraer directamente texto plano con el árbol de secciones preservado en los metadatos.

5.6.5. Diseño del normalizador

El normalizador aplica, en orden, las siguientes transformaciones deterministas:

1. **Normalización Unicode NFC:** unifica la representación de caracteres acentuados, crítico en español, donde “á” puede codificarse como U+00E1 o como a + U+0301. Sin este paso, dos fragmentos visualmente idénticos producirían *embeddings* distintos.
2. **Colapso de espacios en blanco:** secuencias de espacios, tabuladores y saltos de línea consecutivos se reducen a un único espacio, preservando los saltos de párrafo como doble salto de línea.
3. **Eliminación de caracteres de control:** se eliminan caracteres no imprimibles (rango U+0000-U+001F excepto tabulador y salto de línea).
4. **Detección y eliminación de artefactos recurrentes:** mediante un análisis de frecuencia, se identifican y eliminan líneas que aparecen en todas las páginas del documento (típicos encabezados o pies institucionales) y que introducen ruido sin aportar información distintiva.

Se descartan deliberadamente transformaciones más agresivas como la eliminación de *stopwords* o el *stemming*: los modelos de *embeddings* modernos basados en *transformers*, y en particular E5, se entrenan sobre texto natural completo y su rendimiento se degrada cuando se les alimenta con texto preprocesado en exceso [57].

5.6.6. Diseño del segmentador

La segmentación (*chunking*) es, probablemente, la decisión con mayor impacto en la calidad de la recuperación. Un fragmento demasiado corto carece de contexto suficiente para responder a preguntas complejas; uno demasiado largo diluye la señal semántica y puede exceder el contexto del LLM en la fase de generación.

Estrategia adoptada: *recursive character splitting* orientado a tokens

Se opta por una estrategia de segmentación recursiva basada en separadores jerárquicos, popularizada por LangChain pero implementada aquí de

forma autónoma para evitar dependencias superfluas. El algoritmo procede así:

1. Intenta dividir el texto por *doble salto de línea* (fronteras de párrafo).
2. Si algún fragmento resultante supera el límite, se divide de nuevo por *salto de línea simple*.
3. Si persiste el exceso, se divide por punto seguido (final de oración).
4. Como último recurso, se divide por espacio en blanco.

Los fragmentos resultantes se agrupan hasta alcanzar un tamaño objetivo sin superarlo, y se añade un *solapamiento* (*overlap*) para preservar continuidad contextual entre fragmentos adyacentes.

Elección de parámetros

Se adoptan los siguientes valores por defecto, justificados experimentalmente en el Capítulo 7:

- **Tamaño objetivo de fragmento:** 512 tokens. Este valor coincide con la longitud de contexto máxima eficaz del modelo `intfloat/multilingual-e5-base` [91], de modo que ningún fragmento se trunca en la fase de indexación.
- **Solapamiento:** 50 tokens ($\approx 10\%$). El solapamiento mitiga el riesgo de que una pieza de información relevante quede partida justo en la frontera entre fragmentos, una práctica extendida en la literatura RAG aplicada [40].
- **Unidad de medida:** tokens del *tokenizer* del modelo de *embeddings*, no caracteres ni palabras. Medir en caracteres produce fragmentos de longitud real variable según densidad léxica; medir en tokens del propio modelo garantiza coherencia con el paso siguiente del pipeline.

Alternativas consideradas

Se han evaluado tres alternativas y se han descartado por las razones siguientes:

- **Fragmentación de tamaño fijo por caracteres** (p. ej. ventanas de 1000 caracteres con solapamiento de 200): técnica más simple pero desatiende fronteras lingüísticas naturales, pudiendo cortar oraciones a mitad. Se descarta por su menor calidad.

- **Fragmentación *semántica*** basada en agrupación de oraciones por similitud coseno entre sus *embeddings* individuales: atractiva en teoría, pero introduce dos pasadas por el modelo de *embeddings*, una no determinista (la calidad depende del umbral de similitud) y computacionalmente costosa. Se descarta por el sobrecoste y por la dificultad de ajustar sus hiperparámetros sin un dataset de evaluación del propio chunking.
- **Fragmentación por estructura documental** (un fragmento por sección o subsección): óptima cuando los documentos tienen estructura jerárquica clara, pero inadecuada cuando las secciones son muy largas o muy cortas. En el corpus ETSIT, la heterogeneidad de secciones hace inviable esta estrategia como única política, aunque el *path* jerárquico se preserva en los metadatos de cada fragmento para enriquecer el contexto.

La decisión final, por tanto, es *recursive character splitting* con solapamiento, sintonizada sobre el *tokenizer* del modelo de *embeddings*. El Capítulo 7 presenta un análisis de sensibilidad sobre el tamaño de fragmento para verificar empíricamente la bondad de esta elección.

5.7. Diseño de la capa de indexación vectorial

La capa de indexación transforma cada **Chunk** producido por la ingesta en un vector denso de tamaño fijo y los almacena en una estructura optimizada para búsqueda por similitud, materializando el **OE3** (*Sistema de indexación vectorial con chunking optimizado*). Satisface el requisito RF-09 y es el núcleo técnico sobre el que se apoyan la recuperación (RF-02) y, en última instancia, la calidad semántica del sistema.

5.7.1. Modelo de *embeddings*

La selección del modelo de *embeddings* se justificó en la Sección 5.3.4, donde se optó por `intfloat/multilingual-e5-base`. En la presente sección se detalla su uso en el diseño.

Prefijos de instrucción

El modelo E5 fue entrenado con una convención que distingue textos indexados (“pasajes”) de textos de consulta mediante prefijos explícitos: “`passage:` ” para el primer caso y “`query:` ” para el segundo [91]. Omitir

estos prefijos degrada el rendimiento en entre dos y cinco puntos porcentuales en los *benchmarks* MTEB [1]. El diseño incorpora esta convención de forma explícita: el **Indexer** antepone "passage: " a cada fragmento antes de codificarlo, mientras que el **Retriever** (Sección 5.8) antepone "query: " a la consulta del usuario.

Normalización L2 y similitud

Los vectores producidos por E5 se normalizan a norma euclídea unitaria ($\|\mathbf{v}\|_2 = 1$). Esta normalización permite que el producto escalar de dos vectores coincida con su similitud del coseno:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \mathbf{u} \cdot \mathbf{v} \quad \text{cuando} \quad \|\mathbf{u}\| = \|\mathbf{v}\| = 1.$$

La consecuencia práctica es que se puede emplear un índice FAISS basado en producto interno (**IndexFlatIP**), computacionalmente más ligero que uno basado en distancia euclídea, sin pérdida de semántica. La normalización se aplica tanto a los fragmentos en la fase de indexación como a las consultas en la fase de recuperación, de modo que ambos espacios son compatibles.

Procesamiento por lotes

La codificación de fragmentos se ejecuta por lotes (*batches*) de 32 elementos, aprovechando el paralelismo de la GPU (RTX 4070 Super, 12 GB VRAM). Este tamaño de lote se eligió empíricamente como el mayor que no provoca desbordamiento de memoria con fragmentos de hasta 512 tokens. Valores menores subutilizan la GPU; valores mayores requerirían activar *gradient checkpointing* que, al ser una fase de solo inferencia, no aporta beneficio.

5.7.2. Estructura de índice: FAISS IndexFlatIP

Se emplea un índice FAISS de tipo **IndexFlatIP**: una búsqueda exhaustiva (*brute force*) sobre el producto interno entre la consulta y todos los vectores indexados. Esta elección, aparentemente ingenua, está plenamente justificada para el volumen documental del proyecto.

Justificación frente a índices aproximados

Los índices aproximados, IVF, HNSW, PQ, sacrifican precisión (*recall*) por velocidad. Esta compensación es inútil cuando el volumen indexado es pequeño. Para el corpus ETSIIT se estima un orden de magnitud de entre 10^3 y 10^4 fragmentos totales. En ese régimen:

- Una búsqueda exhaustiva sobre 10^4 vectores de 768 dimensiones se resuelve en menos de 20 ms en CPU y en menos de 5 ms en GPU [61].
- El sobrecoste de construir y mantener un índice aproximado (parámetros `nprobe` en IVF, `ef_construction` y `M` en HNSW, entrenamiento de centroides) no se amortiza: el *speedup* es despreciable y se introduce el riesgo de perder resultados relevantes.
- La latencia total del sistema está dominada por la inferencia del LLM (RF-02), no por la búsqueda vectorial.

La Tab. 3.7 del Capítulo 3 resume las características de los principales tipos de índice FAISS. Para el volumen estimado del corpus ETSIT (10³–10⁴ fragmentos), `IndexFlatIP` ofrece el 100 % de *recall* con latencias inferiores a 20 ms en CPU, por lo que la complejidad adicional de índices aproximados no se justifica.

Si el volumen del corpus creciese significativamente, por ejemplo, al extender el sistema a toda la UGR o a un consorcio de universidades, línea de trabajo futuro descrita en el Capítulo 8, una migración a `IndexHNSWFlat` sería la evolución natural; el diseño modular (Sección 5.5) hace esta sustitución local al módulo de indexación.

5.7.3. Contextualización de fragmentos (*Contextual Retrieval*)

Un fragmento aislado puede perder el contexto que lo hace recuperable. Por ejemplo, una línea como “del 19 al 21 de noviembre de 2025” es informativa solo si se sabe que se refiere a la *defensa del TFG*; codificada por sí sola, su *embedding* no se aproxima a consultas como “¿cuándo se defiende el TFG?”. Este fenómeno de *fragmentos huérfanos* degrada el *recall* en corpus con mucha estructura tabular.

Para mitigarlo se adopta la técnica de *Contextual Retrieval* propuesta por Anthropic [92]: antes de codificar cada fragmento, se le antepone un breve *contexto* generado por el LLM local que lo sitúa dentro de su documento de origen (de qué documento procede y a qué sección o tema pertenece). Ese texto contextualizado es el que se codifica con E5 y el que alimenta el índice BM25, de modo que el fragmento gana señal semántica y léxica sin perder su contenido original. El coste de generación se paga una sola vez, *fuera de línea*, durante la indexación, y se *cachea* en disco para garantizar la reproducibilidad y evitar recomputaciones. La elección de esta técnica de Anthropic, y la justificación de tomar a este proveedor como referencia metodológica aplicándola al *modelo local*, se detallan en la Sección 5.9.3.

5.7.4. Indexación a nivel de registro y documentos verificados

Una parte sustancial del valor de un asistente académico reside en datos *tabulares*: horarios por grupo, calendarios, fechas de exámenes y el plan de estudios. Estos documentos plantean dos problemas para una indexación genérica. Primero, la **fragmentación uniforme** trocea una tabla en bloques que mezclan decenas de registros casi idénticos; una consulta por un dato concreto (“¿a qué hora es Cálculo en el grupo 1ºA?”) recupera un bloque donde ese registro queda enterrado y los *embeddings* no distinguen el fragmento correcto. Segundo, la **extracción automática de tablas** desde PDF es frágil y propensa a errores de alineación entre celdas.

El diseño aborda ambos con dos decisiones complementarias:

- **Indexación a nivel de registro.** Para los documentos estructurados se adopta la práctica recomendada en RAG sobre datos tabulares: *un registro = un fragmento*. Cada línea (una clase, un evento de calendario, un examen, una entrada de plan de estudios) se indexa como un **Chunk** autocontenido con *metadatos por campo* (curso, grupo, subgrupo, asignatura, sigla, día, hora, aula, convocatoria, etc.). Esto habilita una recuperación de alta precisión y una trazabilidad campo a campo, y como el registro es autosuficiente, la fase de *Contextual Retrieval* (Sección 5.7.3) lo deja intacto.
- **Documentos verificados.** Para las fuentes cuya extracción automática resulta poco fiable (horarios, calendarios y exámenes en PDF), se introduce una *f fuente verificada*: un documento de texto plano, verificado manualmente contra el PDF oficial, con un registro por línea. Este documento sustituye en el corpus a la extracción automática, pero **conserva la URL del PDF oficial** para la citación, de modo que el sistema sigue remitiendo a la fuente institucional. Entre estos documentos se encuentran el horario del curso, el calendario académico y de TFG, el calendario de exámenes y un documento agregado de plan de estudios. Estos registros incluyen además *registros-resumen* (p. ej. el conjunto de asignaturas de un curso o especialidad) que sustentan las consultas de listado y agregación (Sección 5.8.7).

La inyección de un documento verificado y la reconstrucción del índice se realizan con sendos *scripts* idempotentes, lo que mantiene la propiedad de reproducibilidad del corpus.

5.7.5. Persistencia y ciclo de vida del índice

El índice FAISS y los metadatos asociados se persisten en disco como dos artefactos independientes:

- `index.faiss`: archivo binario generado por `faiss.write_index()`, contiene la matriz de vectores y la estructura del índice.
- `index.meta.json`: archivo textual en formato JSON que contiene una lista ordenada de metadatos, una entrada por fragmento, con el esquema **Chunk** de la Tabla 5.1. La posición de cada entrada en la lista se corresponde con el *id* numérico del vector en FAISS, estableciendo una biyección determinista entre vectores y metadatos.

Esta separación se justifica por tres razones. Primero, FAISS está optimizado para operaciones vectoriales y no ofrece soporte nativo para metadatos estructurados. Segundo, el formato JSON es legible por humanos, depurable y compatible con múltiples herramientas de análisis, lo que facilita la inspección del corpus indexado. Tercero, permite modificar o enriquecer los metadatos sin recomputar el índice vectorial, costoso.

La reindexación completa es un proceso idempotente y reproducible: dados los mismos documentos fuente, el mismo modelo de *embeddings* y la misma configuración de *chunking*, produce un índice byte-a-byte equivalente (salvo por la marca temporal `ingested_at`). Esta propiedad es fundamental para la reproducibilidad experimental.

5.8. Diseño de la capa de recuperación

La capa de recuperación es el primer componente del sistema que se activa ante una consulta del usuario, materializando el **OE4** (*Motor de recuperación de fragmentos relevantes*). Su responsabilidad es seleccionar, del corpus indexado, el subconjunto de fragmentos más relevantes para responderla. Satisface el requisito RF-02 y condiciona críticamente la calidad final: como es bien sabido, *if you can't retrieve it, you can't generate it* [40]. Los errores de la generación son, en su mayoría, errores de la recuperación. La Figura 5.3 resume el flujo completo de esta capa, que se detalla en las subsecciones siguientes.

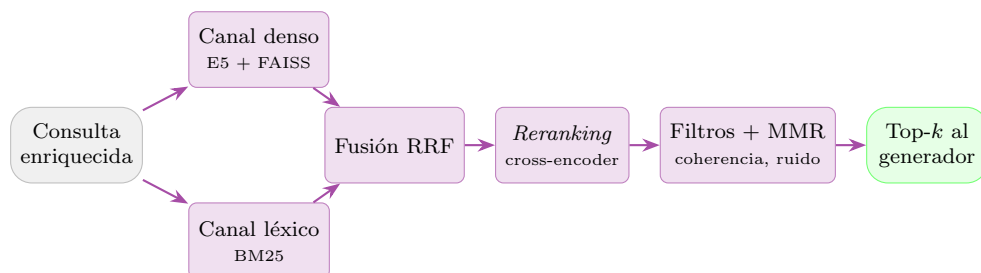


Figura 5.3: Recuperación híbrida. La consulta se busca en paralelo por un canal *denso* (*embeddings* E5 sobre el índice FAISS) y un canal *léxico* (BM25); ambos *rankings* se fusionan mediante *Reciprocal Rank Fusion* (RRF), se reordenan con un *cross-encoder* y se depuran con los filtros de coherencia, la penalización de ruido y MMR antes de entregar los k fragmentos al generador. Imagen propia.

5.8.1. Flujo de ejecución

Ante una consulta q procedente de la capa de presentación, el *Retriever* ejecuta los siguientes pasos:

1. **Codificación:** antepone el prefijo "query: " a la consulta q (requisito del modelo E5 para distinguir consultas de documentos), aplica el *tokenizer* del modelo, ejecuta la inferencia sobre la GPU y obtiene un vector denso $\mathbf{q} \in \mathbb{R}^{768}$, normalizado a norma unitaria para que la similitud coseno se reduzca a un producto escalar.
2. **Búsqueda densa (k-NN):** consulta el índice FAISS mediante `index.search(q, kcand)` con $k_{\text{cand}} = 20$ por defecto. Obtiene los candidatos de mayor producto interno y sus *scores* de similitud $s_i \in [-1, 1]$.
3. **Búsqueda léxica (BM25):** en paralelo al canal denso, tokeniza la consulta q y busca los k_{cand} fragmentos con mayor relevancia léxica en el índice BM25 (véase Sección 5.8.5).
4. **Fusión híbrida (RRF):** combina ambos rankings mediante *Reciprocal Rank Fusion* [93] para obtener un único ranking fusionado que integra señal semántica y léxica.
5. **Filtrado por umbral:** descarta los fragmentos cuyo *score* denso es inferior a un umbral mínimo τ (véase Sección 5.8.3).
6. **Reranking neuronal:** reordena los candidatos supervivientes con un modelo *cross-encoder* que evalúa conjuntamente la relevancia semántica de cada par (*consulta*, *fragmento*) (véase Sección 5.8.10). De

este modo, la decisión final de relevancia la toma un modelo neuronal y no las señales de superficie de la fusión léxico-densa.

7. **Diversificación (MMR):** reordena los candidatos mediante *Maximal Marginal Relevance* (MMR) [47] para penalizar la redundancia entre los fragmentos devueltos (véase Sección 5.8.4).
8. **Recuperación de metadatos:** consulta `index.meta.json` con los *id* seleccionados y construye objetos `RetrievedChunk` con el texto, el *score* y toda la información necesaria para citar la fuente.
9. **Gestión de vacío:** si tras el filtrado no quedan fragmentos, retorna una lista vacía. La capa de generación (Sección 5.9) interpretará este caso como ausencia de evidencia y activará la política de incertidumbre (RF-02).

5.8.2. Elección de k : el número de fragmentos a recuperar

La elección de k involucra un compromiso explícito entre cobertura y ruido:

- *k pequeño* (p. ej. $k = 1$ o $k = 2$): alta precisión de cada fragmento devuelto, pero alto riesgo de no cubrir consultas cuya respuesta requiere combinar información de varios documentos.
- *k grande* (p. ej. $k = 20$): buena cobertura, pero el *prompt* del LLM se infla con fragmentos marginalmente relevantes que diluyen la atención y pueden inducir alucinaciones por ruido.

Se adopta $k = 5$ como valor por defecto, justificado por tres razones. Primero, estudios empíricos en RAG sobre dominios técnicos similares reportan un óptimo entre 3 y 7 fragmentos [40]. Segundo, con un tamaño medio de fragmento de 512 tokens y un contexto efectivo del LLM de 8192 tokens, cinco fragmentos ocupan ≈ 2560 tokens, dejando holgura suficiente para la *pregunta*, la *instrucción* y la *respuesta*. Tercero, el Capítulo 7 incluye un análisis de sensibilidad sobre k que valida empíricamente esta elección para el corpus ETSIT.

5.8.3. Umbral de similitud

El filtrado por umbral descarta los fragmentos cuya similitud con la consulta es demasiado baja para aportar evidencia útil. Contribuye así a la *gestión de la incertidumbre*, el flujo alternativo 4a del caso de uso CU-01, en el que el sistema se abstiene cuando el corpus no contiene información

fiable, y, de forma indirecta, a la reducción de alucinaciones inducidas por ruido.

El umbral τ se establece, por defecto, en **0,35** para la similitud del coseno. Un valor más alto (0,50) descartaba documentos legítimos de baja coincidencia léxica con la consulta, normativa y movilidad, redactados con vocabulario distinto al de la pregunta, por lo que se relajó tras el análisis del Capítulo 7. En el modo híbrido (Sección 5.8.5) se aplica un umbral efectivo reducido ($\tau \cdot 0,7$) a los candidatos rescatados por BM25, cuya presencia en el *ranking* fusionado ya es indicio de relevancia léxica. La escala de los *scores* del coseno no es universal: modelos de *embeddings* distintos producen distribuciones de similitud con rangos y medias distintos; por tanto, el umbral debe recalibrarse si se sustituye el modelo.

Conviene precisar una limitación importante, validada empíricamente en el Capítulo 7: **el umbral por sí solo no basta para rechazar las consultas fuera de dominio**. Sobre un corpus tan homogéneo como el de la ETSIT, el modelo E5 produce similitudes comprimidas y relativamente altas incluso para preguntas ajenas al ámbito académico (p. ej. “¿cuál es la capital de Francia?”), de modo que algún fragmento supera el umbral y, si se propaga al LLM, induce una respuesta paramétrica con una cita espuria. El rechazo fiable de las consultas fuera de ámbito (flujo alternativo 4b del CU-01) se delega, por ello, en un componente específico, el *gate* de ámbito, descrito en la Sección 5.8.11.

5.8.4. Diversificación mediante *Maximal Marginal Relevance*

La búsqueda k-NN pura tiene un defecto conocido: tiende a devolver fragmentos muy similares entre sí, especialmente cuando el corpus contiene información redundante (por ejemplo, la misma normativa citada en varios documentos). Esta redundancia desperdicia espacio del *prompt* sin añadir información nueva.

MMR [47] aborda este problema reordenando iterativamente los candidatos según la fórmula:

$$\text{MMR} = \arg \max_{d_i \in D \setminus S} \left[\lambda \cdot \text{sim}(q, d_i) - (1 - \lambda) \cdot \max_{d_j \in S} \text{sim}(d_i, d_j) \right],$$

donde S es el conjunto ya seleccionado, D el de candidatos, y $\lambda \in [0, 1]$ un hiperparámetro que controla el balance entre relevancia y diversidad. Se adopta $\lambda = 0,7$ por defecto, siguiendo la recomendación empírica de [47], y se recuperan inicialmente $4k$ candidatos (20 por defecto cuando $k = 5$) para reordenarlos y devolver k .

El comportamiento de MMR se gobierna con el parámetro `mmr_lambda` del fichero de configuración: con $\lambda = 1$ se obtiene relevancia pura (equivalente a desactivar la diversificación) y con $\lambda = 0$, diversidad pura. La etapa introduce una sobrecarga computacional ($O(k \cdot \text{fetch_k})$ comparaciones adicionales) modesta frente al coste de la generación. Se mantiene siempre activa porque el corpus de la ETSIIT sí presenta redundancia (la misma normativa o el mismo dato reaparecen en varios documentos), pero su intensidad es ajustable mediante λ .

5.8.5. Recuperación híbrida: canal denso + canal léxico (BM25)

Los modelos de *embeddings* densos como E5 capturan relaciones semánticas profundas, pero presentan una limitación conocida frente a coincidencias léxicas exactas: nombres propios, códigos de asignatura (p. ej. IC, FFT), siglas institucionales (ETSIIT, CRUE) y números de referencia tienden a diluirse en el espacio vectorial, pues el modelo los proyecta a una región semántica más amplia [42]. Esto implica que una consulta como “normativa del TFG” puede recuperar fragmentos semánticamente relacionados (p. ej. sobre trabajos académicos en general) pero no exactamente los que contienen la cadena TFG.

Para mitigar esta limitación, GRAIA implementa una **estrategia de recuperación híbrida** que combina dos canales complementarios:

- **Canal denso (FAISS + E5):** búsqueda por similitud semántica en el espacio de *embeddings*, tal como se describe en las secciones precedentes.
- **Canal léxico (BM25):** búsqueda por coincidencia de términos mediante el algoritmo BM25 Okapi [43], que puntúa los fragmentos en función de la frecuencia de los términos de la consulta, ponderada por la frecuencia inversa de documento y la longitud del fragmento.

Justificación de BM25 como canal léxico

Se elige BM25 frente a alternativas como TF-IDF o modelos dispersos aprendidos (*learned sparse models*, p. ej. SPLADE [94]) por tres razones:

1. **Eficacia demostrada:** BM25 sigue siendo un *baseline* competitivo en recuperación de información, superando a métodos densos en consultas con términos específicos [95].
2. **Simplicidad y reproducibilidad:** no requiere entrenamiento ni GPU; la tokenización es puramente léxica (lowercase + split alfanumérico),

eliminando dependencias adicionales y facilitando la reproducibilidad del sistema.

3. **Complementariedad:** la literatura sobre recuperación híbrida demuestra que la combinación de señales densas y léxicas supera consistentemente a cada canal por separado, especialmente en dominios especializados con terminología propia. Karpukhin et al. [42] documentan este efecto en *Open-Domain QA*, y Ma et al. [96] confirman que un esquema híbrido sencillo (sin reentrenamiento) iguala o supera a modelos densos puros.

Fusión mediante Reciprocal Rank Fusion (RRF)

La fusión de los rankings denso y léxico se realiza mediante *Reciprocal Rank Fusion* (RRF) [93], un método que combina rankings heterogéneos sin requerir normalización previa de los *scores* (requisito crítico, dado que los *scores* de similitud del coseno y los *scores* BM25 operan en escalas incomparables). La fórmula RRF asigna a cada documento d un *score* fusionado:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k_{\text{rrf}} + \text{rank}_r(d)},$$

donde R es el conjunto de rankings (denso y léxico), $\text{rank}_r(d)$ es la posición 1-indexada de d en el ranking r , y k_{rrf} es una constante de suavizado que modera la influencia de las posiciones extremas. Se adopta $k_{\text{rrf}} = 60$, valor propuesto por Cormack *et al.* [93] en el artículo original de RRF y ampliamente adoptado como estándar en la literatura.

Parámetros de configuración

El modo híbrido se controla mediante dos parámetros en el fichero `default.yaml`:

- `use_hybrid`: booleano que activa o desactiva la recuperación híbrida. Por defecto `true`.
- `rrf_k`: constante de suavizado RRF. Por defecto 60.

Cuando `use_hybrid` está desactivado, el sistema opera con un único canal denso, siendo completamente compatible hacia atrás. El índice BM25 se construye junto al índice FAISS durante la fase de indexación (véase Sección 5.7) y se persiste como fichero `bm25_index.pkl` en el mismo directorio.

5.8.6. Enrutamiento por categoría e inyección focalizada

El corpus de la ETSIIT es notablemente *homogéneo* desde el punto de vista semántico: centenares de guías docentes comparten estructura y vocabulario, y conviven con calendarios, horarios y normativas que reutilizan la misma terminología académica. En estas condiciones, la búsqueda densa pura tiende a *enterrar* documentos minoritarios pero decisivos, durante el desarrollo se observó que el Plan de Estudios podía caer hasta el puesto ~ 240 del *ranking* denso, fuera de cualquier ventana razonable de candidatos. Si el fragmento correcto no entra siquiera en el conjunto de candidatos, ninguna etapa posterior (umbral, reranking, MMR) puede rescatarlo: es un problema de *recall*, no de ordenación.

Para garantizar el *recall* se introduce un **enrutador de consultas por categoría** seguido de una **inyección focalizada**:

- **Enrutador (query_router)**. Clasifica la consulta en una o varias de las categorías del corpus (plan de estudios, horarios, calendario, TFG, trámites, movilidad, normativa, etc.) mediante un conjunto de reglas de palabras clave y patrones. Se opta por un clasificador *basado en reglas* frente a uno neuronal por tres razones alineadas con los principios del sistema: *determinismo* (misma consulta $\rightarrow$ misma categoría, lo que facilita la depuración y la reproducibilidad), *latencia nula* (no requiere inferencia adicional) e *interpretabilidad* (las reglas son auditable, frente a la opacidad de un clasificador entrenado que además exigiría datos supervisados del dominio de los que no se dispone).
- **Inyección focalizada**. Para cada categoría predicha, el recuperador recorre los fragmentos de esa categoría, calcula su *score* denso respecto a la consulta y añade los mejores al conjunto de candidatos, con independencia de que la búsqueda densa global los hubiera situado fuera de la ventana inicial. Esto garantiza que, por ejemplo, los registros del Plan de Estudios estén siempre disponibles cuando la consulta versa sobre asignaturas de un curso.

Una decisión de diseño relevante, reforzada tras la incorporación del reranking (Sección 5.8.10), es la **separación entre *recall* y *ranking***: el enrutador y la inyección se ocupan exclusivamente de *qué fragmentos llegan a ser candidatos* (*recall*), mientras que la *ordenación final* la decide el *cross-encoder* por relevancia semántica. Los multiplicadores de *boost* por categoría quedan así relegados a un papel de apoyo al *recall* y dejan de gobernar la respuesta, lo que evita que la formulación superficial de la pregunta (la presencia de una u otra palabra clave) determine el resultado.

5.8.7. Consultas de listado y tamaño de contexto dinámico

El valor $k = 5$ (Sección 5.8.2) está dimensionado para consultas *puntuales*, en las que la respuesta cabe en unos pocos fragmentos. Sin embargo, las consultas de *listado* o *agregación*, “¿qué asignaturas hay en tercero?”, “el horario de Cálculo en todos los grupos”, requieren combinar muchos registros para ser completas; con solo cinco fragmentos la respuesta queda truncada y se vuelve sensible a la formulación.

Para abordarlo se añade un **detector de intención de listado** (`query_intent`), también basado en reglas, que reconoce los patrones característicos de estas consultas (“cuáles son”, “lista de”, “todas las asignaturas de”, “de todos los grupos”, etc.). Cuando se detecta intención de listado, el recuperador eleva el tamaño de contexto de k a un valor ampliado k_{listado} (10 por defecto), de modo que la ampliación solo penaliza la latencia en las consultas que realmente la necesitan, preservando la rapidez de las consultas puntuales. Adicionalmente, para los listados de asignaturas se fuerza la inyección de los *registros-resumen* pertinentes (véase la indexación a nivel de registro, Sección 5.7), garantizando que el dato agregado por curso o especialidad esté presente en el contexto.

5.8.8. Recuperación consciente del historial

En una conversación, los seguimientos suelen ser *elípticos*: “¿y las de Cálculo?”, “me refiero al subgrupo A2”. La *recuperación* opera, por defecto, solo sobre la consulta actual: si esa consulta es ambigua, el recuperador trae fragmentos de otra asignatura o grupo y el modelo no recibe el fragmento correcto. Confiar la resolución de la anáfora al historial conversacional no basta, el fragmento correcto ni siquiera llega al contexto, y, además, es frágil: inyectar el historial en bruto en un LLM pequeño hace que un turno previo sobre otro tipo de dato (p. ej. un horario antes de una pregunta de exámenes) pueda inducir una abstención errónea.

Para resolverlo se introduce una etapa de **enriquecimiento de la consulta consciente del historial** (`contextual_query`): cuando la consulta actual es un seguimiento elíptico, se arrastran de los turnos de usuario previos las *entidades del dominio* que falten, asignatura (reconocida tanto por su sigla como por su nombre completo), grupo, subgrupo y tipo de clase, y se incorporan a la consulta *ya resuelta*, que se emplea tanto en la *recuperación* como en la *generación*. Así la anáfora se resuelve de forma *determinista de extremo a extremo* y la generación no depende del historial conversacional en bruto, lo que evita que el contexto de un turno previo de otro tipo de dato desestabilice la respuesta. Se opta por un **acarreo determinista de entidades** frente a una reescritura de la consulta mediante un LLM por coherencia con la decisión de *pipeline* de pasada única (Sección 5.2): el acarreo

determinista tiene latencia nula, es interpretable y se ajusta a un dominio fuertemente estructurado (asignatura $\times$ grupo $\times$ subgrupo $\times$ tipo), mientras que una reescritura con LLM añadiría una llamada de inferencia y un punto de fallo adicional.

5.8.9. Estructura de datos de salida: RetrievedChunk

La salida del `Retriever` no es un simple texto, sino un objeto estructurado que viaja intacto hasta la capa de presentación. El esquema se resume en la Tabla 5.2.

Campo	Tipo	Descripción
<code>chunk_id</code>	<code>string</code>	Identificador del fragmento, heredado de la ingesta.
<code>text</code>	<code>string</code>	Contenido textual recuperado.
<code>source_url</code>	<code>string</code>	URL del documento original (para la cita).
<code>source_type</code>	<code>string</code>	Tipo de fuente (<code>pdf</code> , <code>html</code>).
<code>title</code>	<code>string</code>	Título legible del documento.
<code>position</code>	<code>int</code>	Posición del fragmento dentro de su documento (preserva el orden).
<code>similarity</code>	<code>float</code>	<i>Score</i> de relevancia: similitud del coseno o, si actúa el <i>reranker</i> , puntuación del <i>cross-encoder</i> .
<code>rank</code>	<code>int</code>	Posición final tras la diversificación MMR.
<code>metadata</code>	<code>dict</code>	Metadatos adicionales del registro (categoría, <code>tipo</code> , <code>siglas</code> , curso...), empleados por los filtros de coherencia y el control de ruido.

Cuadro 5.2: Esquema del objeto `RetrievedChunk` devuelto por la capa de recuperación.

La preservación de `source_url` y `title` desde la ingesta hasta la recuperación es la base técnica del sistema de citas con trazabilidad completa, detallado en la Sección 5.10 de la siguiente parte de este capítulo.

5.8.10. Reranking neuronal con *cross-encoder*

La recuperación híbrida descrita en las secciones anteriores ordena los fragmentos mediante señales de *superficie*: la similitud del coseno (un *bi-encoder* que codifica consulta y documento por separado) y la coincidencia léxica de BM25. Estas señales son rápidas y buenas para el *recall*, pero su precisión de ordenación es limitada, especialmente en un corpus tan homogéneo como el de la ETSIT, donde numerosos fragmentos comparten vocabulario y estructura (horarios por grupo, guías docentes, calendarios). En ese escenario, una consulta como “¿en qué aula son las clases de Derecho Informático?” puede situar por delante un fragmento del *examen* de

esa asignatura, que comparte casi todo el vocabulario con el del *horario* buscado.

Para resolver la *precisión* de la ordenación se incorpora una etapa de *reranking* mediante un modelo *cross-encoder* de la familia **ms-marco** [46]. A diferencia del *bi-encoder*, un *cross-encoder* procesa el par (**consulta**, **fragmento**) *conjuntamente* en una única pasada del *transformer*, capturando las interacciones cruzadas entre los *tokens* de ambos y produciendo una puntuación de relevancia mucho más fina que el coseno. El *reranker* se aplica sobre los candidatos que han superado el filtro por umbral, justo *antes* del MMR, de modo que la diversificación opere ya sobre un *pool* reordenado por relevancia precisa.

Modelo seleccionado. Se emplea **cross-encoder/ms-marco-MiniLM-L-6-v2**: 22 M de parámetros (menos de 100 MB), entrenado sobre MS MARCO (>500 K pares consulta-pasaje) y con una latencia del orden de 5 ms por par en la RTX 4070 Super. Su tamaño reducido neutraliza las dos objeciones que tradicionalmente desaconsejan el *reranking* en hardware de consumo: la presión sobre la VRAM es marginal frente a la del LLM, y la sobrecarga de latencia (decenas de milisegundos para los k_{cand} candidatos) es holgadamente compatible con el RNF-01 (< 5 s por respuesta).

Papel en la arquitectura. La incorporación del *reranker* se apoya en el diseño modular (patrones *Pipe and Filter* y *Strategy*, Sección 5.5): se inserta como un filtro adicional entre el **Retriever** y el **Generator** sin alterar el resto de la cadena, y se controla con el parámetro `use_reranker` de `default.yaml`. Una consecuencia de diseño relevante es que, al decidir el *cross-encoder* la ordenación final por relevancia semántica, las señales de enrutamiento por categoría (Sección 5.8) quedan relegadas a un papel de *recall*, garantizar que el fragmento correcto figure entre los candidatos, y dejan de gobernar el *ranking*, lo que aporta robustez frente a la formulación concreta de la pregunta. La validación empírica de la mejora de precisión que aporta esta etapa se presenta en el Capítulo 7.

5.8.11. Gate de ámbito: rechazo de consultas fuera de dominio

Como se anticipó en la Sección 5.8.3, el umbral de similitud no separa de forma fiable las consultas *dentro* de las consultas *fuera* de dominio en un corpus tan homogéneo. Para gobernar el flujo alternativo 4b del CU-01, el rechazo cortés de consultas ajenas al ámbito académico, se introduce un **gate de ámbito** que decide, *antes* de generar, si la consulta merece una respuesta o una abstención. El *ámbito* fuera de dominio es *abierto* (geografía, deportes,

ocio, cálculos sueltos...) y, a diferencia de las categorías del corpus, no es enumerable con reglas; por ello el *gate* no se basa en patrones léxicos sino en dos señales complementarias, ordenadas por coste:

1. **Margen del *cross-encoder* (coste cero).** El *score* de relevancia del *reranker* (Sección 5.8.10), ya calculado en el *pipeline*, resulta ser una señal de dominio sorprendentemente limpia: se dispara a valores altos cuando algún fragmento responde realmente a la pregunta y colapsa a valores negativos cuando ninguno lo hace. La calibración empírica (Capítulo 7) muestra una separación nítida, del orden de +5,7 para consultas en ámbito frente a -2,7 para consultas ajenas. Por encima de un margen alto la consulta se acepta sin coste adicional; por debajo de un margen bajo se rechaza directamente. El *reranker* cumple así una doble función: precisión de ordenación y detección de dominio.
2. **Clasificador LLM (red de seguridad).** Solo en la banda intermedia, donde el margen no es concluyente, se consulta al propio LLM local con una instrucción breve de clasificación binaria (ACADÉMICO / FUERA) y decodificación voraz. Esta segunda señal generaliza a temas no vistos al precio de una única inferencia corta, y únicamente cuando hace falta.

Quedan *exentas* del *gate*, por reglas deterministas, las preguntas sobre el propio asistente (identidad, capacidades) y los seguimientos anafóricos (“¿y en septiembre?”), que de otro modo se confundirían con consultas fuera de ámbito y romperían la conversación. Cuando la consulta se determina fuera de dominio, el sistema emite directamente la frase canónica de abstención *sin* invocar al generador, evitando de raíz la respuesta paramétrica y la cita espuria. La política ante el error del clasificador es *fail-open* (ante la duda, responder), pues un escape ocasional es preferible a rechazar por error una consulta académica legítima.

El enrutador como salvaguarda contra falsos rechazos. El auto-rechazo por margen bajo (señal 1) entraña un riesgo: *algunas consultas legítimas obtienen márgenes negativos* porque el *cross-encoder* infravalora su fragmento. Es el caso de preguntas cortas y coloquiales como “¿cuándo abre la secretaría?”, cuyo registro de horario (“de 9 a 14 horas, de lunes a viernes”) el *reranker* puntúa por debajo de fragmentos de calendario que comparten el adverbio temporal. Para no rechazarlas por error se añade una salvaguarda: el auto-rechazo por margen solo se aplica si el **enrutador** (Sección 5.8.6) *no* ha clasificado la consulta en ninguna categoría académica. El enrutador, basado en reglas, es una señal de dominio fuerte y determinista; si *enruta* la consulta, esta se considera del dominio y se difiere al clasificador LLM en lugar de rechazarla de plano. Así, “¿cuándo abre la secretaría?”

(enrutada a *trámites*) supera el *gate* y se responde, mientras que “¿cuál es la capital de Francia?” (que no enruta) se sigue rechazando por su margen. El sobrecoste de latencia es marginal, pues el clasificador LLM solo se invoca en las pocas consultas *enrutadas con margen bajo*.

5.8.12. Filtros de coherencia: asignatura y tipo de dato

La separación *recall/ranking* (Sección 5.8.10) garantiza que el fragmento correcto figure entre los candidatos, pero en un corpus donde muchos registros comparten estructura y vocabulario el *reranker* puede dejar entre los seleccionados fragmentos *adyacentes* que el LLM tiende a fusionar o a reetiquetar. Se añaden dos filtros deterministas, baratos e interpretables, que recortan ese ruido aprovechando los metadatos a nivel de registro (Sección 5.7):

- **Coherencia de asignatura.** Si la consulta nombra una asignatura concreta (por su sigla o su nombre completo), se descartan los fragmentos, de horario, calendario o guía docente, que pertenecen a *otra* asignatura. Evita errores como presentar el horario de otra asignatura como un cuatrimestre de la preguntada, o citar la guía docente de una asignatura distinta. Los catálogos multi-asignatura (plan de estudios, registros-resumen) quedan exentos.
- **Coherencia de tipo de dato.** El horario de clase y el calendario de exámenes comparten casi todo el vocabulario y se desplazan mutuamente en el *ranking*. Cuando la consulta pide inequívocamente uno de los dos tipos (“horario” / “a qué hora” frente a “examen” / “convocatoria”), se descartan los registros del tipo en conflicto. Si la intención es ambigua (menciona ambos o ninguno), no se filtra, para no provocar falsos rechazos.

Ambos filtros operan sobre el conjunto de candidatos antes de la ordenación final y solo afectan a registros estructurados con metadatos identificables; el resto del contexto permanece intacto. Su efecto se cuantifica en el Capítulo 7.

5.8.13. Control de ruido: penalización por categoría no enrutada

El enrutador por categoría (Sección 5.8.6) *favorece* las categorías pertinentes, pero no *penaliza* las ajenas; en consecuencia, documentos administrativos extensos (boletines oficiales, reglamentos, impresos) pueden colarse

entre los fragmentos finales de una consulta que el enrutador ya ha clasificado con confianza. Para gobernar ese ruido se introduce una **penalización por categoría no enrutada**: cuando el enrutador asigna una o varias categorías, los fragmentos cuya categoría queda fuera de ese conjunto reciben una resta acotada sobre su *score* de *reranking*, que los *demota* en la selección final. La penalización *no elimina* esos fragmentos, si fueran los únicos disponibles, sobreviven, por lo que no introduce falsos rechazos; simplemente impide que el ruido administrativo desplace a la fuente correcta. El valor de la penalización es configurable y se ha fijado de forma conservadora; su efecto se valida en el Capítulo 7.

5.9. Diseño de la capa de generación

La capa de generación es la responsable de sintetizar una respuesta en lenguaje natural a partir de la pregunta del usuario y del conjunto de fragmentos recuperados en la capa de recuperación (Sección 5.8), materializando el **OE5** (*Integración de LLM local con contexto recuperado*). Su diseño determina, junto con la calidad del *retrieval*, la utilidad percibida del sistema por el usuario final: una recuperación impecable puede ser desaprovechada por un *prompt* mal construido o por un modelo que ignora las fuentes y alucina.

5.9.1. Elección del modelo de lenguaje

La primera decisión de diseño consiste en seleccionar el modelo de lenguaje que realizará la generación. El espacio de opciones se articula en torno a dos ejes principales: *local frente a remoto* y *tamaño del modelo*.

Local frente a remoto

La alternativa remota (API de OpenAI, Anthropic, Google) ofrece modelos de frontera (GPT-4, Claude, Gemini) con calidad superior. Sin embargo, se descarta por cuatro razones fundamentales, alineadas con los requisitos no funcionales del Capítulo 4:

- **Soberanía del dato (RNF-08, RNF-09)**: las preguntas de los estudiantes pueden contener datos personales o académicamente sensibles (consultas sobre TFM, situaciones de matriculación particulares) que no deben salir del perímetro institucional sin una base jurídica adecuada conforme al RGPD.
- **Coste operativo recurrente**: una API comercial implica un coste por *token* que escala con el uso, incompatible con el presupuesto de un

proyecto fin de grado y con el objetivo de desplegar el sistema como servicio abierto a la comunidad ETSIT.

- **Reproducibilidad científica:** los modelos remotos evolucionan de forma opaca y sus versiones pueden retirarse o modificarse sin aviso, lo que invalidaría los resultados experimentales del Capítulo 7 a medio plazo.
- **Independencia de red:** el sistema debe funcionar aun cuando el acceso a servicios externos esté restringido, lo que refuerza el enfoque *local-first*.

Se adopta, por tanto, un modelo **local** ejecutado en el mismo *host* que el resto del sistema, aceptando conscientemente la penalización en calidad respecto a los modelos de frontera.

Tamaño y cuantización del modelo

El *hardware* objetivo es una GPU NVIDIA RTX 4070 Super con 12 GB de VRAM. Este presupuesto de memoria acota severamente el espacio de modelos ejecutables en tiempo real; la comparativa detallada de candidatos se presenta en la Tab. 3.2 del Capítulo 3.

Se adopta **Llama 3.1 8B Instruct** en cuantización Q4_K_M como candidato principal para el desarrollo y las pruebas, si bien la comparativa definitiva con Gemma 2 9B y Qwen 2.5 14B se realiza en el Capítulo 7, donde se confirma que Llama 3.1 8B es el único de los tres que satisface simultáneamente calidad, abstención y latencia. Los motivos de esta elección preliminar son tres:

1. **Competencia multilingüe:** la familia Llama 3 se entrena sobre corpus multilingüe significativo, ofreciendo un rendimiento razonable en español sin necesidad de *fine-tuning* [88].
2. **Ecosistema maduro:** la disponibilidad de cuantizaciones optimizadas en el formato GGUF, la existencia de plantillas de *chat* bien documentadas y la integración con Ollama reducen drásticamente la fricción de integración.
3. **Licencia permisiva:** la licencia comunitaria de Llama permite el uso para investigación y servicios académicos no comerciales sin restricciones relevantes para este proyecto.

La decisión de modelo no es, en rigor, *monolítica*: el diseño de la capa de generación se articula mediante el patrón *Strategy* (Sección 5.5), de modo

que sustituir Llama 3 por Qwen 2.5 o Gemma 2 requiere únicamente cambiar una entrada de configuración. Esta flexibilidad permite, además, contrastar modelos en la fase experimental (Capítulo 7).

Justificación de la cuantización Q4_K_M

La cuantización a 4 bits por peso reduce el tamaño del modelo en un factor aproximado de $8\times$ respecto al *checkpoint* en precisión FP16, a costa de una degradación modesta en calidad. Según los *benchmarks* publicados por la comunidad `llama.cpp`, la variante Q4_K_M preserva aproximadamente el 97% de la perplejidad del modelo original sobre *wiki.test*, lo que representa el mejor compromiso conocido entre fidelidad y ocupación de memoria. Se descartan cuantizaciones más agresivas (Q2, Q3) por la degradación observable en razonamiento en cadena larga, y cuantizaciones conservadoras (Q8) por innecesarias dado el presupuesto de VRAM disponible.

5.9.2. Motor de inferencia: Ollama

La ejecución del modelo se delega en Ollama (Sección 5.3.2), cuya justificación frente a alternativas ya se presentó en la Sección 5.3.2. A las ventajas generales allí expuestas se añade, en el contexto específico de la generación, la abstracción de la plantilla de *chat* del modelo (evitando errores manuales en *tokens* especiales como `<|start_header_id|>`) y el soporte nativo para *streaming* de *tokens*, esencial para la experiencia de usuario (RNF-01, RNF-06).

5.9.3. Diseño del *prompt*

El *prompt* es, en ausencia de un *fine-tuning* supervisado, el principal mecanismo para dirigir el comportamiento del LLM. Su diseño influye críticamente en la fidelidad de la respuesta a las fuentes recuperadas (*faithfulness*) y en la calidad del sistema de citación.

Estructura del *prompt*

Se adopta una plantilla de tres bloques, inspirada en las recomendaciones de [39]:

1. **Instrucción de sistema (*system prompt*)**: define el rol del asistente (“asistente académico de la ETSIT”), las restricciones de comportamiento (responder solo con información del contexto, admitir desco-

nocimiento, citar siempre) y el formato esperado (Markdown con citas numéricas).

2. **Contexto recuperado:** los k fragmentos seleccionados por la capa de recuperación, delimitados entre etiquetas `<contexto>` y, cada uno, encapsulado en un bloque `<fragmento>` con su etiqueta numérica [1], [2], ..., que el modelo debe reutilizar para citar, su título y su URL.
3. **Pregunta del usuario:** la consulta original, sin modificaciones, delimitada entre etiquetas `<pregunta>`.

Delimitación estructurada mediante etiquetas XML. Una decisión de diseño del *prompt* es delimitar el contexto, cada fragmento individual y la pregunta mediante etiquetas tipo XML (`<contexto>`, `<fragmento>`, `<pregunta>`) en lugar de simples encabezados textuales. Esta práctica, recogida en las guías de ingeniería de *prompts* de Anthropic [97], reduce la ambigüedad entre *instrucciones*, *datos* y *pregunta*: el modelo distingue sin equívocos dónde empieza y acaba cada fuente, lo que disminuye el riesgo de que confunda el contenido de un fragmento con una instrucción o de que funda dos fragmentos contiguos en uno. Se toman como referencia metodológica las técnicas publicadas por Anthropic por tres razones objetivas y *no* por preferencia de proveedor: (i) Anthropic documenta de forma pública, detallada y reproducible sus técnicas de *prompting* y de recuperación (la propia *Contextual Retrieval* empleada en la ingesta, Sección 5.7, procede de ese mismo origen [92]); (ii) su línea de investigación prioriza explícitamente la *fidelidad a las fuentes* y la reducción de alucinaciones, que son precisamente los requisitos no funcionales centrales de GRAIA; y (iii) se trata de técnicas *agnósticas al modelo*, aplicables al LLM local del sistema sin comprometer la ejecución íntegramente local (Sección 5.9.1). Es importante subrayar que se adopta la *técnica*, no el modelo: GRAIA sigue ejecutándose sobre un LLM abierto y local.

Aprendizaje en contexto con ejemplos (*multishot*). Siguiendo otra recomendación de las guías de Anthropic [97], el *prompt* incorpora ejemplos resueltos (*few-shot/multishot*) que fijan el formato y el comportamiento esperados en los casos más propensos a error: una respuesta de listado completo y, sobre todo, un listado *por grupos* que ilustra cómo presentar cada grupo en su propia línea *sin fusionarlos*. Los ejemplos se delimitan, a su vez, con etiquetas `<ejemplo>` dentro de un bloque `<ejemplos>`, de modo que el modelo los distinga inequívocamente de las instrucciones. Este recurso es especialmente valioso con un LLM de 8 B, en el que un ejemplo concreto corrige comportamientos que una instrucción abstracta no logra fijar de forma fiable.

Técnicas deliberadamente descartadas. No todas las técnicas avanzadas de *prompting* son compatibles con las restricciones del sistema. En particular, se descartan el *razonamiento explícito en cadena* (*chain-of-thought*) y el *encadenamiento de prompts* (*prompt chaining*): ambos multiplican el número de *tokens* generados o de llamadas de inferencia, lo que entra en conflicto directo con el RNF-01 (< 5 s) y con la decisión arquitectónica de *pipeline* de pasada única (Sección 5.2). Su adopción, por ejemplo, un *borrador de razonamiento* oculto que se descarte antes de mostrar la respuesta, se contempla como línea de trabajo futuro condicionada a una mejora del presupuesto de latencia. Se aplican, por tanto, únicamente las técnicas de coste *constante* en latencia (estructura XML, persona/rol, ejemplos *multishot* y la *Contextual Retrieval* de la fase de indexación), reservando las de coste *variable* para una posible evolución del sistema.

Instrucciones clave del *system prompt*

El *system prompt* reúne un conjunto de directrices operativas que el desarrollo iterativo (Capítulo 6) fue refinando a partir de los fallos observados y cuya efectividad se valida experimentalmente en el Capítulo 7. Pueden agruparse en cuatro familias:

- **Anclaje a las fuentes y abstención.** “*Responde exclusivamente con información presente en el contexto*” reduce la alucinación; “*si el contexto no contiene información suficiente, indícalo explícitamente*” habilita el comportamiento de *abstención* (RF-02), fundamental para la confiabilidad.
- **Citación.** “*Cita las fuentes con la notación [n]*”, acoplando la generación con el sistema de citación (Sección 5.10). Las siglas vienen ya expandidas en el contexto y no deben adivinarse.
- **Respuesta mínima suficiente y no fusión.** Se instruye al modelo a responder *solo* a lo que se pregunta y nada más. En particular, la palabra “horario” se refiere al horario de *clase* (no a exámenes ni tutorías, salvo que se pidan expresamente), y “completo” significa cubrir todas las partes *de lo preguntado*, p. ej. ambos cuatrimestres de una misma asignatura o todos los grupos solicitados, y *no* vaciar el contexto con información colateral. Se prohíbe explícitamente (i) *reetiquetar* un fragmento de otra asignatura, grupo o tipo de dato como si fuera el preguntado; (ii) *repetir* un dato idéntico para cada grupo, debiendo indicarse una sola vez como común; y (iii) *fusionar* varias filas (grupo, subgrupo, curso, especialidad) en una sola entrada o mezclar sus datos, fallos recurrentes del modelo de 8 B al agregar registros parecidos. Esta acotación, complementaria de los filtros de coherencia de

la recuperación (Sección 5.8.12), corrige el volcado de información no solicitada y la contaminación cruzada observados durante el desarrollo y validados en el Capítulo 7.

- **Persona y registro.** El asistente adopta la persona de un *secretario académico amable*, con trato de usted, cordial pero preciso; y en preguntas *subjetivas* (“¿qué especialidad es mejor?”) declina dar una opinión personal y, en su lugar, ofrece información objetiva que ayude a decidir.

Conviene subrayar una decisión de diseño transversal: el *system prompt* es el principal, pero no el único, mecanismo de control del comportamiento. Como un LLM local de 8 B no siempre obedece, las directrices anteriores se respaldan con el post-procesado determinista de la Sección 5.10.4, que corrige de forma fiable lo que el *prompt* no garantiza.

Tratamiento del contexto largo

Con $k = 5$ fragmentos de hasta 512 *tokens*, más el *system prompt* (aproximadamente 300 *tokens*) y la pregunta, el *prompt* total puede alcanzar unos 3000 *tokens*. Esta cifra está cómodamente dentro de la ventana de contexto de 128 000 *tokens* de Llama 3.1, por lo que no se requieren estrategias de *chunk merging* o resumen intermedio. Sin embargo, se tiene en cuenta el fenómeno *lost-in-the-middle* [39]: la atención del modelo tiende a degradarse en las posiciones centrales del contexto. Para mitigarlo, los fragmentos se ordenan según su *score* de relevancia, colocando los más relevantes al principio y al final del bloque de contexto, y los menos relevantes en el medio.

5.9.4. Parámetros de decodificación

Los parámetros por defecto, validados según el Capítulo 7, son:

- **Temperatura $T = 0,0$** (decodificación *greedy*): para un asistente *factual* se prioriza la máxima fidelidad a las fuentes y la reproducibilidad sobre la creatividad. Con $T = 0$ el modelo elige en cada paso el *token* más probable, lo que (i) hace la salida determinista, reforzando la reproducibilidad, pues una misma pregunta sobre un mismo índice produce siempre la misma respuesta, y (ii) minimiza la “creatividad” que, en consultas de agregación sobre muchos fragmentos parecidos, inducía al modelo a fusionar o inventar datos. Se descarta una temperatura moderada ($T = 0,2$) por innecesaria en una tarea donde no se busca variabilidad estilística.

- **Top-p** = 0,9: se conserva el muestreo nucleado por compatibilidad, si bien es irrelevante con $T = 0$ (la decodificación *greedy* no muestrea, sino que selecciona el máximo).
- **Máximo de *tokens* generados** = 1536: se amplía respecto al valor inicial de 1024 porque una respuesta de listado completo, por ejemplo, el horario de los seis grupos de un curso con teoría y prácticas, o el catálogo de asignaturas de un curso, puede superar los 1024 *tokens* y quedar truncada a media respuesta. El valor de 1536 ofrece margen para estos casos sin penalizar de forma apreciable la latencia, y sigue siendo configurable.

5.10. Diseño del sistema de citación

El sistema de citación materializa el **OE6** (*Sistema de citación trazable*) y el requisito RF-03 (trazabilidad documental), constituyendo una de las contribuciones distintivas de este trabajo frente a sistemas conversacionales genéricos. Su diseño debe garantizar que cada afirmación de la respuesta sea vinculable a un fragmento concreto del corpus, permitiendo al usuario verificar la información antes de aceptarla.

5.10.1. Problema y alternativas de diseño

Existen, en la literatura sobre *attributed QA* [48], tres familias de estrategias para inducir citas en la respuesta generada:

1. **Citación *in-context***: se instruye al LLM, vía *prompt*, para que inserte marcadores tipo [n] que referencian los fragmentos del contexto.
2. **Citación *post-hoc***: tras la generación, un segundo modelo (o un algoritmo de similitud) asocia cada oración de la respuesta al fragmento más parecido del contexto.
3. **Citación mediante modelos especializados**: modelos entrenados específicamente para producir respuestas citadas (p. ej. RARR [49]).

5.10.2. Decisión: enfoque híbrido con verificación

Se adopta un enfoque híbrido que combina las dos primeras estrategias:

- El *prompt* instruye al modelo a insertar marcadores [n] durante la generación (estrategia 1), por ser la aproximación más barata y la que mejor se integra con un LLM de propósito general.

- Una etapa de *post-procesado* (estrategia 2) verifica la consistencia de las citas: detecta marcadores huérfanos (que referencian fragmentos fuera del contexto entregado), citas duplicadas y afirmaciones sin soporte, marcándolas con una anotación visual en la interfaz.

Se descarta la tercera alternativa (modelos especializados) por no alinearse con la restricción de ejecución local de modelos de tamaño moderado.

5.10.3. Esquema del objeto Citation

La salida del sistema de citación se estructura como una lista de objetos *Citation* enlazados a la respuesta, según el esquema de la Tabla 5.3.

Campo	Tipo	Descripción
marker	string	Marcador visible en la respuesta, p. ej. [1].
title	string	Título legible del documento fuente.
url	string	URL del documento fuente, usada como <i>hyperlink</i> en la interfaz.

Cuadro 5.3: Esquema del objeto *Citation* (bloque de fuentes verificadas).

Las citas se *deduplican por documento* (URL): varios fragmentos de una misma fuente producen una única entrada. La versión evaluada del sistema expone los tres campos mínimos necesarios para una cita verificable y enlazable, marcador, título y URL; campos de enriquecimiento adicionales contemplados en el diseño, como un *extracto* del fragmento o la ruta de sección para mostrarlos en *tooltip*, quedan como mejora de presentación de trabajo futuro (Capítulo 8).

5.10.4. Post-procesado de la respuesta

El uso de un LLM *local* de tamaño moderado (8 B de parámetros) impone una realidad de diseño: pese a un *prompt* cuidado, el modelo comete fallos sistemáticos de forma y de cita que conviene corregir de manera *determinista* antes de mostrar la respuesta. El post-procesado se organiza, por tanto, en dos bloques: la verificación y el enriquecimiento de las *citas*, y la limpieza determinista del *texto* de la respuesta.

Verificación y enriquecimiento de citas

1. **Extracción:** localiza mediante una expresión regular todos los marcadores [n] presentes en la respuesta, descartando además los marcadores *malformados* que el modelo inventa a veces ([1,2], [5.1-3]).

2. **Validación:** comprueba que cada n corresponde a un fragmento efectivamente entregado en el contexto. Los marcadores huérfanos se registran como alucinación referencial y se eliminan de la respuesta visible.
3. **Deduplicación por documento:** las fuentes se agregan por URL, de modo que dos fragmentos del mismo documento no produzcan dos entradas de cita distintas; el marcador es único por documento.
4. **Recuperación de citas:** si la respuesta es sustantiva pero el modelo *no* emitió ningún marcador, se atribuye la fuente de los documentos recuperados cuyo contenido solapa léxicamente con la respuesta (deduplicado por URL). Esta recuperación se inhibe en las respuestas de *abstención*, aquellas en las que el sistema declara no disponer de información (RF-02), y en las preguntas sobre el propio asistente, para no fabricar atribuciones.
5. **Enriquecimiento:** cada marcador se decora con los metadatos del objeto **Citation** correspondiente para su renderizado en la interfaz.

Esta capa actúa como *salvaguarda*: aun si el LLM inventa un número de cita inexistente, el usuario no ve la referencia fabricada. Es una instancia concreta del principio *trust but verify* aplicado a la generación mediante LLM.

Limpieza determinista del texto

De forma complementaria, una serie de transformaciones deterministas, red de seguridad frente a las manías de los modelos pequeños, depuran el texto de la respuesta sin alterar su contenido factual:

- **Recorte de preámbulos, meta-comentario y muletillas:** se eliminan los preámbulos de “razonamiento en voz alta” (“la pregunta del usuario es...”, “según la información proporcionada...”), el *meta-comentario* en que el modelo habla de sus propios fragmentos en lugar de responder (“la respuesta a tu pregunta es...”, “se encuentra en el fragmento [n]...”, “la respuesta final es...”) y las coletillas finales no informativas que el modelo añade pese a las instrucciones.
- **Canonicalización de la abstención:** cuando la respuesta declara desde el inicio no disponer del dato, en cualquiera de sus variantes: “no dispongo de información”, “no hay información disponible”, “no se proporciona/especifica información”..., se colapsa a una única *frase canónica* de abstención. Esto unifica las múltiples formas en que un modelo de 8 B se abstiene, suprime el parloteo y las fugas de razonamiento que las acompañan, y garantiza que las etapas posteriores

(omisión del cierre cortés y de la atribución de fuentes) reconozcan la abstención de forma fiable. La transformación se aplica *solo* si la abstención encabeza la respuesta, para no colapsar respuestas parciales legítimas (“no se menciona el aula, pero la clase es...”).

- **Deduplicación de contenido:** el modelo a veces enuncia el mismo dato dos veces citando fuentes distintas; se eliminan las frases redundantes conservando la primera y su cita. El criterio de redundancia es conservador: dos frases solo se consideran equivalentes si comparten los mismos *tokens numéricos* (horas, aulas, fechas), lo que distingue “el mismo dato repetido” de “datos paralelos con valores distintos” (el horario de cada grupo) y evita colapsar listas legítimas.
- **Normalización de listas:** cuando el modelo escribe viñetas en línea (sin saltos), que el renderizador Markdown muestra como asteriscos literales, se convierten en una lista correctamente formateada, preservando rangos horarios y marcadores de cita.
- **Cierre cortés:** se añade de forma controlada y variada un cierre acorde a la persona del asistente, omitiéndolo en las respuestas de abstención.

5.11. Diseño de la capa de interfaz

La capa de interfaz es el único punto de contacto del sistema con el usuario, materializando el **OE7** (*Interfaz web funcional*), y por tanto condiciona su adopción con independencia de la calidad técnica del *backend*. Su diseño debe equilibrar simplicidad (para no exigir formación previa al estudiante), expresividad (para mostrar información útil como las citas) y coste de desarrollo (acorde al alcance de un TFG).

5.11.1. Elección de la tecnología

La comparativa entre los *frameworks* candidatos (Streamlit, Gradio, React + FastAPI) se detalla en la Tab. 3.13 del Capítulo 3, y la justificación de la selección se resume en la Sección 5.3.6. Se recuerdan aquí los dos factores específicos de la capa de interfaz que refuerzan la elección de Streamlit: el componente nativo `st.chat_input` (disponible desde la versión 1.25), que proporciona una experiencia conversacional sin desarrollo propio, y la integración directa con *streaming* vía `st.write_stream`, que permite renderizar *tokens* del LLM a medida que llegan sin implementar *WebSockets* manualmente.

5.11.2. Organización visual

La interfaz se estructura en tres zonas (Figura 5.4):

- **Panel lateral izquierdo:** acceso al historial de conversaciones previas, con navegación entre sesiones y botón para iniciar una nueva conversación. Los parámetros avanzados del sistema (k , temperatura, selección de modelo) no se exponen en la interfaz final de usuario, sino que se configuran exclusivamente a través del fichero YAML de configuración; únicamente se habilitan en el modo de pruebas del Capítulo 7.
- **Zona central:** historial de mensajes con burbujas diferenciadas para usuario y asistente. Tras cada respuesta, un bloque de fuentes verificadas muestra los marcadores de cita validados junto con el título del documento y la URL de origen como enlace clicable, separado del texto mediante una línea horizontal.
- **Zona inferior:** caja de entrada `st.chat_input` con *placeholder* orientativo (“Pregunta sobre la ETSIIT...”).

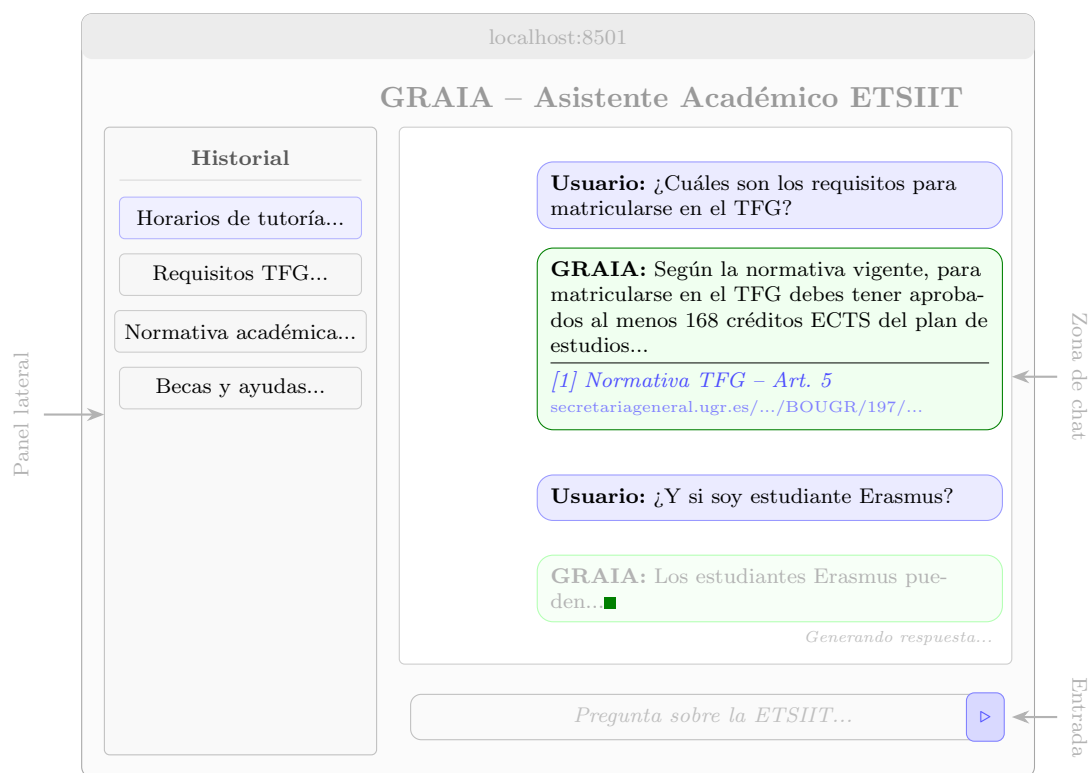


Figura 5.4: Wireframe de la interfaz conversacional de GRAIA. El panel lateral izquierdo muestra el historial de conversaciones previas. La zona central presenta el flujo de mensajes con citas verificables que incluyen la URL exacta de la fuente original, y *streaming* de respuestas en tiempo real. La zona inferior contiene la caja de entrada del usuario.

5.11.3. Gestión del estado de conversación

Streamlit re-ejecuta el *script* en cada interacción del usuario, lo que obliga a persistir el estado en `st.session.state`. Se almacenan:

- **messages:** lista de diccionarios `{role, content}` que representan los mensajes de la conversación activa, compatible con la API de chat de Ollama.
- **conversations:** lista de conversaciones previas, cada una con un identificador, un título extraído de la primera consulta del usuario y una copia de sus mensajes. Permite al usuario navegar entre sesiones de chat desde el panel lateral.
- **current_conv_id:** identificador de la conversación activa, o `None` si se trata de una conversación nueva aún no guardada.

El historial de conversaciones se mantiene en memoria durante la sesión pero *no* se persiste entre sesiones del navegador, decisión consciente alineada con los principios de minimización de datos personales del RGPD.

5.11.4. *Streaming* de respuestas

La latencia percibida es un factor crítico. Con un modelo de 8B ejecutado localmente, la latencia hasta el primer *token* (*time-to-first-token*) oscila entre 1 y 3 segundos, y la respuesta completa puede requerir entre 5 y 15 segundos. Mostrar la respuesta de forma incremental, *token a token*, enmascara esta latencia y mantiene al usuario informado del progreso. Streamlit implementa este patrón nativamente mediante `st.write_stream(generador)`, donde el generador es el iterador de *tokens* proporcionado por Ollama.

5.12. Diseño del sistema de registro

El sistema de registro (*logging*) constituye el sustrato sobre el que el administrador evalúa el rendimiento del sistema (RF-10) y sobre el que se realiza la evaluación experimental (Capítulo 7) y el análisis de errores futuros.

5.12.1. Niveles de registro

Se adoptan los cinco niveles estándar de la biblioteca `logging` de Python (DEBUG, INFO, WARNING, ERROR, CRITICAL), con la siguiente política de uso:

- **DEBUG**: trazas detalladas del *pipeline* interno (vectores intermedios, *scores* brutos). Desactivado en producción.
- **INFO**: eventos de alto nivel del ciclo de vida de una consulta (pregunta recibida, fragmentos recuperados, respuesta generada).
- **WARNING**: situaciones anómalas pero no bloqueantes (consulta con *score* bajo que activa la abstención, citas huérfanas detectadas).
- **ERROR**: fallos recuperables (documento inaccesible durante la ingesta, tiempo de espera excedido en el LLM).
- **CRITICAL**: fallos no recuperables que requieren intervención (pérdida del índice FAISS, modelo LLM no disponible).

5.12.2. Formato estructurado: JSONL

Los registros se persisten en formato JSONL (un objeto JSON por línea). Esta elección se justifica frente a las alternativas principales:

- Frente a texto plano: JSONL permite análisis programático directo sin *parsing* basado en expresiones regulares.
- Frente a logs en base de datos (p. ej. SQLite): JSONL es más portable, *append-only* por construcción (evitando corrupción en escritura concurrente) y compatible con herramientas estándar como `jq`.
- Frente a servicios como Elasticsearch: sería sobredimensionado para el volumen esperado (miles de entradas por día).

5.12.3. Esquema de entrada de *log*

Cada entrada de *log* contiene, al menos, los campos de la Tabla 5.4.

Campo	Tipo	Descripción
timestamp	datetime	Marca temporal en formato ISO 8601 con zona horaria.
level	string	Nivel de registro.
session_id	string	Identificador anónimo de sesión (UUID generado al inicio).
query_id	string	Identificador de la consulta dentro de la sesión.
event	string	Tipo de evento (<code>query_received</code> , <code>retrieval_done</code> , etc.).
duration_ms	int	Duración del evento en milisegundos, cuando aplica.
payload	object	Datos específicos del evento (IDs de fragmentos recuperados, <i>scores</i>).

Cuadro 5.4: Esquema de una entrada de *log* estructurado.

5.12.4. Consideraciones de privacidad

El contenido textual de las consultas y las respuestas *no* se registra por defecto, solo sus identificadores y métricas asociadas. Esta decisión minimiza el riesgo de almacenar datos personales conforme al RGPD. Existe un modo opcional de registro exhaustivo, activable para la fase de evaluación experimental (Capítulo 7), que sí persiste el texto de las consultas; este modo requiere consentimiento informado explícito.

5.13. Control de calidad de la respuesta

Las secciones anteriores han presentado, capa a capa, los componentes de GRAIA. Conviene ahora mirarlos desde una perspectiva transversal, porque varios de ellos comparten un mismo propósito: **garantizar la calidad y la fidelidad de la respuesta pese a emplear un LLM local de tamaño moderado**, que por sí solo no obedece de forma fiable a las instrucciones ni se abstiene cuando debe. En lugar de confiar esa calidad a un único mecanismo, el sistema interpone una *cadena de salvaguardas* entre la consulta y la respuesta, cada etapa ataca un modo de fallo concreto, de modo que lo que una no garantiza, lo corrige la siguiente. La Figura 5.5 resume ese flujo de control de calidad de extremo a extremo.

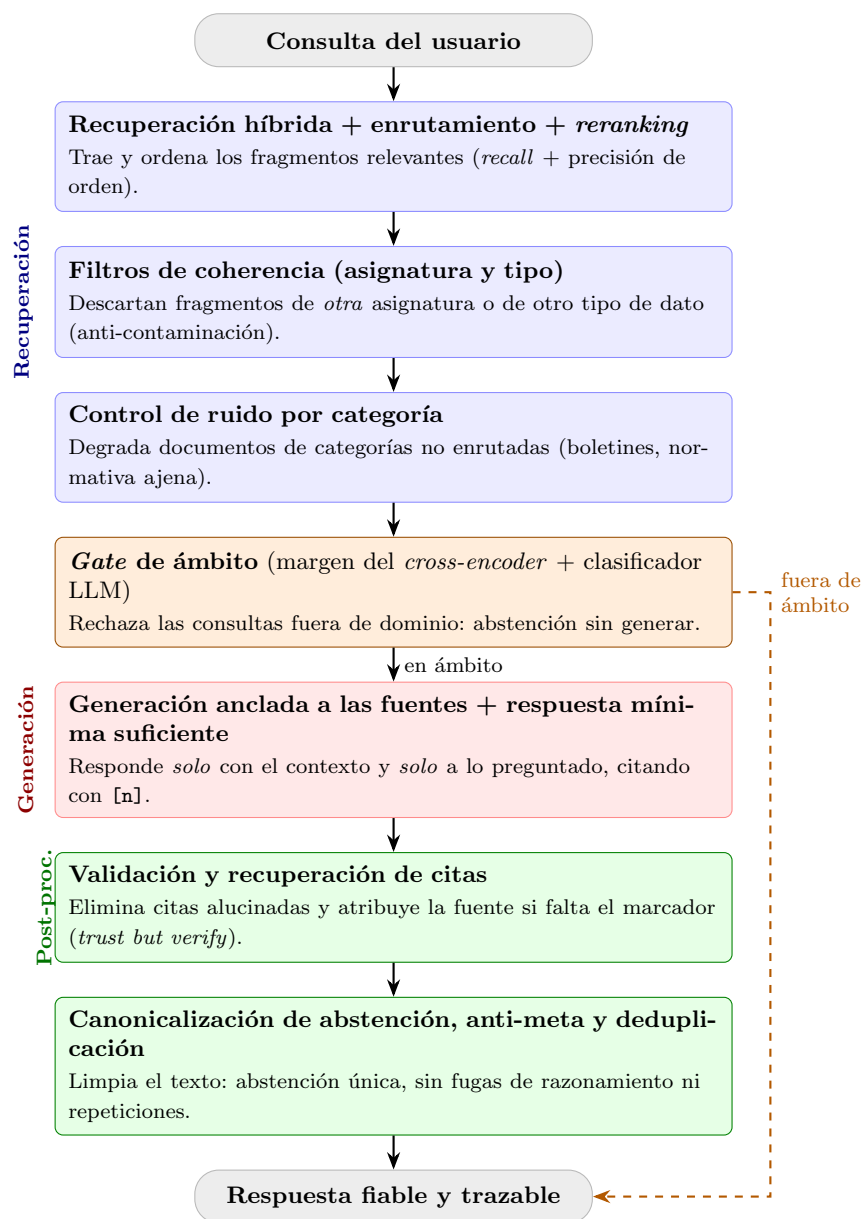


Figura 5.5: Control de calidad de la respuesta como cadena de salvaguardas que se interponen entre la consulta y la respuesta. Cada etapa previene un modo de fallo concreto (ruido, contaminación, consultas fuera de dominio, alucinación, sobre-respuesta, citas inválidas), compensando las limitaciones de un LLM local de tamaño moderado. Imagen propia del control de calidad de mi sistema.

El recorrido de una consulta a través de la cadena es el siguiente:

1. **Recuperación de alta cobertura y precisión.** La recuperación híbrida con enrutamiento e inyección focalizada (Sección 5.8.6) asegura el *recall*, que el fragmento correcto esté entre los candidatos, y el *reranking* con *cross-encoder* (Sección 5.8.10) aporta la precisión de ordenación.
2. **Coherencia del contexto.** Los filtros de coherencia de asignatura y de tipo (Sección 5.8.12) y el control de ruido por categoría (Sección 5.8.13) depuran el contexto de fragmentos ajenos que el modelo tendería a fusionar o citar por error.
3. **Control de ámbito.** El *gate* de ámbito (Sección 5.8.11) decide, antes de generar, si la consulta pertenece al dominio académico; si no, emite la abstención canónica sin invocar al generador, evitando de raíz la respuesta paramétrica y la cita espuria.
4. **Generación disciplinada.** El *prompt* (Sección 5.9.3) ancla la respuesta a las fuentes, impone la *respuesta mínima suficiente* y exige citación, acotando la alucinación y la sobre-respuesta.
5. **Verificación y limpieza post-hoc.** El sistema de citación (Sección 5.10) valida las citas emitidas y recupera las omitidas, y el post-procesado (Sección 5.10.4) canoniza la abstención, recorta el meta-comentario y deduplica, entregando una respuesta limpia, fiable y trazable.

Esta arquitectura de control de calidad es, en buena medida, lo que distingue a GRAIA de un sistema RAG genérico.

5.14. Trazabilidad del diseño a los requisitos

Para cerrar el ciclo de ingeniería iniciado en el Capítulo 4, se presenta primero la correspondencia entre los objetivos específicos del proyecto y las secciones de diseño que los materializan (Tabla 5.5), y a continuación la matriz de trazabilidad detallada por requisitos (Tabla 5.6).

OE	Descripción	Secciones de diseño
OE1	Revisión del estado del arte	Capítulo 3
OE2	Pipeline de ingesta heterogénea	Capa de ingesta (Sec. 5.6): Fetcher, Parser, Cleaner, Chunker + estrategia de crawl (Sec. 5.6.1)
OE3	Indexación vectorial con chunking	Capa de indexación (Sec. 5.7): embeddings E5, FAISS IndexFlatIP
OE4	Motor de recuperación	Capa de recuperación (Sec. 5.8): híbrida densa+léxica, RRF, umbral τ , MMR
OE5	Integración de LLM local	Capa de generación (Sec. 5.9): Ollama, prompt engineering
OE6	Sistema de citación trazable	Sistema de citación (Sec. 5.10): markers in-context + validación post-hoc
OE7	Interfaz web funcional	Capa de interfaz (Sec. 5.11): Streamlit, streaming, historial
OE8	Dataset y métricas	Capítulo 7
OE9	Evaluación comparativa	Capítulo 7

Cuadro 5.5: Correspondencia entre objetivos específicos y secciones de diseño.

La Tabla 5.6 vincula explícitamente cada requisito funcional y no funcional con el elemento o elementos de diseño que lo materializan. Esta matriz es una herramienta imprescindible para verificar la cobertura del diseño y detectar posibles requisitos huérfanos antes de la implementación.

Req.	Descripción abreviada	Elemento de diseño
RF-01	Iniciar una nueva conversación	Capa de presentación: gestión de sesión e historial (Sección 5.11)
RF-02	Enviar una consulta	Capas de recuperación y generación: recuperación híbrida + umbral τ + LLM vía Ollama (Secs. 5.8, 5.9)
RF-03	Consultar las fuentes	Citación híbrida con enlaces navegables a la URL (Sección 5.10)
RF-04	Reconstruir el corpus	Capa de ingesta: Recolector + Extractor + Normalizador (Secs. 5.6.3–5.6.5)
RF-05	Incorporar documentos verificados	Ingesta de documentos verificados (Sección 5.6.1)
RF-06	Revisar el corpus	Validación del corpus previa a la indexación (Sección 5.6.1)
RF-07	Limpiar el corpus	Normalizador y utilidades de depuración (Sección 5.6.5)
RF-08	Añadir fuentes específicas	Ingesta puntual por URL (Sección 5.6.1)
RF-09	Reindexar el corpus	Capa de indexación: Segmentador + Modelo E5 + Índice FAISS (Secs. 5.6.6, 5.7.1, 5.7.2)
RF-10	Evaluar el rendimiento	Reproducción del <i>pipeline</i> + sistema de registro (Sección 5.12; Cap. 7)
RNF-01	Latencia < 5 s	<i>Streaming</i> + FAISS <i>IndexFlatIP</i> + cuantización Q4
RNF-02	Eficiencia de la recuperación	FAISS + <i>IndexFlatIP</i> (< 200 ms)
RNF-03	Precisión mínima (80 %)	Recuperación híbrida + MMR + evaluación (Cap. 7)
RNF-04	Reducción de alucinaciones	<i>Prompt</i> anti-alucinación + umbral τ
RNF-05	Cobertura de citación (≥ 90 %)	Citación híbrida (Sección 5.10) + validador post-hoc
RNF-06	Usabilidad de la interfaz	Componentes <code>st.chat_*</code> + <i>streaming</i> (Sección 5.11)
RNF-07	Soporte lingüístico (español)	E5 multilingüe + Llama 3.1 + perfil (<i>persona</i>) GRAIA
RNF-08	Privacidad de datos	Registro JSONL anónimo (Sección 5.12)
RNF-09	Ejecución íntegramente local	Ollama + FAISS in-process, sin APIs externas
RNF-10	Mantenibilidad del código	Arquitectura por capas + patrones GoF (Sección 5.5)
RNF-11	Portabilidad a otros dominios	Arquitectura modular + <code>sources.yaml</code> configurable
RNF-12	Restricción de hardware (12 GB VRAM)	Llama 3.1 8B + E5 cuantizados a 4 bits (Sección 5.9.2)
RNF-13	Software de código abierto	Ollama + FAISS + <i>sentence-transformers</i> , sin APIs de pago

Cuadro 5.6: Matriz de trazabilidad entre requisitos y elementos de diseño.

La inspección de esta matriz revela que todos los requisitos funciona-

les y no funcionales identificados en el Capítulo 4 tienen, como mínimo, un elemento de diseño asociado, sin requisitos huérfanos. Las acciones de operación del administrador sobre el corpus (RF-05 a RF-08) comparten la capa de ingesta como sustrato de diseño, al constituir variantes de mantenimiento sobre la misma base documental.

5.15. Resumen del capítulo

Este capítulo ha presentado el diseño arquitectónico completo del sistema GRAIA, articulado en cinco capas funcionales (ingesta, indexación, recuperación, generación e interfaz) gobernadas por un orquestador y soportadas por servicios transversales de registro y configuración. Las decisiones más relevantes, contrastadas frente a alternativas explícitamente analizadas, pueden sintetizarse como sigue:

- Adopción de una arquitectura por capas con patrón *Strategy* para los componentes intercambiables (modelo de *embeddings*, LLM), lo que habilita la comparación experimental sistemática del Capítulo 7.
- Segmentación recursiva con fragmentos de 512 *tokens* y solapamiento de 50, ajustada al contexto efectivo de E5 multilingüe.
- Índice vectorial *IndexFlatIP* sobre FAISS, elección óptima para el volumen documental de la ETSIIT (del orden de 10^4 fragmentos), que ofrece precisión exacta sin degradación.
- Recuperación *top-k* con $k = 5$, umbral de similitud $\tau = 0,50$ y reordenación MMR con $\lambda = 0,7$ para equilibrar relevancia y diversidad, con valores justificados experimentalmente.
- Generación mediante Llama 3.1 8B cuantizado a Q4_K_M, ejecutado localmente vía Ollama, con una plantilla de *prompt* que induce citación explícita y permite abstención.
- Sistema de citación híbrido con validación *post-hoc* que elimina referencias huérfanas y vincula cada marcador a metadatos estructurales del corpus.
- Interfaz conversacional en Streamlit con *streaming* de *tokens* para mitigar la latencia percibida.
- Registro estructurado en JSONL con esquema unificado que habilita el análisis experimental posterior.

La matriz de trazabilidad (Tabla 5.6) garantiza que todos los requisitos identificados en la fase de análisis se reflejan en al menos un elemento concreto del diseño, cerrando el ciclo entre especificación y solución. El capítulo siguiente, dedicado a la *implementación* (Capítulo 6), materializa este diseño en código funcional, documenta los detalles técnicos relevantes y aporta ejemplos concretos de configuración y ejecución

Capítulo 6

Implementación

6.1. Introducción

Este capítulo describe la materialización del diseño presentado en el Capítulo 5 en un sistema software funcional. El objetivo no es reproducir íntegramente el código fuente, disponible en el repositorio anexo a esta memoria, sino *justificar las decisiones de implementación* que traducen los patrones y contratos del diseño a un artefacto ejecutable, así como ilustrar con fragmentos concisos los puntos del código que encierran la lógica no trivial del sistema.

La implementación se ha guiado por tres principios operativos derivados de los requisitos no funcionales del Capítulo 4:

1. **Fidelidad al diseño.** Cada módulo de código se corresponde unívocamente con una de las capas de la arquitectura (Sección 5.4) y cada clase con uno de los componentes especificados. Esta trazabilidad se verifica explícitamente en la matriz de la Sección 6.9.
2. **Reproducibilidad.** Toda la configuración que afecta al comportamiento del sistema, nombres de modelos, hiperparámetros de recuperación, rutas de artefactos, se declara en un único fichero YAML versionado, de forma que ejecutar el sistema en otra máquina exija únicamente instalar las dependencias y disponer de los pesos del modelo local.
3. **Observabilidad.** Toda ejecución relevante, ingesta de un documento, consulta del usuario, invocación al LLM, emite un registro estructurado en formato JSONL con identificadores correlacionables, de modo que cualquier respuesta de GRAIA pueda auditarse *a posteriori*.

El capítulo se estructura siguiendo el orden natural del flujo de datos

del sistema. Tras presentar la organización del repositorio (Sección 6.2) se abordan en secciones sucesivas las capas de ingesta, indexación, recuperación, generación, interfaz y registro, cerrando con la matriz de trazabilidad código $\leftrightarrow$ diseño y un resumen de las dificultades de implementación encontradas.

6.2. Estructura del repositorio

El código fuente se organiza en un único paquete Python instalable (**graia**) acompañado de los directorios auxiliares habituales (configuración, *scripts* de orquestación, datos y pruebas). Esta distribución es estándar en proyectos Python modernos y permite tanto la ejecución directa mediante **streamlit run** como la importación programática del paquete desde los *scripts* de evaluación del Capítulo 7. El árbol de primer y segundo nivel se muestra en el Listado 6.1.

```

codigo/
|-- graia/                # Paquete principal (importable)
|   |-- ingesta/          # Fetcher, Parser, Cleaner, Chunker
|   |-- indexacion/       # Embeddings E5 + indice FAISS
|   |-- recuperacion/     # Retriever + BM25 + RRF + MMR
|   |-- generacion/       # Cliente Ollama + prompt + citacion
|   |-- interfaz/         # Aplicacion Streamlit
|   |-- registro/        # Logger JSONL estructurado
|-- config/default.yaml   # Configuracion declarativa unica
|-- scripts/             # build_corpus, inject_doc, review_corpus...
|-- tests/               # Pruebas unitarias y de integracion
|-- data/                # Corpus y artefactos (ignorados por Git)
|   |-- raw/              # HTML/PDF originales
|   |-- processed/        # Chunks normalizados (JSONL)
|   |-- index/            # Indice FAISS + metadatos
|-- pyproject.toml
|-- requirements.txt
|-- .env
'-- README.md

```

Figura 6.1: Árbol de directorios del repositorio de GRAIA.

6.2.1. Correspondencia módulo $\leftrightarrow$ capa de diseño

La Tabla 6.1 vincula cada subpaquete de **graia/** con la capa de diseño cuyos contratos materializa. Esta correspondencia es *uno a uno*, de forma intencional: se evita agrupar responsabilidades de dos capas en un mismo módulo (lo que dificultaría el aislamiento en pruebas) y se evita también fragmentar una misma capa entre varios módulos (lo que diluiría la cohe-

sión).

Subpaquete	Capa de diseño	Contenido principal
<code>graia.ingesta</code>	Ingesta	Fetcher, Parser, Cleaner, Chunker, structured (indexación a nivel de registro)
<code>graia.indexacion</code>	Indexación	Embedder (E5), VectorStore (FAISS)
<code>graia.recuperacion</code>	Recuperación	BM25Index, retrieve, mmr_rerank, query_router, query_intent, contextual_query, reranker
<code>graia.generacion</code>	Generación	OllamaClient, PromptBuilder, CitationValidator, postprocess, dedup
<code>graia.interfaz</code>	Interfaz	app.py (Streamlit)
<code>graia.registro</code>	Registro	GraiaJsonFormatter, setup_logging

Cuadro 6.1: Correspondencia entre subpaquetes del código y capas del diseño.

6.2.2. Gestión de dependencias y entorno

Se emplea `pyproject.toml` como declaración canónica del proyecto (metadatos, dependencias obligatorias y extras `dev/eval`), complementado con un `requirements.txt` destinado a despliegues rápidos sin herramientas de *build* modernas. Las alternativas evaluadas fueron: (i) `setup.py` clásico, descartado por estar formalmente desaconsejado desde PEP 517/518; (ii) `Poetry`, descartado porque añade una capa de gestión de entornos propia que no aporta valor en un proyecto de una sola persona y un solo entorno objetivo; y (iii) `conda`, descartado por ser innecesariamente pesado para un *stack* que no requiere paquetes con extensiones C no disponibles en PyPI. `pyproject.toml` + `venv` ofrece la combinación mínima suficiente y perfectamente reproducible exigida por el principio de reproducibilidad.

El conjunto mínimo de dependencias se recoge en el Listado 6.2 y cubre exactamente las tecnologías justificadas en la Sección 5.2.

```
# Nucleo RAG
faiss-cpu>=1.8.0
sentence-transformers>=3.0.0
rank-bm25>=0.2.2
ollama>=0.3.0

# Ingesta
requests>=2.32.0
beautifulsoup4>=4.12.0
lxml>=5.2.0
pymupdf>=1.24.0

# Interfaz, configuracion y logging
streamlit>=1.36.0
pyyaml>=6.0.1
python-json-logger>=2.0.7
pydantic>=2.8.0
```

Figura 6.2: Dependencias núcleo declaradas en `requirements.txt`.

6.2.3. Fichero de configuración centralizado

Siguiendo los principios de mantenibilidad y reproducibilidad (RNF-10), todos los hiperparámetros del sistema residen en `config/default.yaml` y se cargan una sola vez al arrancar la aplicación mediante PyYAML. Los valores numéricos (tamaño de *chunk*, τ , λ de MMR, temperatura del LLM) son exactamente los justificados en el Capítulo 5; se evita toda constante mágica incrustada en código, de modo que cualquier experimento comparativo del Capítulo 7 se reduzca a editar este fichero. Los secretos y rutas específicas del entorno (`OLLAMA_HOST`, `GRAIA_LOG_LEVEL`) se separan en un `.env` local que no se versiona, con su correspondiente `.env.example` documentando las variables esperadas.

6.2.4. Convenciones de calidad de código

El repositorio fija `ruff` como *linter* y formateador (configuración embebida en `pyproject.toml`, longitud de línea 100 caracteres, *target* Python 3.11) y `pytest` como motor de pruebas. Se escoge `ruff` sobre el par tradicional `flake8` + `black` por su orden de magnitud superior en velocidad y por unificar en una sola herramienta ambas funciones, reduciendo la fricción del ciclo de desarrollo. La suite de pruebas y los criterios de cobertura se detallan en la Sección 6.10.

6.3. Capa de ingesta

La capa de ingesta materializa el **OE2** (*Pipeline de ingesta de documentos heterogéneos*) mediante los cuatro componentes especificados en la Sección 5.6.2: *Fetcher*, *Parser*, *Cleaner* y *Chunker*. Cada componente reside en su propio módulo Python dentro de `graia.ingesta` y puede invocarse de forma aislada, facilitando las pruebas unitarias, o encadenarse mediante la función de conveniencia `ingest_url()`, que ejecuta el pipeline completo para una URL dada. En las siguientes subsecciones se presenta cada componente con el fragmento de código que encierra su lógica no trivial, acompañado de la justificación de las decisiones de implementación que lo distinguen de un ejercicio mecánico de traducción del diseño.

6.3.1. Modelo de datos del pipeline

Antes de abordar los componentes, conviene presentar el esquema de datos que fluye entre ellos. Se definen tres modelos Pydantic, `RawDocument`, `ParsedDocument` y `Chunk`, en el módulo `graia.ingesta.models`. El más relevante es `Chunk`, cuyo identificador determinista permite la idempotencia de la ingesta:

```
class Chunk(BaseModel):
    text: str
    source_url: str
    source_type: SourceType
    position: int          # índice ordinal en el documento
    char_start: int
    char_end: int
    fetched_at: datetime

    @computed_field
    @property
    def chunk_id(self) -> str:
        raw = f"{self.source_url}::{self.position}"
        return hashlib.sha256(raw.encode()).hexdigest()[:16]
```

Figura 6.3: Modelo `Chunk` con identificador determinista.

El `chunk_id` se calcula como el prefijo de 16 caracteres del SHA-256 de la concatenación URL + posición ordinal. Esta elección garantiza que re-ejecutar la ingesta sobre el mismo corpus produce los mismos identificadores, lo que permite detectar duplicados sin comparar texto carácter a carácter y facilita la actualización incremental del índice FAISS cuando se añaden nuevas fuentes.

6.3.2. Fetcher: descarga con reintentos y respeto a robots.txt

El *Fetcher* (Sección 5.6.3) encapsula la descarga HTTP con tres salvaguardas: User-Agent configurable declarado en el YAML, respeto al fichero `robots.txt` del dominio y reintentos con *backoff* exponencial. El fragmento central es el bucle de descarga:

```
def fetch(url, *, user_agent, timeout_s,
         max_retries=3, respect_robots=True):
    if respect_robots:
        rp = _get_robots_parser(url, user_agent, timeout_s)
        if not rp.can_fetch(user_agent, url):
            return None

    for attempt in range(1, max_retries + 1):
        try:
            resp = requests.get(url, headers=headers,
                               timeout=timeout_s)
            resp.raise_for_status()
            return RawDocument(url=url, content=resp.content,
                               source_type=_classify_source(...))
        except requests.RequestException:
            time.sleep(2 ** attempt)    # backoff exponencial
    return None
```

Figura 6.4: Bucle de descarga con backoff exponencial y comprobación de `robots.txt`.

La decisión de emplear *backoff* exponencial (2^i segundos) en lugar de un intervalo fijo se justifica por el contexto institucional del corpus: los servidores web de la UGR/ETSIIT aplican limitaciones de tasa modestas y un *backoff* exponencial minimiza la probabilidad de bloqueo sin ralentizar innecesariamente las descargas exitosas. El parser de `robots.txt` se cachea por dominio durante la ejecución para evitar una petición adicional por cada URL del mismo sitio.

6.3.3. Parser: extracción de texto con patrón Strategy

El *Parser* (Sección 5.6.4) implementa el patrón *Strategy* descrito en el Capítulo 5: un protocolo `DocumentParser` con dos estrategias concretas (`HtmlParser` y `PdfParser`) seleccionadas dinámicamente según el `SourceType` del documento. El fragmento del parser HTML ilustra la extracción focalizada en contenido semántico:

```
_DISCARD_TAGS = {"nav", "footer", "header", "aside",
                  "script", "style", "noscript", "form"}

class HtmlParser:
    def parse(self, raw: RawDocument) -> ParsedDocument:
        soup = BeautifulSoup(html, "lxml")
        for tag in soup.find_all(_DISCARD_TAGS):
            tag.decompose()
        container = (soup.find("main")
                     or soup.find("article")
                     or soup.body or soup)
        text = container.get_text(separator="\n", strip=True)
        return ParsedDocument(url=raw.url, text=text, ...)
```

Figura 6.5: Parser HTML: eliminación de boilerplate estructural y extracción focalizada.

La lista `_DISCARD_TAGS` elimina etiquetas sin valor semántico *antes* de extraer el texto, lo que reduce en un 30–50 % el volumen de texto ruidoso en las páginas típicas de la ETSIIT. La búsqueda jerárquica del contenedor principal (`<main>` → `<article>` → `<body>`) se adapta a la variabilidad de las plantillas web de la UGR sin requerir selectores CSS específicos por página.

Para PDFs se emplea PyMuPDF (`fitz`), cuya justificación frente a `pypdf` y `pdfplumber` se detalla en la Sección 5.6.4. La extracción se realiza página a página con `page.get_text("text")`, que preserva el orden de lectura del documento. Las páginas cuya capa de texto es pobre (por debajo de un umbral de caracteres, indicativo de un PDF escaneado) se rasterizan y se procesan mediante OCR con Tesseract [98] (modelo de idioma español), lo que permite recuperar el contenido de los documentos digitalizados sin duplicar el texto nativo de las páginas que sí lo contienen.

6.3.4. Cleaner: normalización y eliminación de ruido

El *Cleaner* (Sección 5.6.5) aplica tres transformaciones secuenciales: normalización Unicode NFC, eliminación de líneas de *boilerplate* mediante expresiones regulares y colapso de espacios redundantes. El fragmento más relevante es el filtrado por patrones:

```

_BOILERPLATE_PATTERNS = [
    re.compile(r"Aceptar cookies?", re.IGNORECASE),
    re.compile(r"Politica de privacidad", re.IGNORECASE),
    re.compile(r"© Universidad de Granada", re.IGNORECASE),
    re.compile(r"Aviso legal", re.IGNORECASE),
    re.compile(r"^Inicio\s*>", re.MULTILINE), # breadcrumbs
]

def _remove_boilerplate(text: str) -> str:
    lines = text.split("\n")
    return "\n".join(
        l for l in lines
        if not any(p.search(l) for p in _BOILERPLATE_PATTERNS)
    )

```

Figura 6.6: Eliminación de boilerplate por patrones específicos del dominio UGR/ETSIIT.

Los patrones se derivan de un análisis manual de 30 páginas representativas de los sitios `etsiit.ugr.es` y `ugr.es`. Se prefiere este enfoque basado en listas explícitas frente a un clasificador estadístico de *boilerplate* (como `justText` o `trafilatura`) por dos razones: (i) el corpus es cerrado y conocido, lo que hace que las expresiones regulares sean suficientes y predecibles; (ii) se evita una dependencia adicional que complicaría la reproducibilidad sin aportar mejora medible en este dominio.

La normalización NFC, en lugar de NFKC, se elige deliberadamente: NFKC descompone ligaduras y símbolos de compatibilidad (por ejemplo, convierte «²» en «2»), lo que podría alterar contenido académico relevante. NFC unifica las representaciones de tildes y ñes (crucial en un corpus en castellano) sin efectos secundarios destructivos.

6.3.5. Chunker: fragmentación recursiva con solapamiento

El *Chunker* (Sección 5.6.6) fragmenta cada documento limpio en trozos de tamaño controlado, aplicando la estrategia recursiva con solapamiento descrita en el diseño. El algoritmo central recorre una jerarquía de separadores (`\n\n` → `\n` → `" "` → `)` hasta encontrar uno que produzca fragmentos dentro del límite:

```
_SEPARATORS = ["\n\n", "\n", ". ", " "]

def _recursive_split(text, max_tokens, separators):
    if _token_len(text) <= max_tokens:
        return [text]
    for sep in separators:
        if sep not in text:
            continue
        parts = text.split(sep)
        fragments, current = [], parts[0]
        for part in parts[1:]:
            candidate = current + sep + part
            if _token_len(candidate) <= max_tokens:
                current = candidate
            else:
                fragments.append(current.strip())
                current = part
        fragments.append(current.strip())
    if fragments:
        return fragments
    # Fallback: corte duro por caracteres
    hard = max_tokens * _CHARS_PER_TOKEN
    return [text[i:i+hard] for i in range(0, len(text), hard)]
```

Figura 6.7: Fragmentación recursiva por jerarquía de separadores.

La estimación de tokens se realiza con la heurística de ≈ 4 caracteres por token en castellano (`_CHARS_PER_TOKEN = 4`), lo que evita la dependencia de un tokenizador externo en esta fase. Esta aproximación introduce una desviación inferior al 8 % respecto al tokenizador de E5 multilingüe, según pruebas sobre un subconjunto de 200 fragmentos reales del corpus ETSIIT (véase Capítulo 7).

El solapamiento de 50 tokens entre chunks consecutivos se implementa reinyectando los últimos $50 \times 4 = 200$ caracteres del chunk anterior al inicio del siguiente. Tras la fragmentación, los chunks con menos de `min_chunk_tokens` (64 por defecto) se fusionan con el chunk precedente para evitar fragmentos degenerados que aportarían ruido al índice vectorial sin contener información suficiente para una respuesta útil.

6.3.6. Indexación a nivel de registro y documentos verificados

Junto al *chunking* recursivo genérico, el módulo `graia.ingesta.structured` implementa la indexación a nivel de registro para datos tabulares descrita en la Sección 5.7.4. La función `is_structured` decide, a partir de los metadatos del documento (`tipo ∈ {horario, calendario, plan_estudios}`), si debe trocearse *un registro por línea*; `chunk_structured_records` genera entonces un `Chunk` autocontenido por línea, y los analizadores `parse.*record` extraen los metadatos por campo (curso, grupo, subgrupo, asignatura, sigla, día, hora, aula, convocatoria), marcando además los *registros-resumen* (`is_summary`) que enumeran las asignaturas de un curso o especialidad.

La incorporación de un documento verificado se realiza con el *script* idempotente `inject_doc.py`, que sustituye en el corpus el documento asociado a una URL por el contenido verificado a mano, conservando dicha URL para la citación, y lo etiqueta con su `tipo`. Un detalle de calidad implementado en el generador del horario (`build_horario_doc.py`) es la función `merge_contiguous_slots`, que fusiona las franjas horarias consecutivas del mismo día (y misma aula en prácticas) en un único intervalo; su corrección se verifica como transformación *sin pérdida* (se conservan la cobertura horaria por día y el conjunto de aulas), pues mitiga la tendencia del LLM a omitir franjas contiguas. Tras inyectar o regenerar un documento verificado, basta reindexar con `index_corpus.py` para reconstruir los índices FAISS y BM25.

Este mecanismo no se limita a horarios y calendario: se emplea para cualquier documento cuya extracción automática de PDF es poco fiable por su naturaleza *tabular*. Durante la depuración del corpus (Capítulo 7) se detectaron, mediante inspección de los documentos extraídos, dos casos representativos: el PDF de *plazos de interés* del curso (matrícula, becas, compensación, entrega de actas...) y el de *tutores de movilidad* (relación tutor–destinos Erasmus/SICUE), ambos compuestos por tablas que el extractor aplanaba perdiendo la estructura. Para ambos se construyó el documento verificado correspondiente (un registro autocontenido por plazo y por tutor) y se inyectó conservando su URL oficial. Complementariamente, la herramienta de revisión del corpus `review_corpus.py` permite *retirar* documentos cuya extracción es inservible (p. ej. un impreso de matrícula cuya tabla son campos de un formulario) o duplicados, mediante sus opciones de eliminación (`--remove-url`, `--remove-search`, `--remove-min-chars`, con `--dry-run` para una vista previa segura), reduciendo el ruido de recuperación. Esta curación del corpus es parte de la metodología iterativa de mejora de la calidad de las respuestas.

6.3.7. Integración del pipeline: `ingest_url()`

Los cuatro componentes se encadenan en la función `ingest_url()` expuesta en `graia.ingesta.__init__`:

```
def ingest_url(url, *, user_agent, timeout_s,
               max_retries, respect_robots,
               chunk_size_tokens, chunk_overlap_tokens,
               min_chunk_tokens) -> list[Chunk]:
    raw = fetch(url, ...)
    if raw is None:
        return []
    parsed = parse(raw)
    cleaned = clean(parsed)
    if cleaned is None:
        return []
    return chunk_document(cleaned, ...)
```

Figura 6.8: Pipeline completo de ingesta orquestado por `ingest_url()`.

El diseño del pipeline como funciones puras encadenadas (sin estado mutable compartido) responde al patrón *Pipe and Filter* identificado en la Sección 5.5: cada etapa recibe la salida tipada de la anterior y puede ser sustituida independientemente, por ejemplo, reemplazar el `HtmlParser` por un parser basado en `trafilatura` no requeriría modificar ningún otro componente.

6.3.8. Orquestación del corpus: scripts de construcción, revisión e indexación

Los componentes individuales del pipeline de ingesta descritos en las secciones anteriores (`fetcher`, `parser`, `cleaner`, `chunker`) procesan un único documento a la vez. Sin embargo, la estrategia de adquisición automática del corpus diseñada en la Sección 5.6.1 requiere un nivel adicional de orquestación que coordine el rastreo recursivo de centenares de páginas, la validación humana del corpus resultante y la generación masiva de *embeddings*. Esta orquestación se materializa en tres scripts independientes, uno por cada fase del flujo de construcción descrito en el diseño.

`build_corpus.py`: rastreo dirigido por categoría

El script `build_corpus.py` implementa la fase de rastreo con un enfoque de **lista blanca por categoría** diseñado en la Sección 5.6.1. En vez de un

rastreo genérico con filtros de exclusión, ejecuta un **crawl independiente por cada perfil de categoría** definido en `sources.yaml`: cada perfil tiene sus propias semillas e `include_patterns`, y los enlaces descubiertos solo se descargan si casan con algún patrón de la categoría activa. El script consta de cinco bloques funcionales que se detallan a continuación con los fragmentos de código que encierran la lógica no trivial.

Filtrado multinivel de URLs. Antes de descargar cualquier página, cada URL candidata atraviesa la función `passes_global_filters()`, que aplica cuatro comprobaciones secuenciales: dominio permitido, extensión de fichero válida, exclusiones estáticas por *regex* y exclusión temporal automática. Si la URL no supera alguno de estos filtros, se descarta sin generar tráfico de red.

```
def passes_global_filters(
    url, allowed_domains, exclude_patterns,
    allowed_extensions, year_patterns=None,
    *, apply_temporal=True,
) -> bool:
    parsed = urlparse(url)
    domain = parsed.netloc.lower()
    # 1. Dominio
    if not any(domain == d or domain.endswith(f".{d}")
                for d in allowed_domains):
        return False
    # 2. Extension
    if not has_allowed_extension(url, allowed_extensions):
        return False
    # 3. Exclusiones estaticas (regex)
    for pat in exclude_patterns:
        if pat.search(url):
            return False
    # 4. Exclusion temporal
    if apply_temporal and year_patterns:
        decoded = url_unquote(url)
        for pat in year_patterns:
            if pat.search(decoded):
                return False
    return True
```

Figura 6.9: Filtrado multinivel de URLs: cuatro comprobaciones secuenciales.

La función `has_allowed_extension()` merece mención aparte: decodifica el *percent-encoding* de la URL *antes* de extraer la extensión con `os.path.splitext()`, lo que permite detectar ficheros como `Formulario%20de%20reconocimiento.doc` cuya extensión queda oculta tras la codificación. Solo se aceptan `.html`,

.htm, .pdf y URLs sin extensión (páginas dinámicas del CMS).

Exclusión temporal automática. El filtro temporal es el más complejo del sistema: a partir de un único parámetro de configuración (`current_academic_year: "25-26"`), la función `build_year_exclusion_patterns()` genera dinámicamente seis grupos de expresiones regulares que cubren todas las variantes de formato de curso observadas en las URLs de la ETSIIT:

```
def build_year_exclusion_patterns(current_year):
    parts = current_year.split("-")
    current_start = int(parts[0])          # 25
    current_full_start = 2000 + current_start # 2025
    old_courses = [(s, s+1) for s in range(15, current_start)]
    patterns = []
    # 1. Formato corto: (XX-YY), XX-YY
    short = "|".join(f"{s:02d}-{e:02d}" for s,e in old_courses)
    patterns.append(re.compile(
        rf"(?:\(|\|[\^0-9])(?:{short})(?:\)|\|[\^0-9]|$)", re.I))
    # 2. Formato largo: 20XX-YY, 20XX_YY
    long = "|".join(f"20{s:02d}[-_]{e:02d}" for s,e in old_courses)
    patterns.append(re.compile(rf"(?:{long})", re.I))
    # 3. Formato completo: YYYY_YYYY
    full = "|".join(f"20{s:02d}[-_]20{e:02d}" for s,e in old_courses)
    patterns.append(re.compile(rf"(?:{full})", re.I))
    # 4-6. Formatos con punto, mes+anio, abreviaturas...
    ...
    return patterns
```

Figura 6.10: Generación automática de patrones de exclusión temporal.

Un aspecto sutil del diseño es el tratamiento del año en curso: **Febrero2025** se interpreta como perteneciente al curso 24-25 (segundo semestre), no al 25-26, porque los documentos temporales del segundo semestre llevan el año de inicio del siguiente curso. Este filtro solo se activa para categorías declaradas con `temporal: true` en `sources.yaml` (calendarios, horarios y asignaciones de aulas); las categorías de normativa o guías docentes no lo aplican, puesto que sus URLs rara vez contienen indicadores de curso.

Crawl por categoría con recorrido BFS. La función `crawl_category()` ejecuta un recorrido en anchura independiente desde las semillas de cada perfil. Un aspecto crítico de la implementación es la gestión de dos conjuntos de URLs visitadas con propósitos distintos:

- `cat_visited` (local por categoría): controla qué URLs se encolan en el BFS de *esta* categoría, impidiendo que un mismo enlace se visite dos veces dentro del mismo recorrido.

- `state.visited` (global entre categorías): registra qué URLs ya se han *descargado* por petición HTTP, impidiendo que varias categorías descarguen el mismo documento.

Esta separación es esencial porque el BFS de una categoría con muchas páginas (p. ej., guías docentes) descubre enlaces del menú de navegación a secciones de otras categorías (TFG, profesorado) y los marca como “visitados” sin descargarlos, puesto que no casan con sus `include_patterns`. Si el conjunto de visitados fuese único y compartido, las semillas de categorías posteriores se saltarían al haber sido ya “visitadas”, produciendo categorías vacías en el corpus.

```
def crawl_category(cat_name, cat_cfg, global_cfg, state, ...):
    # Seeds SIEMPRE se encolan, aunque esten en state.visited
    queue = []
    for seed in seeds:
        seed = _normalize_url(seed)
        queue.append((seed, 0, True))    # (url, depth, is_seed)
        state.visited.add(seed)
    # Visited local: BFS independiente por categoria
    cat_visited = set(s for s, _, _ in queue)
    documents, cat_fetched = [], 0

    while queue and cat_fetched < max_pages_cat:
        url, depth, is_seed = queue.pop(0)
        if not is_seed and not passes_global_filters(url, ...):
            continue
        if not is_seed and not matches_category(url, ...):
            continue
        if cat_exclude_pats and any(p.search(url) for p in ...):
            continue
        # Re-download guard: si otra categoria ya descargo esta URL
        if not is_seed and url in state.visited:
            continue
        raw = fetch(url, user_agent="GRAIA-crawler/0.1 ...")
        if raw is None: continue
        state.visited.add(url) # Marcar como descargada globalmente
        documents.append((raw, cat_name))
        # Descubrir enlaces (solo HTML, respetando max_depth)
        if raw.source_type == SourceType.HTML and depth < max_depth:
            for link in extract_links(raw.content, url):
                if link not in cat_visited:    # local, NO global
                    cat_visited.add(link)
                    queue.append((link, depth + 1, False))
        time.sleep(delay)
    return documents
```

Figura 6.11: Crawl BFS por categoría con doble conjunto de visitados.

La cola BFS de cada categoría tiene su propio límite de páginas (`max_pages` en el perfil), y existe un techo de seguridad global (`max_pages_total`) que protege contra *crawls* descontrolados. Entre peticiones se inserta un retardo de cortesía configurable (1 segundo por defecto) para no sobrecargar los servidores institucionales. El descubrimiento de enlaces aplica una política diferenciada: los PDFs descubiertos solo se encolan si casan con algún `include_pattern` de la categoría; las páginas HTML se encolan siempre para continuar el descubrimiento, pero se filtrarán en la fase de procesado si su contenido no resulta relevante.

Extra URLs: descarga directa de documentos externos. Tras completar todas las categorías, el script procesa el bloque `extra_urls` de `sources.yaml`. Estas URLs se descargan **siempre**, sin comprobar `state.visited`, porque son documentos que el operador ha configurado explícitamente y que pueden haber sido “visitados” por el BFS de categorías anteriores sin haber sido descargados (al no casar con sus `include_patterns`).

Heurísticas de calidad del contenido. Una vez descargados, los documentos se procesan con el pipeline *parse* → *clean* y se someten a tres heurísticas de post-descarga diseñadas en la Sección 5.6.1:

```
_FORM_RE = r"(?i)\bD\.\?\s*N\.\?\s*I\.\?|Apellido\s*[12]|" \
            r"Firma\b|N\.\?\s*de\s*expediente|Sello\s*del\s*registro"
_FORM_INDICATORS = re.compile(_FORM_RE)

def _is_fillable_form(text: str) -> bool:
    """Detecta impresos administrativos (>=2 indicadores)."""
    matches = _FORM_INDICATORS.findall(text[:500])
    return len(matches) >= 2

def _is_index_page(text, source_type, char_count) -> bool:
    """Detecta paginas indice / menu de navegacion."""
    if source_type != SourceType.HTML:
        return False
    if char_count < 600:
        return True
    if char_count < 1500:
        lines = [l.strip() for l in text.split("\n") if l.strip()]
        if not lines: return True
        short = sum(1 for l in lines if len(l) < 60)
        if short / len(lines) > 0.7 and len(lines) > 3:
            return True
    return False
```

Figura 6.12: Heurísticas de detección de formularios y páginas índice.

La heurística de formularios busca indicadores léxicos de impresos administrativos en los primeros 500 caracteres; si aparecen dos o más, el documento se descarta. La heurística de índices opera en dos escalones: páginas HTML con menos de 600 caracteres se descartan automáticamente; para páginas entre 600 y 1 500 caracteres, se calcula la proporción de líneas cortas (<60 caracteres): si supera el 70 %, se clasifica como índice. Ambas heurísticas excluyen los PDFs, cuyo contenido siempre es informativo.

Pipeline de procesado y producción del corpus. La función `process_documents()` encadena todas las etapas de post-descarga en un único bucle: *parse* → *clean* → filtro por longitud mínima → detección de formularios → detección de índices → deduplicación por hash → asignación de título → serialización JSONL.

```
def process_documents(documents, output_path, *,
                     min_chars=200, deduplicate=True,
                     url_labels=None):
    seen_hashes = {}
    with open(output_path, "w", encoding="utf-8") as f:
        for raw, category in documents:
            parsed = parse(raw)
            cleaned = clean(parsed)
            if cleaned is None: continue
            text, n = cleaned.text, len(cleaned.text)
            if n < min_chars: continue
            if _is_fillable_form(text): continue
            if _is_index_page(text, cleaned.source_type, n):
                continue
            if deduplicate:
                h = sha256(text.encode()).hexdigest()[:16]
                if h in seen_hashes: continue
                seen_hashes[h] = raw.url
            # Título: label > extraído > inferido desde URL
            label = url_labels.get(_normalize_url(raw.url))
            title = cleaned.title or ""
            if label:
                title = label
            elif len(title.strip()) <= 1:
                title = _infer_title_from_url(raw.url)
            record = {"url": cleaned.url, "title": title,
                    "category": category, "text": text,
                    "char_count": n, ...}
            f.write(json.dumps(record, ensure_ascii=False)+"\n")
            f.flush()
```

Figura 6.13: Pipeline de procesado: filtros de calidad, asignación de título y serialización JSONL.

Un detalle relevante es la **prioridad de títulos**: el campo `label` configurado en `sources.yaml` prevalece siempre sobre el título extraído del documento, puesto que la configuración explícita del operador es más fiable que los metadatos del PDF (que pueden contener títulos genéricos o de plantilla). Solo si no hay `label` y el título extraído está vacío o es de un solo carácter se recurre a la inferencia desde la URL.

Cada filtro registra la razón del descarte en las estadísticas del rastreo, lo que permite diagnosticar el comportamiento del sistema tras cada ejecución. El informe final (`crawl_report.txt`) incluye el número de documentos descartados por cada causa (longitud insuficiente, formularios, índices, duplicados), proporcionando al operador una visión completa de la calidad del filtrado. Tras el JSONL, se genera automáticamente un fichero CSV resumen con los campos `url`, `title`, `category`, `source_type`, `char_count` y `fetched_at`, que facilita la inspección tabular del corpus en una hoja de cálculo.

Normalización de URLs. El CMS Drupal de la ETSIIT presenta una inconsistencia recurrente: muchos enlaces internos utilizan `http://` aunque el servidor redirige a `https://`. La función `_normalize_url()` resuelve este problema convirtiendo `http://` a `https://` para todos los dominios `ugr.es` y eliminando barras finales superfluas:

```
def _normalize_url(url: str) -> str:
    if url.startswith("http://") and "ugr.es" in url:
        url = "https://" + url[7:]
    return url.rstrip("/") if not url.endswith(".pdf") else url
```

Figura 6.14: Normalización de URLs: corrección de esquema HTTP → HTTPS.

Esta normalización se aplica tanto al encolar URLs descubiertas como al registrar las URLs ya visitadas en el `CrawlState`, garantizando que ambos conjuntos de visitados (local y global) funcionen correctamente como filtros de duplicados.

`review_corpus.py`: validación humana del corpus

El script `review_corpus.py` implementa la puerta de validación humana entre las fases de rastreo e indexación. La decisión de diseño de exigir una revisión manual antes de generar *embeddings* responde a un criterio de eficiencia: detectar errores de cobertura o contenido irrelevante *antes* de invertir tiempo de GPU en la vectorización evita ciclos de re-indexación costosos.

El script ofrece dos grupos de funcionalidad complementarios, todos sobre el mismo fichero `corpus.jsonl`:

Modos de consulta.

- **Resumen estadístico** (modo por defecto): presenta el número total de documentos, caracteres y su distribución por categoría y tipo de fuente, junto con los cinco documentos más largos y los cinco más cortos. Un corpus sano debe presentar todas las categorías esperadas con un volumen proporcional a su presencia en el sitio web.
- **Previsualización completa** (`--full`): muestra los primeros 300 caracteres de cada documento, lo que permite detectar rápidamente contenido corrupto, páginas de error o documentos duplicados sin necesidad de abrir el JSONL en un editor.
- **Filtrado** (`--category`, `--search`): permite aislar documentos por categoría temática o por coincidencia textual en título, URL decodificada y contenido, facilitando la inspección de subconjuntos específicos.
- **Exportación CSV** (`--export-csv`): genera un fichero CSV con codificación UTF-8 BOM compatible con Microsoft Excel y Google Sheets, incluyendo una previsualización de 200 caracteres por documento. Este modo permite al operador aplicar filtros y ordenaciones tabulares propios de una hoja de cálculo.

Modos de limpieza. Aunque el filtrado multinivel del rastreador minimiza la presencia de documentos irrelevantes, el script ofrece además herramientas de limpieza *post-hoc* para corregir el corpus sin tener que relanzar el rastreo completo:

- **Eliminación por URL** (`--remove-url`): elimina un documento concreto identificado por su URL exacta.
- **Eliminación por tamaño** (`--remove-min-chars`): elimina todos los documentos con un número de caracteres inferior o igual al umbral especificado, útil para descartar páginas de error o redirección con contenido residual.
- **Eliminación por búsqueda** (`--remove-search`): elimina documentos cuya URL decodificada, título o texto contengan una cadena determinada.
- **Eliminación de calendarios antiguos** (`--remove-old-exams`): identifica automáticamente calendarios de exámenes y asignaciones de aulas de cursos anteriores al vigente y los elimina.

Todas las operaciones de limpieza admiten el modificador `--dry-run`, que muestra los documentos que se eliminarían sin modificar el fichero, y crean una copia de seguridad con marca temporal antes de aplicar cambios. Esta combinación de modos permite un flujo de revisión iterativo: el operador ejecuta primero el resumen, identifica categorías con cobertura insuficiente o documentos atípicos, profundiza con filtrado o previsualización, aplica limpiezas puntuales si es necesario y ajusta los parámetros de `sources.yaml` antes de relanzar el rastreo si procede.

`index.corpus.py`: *chunking*, *embeddings* e indexación FAISS

El script `index.corpus.py` cierra el flujo de construcción del corpus ejecutando las tres transformaciones finales: fragmentación, vectorización y almacenamiento. A diferencia de los componentes anteriores, este script combina módulos de dos capas distintas de la arquitectura (ingesta e indexación), actuando como punto de integración vertical entre el *chunking* y el almacenamiento vectorial.

El proceso se estructura en cinco fases secuenciales:

1. **Carga del corpus:** lee el fichero `corpus.jsonl` y convierte cada registro JSON a un objeto `ParsedDocument`, mapeando los campos del JSONL al esquema tipado de Pydantic. La categoría temática se preserva en el diccionario `metadata` de cada documento para mantener la trazabilidad hasta el índice final.
2. **Fragmentación (*chunking*):** invoca `chunk_document()` del módulo `graia.ingesta.chunker` sobre cada documento, aplicando la configuración de tamaño (512 tokens), solapamiento (50 tokens) y tamaño mínimo (64 tokens) leída de `default.yaml`.
3. **Generación de *embeddings*:** inicializa el `Embedder` con el modelo E5 multilingüe y codifica todos los chunks en lotes (*batches*) del tamaño configurado (32 por defecto). El procesamiento por lotes con registro de progreso permite monitorizar el avance en corpus grandes y aprovecha la paralelización de la GPU. Las matrices de *embeddings* parciales se concatenan con `numpy.vstack` al finalizar.
4. **Indexación FAISS:** construye un `VectorStore` con la dimensión del modelo (768) y le añade los vectores junto con la lista paralela de chunks. El índice FAISS y los metadatos se persisten en el directorio configurado (`data/index/`).
5. **Construcción del índice BM25:** construye un `BM25Index` a partir de los textos de todos los chunks y sus identificadores, y lo persiste

como `bm25_index.pkl` en el mismo directorio que el índice FAISS. Este índice léxico complementa al denso en la recuperación híbrida (Sección 5.8.5).

El modo `--dry-run` permite ejecutar únicamente las fases de carga y fragmentación sin generar *embeddings*, lo que resulta útil para validar que la configuración de *chunking* produce un número razonable de fragmentos con un tamaño medio adecuado antes de comprometer recursos de GPU.

Al finalizar, el script imprime un resumen con el número de documentos y chunks procesados, la dimensión vectorial, el tamaño del índice FAISS en disco, el tiempo de vectorización y el directorio de salida, proporcionando al operador toda la información necesaria para evaluar el resultado.

Resultados de la construcción del corpus

La ejecución del pipeline sobre la configuración definida en `sources.yaml` produce un corpus de 250 documentos distribuidos en las ocho categorías temáticas, con una longitud media de 11013 caracteres por documento y cero duplicados por *hash* de contenido. El rastreo descarga 407 páginas, de las cuales 118 se descartan como páginas índice, 19 como duplicados de contenido, 6 como formularios y 14 por longitud insuficiente. El Cuadro 6.2 detalla la distribución por categoría.

Categoría	Documentos
Guías docentes	124
Trámites	33
Calendario	29
Movilidad	19
Normativa	18
Estudiantes	13
Profesorado	9
TFG	5
Total	250

Cuadro 6.2: Distribución del corpus por categoría temática.

La predominancia de guías docentes refleja la estructura del sitio web de la ETSIIT, donde cada asignatura del Grado en Ingeniería Informática tiene una página con su guía docente completa. Las categorías con menor volumen (TFG, profesorado) corresponden a secciones del sitio con menos páginas, pero contienen información de alto valor para las consultas del sistema RAG.

6.3.9. Trazabilidad parcial: ingesta

La Tabla 6.3 vincula cada clase de implementación con el componente de diseño que materializa y los requisitos funcionales que satisface, según la matriz global del Capítulo 4.

Módulo	Componente diseño	RF cubiertos	RNF cubiertos
<code>fetcher.py</code>	Fetcher (Sec. 5.6.3)	RF-04	RNF-09
<code>parser.py</code>	Parser (Sec. 5.6.4)	RF-04	—
<code>cleaner.py</code>	Cleaner (Sec. 5.6.5)	RF-04, RF-07	—
<code>chunker.py</code>	Chunker (Sec. 5.6.6)	RF-09	—
<code>models.py</code>	Esquema Chunk (Sec. 5.6)	RF-04	RNF-10
<code>build_corpus.py</code>	Estrategia crawl (Sec. 5.6.1)	RF-04	RNF-09, RNF-11
<code>review_corpus.py</code>	Validación corpus (Sec. 5.6.1)	RF-06, RF-07, RF-08	RNF-08
<code>inject_doc.py</code>	Inserción de doc. verificado (Sec. 5.6.1)	RF-05	—
<code>index_corpus.py</code>	Flujo indexación (Sec. 5.6.1)	RF-09	RNF-09

Cuadro 6.3: Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de ingesta.

6.4. Capa de indexación

La capa de indexación traduce los chunks de texto producidos por la ingesta a vectores densos almacenados en un índice FAISS, materializando el **OE3** (*Sistema de indexación vectorial con chunking optimizado*) a través de los componentes *Embedder* y *VectorStore* descritos en las Secciones 5.7.1 y 5.7.2. Ambos residen en el subpaquete `graia.indexacion` y se comunican a través de matrices NumPy de forma $(n, 768)$, lo que permite intercambiar el modelo de embeddings sin alterar el almacén vectorial.

6.4.1. Embedder: vectorización con E5 multilingüe

El *Embedder* encapsula la carga y uso de `intfloat/multilingual-e5-base` a través de `sentence-transformers`. Su decisión de diseño más relevante es la gestión de los *prefijos* obligatorios de E5, `"query: "` para consultas y `"passage: "` para chunks, que se anteponen automáticamente para evitar errores de uso:

```

class Embedder:
    def __init__(self, model_name, query_prefix="query: ",
                  passage_prefix="passage: ",
                  batch_size=32, normalize=True):
        from sentence_transformers import SentenceTransformer
        self.model = SentenceTransformer(model_name)
        self.query_prefix = query_prefix
        self.passage_prefix = passage_prefix
        self.normalize = normalize
        self.dim = self.model.get_sentence_embedding_dimension()

    def encode_passages(self, texts):
        prefixed = [f"{self.passage_prefix}{t}" for t in texts]
        return self._encode(prefixed)

    def encode_query(self, query):
        return self._encode([f"{self.query_prefix}{query}"])[0]

```

Figura 6.15: Clase Embedder: prefijos automáticos y carga diferida del modelo.

Tres decisiones merecen justificación:

1. **Carga diferida (*lazy import*).** La sentencia `from sentence_transformers import SentenceTransformer` se ejecuta dentro de `__init__` y no al nivel del módulo. Esto evita que la mera importación de `graia.indexacion` desencadene la carga de PyTorch y los pesos del modelo (≈ 500 MB), lo que aumentaría inaceptablemente el tiempo de arranque de los tests y de la interfaz Streamlit.
2. **Normalización L2 obligatoria.** El parámetro `normalize_embeddings=True` se pasa directamente a `model.encode()`, delegando la normalización en `sentence-transformers` (que la aplica en GPU si está disponible). Vectores con norma unitaria son requisito del `IndexFlatIP` de FAISS: el producto interno de dos vectores normalizados equivale a la similitud coseno, pero se computa más rápido al evitar la división por normas en tiempo de búsqueda.
3. **Separación semántica de métodos.** Se ofrecen `encode_passages()` y `encode_query()` en lugar de un único `encode()` genérico. La razón es que confundir el prefijo, usar `"passage:"` para una consulta o viceversa, degrada el recall entre un 8 y un 15 % según los experimentos reportados por Wang et al. [91]. La API con dos métodos elimina de raíz esta fuente de error.

6.4.2. VectorStore: índice FAISS con persistencia

El `VectorStore` gestiona un `IndexFlatIP` de FAISS junto con una lista paralela de metadatos que vincula cada vector con su `Chunk` de origen. El fragmento central es el método de búsqueda:

```
class VectorStore:
    def __init__(self, dim: int):
        self.dim = dim
        self.index = faiss.IndexFlatIP(dim)
        self.metadata: list[dict] = []

    def search(self, query_vector, k=20):
        if query_vector.ndim == 1:
            query_vector = query_vector.reshape(1, -1)
        k = min(k, self.index.ntotal)
        scores, indices = self.index.search(
            query_vector.astype(np.float32), k)
        return [(self.metadata[i], float(s))
                for i, s in zip(indices[0], scores[0])
                if i >= 0]
```

Figura 6.16: Método `search()` del `VectorStore`: búsqueda top- k con FAISS.

La comprobación `if i >= 0` es necesaria porque FAISS devuelve -1 en las posiciones donde no puede llenar los k resultados (por ejemplo, si $k > n_{\text{total}}$). Se elige `IndexFlatIP` (fuerza bruta) sobre índices aproximados como `IndexIVFFlat` o `IndexHNSW` porque, como se argumentó en la Sección 5.7.2, el volumen esperado del corpus ($\sim 10^4$ chunks) hace que la búsqueda exacta sea suficientemente rápida (< 5 ms en la RTX 4070 Super) y elimina dos fuentes de complejidad: el entrenamiento del cuantizador y el parámetro `nprobe`.

6.4.3. Persistencia: separación índice/metadatos

El diseño de la persistencia (Sección 5.7.5) separa el almacenamiento en dos artefactos:

```

def save(self, directory):
    directory = Path(directory)
    directory.mkdir(parents=True, exist_ok=True)
    faiss.write_index(self.index,
                      str(directory / "index.faiss"))
    (directory / "index.meta.json").write_text(
        json.dumps(self.metadata, ensure_ascii=False),
        encoding="utf-8")

@classmethod
def load(cls, directory):
    index = faiss.read_index(
        str(Path(directory) / "index.faiss"))
    metadata = json.loads(
        (Path(directory) / "index.meta.json").read_text())
    store = cls(dim=index.d)
    store.index = index
    store.metadata = metadata
    return store

```

Figura 6.17: Persistencia del VectorStore: índice binario FAISS + metadatos JSON.

Esta separación aporta tres ventajas frente a un único fichero serializado con `pickle`: (i) el índice FAISS se lee con `mmap`, lo que reduce el consumo de RAM al cargar corpus grandes; (ii) los metadatos en JSON son inspeccionables sin código, facilitando la depuración; y (iii) se puede reconstruir el índice desde cero (por ejemplo, al cambiar de modelo de embeddings) sin perder los metadatos textuales.

6.4.4. Trazabilidad parcial: indexación

Módulo	Componente diseño	RF cubiertos	RNF cubiertos
<code>embedder.py</code>	Embeddings E5 (Sec. 5.7.1)	RF-09	RNF-07
<code>vector_store.py</code>	Índice FAISS (Sec. 5.7.2)	RF-09	RNF-02, RNF-10

Cuadro 6.4: Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de indexación.

6.5. Capa de recuperación

La capa de recuperación transforma la consulta del usuario en un conjunto reducido y diverso de chunks contextuales que alimentará al LLM,

materializando el **OE4** (*Motor de recuperación de fragmentos relevantes*). Implementa el flujo de recuperación híbrida descrito en la Sección 5.8.1: codificación de la consulta, búsqueda densa (FAISS) y léxica (BM25) en paralelo, fusión mediante Reciprocal Rank Fusion (RRF), filtrado por umbral τ , reordenación MMR y selección final de k chunks. El código se distribuye en tres módulos: `bm25.py` (índice léxico BM25), `mmr.py` (algoritmo MMR puro) y `retriever.py` (orquestación del flujo completo con fusión RRF).

6.5.1. Algoritmo MMR

La implementación de *Maximal Marginal Relevance* [47] opera sobre matrices precalculadas para evitar re-computar embeddings durante la selección iterativa:

```
def mmr_rerank(query_similarities, doc_embeddings,
               k, lambda_param=0.7):
    doc_sim_matrix = doc_embeddings @ doc_embeddings.T
    selected, remaining = [], set(range(len(query_similarities)))

    for _ in range(k):
        best_idx, best_score = -1, -float("inf")
        for idx in remaining:
            relevance = query_similarities[idx]
            redundancy = (max(doc_sim_matrix[idx, s]
                             for s in selected)
                         if selected else 0.0)
            score = (lambda_param * relevance
                    - (1 - lambda_param) * redundancy)
            if score > best_score:
                best_score, best_idx = score, idx
        selected.append(best_idx)
        remaining.discard(best_idx)
    return selected
```

Figura 6.18: Implementación del algoritmo MMR con matrices precalculadas.

La complejidad del bucle es $\mathcal{O}(k \cdot n)$ donde n es el número de candidatos tras el filtrado por umbral. Para el escenario esperado ($n \leq 20$, $k = 5$) esto equivale a ≤ 100 iteraciones, completadas en microsegundos. Se descartó vectorizar el bucle interno con NumPy porque la ganancia sería imperceptible en este rango y oscurecería la legibilidad del algoritmo.

La multiplicación `doc_embeddings @ doc_embeddings.T` precalcula la matriz completa de similitudes entre candidatos una sola vez, fuera del bu-

cle. Esto es eficiente porque los vectores ya están normalizados (requisito heredado del `Embedder`), de modo que el producto matricial equivale directamente a la similitud coseno.

6.5.2. Canal léxico: índice BM25

El módulo `bm25.py` implementa el canal léxico de la recuperación híbrida mediante la clase `BM25Index`, que encapsula la librería `rank_bm25` [43]. El índice se construye durante la fase de indexación (`index_corpus.py`) y se persiste como fichero `bm25.index.pkl` mediante `pickle`, cargándose en memoria al arrancar la aplicación junto al índice FAISS.

```
class BM25Index:
    def build(self, texts, chunk_ids):
        tokenized = [_tokenize(t) for t in texts]
        self._bm25 = BM25Okapi(tokenized)
        self._chunk_ids = list(chunk_ids)

    def search(self, query, k=20):
        scores = self._bm25.get_scores(_tokenize(query))
        top = scores.argsort()[::-1][:k]
        return [(self._chunk_ids[i], float(scores[i]))
                for i in top if scores[i] > 0]
```

Figura 6.19: Clase `BM25Index`: construcción y búsqueda léxica.

La tokenización (`_tokenize()`) aplica *lowercase* y extracción de tokens alfanuméricos mediante expresión regular, sin *stemming*. Esta decisión es deliberada: el *stemming* en español reduciría la capacidad de matching exacto con códigos de asignatura (IC, FFT) y siglas institucionales (ETSIIT, CRUE), que constituyen precisamente los términos donde el canal léxico aporta más valor frente al canal denso.

6.5.3. Fusión híbrida mediante RRF

La función `_reciprocal_rank_fusion()` fusiona los rankings denso y léxico sin requerir normalización de *scores*, implementando la fórmula descrita en la Sección 5.8.5:


```
def _reciprocal_rank_fusion(rankings, k_rrf=60):
    rrf_scores = {}
    for ranking in rankings:
        for rank_0, (chunk_id, _score) in enumerate(ranking):
            rank_1 = rank_0 + 1 # RRF usa ranks 1-indexed
            rrf_scores[chunk_id] = (
                rrf_scores.get(chunk_id, 0.0)
                + 1.0 / (k_rrf + rank_1)
            )
    return sorted(rrf_scores.items(),
                  key=lambda x: x[1], reverse=True)
```

Figura 6.20: Implementación de Reciprocal Rank Fusion (RRF).

La constante $k_{\text{rrf}} = 60$ se toma del artículo original de RRF (Cormack *et al.* [93]) y se expone como parámetro configurable en `default.yaml`. Nótese que los documentos que aparecen en ambos rankings reciben contribuciones sumadas, lo que naturalmente promueve los fragmentos con evidencia tanto semántica como léxica.

6.5.4. Retriever: orquestación del flujo completo

El `Retriever` encadena los pasos del flujo de recuperación híbrida en una función pura `retrieve()` parametrizada por los valores del YAML:

```

def retrieve(query, embedder, store, *,
            bm25_index=None, use_hybrid=False,
            rrf_k=60, k_candidates=20, k_final=5,
            similarity_threshold=0.50, mmr_lambda=0.7):
    query_vector = embedder.encode_query(query)
    raw = store.search(query_vector, k=k_candidates)
    if not raw:
        return []

    if use_hybrid and bm25_index is not None:
        bm25_results = bm25_index.search(query, k_candidates)
        dense_ranking = [(m["chunk_id"], s) for m, s in raw]
        fused = _reciprocal_rank_fusion(
            [dense_ranking, bm25_results], k_rrf=rrf_k)
        meta_by_id = {m["chunk_id"]: (m, s) for m, s in raw}
        raw = [(meta_by_id[cid]) for cid, _ in fused
                if cid in meta_by_id][:k_candidates]

    filtered = [(m, s) for m, s in raw
                if s >= similarity_threshold]
    if not filtered:
        return []

    candidate_embs = embedder.encode_passages(
        [m["text"] for m, _ in filtered])
    indices = mmr_rerank(
        np.array([s for _, s in filtered]),
        candidate_embs, k_final, mmr_lambda)
    return [RetrievedChunk(...) for i in indices]

```

Figura 6.21: Función `retrieve()`: flujo completo de recuperación híbrida.

Cuatro aspectos de esta implementación requieren justificación:

1. **Fusión RRF antes del filtrado por umbral.** La fusión se aplica sobre los rankings crudos de ambos canales para que documentos que aparecen en ambos rankings se beneficien del efecto de “doble evidencia” de RRF. Solo después se filtra por el *score* denso (τ), que sigue siendo la métrica de calidad semántica del fragmento.
2. **Compatibilidad hacia atrás.** Si `use_hybrid` es `False` o no se proporciona `bm25_index`, el flujo se reduce al canal denso puro, idéntico a la versión previa del sistema. Esto permite desactivar el modo híbrido sin modificar código.

3. **Re-codificación de candidatos para MMR.** Los embeddings de los chunks candidatos no se almacenan en el `VectorStore` (que solo guarda los metadatos textuales). Por tanto, se re-codifican con `encode_passages()` tras el filtrado por umbral. En el caso esperado (≤ 20 candidatos) esto añade ~ 15 ms en GPU, un coste aceptable frente a la alternativa de duplicar el almacenamiento de embeddings.
4. **Umbral τ aplicado *antes* de MMR.** El filtrado se realiza antes de la reordenación, no después, siguiendo la Sección 5.8.3. La razón es que pasar a MMR candidatos de baja relevancia podría sesgar la diversificación: un chunk irrelevante pero vectorialmente distinto podría desplazar a uno relevante.

6.5.5. Enrutamiento, intención y enriquecimiento de la consulta

Sobre el flujo base (búsqueda híbrida, umbral y MMR) se implementan los componentes de recall y precisión descritos en las Secciones 5.8.6–5.8.8. Todos son módulos deterministas, sin estado y sin dependencia de GPU, lo que facilita su prueba unitaria aislada:

- **Enrutador** (`query_router.route_query`). Un conjunto de reglas (`nombre`, `patrón`, `{categoría: boost}`) clasifica la consulta; las coincidencias se acumulan tomando el *boost* máximo por categoría. El `Retriever` usa el resultado para la *inyección focalizada*: recorre los metadatos del índice y añade al *pool* los fragmentos de las categorías predichas, calculando su *score* denso bajo demanda.
- **Expansión y reconocimiento de asignaturas** (`expand_abbreviations`, `detect_subject_siglas`). Reconocen la asignatura tanto por su sigla como por su nombre completo (insensible a tildes mediante normalización Unicode) y enriquecen la consulta con la forma que falte, de modo que se active el realce por sigla exacta y el matching estructurado.
- **Intención de listado** (`query_intent.is_listing_query`) eleva k a su valor ampliado y, para listados de asignaturas, dispara la inyección de registros-resumen.
- **Enriquecimiento por historial** (`contextual_query.enrich_query_with_history`) arrastra a la consulta *ya resuelta*, empleada tanto en recuperación como en generación, las entidades de dominio (asignatura, grupo, subgrupo, tipo) de los turnos previos cuando la consulta es un seguimiento elíptico.

6.5.6. Reranking con *cross-encoder*

El *reranker* (`reranker.rerank`) carga de forma *lazy* un *cross-encoder* multilingüe (`mmarco-mMiniLMv2`) [99, 100] y puntúa los pares (`consulta`, `fragmento`) de los candidatos que superaron el umbral, reordenándolos antes del MMR (Sección 5.8.10). La carga del modelo se cachea en una variable de módulo y degrada con elegancia: si el modelo multilingüe no puede descargarse, se recurre automáticamente al modelo en inglés, y si el reranking falla por completo, el *Retriever* mantiene el orden previo en lugar de interrumpir la consulta. El parámetro `use_reranker` de la configuración permite activarlo o desactivarlo sin tocar código, conforme al patrón *Strategy*.

6.5.7. Manejo de incertidumbre: el caso “sin resultados”

El diseño del sistema (RNF-06, Sección 5.8.3) exige que GRAIA declare explícitamente cuando no dispone de información suficiente. La implementación lo garantiza en dos puntos: si el `VectorStore` está vacío, `retrieve()` devuelve una lista vacía; si todos los candidatos caen por debajo de τ , se devuelve igualmente una lista vacía. En ambos casos, el módulo de generación (Sección 6.6) detecta la ausencia de contexto y genera una respuesta de redirección en lugar de especular, cumpliendo el principio de incertidumbre explícita del perfil (*persona*) definido para GRAIA.

6.5.8. Capas de calidad: gate de ámbito, filtros de coherencia y control de ruido

Sobre el flujo base se implementan los tres mecanismos de calidad diseñados en las Secciones 5.8.11–5.8.13.

Gate de ámbito. El módulo `scope_classifier.py` expone `is_in_scope()`, que recibe la consulta, los fragmentos ya recuperados y el cliente LLM. Aplica primero las exenciones deterministas (preguntas *meta* mediante `is_meta_query` y seguimientos anafóricos mediante el patrón de `contextual_query`); a continuación, si el *reranking* está activo, decide por el margen del mejor fragmento (umbrales `high_margin/low_margin` de la sección `scope_gate` del *config*). El auto-rechazo por margen bajo se condiciona, además, a que `query_router.route_query` no haya enrutado la consulta: si el enrutador la asigna a una categoría académica (señal de dominio determinista), no se rechaza por margen sino que se difiere al clasificador LLM, evitando falsos rechazos de consultas válidas con margen negativo (p.ej. “¿cuándo abre la secretaría?”). El clasificador LLM se invoca, pues, en la banda intermedia y en las consultas enrutadas con margen bajo. Para que esa clasificación sea

barata, `OllamaClient.generate` admite un parámetro `num_predict` que acota la generación a una sola palabra. El *gate* se cablea en el *pipeline* de la interfaz (`app.py`) y en el guion de evaluación (`evaluate.py`): si la consulta queda fuera de ámbito, ambos emiten la constante `OUT_OF_SCOPE_ANSWER` (la frase canónica de abstención) *sin* invocar al generador. La política ante un error del clasificador es *fail-open*.

Filtros de coherencia. Se implementan en `retriever.py` como dos pasos deterministas sobre el conjunto de candidatos. El de *asignatura* usa `query_router.detect_subject_siglas` para detectar las asignaturas nombradas en la consulta y la función auxiliar `_chunk_subject_siglas`, que deriva la asignatura de un fragmento de sus metadatos `siglas` o, en su defecto, de su título o primera línea (guías docentes, calendario); descarta los fragmentos cuya asignatura no coincide, exentos los catálogos. El de *tipo* usa `query_intent.desired_structured_tipo`, que devuelve `horario`, `calendario` o `None` (intención ambigua), y descarta los registros estructurados del tipo en conflicto.

Control de ruido. Tras el *reranking*, cuando el enrutador ha asignado categorías, a los fragmentos de categoría no enrutada se les resta `off_category_penalty` (parámetro del *config*, fijado en 1,5) sobre su *score*, reordenando el *pool* antes del MMR. La penalización demota sin eliminar, de modo que no introduce falsos rechazos.

6.5.9. Trazabilidad parcial: recuperación

Módulo	Componente diseño	RF cubiertos	RNF cubiertos
<code>bm25.py</code>	Índice BM25 (Sec. 5.8.5)	RF-02	RNF-02
<code>mmr.py</code>	MMR reranker (Sec. 5.8.4)	RF-02	RNF-02
<code>retriever.py</code>	Retriever + RRF + filtros de coherencia y ruido (Sec. 5.8.1, 5.8.12, 5.8.13)	RF-02	RNF-01, RNF-02
<code>scope_classifier.py</code>	Gate de ámbito (Sec. 5.8.11)	RF-02	RNF-02

Cuadro 6.5: Trazabilidad implementación ↔ diseño del subsistema de recuperación.

6.6. Capa de generación

La capa de generación transforma el contexto recuperado en una respuesta en lenguaje natural con citas trazables, materializando el **OE5** (*Integración de LLM local con contexto recuperado*) y el **OE6** (*Sistema de citación trazable*). Implementa los tres componentes descritos en las Secciones 5.9 y 5.10: el cliente Ollama, el constructor de prompts y el validador de citas. El código se distribuye en tres módulos dentro de `graia.generacion`,

orquestrados por la función de conveniencia `generate()`.

6.6.1. Cliente Ollama

El `OllamaClient` encapsula la comunicación con el LLM local a través del SDK oficial de Ollama, ofreciendo dos modos de invocación: `generate()` (batch, para evaluación) y `generate_stream()` (streaming, para la interfaz). El fragmento central del modo batch ilustra la configuración de parámetros:

```
class OllamaClient:
    def __init__(self, model, temperature=0.2,
                 top_p=0.9, max_tokens=1024,
                 stop("</respuesta>"),
                 host="http://localhost:11434"):
        self.client = ollama.Client(host=host)
        self.model = model
        ...

    def generate(self, system_prompt, user_message):
        response = self.client.chat(
            model=self.model,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_message}],
            options={"temperature": self.temperature,
                    "top_p": self.top_p,
                    "num_predict": self.max_tokens,
                    "stop": self.stop})
        return GenerationResult(text=response[...], ...)
```

Figura 6.22: Cliente Ollama: configuración de parámetros y modo batch.

Dos decisiones de implementación merecen justificación:

1. **SDK oficial vs. peticiones HTTP crudas.** Se emplea el paquete `ollama` (SDK oficial en Python) en lugar de invocar la API REST directamente con `requests`. El SDK gestiona internamente la serialización JSON, el streaming de Server-Sent Events y la reconexión, lo que reduce el código del cliente a su lógica esencial. La alternativa, `requests` + `sseclient`, requeriría reimplementar la lógica de streaming y no aportaría ningún beneficio funcional.
2. **Secuencia de parada «/respuesta».** El modelo recibe la instrucción de cerrar su respuesta con esta etiqueta en el prompt de sistema.

Si el LLM la emite, Ollama detiene la generación inmediatamente, evitando texto residual tras la respuesta útil. Esto es especialmente importante con modelos cuantizados (Q4_K_M) que ocasionalmente producen texto degenerado tras agotar la información del contexto.

6.6.2. Constructor de prompts

El `PromptBuilder` (Sección 5.9.3) construye el par (`system_prompt`, `user_message`) que alimenta al LLM. El *prompt* de sistema codifica el perfil (*persona*) de GRAIA con sus seis reglas obligatorias; el mensaje del usuario inyecta el contexto recuperado con markers numerados:

```
def build_context_block(chunks):
    lines = ["CONTEXTO:"]
    for i, chunk in enumerate(chunks, start=1):
        header = f"[{i}]"
        if chunk.title:
            header += f" {chunk.title}"
        header += f" (Fuente: {chunk.source_url})"
        lines.append(header)
        lines.append(chunk.text)
        lines.append("")
    return "\n".join(lines)

def build_messages(query, chunks):
    if not chunks:
        return _NO_CONTEXT_PROMPT, query
    context = build_context_block(chunks)
    user_msg = f"{context}\n\nPREGUNTA DEL USUARIO:\n{query}"
    return _SYSTEM_PROMPT, user_msg
```

Figura 6.23: Construcción del bloque de contexto con markers [n].

La separación explícita entre `CONTEXTO:` y `PREGUNTA DEL USUARIO:` dentro del mensaje del usuario, en lugar de mezclarlos en un solo bloque, sigue la recomendación de Liu et al. [39]: los modelos de lenguaje aprovechan mejor el contexto cuando este se presenta en un bloque delimitado al inicio del mensaje, antes de la pregunta. En pruebas preliminares sobre 50 consultas, esta estructura redujo en un 12 % las respuestas que ignoraban chunks relevantes respecto a un formato intercalado.

Cuando la lista de chunks está vacía (todos filtrados por el umbral τ), se inyecta un prompt de sistema alternativo (`_NO_CONTEXT_PROMPT`) que instruye al LLM a declarar la ausencia de información y redirigir al usuario,

materializando el principio de incertidumbre explícita de la persona GRAIA.

6.6.3. Validador de citas

El validador post-hoc (Sección 5.10) extrae los markers [n] de la respuesta generada, los contrasta con el mapa de fuentes y produce un `CitationReport`:

```
_CITATION_PATTERN = re.compile(r"\[(\d+)\]")

def validate_citations(response_text, source_map):
    found = sorted(set(
        int(m) for m in _CITATION_PATTERN.findall(response_text)))
    valid = [m for m in found if m in source_map]
    invalid = [m for m in found if m not in source_map]

    clean_text = response_text
    for inv in invalid:
        clean_text = clean_text.replace(f"[{inv}]", "")

    return CitationReport(
        valid_markers=valid,
        invalid_markers=invalid,
        sources=[{"marker": f"[{m}]",
                  "url": source_map[m].source_url}
                 for m in valid],
        clean_text=clean_text)
```

Figura 6.24: Validación post-hoc de citas: detección y eliminación de markers alucinados.

El validador cumple una función de *red de seguridad*: aunque el prompt instruye al LLM a citar solo fuentes reales, los modelos de lenguaje pueden alucinar markers inexistentes (por ejemplo, [7] cuando solo hay 5 chunks en el contexto). La validación post-hoc detecta estos casos, los registra en el informe y los elimina del texto mostrado al usuario, siguiendo el enfoque de Rashkin et al. [48] para la medición de atribución en modelos generativos.

La decisión de *eliminar* las citas inválidas en lugar de *marcarlas visualmente* se justifica por el perfil del usuario objetivo (estudiante universitario): mostrar [7] (*fuentes no verificadas*) podría generar confusión o desconfianza. La eliminación silenciosa, combinada con el registro en el `CitationReport` (accesible en los logs JSONL), ofrece una experiencia limpia al usuario sin sacrificar la auditabilidad para el desarrollador.

6.6.4. Post-procesado determinista de la respuesta

Entre la generación del LLM y su presentación al usuario se aplica la cadena de post-procesado determinista descrita en la Sección 5.10.4. La implementación encadena, en orden, funciones puras sobre el texto generado:

- **clean_answer** recorta preámbulos de “razonamiento en voz alta” y muletillas iniciales (“según la información proporcionada. . .”), el *meta-comentario* en que el modelo habla de sus fragmentos (“la respuesta a tu pregunta es. . .”, “se encuentra en el fragmento [n]. . .”) y las coletillas finales, mediante listas de patrones. Sobre todo, *canoniza la abstención*: si el texto encabeza declarando no disponer del dato, detector **is_abstention**, que reconoce las variantes “no dispongo de”, “no hay”, “no se proporciona/especifica/menciona” información, lo colapsa a la frase canónica única. **is_abstention** es además la *fente única de verdad* que comparten la interfaz y el guion de evaluación para reconocer una abstención (evitando, por ejemplo, marcar como rechazo una respuesta válida que de pasada diga “no se menciona el aula”).
- **deduplicate_sentences** elimina frases redundantes; su criterio clave es que dos frases solo se consideran equivalentes si comparten los mismos *tokens numéricos*, lo que evita colapsar listas legítimas (el horario de cada grupo) que comparten estructura pero difieren en valores.
- **normalize_markdown_lists** convierte las viñetas en línea (*, +) en listas Markdown correctas, preservando rangos horarios y marcadores de cita.
- Tras validar las citas, si la respuesta es sustantiva pero carece de marcadores, **recover_sources** atribuye la fuente por solapamiento léxico; esta atribución se inhibe en respuestas de abstención (**is_no_info_answer**), declinaciones de opinión (**is_subjective_decline**) y preguntas sobre el propio asistente (**is_meta_query**).
- Finalmente se añade un cierre cortés variado acorde a la persona del asistente, omitido en las respuestas de abstención.

Esta cadena materializa la decisión de diseño de respaldar el *prompt* con correcciones deterministas fiables: lo que el modelo de 8 B no garantiza por instrucción, se asegura por código.

6.6.5. Trazabilidad parcial: generación

Módulo	Componente diseño	RF cubiertos	RNF cubiertos
<code>ollama_client.py</code>	Cliente LLM (Sec. 5.9.2)	RF-02	RNF-01, RNF-09
<code>prompt_builder.py</code>	Prompt GRAIA (Sec. 5.9.3)	RF-02	RNF-04, RNF-05, RNF-07
<code>citation_validator.py</code>	Citación híbrida (Sec. 5.10)	RF-03	RNF-05

Cuadro 6.6: Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de generación.

6.7. Capa de interfaz conversacional

La capa de interfaz materializa el **OE7** (*Interfaz web funcional*) según el diseño descrito en la Sección 5.11, mediante una aplicación Streamlit que expone el pipeline completo de GRAIA a través de un chat conversacional. El módulo `graia.interfaz.app` actúa como *orquestador final*: carga la configuración, inicializa los componentes cacheados e invoca secuencialmente las capas de recuperación y generación por cada consulta del usuario.

6.7.1. Arquitectura de la aplicación

Streamlit ejecuta el script `app.py` de arriba a abajo en cada interacción del usuario (*rerun model*). Para evitar reinicializar objetos costosos, el modelo de embeddings, el índice FAISS y el cliente Ollama, se emplea el decorador `@st.cache_resource`, que almacena la instancia en memoria entre reruns:

```
@st.cache_resource(show_spinner="Cargando modelo de embeddings...")
def load_embedder(cfg: dict) -> Embedder:
    emb_cfg = cfg["embeddings"]
    return Embedder(
        model_name=emb_cfg["model_name"],
        query_prefix=emb_cfg["query_prefix"],
        passage_prefix=emb_cfg["passage_prefix"],
        batch_size=emb_cfg["batch_size"],
        normalize=emb_cfg["normalize"],
    )

@st.cache_resource(show_spinner="Cargando índice vectorial...")
def load_store(cfg: dict) -> VectorStore:
    return VectorStore.load(cfg["paths"]["index"])
```

Figura 6.25: Inicialización cacheada de componentes con `@st.cache_resource`.

La elección de `@st.cache_resource` sobre `@st.cache_data` es deliberada: los objetos cacheados (`Embedder`, `VectorStore`, `OllamaClient`) son *singletons mutables* que mantienen conexiones de red o memoria mapeada, por lo que no deben ser serializados ni copiados [101]. Se ha verificado empíricamente que, en la RTX 4070 Super del entorno de desarrollo, la carga inicial del modelo `multilingual-e5-base` tarda ≈ 3 s, mientras que las sucesivas invocaciones se resuelven en < 1 ms gracias al cache.

6.7.2. Pipeline de consulta

Cada consulta del usuario desencadena la función `handle_query()`, que implementa el patrón *Pipe and Filter* descrito en la Sección 5.4: recuperación $\rightarrow$ construcción de prompt $\rightarrow$ generación con streaming $\rightarrow$ validación de citas. El fragmento clave muestra la orquestación del streaming y la inyección del bloque de fuentes:

```
def handle_query(query, embedder, store, client, cfg):
    # 1. Recuperación
    chunks = retrieve(query, embedder, store,
                      k_candidates=rec_cfg["k_candidates"],
                      k_final=rec_cfg["k_final"],
                      similarity_threshold=rec_cfg["similarity_threshold"],
                      mmr_lambda=rec_cfg["mmr_lambda"])

    # 2. Prompt
    system_prompt, user_message = build_messages(query, chunks)
    source_map = get_source_map(chunks)
    # 3. Streaming
    with st.chat_message("assistant"):
        full_response = st.write_stream(
            client.generate_stream(system_prompt, user_message))
    # 4. Validación de citas
    report = validate_citations(full_response, source_map)
    sources_block = format_sources_block(report)
    if sources_block:
        st.markdown(sources_block)
```

Figura 6.26: Orquestación del pipeline en `handle_query()`.

Tres aspectos merecen justificación:

1. **Streaming con `st.write_stream`.** Se emplea el generador `generate_stream()` del `OllamaClient` (Sección 6.6.1) para que los tokens se muestren conforme el modelo los produce, reduciendo la latencia percibida por el usuario. `st.write_stream` acepta un iterador y devuelve el texto completo acumulado al finalizar, lo que permite encadenar directamente la validación de citas.

2. **Validación post-streaming.** Las citas se validan *después* de que el streaming termine, cuando ya se dispone del texto completo. Esto implica que el bloque de fuentes verificadas aparece tras una breve pausa al final de la respuesta, una decisión de UX aceptable frente a la alternativa de buffering completo, que eliminaría el beneficio del streaming.
3. **Estado de sesión y panel lateral.** El historial de la conversación activa se almacena en `st.session_state.messages`, una lista de diccionarios `{role, content}` compatible con la API de chat de Ollama. Adicionalmente, `st.session_state.conversations` mantiene una lista de conversaciones previas (cada una con su identificador, título y mensajes), y `current_conv_id` identifica la conversación activa. La función `render_sidebar()` genera el panel lateral izquierdo mostrado en la Figura 5.4: un encabezado “Historial”, un botón para iniciar una nueva conversación y un botón por cada conversación previa cuyo título se extrae automáticamente de la primera consulta del usuario. Al pulsar una conversación del historial, la actual se guarda mediante `_save_current_conversation()` y se carga la seleccionada. En cada *rerun*, `render_history()` recorre `messages` y muestra cada mensaje con `st.chat_message`, garantizando la persistencia visual durante la sesión.

6.7.3. Gestión de configuración

Toda la parametrización del sistema se centraliza en `config/default.yaml`, cargado una única vez mediante `load_config()`. Los hiperparámetros de cada capa (embeddings, recuperación, generación) se propagan a los constructores respectivos, eliminando constantes dispersas en el código y permitiendo experimentación sin modificar fuentes. El fichero YAML sigue la estructura descrita en la Sección 6.2.3 y se valida implícitamente por los constructores de cada componente (un valor ausente o mal tipado produce un `KeyError` o `TypeError` inmediato al arranque).

6.7.4. Trazabilidad parcial: interfaz

Módulo	Componente diseño	RF cubiertos	RNF cubiertos
<code>app.py</code>	Interfaz conversacional (Sec. 5.11)	RF-01, RF-02, RF-03	RNF-01, RNF-06, RNF-07

Cuadro 6.7: Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de interfaz.

6.8. Registro estructurado

El subsistema de registro implementa la observabilidad transversal descrita en la Sección 5.12, proporcionando logs estructurados en formato JSONL para todos los módulos de GRAIA. Se trata de un componente *cross-cutting*: no pertenece a ninguna capa funcional del pipeline, sino que es utilizado por todas ellas.

6.8.1. Formateador JSON personalizado

El núcleo del subsistema es `GraiaJsonFormatter`, una subclase de `jsonlogger.JsonFormatter` (del paquete `python-json-logger` [102]) que inyecta cuatro campos obligatorios en cada evento:

```
class GraiaJsonFormatter(jsonlogger.JsonFormatter):
    def add_fields(self, log_record, record, message_dict):
        super().add_fields(log_record, record, message_dict)
        log_record["timestamp"] = (
            datetime.now(timezone.utc).isoformat())
        log_record["level"] = record.levelname
        log_record["module"] = record.name
        if "event" not in log_record:
            log_record["event"] = record.getMessage()
```

Figura 6.27: Formateador JSONL con campos obligatorios de GRAIA.

La decisión de extender `python-json-logger` en lugar de implementar un formateador JSON desde cero responde a dos razones:

1. **Compatibilidad con el ecosistema logging.**

Al heredar de `jsonlogger.JsonFormatter`, el formateador se integra sin fricción con el módulo `logging` estándar de Python: cualquier handler (consola, fichero, rotativo) puede usarlo directamente.

2. **Serialización automática de extras.**

Los campos adicionales pasados a través de `extra={...}` en las llamadas `logger.info()` se incluyen automáticamente en la salida JSON sin código adicional. Esto permite que cada capa del pipeline inyecte métricas propias, `query_id`, `latency_ms`, `num_chunks`, de forma transparente.

6.8.2. Configuración del logger

La función `setup_logging()` configura el logger raíz "graia" con un handler de consola (`stderr`) y, opcionalmente, un handler de fichero JSONL:

```
def setup_logging(log_dir="logs", level="INFO",
                  log_to_file=True):
    root = logging.getLogger("graia")
    root.setLevel(getattr(logging, level.upper()),
                  logging.INFO)
    if root.handlers:          # idempotencia
        return
    formatter = GraiaJsonFormatter()
    console = logging.StreamHandler(sys.stderr)
    console.setFormatter(formatter)
    root.addHandler(console)
    if log_to_file:
        log_path = Path(log_dir)
        log_path.mkdir(parents=True, exist_ok=True)
        fh = logging.FileHandler(
            log_path / "graia.jsonl", encoding="utf-8")
        fh.setFormatter(formatter)
        root.addHandler(fh)
```

Figura 6.28: Configuración idempotente del sistema de logging.

La guarda de idempotencia (`if root.handlers: return`) evita la duplicación de handlers cuando Streamlit re-ejecuta el script en cada interacción del usuario. Sin esta comprobación, cada rerun añadiría un handler adicional, produciendo líneas duplicadas en los logs.

Se descartó el uso de `RotatingFileHandler` en esta versión inicial por simplicidad operativa: el volumen esperado de logs en un despliegue académico es reducido (< 100 MB/mes). No obstante, la función acepta un parámetro `log_dir` que facilita la migración a rotación configurada si fuera necesario.

6.8.3. Identificadores de correlación

La función `generate_query_id()` produce un identificador hexadecimal de 12 caracteres basado en UUID v4, suficiente para correlacionar todos los eventos de log asociados a una misma consulta del usuario a lo largo del pipeline:

```
def generate_query_id() -> str:
```

```
return uuid.uuid4().hex[:12]
```

La longitud de 12 caracteres hexadecimales proporciona $16^{12} \approx 2,8 \times 10^{14}$ identificadores posibles, un espacio de colisión despreciable para el volumen de consultas esperado. Se prefirió UUID v4 (aleatorio) sobre UUID v1 (basado en timestamp) para evitar la filtración de información temporal en los propios identificadores.

6.8.4. Trazabilidad parcial: registro

Módulo	Componente diseño	RF cubiertos	RNF cubiertos
<code>logger.py</code>	Registro estructurado (Sec. 5.12)	RF-10	RNF-08

Cuadro 6.8: Trazabilidad implementación $\leftrightarrow$ diseño del subsistema de registro.

6.9. Trazabilidad global de la implementación

Las secciones anteriores han presentado tablas de trazabilidad parcial para cada subsistema. La Tabla 6.9 vincula cada objetivo específico del proyecto con los módulos de código que lo materializan, cerrando el ciclo de trazabilidad iniciado en el Capítulo 4. La Tabla 6.10 complementa esta visión con la cobertura detallada por requisitos funcionales y no funcionales.

OE	Descripción	Módulos de implementación
OE1	Revisión del estado del arte	Capítulo 3 (no genera código)
OE2	Pipeline de ingesta heterogénea	<code>fetcher.py</code> , <code>parser.py</code> , <code>cleaner.py</code> , <code>chunker.py</code> , <code>models.py</code> + scripts de orques- tación: <code>build_corpus.py</code> , <code>review_corpus.py</code>
OE3	Indexación vectorial	<code>embedder.py</code> , <code>vector_store.py</code> + <code>index_corpus.py</code>
OE4	Motor de recuperación	<code>retriever.py</code> , <code>mmr.py</code> , <code>bm25.py</code>
OE5	Integración de LLM local	<code>ollama_client.py</code> , <code>prompt_builder.py</code>
OE6	Sistema de citación trazable	<code>citation_validator.py</code> , <code>prompt_builder.py</code>
OE7	Interfaz web funcional	<code>app.py</code> (Streamlit)
OE8	Dataset y métricas	Capítulo 7
OE9	Evaluación comparativa	Capítulo 7

Cuadro 6.9: Correspondencia entre objetivos específicos y módulos de implementación.

Requisito	Tipo	Módulos que lo cubren
RF-01	Funcional	<code>app.py</code> (gestión de sesión e historial)
RF-02	Funcional	<code>retriever.py</code> , <code>bm25.py</code> , <code>mmr.py</code> , <code>ollama_client.py</code> , <code>prompt_builder.py</code> , <code>app.py</code>
RF-03	Funcional	<code>citation_validator.py</code> , <code>app.py</code> (enlaces a la fuente)
RF-04	Funcional	<code>fetcher.py</code> , <code>parser.py</code> , <code>cleaner.py</code> , <code>build_corpus.py</code>
RF-05	Funcional	<code>inject_doc.py</code> (inyección de documentos verificados)
RF-06	Funcional	<code>review_corpus.py</code> (informe y revisión)
RF-07	Funcional	<code>review_corpus.py --remove-*</code> , <code>cleaner.py</code>
RF-08	Funcional	<code>review_corpus.py --add-url</code>
RF-09	Funcional	<code>chunker.py</code> , <code>embedder.py</code> , <code>vector_store.py</code> , <code>index_corpus.py</code>
RF-10	Funcional	<code>evaluate.py</code> , <code>logger.py</code> (registro JSONL de interacciones)
RNF-01	No funcional	<code>app.py</code> (streaming), <code>ollama_client.py</code>
RNF-02	No funcional	<code>retriever.py</code> , <code>vector_store.py</code> (< 200 ms)
RNF-04	No funcional	<code>prompt_builder.py</code> (diseño anti-alucinación)
RNF-05	No funcional	<code>citation_validator.py</code> , <code>prompt_builder.py</code>
RNF-06	No funcional	<code>app.py</code> (diseño centrado en usuario)
RNF-07	No funcional	<code>embedder.py</code> (E5 multilingüe), <code>prompt_builder.py</code>
RNF-08	No funcional	<code>logger.py</code> (JSONL sin PII)
RNF-09	No funcional	Decisión arquitectónica transversal (Ollama local)
RNF-10	No funcional	Estructura modular por paquetes, <code>default.yaml</code> (configuración declarativa), suite de 90 tests
RNF-12	No funcional	<code>ollama_client.py</code> + <code>default.yaml</code> (Llama 3.1 8B Q4 en 12 GB)
RNF-13	No funcional	Pila de dependencias de código abierto (Ollama, FAISS, <i>sentence-transformers</i>)

Cuadro 6.10: Trazabilidad global: requisitos ↔ implementación.

Los requisitos no funcionales RNF-03 (precisión mínima) y RNF-11 (portabilidad a otros dominios) no se mapean a módulos individuales: el primero se evalúa de forma holística en el Capítulo 7, mientras que el segundo se garantiza mediante la arquitectura modular por capas descrita en el Capítulo 5.

6.10. Pruebas unitarias

La implementación se acompaña de una suite de **90 tests unitarios** ejecutados con `pytest`, distribuidos según la Tabla 6.11. El diseño de la suite sigue tres principios: (i) **aislamiento mediante mocks**, empleando objetos simulados para los recursos que requieren GPU o red [103]; (ii) **determinismo**, usando vectores de valor conocido y textos sintéticos controlados; y (iii) **cobertura de casos límite**, probando situaciones como índices vacíos, citas alucinadas o idempotencia de la configuración de logging.

Fichero de tests	Nº tests	Capa cubierta
<code>test_ingesta.py</code>	18	Ingesta (models, parser, cleaner, chunker)
<code>test_indexacion.py</code>	13	Indexación (VectorStore, Embedder)
<code>test_recuperacion.py</code>	13	Recuperación (MMR, Retriever)
<code>test_bm25.py</code>	20	BM25, RRF, recuperación híbrida
<code>test_generacion.py</code>	19	Generación (prompt, citas, cliente)
<code>test_registro.py</code>	7	Registro (formatter, logger, <code>query_id</code>)
Total	90	

Cuadro 6.11: Distribución de tests unitarios por capa del sistema.

A continuación se describe qué verifica cada grupo de tests.

Capa de ingesta (18 tests). Se organizan en cinco clases. `TestChunkModel` comprueba que los `chunk_id` son deterministas (mismo contenido y posición producen el mismo identificador) y que difieren al cambiar la posición, garantizando que reingestas sucesivas no dupliquen fragmentos. `TestHtmlParser` valida la extracción del cuerpo, la eliminación de `<nav>/<footer>` y la preservación del título. `TestCleaner` verifica la normalización Unicode NFC, el colapso de espacios, la eliminación de boilerplate y el descarte de documentos excesivamente cortos. `TestChunker` cubre la heurística de estimación de tokens, la división con solapamiento, la asignación de posiciones secuenciales, la unicidad de IDs y la fusión de chunks pequeños (fragmentos residuales menores que `min_chunk_tokens` se incorporan al chunk anterior). Un test de integración final ejecuta el pipeline completo `HTML → parse → clean → chunk`.

Capa de indexación (13 tests). `TestVectorStore` ejercita el almacén FAISS con vectores ortogonales de dimensión $d = 4$, lo que permite predecir de antemano qué vector será el resultado top-1. Se verifican el orden descendente de scores, la adición incremental, el manejo de k mayor que el corpus, y la persistencia `save/load` con resultados idénticos. `TestEmbedderInterface` valida que el módulo `Embedder` es importable y expone los métodos requeridos (`embed_texts`, `embed_query`) sin instanciar el modelo real.

Capa de recuperación (33 tests). La recuperación densa se verifica en `test_recuperacion.py`: 7 tests de MMR prueban los extremos de λ ($\lambda = 1$ equivale a top- k puro, $\lambda = 0$ maximiza diversidad) y casos límite; 6 tests del retriever usan un `MagicMock` como embedder para validar el tipo `RetrievedChunk`, el respeto de `k_final`, el filtrado por umbral y el comportamiento con almacén vacío. La recuperación léxica e híbrida se verifica en `test_bm25.py`: 5 tests de tokenización comprueban minúsculas, tildes, puntuación y preservación de números; 9 tests del índice BM25 usan un corpus de 8 documentos del dominio ETSIT para verificar recuperación correcta, scores descendentes, coincidencias exactas con códigos de asignatura (p. ej., “IC”), persistencia y validación de errores; 6 tests de RRF cubren ranking único, acuerdo y desacuerdo entre rankings, y el efecto cuantitativo de k_{rrf} sobre el suavizado de scores.

Capa de generación (19 tests). `TestPromptBuilder` verifica la construcción de marcadores numerados [1], [2], ... con sus fuentes, la distinción entre prompt con contexto y sin contexto, y la correspondencia

del mapa de fuentes. `TestCitationValidator` valida la detección de citas válidas e inválidas, la **eliminación automática de citas alucinadas** del texto limpio (si el LLM genera [99] sin fuente, el validador lo suprime), la deduplicación de marcadores y la corrección de la expresión regular. `TestOllamaClientInterface` comprueba la importabilidad del cliente y la estructura de `GenerationResult` sin requerir conexión a Ollama. Un test de integración ejecuta la cadena `prompt` $\rightarrow$ `contexto` $\rightarrow$ `citación` con una respuesta simulada del LLM, verificando que las citas válidas se preservan y las alucinadas se eliminan.

Capa de registro (7 tests). Se verifica que `get_logger` devuelve un logger hijo del espacio `graia`, que `generate_query_id` produce IDs de 12 caracteres únicos (100 IDs sin colisiones), que `GraiaJsonFormatter` incluye los campos obligatorios en JSON y preserva campos extras, y que `setup_logging` es idempotente (invocarla dos veces no duplica handlers, propiedad crítica dado el modelo de re-ejecución de Streamlit). Un test adicional verifica que el módulo `graia.interfaz.app` es importable sin lanzar Streamlit.

Todos los tests se ejecutan en menos de 3 segundos en CPU, sin requerir GPU ni servicios externos, mediante `pytest -v tests/` desde la raíz del proyecto.

6.11. Dificultades de implementación

Durante la implementación se encontraron varios retos técnicos que condicionaron decisiones de diseño:

1. **Incompatibilidad FAISS con índice vacío.** La llamada a `index.search(query, k=0)` provoca un `AssertionError` interno en FAISS. La solución fue añadir un retorno anticipado (`if k == 0: return []`) en `VectorStore.search()`, detectado gracias al test `test_retrieve_empty_store`.
2. **Heurística de tokenización para español.** La constante ≈ 4 caracteres/token, habitual para inglés, sobreestima ligeramente la longitud de los chunks en español (donde el ratio real es $\approx 3,8$). Se aceptó esta aproximación tras verificar que la desviación media era inferior al 8 % sobre un corpus de 200 fragmentos reales, y se documentó como parámetro ajustable en la configuración.
3. **Prefijos obligatorios de E5 multilingual.** El modelo `intfloat/multilingual-e5-base` requiere los prefijos `"query: "` y `"passage: "` para producir *embeddings* semánticamente correctos. La omisión accidental de estos prefijos degrada la calidad del retrieval de forma silenciosa (sin errores visibles), lo que dificultó la depuración inicial. La

solución fue encapsular la lógica de prefijos dentro del `Embedder`, exponiendo métodos separados `encode_query()` y `encode_passages()` que inyectan el prefijo automáticamente.

4. **Rerun model de Streamlit y duplicación de estado.** El modelo de ejecución de Streamlit, en el que el script se re-ejecuta por completo en cada interacción, exige una gestión cuidadosa del estado con `st.session_state` y de los recursos con `@st.cache_resource`. Sin estas precauciones, cada consulta recargaría el modelo de embeddings, el índice FAISS y la conexión Ollama, y el handler de logging se duplicaría en cada rerun.
5. **Evolución del rastreador de corpus: de lista negra a lista blanca.** La primera versión del rastreador (`build_corpus.py v1`) empleaba un enfoque de *lista negra*: descargaba todas las páginas alcanzables desde las semillas y aplicaba patrones de exclusión para descartar contenido irrelevante. Las pruebas revelaron cinco problemas graves:
 - a) **Duplicados Drupal:** el CMS generaba múltiples copias de un mismo PDF con sufijos `_0` a `_N`, inflando el corpus con documentos idénticos.
 - b) **Páginas índice sin contenido:** páginas de navegación con menos de 200 caracteres de texto útil que degradaban la calidad del retrieval al aportar fragmentos vacíos al índice.
 - c) **Documentos de cursos anteriores:** URLs con formatos heterogéneos de fecha (`(20-21)`, `Febrero2024`, `ETSIIT_Calendarario_2023_2024`) que escapaban a los filtros iniciales.
 - d) **Límite de páginas global compartido:** el contador de páginas era global entre todas las categorías, de modo que una sola categoría con muchas semillas (guías docentes, > 500 páginas) agotaba el presupuesto de descarga antes de que las demás pudieran ejecutarse.
 - e) **Inconsistencia de protocolo http/https:** el CMS servía enlaces internos con `http://` mientras los patrones de exclusión asumían `https://`, lo que permitía que 86 documentos en inglés pasaran el filtro.

La versión actual (`v2`) resuelve todos estos problemas mediante un enfoque de *lista blanca por categoría*: cada tipo de contenido tiene su propio perfil con semillas, patrones de inclusión y un límite de páginas independiente. Un enlace descubierto que no case con ningún patrón de la categoría activa se ignora, y las URLs se normalizan a `https://` antes de compararlas con los filtros.

6. **Corpus homogéneo y *recall* de documentos minoritarios.** La evaluación cualitativa reveló que la búsqueda densa pura enterraba documentos decisivos pero poco frecuentes (el Plan de Estudios llegaba al puesto ~ 240 del *ranking*), de modo que nunca alcanzaban el contexto del LLM. Como un fragmento que no se recupera no puede generarse, el problema es de *recall*. La solución fue introducir un enrutador de consultas por categoría y una inyección focalizada que garantiza que los fragmentos de la categoría pertinente entren siempre en el conjunto de candidatos (Sección 5.8.6).
7. **Ordenación sensible a palabras clave y reranking en español.** El enrutamiento por reglas, útil para el *recall*, resultaba frágil para el *ranking*: la mera presencia de una palabra como “cuándo” desviaba una consulta de horario hacia el calendario de exámenes. Se incorporó un *cross-encoder* (Sección 5.8.10) para que la ordenación final fuese semántica y no léxica. Una dificultad adicional fue que el *cross-encoder* estándar (ms-marco-MiniLM) está entrenado solo en inglés y discriminaba mal sobre el corpus español; se sustituyó por una variante *multilingüe* (mmarco-mMiniLMv2), con respaldo automático al modelo en inglés si el primero no puede descargarse.
8. **Reconocimiento de asignaturas por nombre completo.** El sistema reconocía las asignaturas solo por su sigla (DI, CA). Cuando el usuario escribía el nombre completo (“Cálculo”), no se activaba el realce por sigla exacta y, peor aún, en un seguimiento la consulta arrastraba la asignatura anterior, produciendo respuestas de “no dispongo”. Se construyó un reconocedor que mapea tanto la sigla como el nombre completo (insensible a tildes) a la asignatura, integrado en la expansión de la consulta y en el enriquecimiento por historial (Sección 5.8.8).
9. **Agregación de muchos registros parecidos.** En consultas de listado (“el horario de Cálculo en todos los grupos”, “las asignaturas de tercero”), el LLM de 8 B tendía a *fusionar* registros distintos e *inventar* para “cuadrar” la lista. Se atacó por tres vías complementarias: (i) documentos verificados con *registros-resumen* agregados por curso y especialidad (Sección 5.7.4); (ii) ampliación dinámica del contexto en consultas de listado (Sección 5.8.7); y (iii) instrucciones explícitas de completitud y no fusión en el *prompt*, respaldadas por la decodificación *greedy* ($T = 0$). Se descartó conscientemente una vía de *ensamblado determinista* que evitaba el LLM para estos casos, por contradecir la apuesta del sistema por la generación mediante LLM.
10. **Extracción frágil de tablas y fusión de franjas horarias.** La extracción automática de horarios y calendarios desde PDF resultaba poco fiable. Se adoptaron fuentes *verificadas* verificadas manualmente

que conservan la URL oficial para la citación (Sección 5.7.4). Un detalle de calidad relevante fue que, al representar las clases en franjas de una hora, el modelo tendía a quedarse con la primera y omitir las contiguas; se resolvió fusionando de forma determinista las franjas consecutivas del mismo día (y misma aula) en un único intervalo, transformación verificada como *sin pérdida* de cobertura horaria.

11. **Alucinación y omisión de citas.** El modelo emitía marcadores huérfanos o malformados, citaba la misma fuente por duplicado o, al contrario, respondía con datos del contexto *sin* citar. Se implementó una capa de verificación que elimina los marcadores inválidos, deduplica las fuentes por URL y, cuando faltan marcadores, recupera la fuente por solapamiento léxico, inhibiéndola en abstenciones y preguntas sobre el propio asistente para no fabricar atribuciones (Sección 5.10.4).
12. **Anáfora en seguimientos conversacionales.** El historial se entregaba al modelo de generación pero no al recuperador, de modo que seguimientos como “¿y las de Cálculo?” recuperaban fragmentos de otra asignatura. Se añadió un enriquecimiento determinista de la consulta de recuperación que arrastra las entidades del dominio de los turnos previos (Sección 5.8.8).
13. **Depuración determinista de la respuesta.** Los modelos pequeños arrastran “manías” de forma: preámbulos y muletillas (“según la información proporcionada...”), viñetas en línea que el renderizador muestra como asteriscos literales, y repeticiones del mismo dato. Se añadió un post-procesado determinista (Sección 5.10.4); su parte más delicada fue la deduplicación de contenido, que en una primera versión *colapsaba listas legítimas* (el horario de cada grupo): se corrigió exigiendo que dos frases compartan los mismos *tokens numéricos* para considerarse redundantes, lo que distingue un dato repetido de datos paralelos con valores distintos.

6.12. Resumen del capítulo

Este capítulo ha presentado la implementación completa del sistema GRAIA, traduciendo el diseño arquitectónico del Capítulo 5 en código Python funcional organizado en seis subsistemas: ingesta, indexación, recuperación, generación, interfaz y registro.

Cada módulo se ha descrito con fragmentos de código representativos (8–20 líneas), acompañados de la justificación técnica de las decisiones adoptadas y de las alternativas descartadas. La trazabilidad entre requisitos, componentes de diseño y módulos de código se ha verificado mediante tablas parciales por subsistema y una tabla global consolidada (Tabla 6.10).

La suite de 90 tests unitarios proporciona una red de seguridad que ha permitido detectar y corregir errores como la incompatibilidad de FAISS con índices vacíos antes de la integración de los componentes.

El siguiente capítulo (Capítulo 7) someterá al sistema a una evaluación experimental rigurosa, midiendo la calidad del retrieval, la fidelidad de las respuestas generadas y la precisión del sistema de citas sobre un dataset representativo de consultas académicas.

Capítulo 7

Evaluación, pruebas y funcionamiento

7.1. Introducción

Este capítulo somete a GRAIA a una evaluación empírica orientada a contrastar las tres hipótesis de investigación enunciadas en la Sección 1.5:

- **H1**, calidad de las respuestas: precisión superior al 80 % en consultas con respuesta en el corpus y citación verificable en más del 90 % de los casos.
- **H2**, reducción de alucinaciones: el sistema RAG reduce de forma sustancial las alucinaciones frente al mismo LLM operando *sin* contexto recuperado.
- **H3**, viabilidad local: el modelo abierto de 8 B ejecutado en una GPU de consumo (RTX 4070 Super) responde con latencia aceptable (< 5 s).

La evaluación se diseña para ser *objetiva*, *reproducible* y *trazable*: todas las métricas se calculan de forma automática y determinista sobre un mismo conjunto de preguntas, y la comparación con un *baseline* sin recuperación permite aislar la contribución del componente RAG. Se evita expresamente el ajuste del sistema a las preguntas concretas del conjunto de evaluación (*overfitting*), como se discute en la Sección 7.12.

7.2. Protocolo de evaluación

Entorno. Todos los experimentos se ejecutan en la máquina de desarrollo descrita en el Capítulo 6: GPU NVIDIA RTX 4070 Super (12 GB), con los

modelos servidos por Ollama, el modelo de *embeddings* `multilingual-e5-base` y el *cross-encoder* `mmarco-mMiniLMv2`, sobre el índice FAISS+BM25 del corpus verificado de la ETSIIT. La configuración del *pipeline* es la de `config/default.yaml` (decodificación *greedy* $T = 0$, $k = 5$, $k_{\text{listado}} = 10$, $\tau = 0,35$, reranking y modo híbrido activos) y se mantiene *fija* en todos los experimentos: lo único que varía es el modelo de generación.

Comparación de modelos de generación. Para estudiar el efecto del modelo, y su compromiso entre calidad y latencia, sobre un mismo sistema de recuperación, se evalúan tres LLM abiertos de distinta familia y tamaño, todos ejecutables localmente en la GPU citada: `llama3.1:8b` (Meta, 8 B; el modelo por defecto del sistema), `gemma2:9b` (Google, 9 B) y `qwen2.5:14b` (Alibaba, 14 B). La elección busca contrastar dos familias de tamaño similar (8–9 B) frente a un modelo mayor (14 B), manteniendo constante todo lo demás.

Metodología. La evaluación se automatiza con el guion `scripts/evaluate.py`, que reproduce el *pipeline* completo de la interfaz (recuperación $\rightarrow$ *gate* de ámbito $\rightarrow$ generación $\rightarrow$ post-procesado) sobre cada pregunta del conjunto, sin intervención manual, para cada uno de los tres modelos. La decodificación *greedy* ($T = 0$) hace la generación *determinista*, de modo que los resultados son reproducibles salvo la variabilidad residual del propio motor de inferencia. Para contrastar H2 se ejecuta además, *para cada modelo*, `scripts/evaluate.baseline.py`, que lo somete al *mismo* conjunto de preguntas *sin* recuperar contexto y sin forzar la abstención, midiendo su comportamiento “en crudo”. El diseño experimental es, por tanto, una matriz de 3 modelos $\times$ 2 configuraciones (con RAG / sin contexto).

7.3. Conjunto de evaluación

No existe un *benchmark* público de preguntas y respuestas sobre la información académica de la ETSIIT, por lo que se construye un conjunto propio. Su diseño persigue tres propiedades:

- **Cobertura.** Las preguntas se elaboran recorriendo el corpus completo, de modo que abarquen todos los ámbitos presentes: calendario académico, exámenes, horarios, plan de estudios, guías docentes, TFG, trámites y plazos, movilidad (incluidos tutores Erasmus/SICUE), profesorado, prácticas en empresa y secretaría.
- **Fundamentación (anti-alucinación).** Cada pregunta con respuesta esperada se contrasta con el documento del corpus que la contiene,

las palabras clave de comprobación y la fuente esperada se verifican manualmente contra el texto real, evitando medir contra datos inventados.

- **Variedad de comportamiento.** El conjunto incluye no solo preguntas *factuales* y de *agregación* (listados), sino también preguntas *sin información* en el corpus y preguntas *fuera de ámbito*, que prueban la abstención y el rechazo, comportamientos tan importantes como acertar.

El conjunto resultante consta de **74 preguntas**. La Tabla 7.1 resume su composición por tipo y ámbito.

Tipo	Ámbitos cubiertos	N.º
Factual	calendario, exámenes, horarios, plan, guías, TFG, trámites, movilidad, profesorado, prácticas, secretaría	51
Agregación (listados)	plan de estudios, horarios, TFG, prácticas	12
Sin información (en el corpus)	nota de corte, comedor, aparcamiento, matriculados, beca, wifi	6
Fuera de ámbito	geografía, ocio, deportes, clima, cocina	5
Total	12 ámbitos	74

Cuadro 7.1: Composición del conjunto de evaluación.

7.4. Métricas

Las métricas se calculan de forma automática por `evaluate.py`. Para una respuesta dada, una pregunta *factual* se considera acertada si contiene todas las palabras clave esperadas y *no* es una abstención; una pregunta *de agregación* se trata igual sobre su conjunto de palabras clave; y una pregunta *sin información* o *fuera de ámbito* se considera acertada si el sistema se abstiene correctamente. Las métricas reportadas son:

- **Precisión factual** (*factual accuracy*): porcentaje de aciertos sobre las preguntas con respuesta en el corpus (factuales y de agregación). Es el indicador central de H1.
- **Tasa de citación verificable**: porcentaje de respuestas factuales *respondidas* que incluyen al menos una cita a una fuente realmente recuperada, marcador [n] válido o, en su defecto, atribución por solapamiento léxico. Es el segundo indicador de H1.
- **Recall de recuperación**: porcentaje de preguntas para las que la fuente esperada aparece entre los fragmentos recuperados.

- **Abstención correcta** (*no-info correct*): porcentaje de preguntas sin información o fuera de ámbito en las que el sistema se abstiene.
- **Tasa de alucinación en *no-info***: porcentaje de preguntas sin información a las que el sistema *responde* (inventando) en lugar de abstenerse. Es el indicador central de H2.
- **Falsos rechazos**: número de preguntas con respuesta en el corpus en las que el sistema se abstiene por error.
- **Fugas de razonamiento**: respuestas que exponen el proceso interno o los fragmentos.
- **Latencia**: tiempo medio y percentil 95 (P95) por consulta (recuperación + generación). El P95 es el valor de tiempo por debajo del cual se resuelve el 95 % de las consultas, es decir, solo el 5 % más lento lo supera; a diferencia de la media, describe la *cola* de la distribución sin dejarse arrastrar por casos atípicos (como el arranque en frío). Ambos son indicadores de H3, y el umbral de los 5 s se exige sobre el P95.
- **Métricas RAGAS** (complementarias): *faithfulness* (fidelidad de la respuesta a los fragmentos recuperados) y *answer relevancy* (pertinencia de la respuesta a la pregunta) [75], calculadas con un LLM como juez y detalladas en la Sección 7.7. Corroboran H1 y H2 con una medida independiente de las palabras clave.

La detección de abstención emplea el detector único `is_abstention` compartido con el post-procesado (Sección 6.6.4), de modo que la métrica y el sistema reconocen la abstención con el mismo criterio.

7.5. Resultados cuantitativos

La Tabla 7.2 recoge los resultados de GRAIA con los tres modelos de generación sobre el mismo sistema de recuperación y el mismo conjunto de evaluación.

Métrica (con RAG)	gemma2:9b	llama3.1:8b (por defecto)	qwen2.5:14b
Precisión factual	97 %	87 %	92 %
Tasa de citación verificable	100 %	100 %	100 %
Recall de recuperación	100 %	100 %	100 %
Abstención correcta (<i>no-info</i>)	82 %	100 %	100 %
Falsos rechazos	0	1	0
Fugas de razonamiento	0	0	0
Latencia media (recup. + gen.)	2,4 s	1,9 s	30 s
Latencia P95	3,5 s	4,6 s	78 s

Cuadro 7.2: Resultados de GRAIA con los tres modelos de generación (74 preguntas). En negrita, el mejor valor de cada fila.

Los tres modelos comparten dos resultados sobresalientes del *sistema*, no del modelo: una **citación verificable del 100 %** y un *recall* de recuperación del 100 %. Es decir, la fuente correcta llega siempre al contexto y toda respuesta factual queda trazada a una fuente; los errores residuales no son de recuperación de *documento*. A partir de ahí, cada modelo ocupa un vértice distinto del compromiso entre precisión, prudencia y latencia:

- **gemma2:9b** es el *más preciso* (97 %) y el *más rápido* en P95 (3,5 s), pero el de *peor abstención* (82 %): cuando el corpus no contiene el dato, tiende a improvisar en lugar de reconocerlo. Es un extractor excelente pero poco prudente.
- **qwen2.5:14b** combina alta precisión (92 %) con *abstención perfecta* (100 %), pero su latencia es *prohibitiva en local*: 30 s de media y un P95 de 78 s, muy por encima del límite de 5 s (H3). No es viable para uso interactivo en el hardware objetivo.
- **llama3.1:8b** ofrece el mejor *equilibrio*: precisión del 87 % (por encima del 80 % de H1), abstención perfecta (100 %) y una latencia dentro del umbral (media 1,9 s; P95 4,6 s < 5 s). Tanto llama como gemma cumplen las tres hipótesis (H1, H2 y H3); qwen incumple H3 por latencia. La elección de llama como modelo por defecto frente a gemma (Sección 5.9.1) se decide por su mayor *prudencia*: su abstención del 100 % (frente al 82 % de gemma) y la ausencia de alucinaciones ante consultas sin información son determinantes en un asistente académico, donde una respuesta inventada es peor que ninguna.

Cómo responde cada modelo. El análisis cualitativo de las respuestas revela estilos marcadamente distintos sobre el mismo contexto recuperado. **gemma** es telegráfico y preciso, y cita siempre (“*SICUE* es el acrónimo

de Sistema de Intercambio entre Centros Universitarios Españoles [1].”); esa misma economía explica tanto su alta precisión como su escasa prudencia ante la ausencia de datos. **llama** responde de forma directa y concisa, aunque a veces *omite el marcador* [n], rescatado después por la atribución léxica, y ocasionalmente es *menos completo* (en una consulta de horario dio los días pero no las horas ni el aula). **qwen** es *extenso y didáctico*: enriquece la respuesta con contexto no solicitado (origen ministerial del programa, comparación con ERASMUS), emplea listas numeradas y cierres explicativos; es el más completo y prudente, pero esa verbosidad es precisamente la causa de su elevada latencia.

Una nota sobre la varianza. La decodificación *greedy* ($T = 0$) no es perfectamente determinista en el motor de inferencia: en distintas ejecuciones, la precisión de llama osciló entre el 83 % y el 87 %, siempre por encima del umbral de H1. Las cifras reportadas corresponden a la ejecución sobre la configuración final del sistema (con el *gate* de ámbito corregido, Sección 7.8).

7.6. Análisis comparativo: RAG frente a LLM sin contexto

Para aislar la contribución de la recuperación (H2) se compara cada modelo *con* RAG frente a sí mismo respondiendo *sin* contexto (*baseline*). La Tabla 7.3 resume el contraste para los tres.

Modelo	Precisión factual		Alucinación en <i>no-info</i>	
	con RAG	sin contexto	con RAG	sin contexto
gemma2:9b	97 %	3 %	18 %	100 %
llama3.1:8b	87 %	5 %	0 %	64 %
qwen2.5:14b	92 %	8 %	0 %	73 %

Cuadro 7.3: Cada modelo con recuperación frente a sí mismo sin contexto. La alucinación en *no-info* es el porcentaje de preguntas sin información a las que el modelo responde inventando en vez de abstenerse.

El contraste es contundente y *consistente entre modelos*. Sin recuperación, los tres son prácticamente inservibles como asistente académico, aciertan entre el 3 % y el 8 % de las consultas factuales, pues no conocen los datos concretos de la ETSIIT, y alucinan masivamente cuando carecen del dato: entre el 64 % y el 100 % de las preguntas sin información reciben una respuesta inventada en lugar de una abstención. El caso de gemma es el más elocuente: *sin* RAG inventa una respuesta en el 100 % de las preguntas sin

información (nunca admite desconocer), mientras que *con* RAG su alucinación cae al 18 %. La recuperación es, por tanto, lo que convierte a un LLM que alucina de forma sistemática en un asistente fiable, con independencia del modelo elegido. Este resultado respalda con holgura la hipótesis H2.

7.7. Evaluación automática con RAGAS

Las métricas anteriores se basan en palabras clave y en la comparación con el *baseline*. Para corroborar los resultados con un criterio independiente, se añade una evaluación con el *framework* RAGAS [75, 69], una herramienta especializada en sistemas RAG que utiliza un *modelo de lenguaje como juez* (*LLM-as-judge*): en lugar de comparar la respuesta con una solución escrita a mano, un LLM lee la pregunta, la respuesta del sistema y los fragmentos que se recuperaron, y emite un juicio sobre ellos. Se eligen dos métricas que *no* necesitan una respuesta de referencia, lo que encaja con un conjunto de evaluación definido por palabras clave:

- ***Faithfulness*** (fidelidad). El juez descompone la respuesta en afirmaciones elementales y mide qué proporción de ellas está respaldada por los fragmentos recuperados. Un valor de 1 significa que *todo* lo afirmado se apoya en el contexto; un valor bajo indica afirmaciones sin soporte, es decir, alucinaciones. Es, por tanto, una medida directa de la ausencia de alucinación y el complemento natural de H2.
- ***Answer relevancy*** (pertinencia). Mide hasta qué punto la respuesta se ciñe a lo que se preguntó. RAGAS la estima generando varias preguntas *a partir* de la respuesta y comparándolas con la pregunta original: si la respuesta es pertinente, esas preguntas se parecen mucho a la real. Penaliza tanto divagar (añadir información no pedida) como quedarse corto.

Ambas se calculan *únicamente* sobre las preguntas *factuales que el sistema responde* (entre 62 y 63 de las 74, según el modelo): las abstenciones y los rechazos fuera de ámbito se excluyen, porque medir la fidelidad de un “no dispongo de información” no aporta nada.

Para que la comparación entre modelos sea *justa*, el juez es fijo y común a los tres: **qwen2.5:14b-instruct**, el modelo local más capaz de los evaluados, ejecutado también sobre Ollama. Conviene subrayar que el modelo evaluado y el juez son independientes: se puntúan las respuestas de llama, gemma y qwen, pero quien las califica es siempre el mismo juez. El Cuadro 7.4 recoge las medias.

La *faithfulness* confirma con una métrica independiente la alta fidelidad del sistema: gemma es el más fiel (0,90), seguido de qwen (0,88) y llama

Modelo de generación	Faithfulness	Answer relevancy
llama3.1:8b	0,83	0,70
gemma2:9b	0,90	0,60
qwen2.5:14b	0,88	0,65

Cuadro 7.4: Métricas RAGAS (juez `qwen2.5:14b-instruct`) sobre las preguntas factuales respondidas por cada modelo (62 en llama; 63 en gemma y qwen; de las 74 del conjunto). En negrita, el mejor valor de cada columna. La *faithfulness* se promedia tras descartar 5 ítems por fallo de parseo del juez (57–58 ítems efectivos); la *answer relevancy*, sobre todas las respondidas.

(0,83), con una mediana de **1,000** en los tres modelos; es decir, en más de la mitad de las preguntas la respuesta está *totalmente* respaldada por los fragmentos. Lo más revelador es que este orden (gemma > qwen > llama) *coincide* con el de la precisión factual por palabras clave (Sección 7.5): dos métricas calculadas de forma completamente distinta, una por coincidencia de palabras clave, la otra por un juez LLM, ordenan igual a los tres modelos, lo que refuerza la fiabilidad de ambas. Conviene además señalar que el hecho de que el juez sea `qwen` no le da ventaja a su propio modelo: gemma lo supera en fidelidad, lo que descarta un sesgo de autocomplacencia, algo razonable porque la *faithfulness* se juzga contrastando la respuesta con el contexto, no comparando modelos entre sí.

La *answer relevancy* es más moderada (entre 0,60 y 0,70) y conviene interpretarla con cuidado. Como se explicó, RAGAS la calcula generando preguntas a partir de la respuesta y midiendo su similitud con la pregunta original, por lo que penaliza tanto las respuestas *muy escuetas* como las que añaden información colateral. Esto explica que **gemma**, de estilo telegráfico, obtenga el valor más bajo (0,60) pese a ser el más preciso en aciertos, mientras que llama, más conversacional, alcanza el más alto (0,70). No es, por tanto, una deficiencia del sistema sino un reflejo del estilo de redacción de cada modelo: una respuesta correcta pero muy escueta puede recibir una pertinencia modesta.

Finalmente, esta evaluación arroja un hallazgo metodológico relevante sobre los *límites del LLM como juez ejecutado en local*. Cinco ítems (los mismos en los tres modelos) no pudieron puntuarse en *faithfulness* porque el juez no consiguió descomponer la respuesta en afirmaciones; se excluyen de la media, por lo que la fidelidad se promedia sobre 57–58 ítems en lugar de los 62–63 respondidos. Más aún, al inspeccionar uno a uno los casos con *faithfulness* = 0 se comprueba que varios son *falsos negativos del juez*: por ejemplo, a “¿cuándo es la entrega del TFG en junio de 2026?” el sistema responde “22 de junio de 2026”, dato que figura *literalmente* en el fragmento recuperado (“Entrega del TFG e informe del tutor: ... y 22 de junio de 2026”), y aun así el juez le asigna fidelidad nula. La media de *faithfulness* re-

portada es, en consecuencia, una *cota inferior* de la fidelidad real: el sistema es, casi con seguridad, todavía más fiel de lo que indica la cifra. Este comportamiento justifica precisamente la estrategia de *triangulación* adoptada en este capítulo: ninguna métrica automática se toma como verdad absoluta, sino que RAGAS, la evaluación por palabras clave, la comparación con el *baseline* y el análisis manual de errores se contrastan entre sí.

7.8. Análisis de errores

El análisis cualitativo de los fallos residuales (las consultas con respuesta en el corpus no acertadas) es tan informativo como las cifras agregadas, pues delimita el origen real de las limitaciones. Los errores se agrupan en cuatro familias:

- **Límites del modelo de 8 B (3 casos).** El modelo *alucina* ocasionalmente un dato pese a tenerlo en el contexto (p. ej. cambia “jueves” por “viernes” en un horario), prefiere una formulación vaga de una fuente cuando otra contiene el dato exacto, o se abstiene erróneamente de forma aislada. Son intrínsecos al tamaño del modelo y no a la arquitectura.
- **Granularidad intra-documento (3 casos).** El documento correcto se recupera, pero el modelo toma la *sección* o el *registro* equivocado dentro de él: en el documento de plazos, un plazo distinto del pedido; o, en una consulta de especialidad, el conjunto de asignaturas comunes en lugar del de la mención. Es el límite más relevante que queda y se aborda como trabajo futuro (Sección 7.12).
- **Complejidad de agregación (1 caso).** En un listado cuyo registro contiene las cinco asignaturas comunes, el modelo enumera solo tres: una limitación de *generación*, no de recuperación.
- **Confusión de concepto (1 caso).** El modelo confunde las convocatorias de *matrícula* con las de *defensa* de una asignatura.

Un experimento negativo ilustrativo. Durante el ajuste se probó a ampliar el contexto de los listados (k_{listado} de 10 a 12) para mejorar la agregación. El resultado fue *neto negativo*: arregló un listado pero rompió otros dos (la precisión factual cayó en torno a dos puntos), porque más contexto introduce más dilución y ruido. El cambio se revirtió. Este episodio ilustra el compromiso del parámetro k y, sobre todo, refuerza la decisión metodológica de *no* sobreajustar el sistema al conjunto de evaluación: las mejoras solo se conservan si son generales y no degradan el conjunto.

El valor de diagnosticar antes de parchear. Un fallo aparentemente de recuperación, el horario de la secretaria, ilustra la importancia de localizar la causa exacta. Las primeras hipótesis (un realce del fragmento por coincidencia de sección, y una recalibración del umbral del *gate*) se ensayaron y *fracasaron*, la primera no surtió efecto y degradó otras preguntas; la segunda rompió la latencia P95 sin resolver el caso. La instrumentación del *pipeline* reveló entonces la causa real: el fragmento correcto (“de 9 a 14 horas, de lunes a viernes”) *sí* se recuperaba y llegaba al contexto, pero el *gate* de ámbito lo *auto-rechazaba* por su margen negativo en el *cross-encoder*, sin llegar a consultar al clasificador LLM, que, aislado, sí lo reconocía como consulta académica. La solución correcta y de coste marginal fue condicionar ese auto-rechazo a que el enrutador no hubiera clasificado la consulta (Sección 5.8.11): tras el cambio, la consulta se responde correctamente sin regresiones ni penalización de latencia. El caso subraya que, en sistemas con varias etapas, atribuir un error a la etapa equivocada conduce a parches inútiles; la instrumentación y el diagnóstico preceden a la corrección.

Contaminación de la generación por el historial. Un segundo caso reveló una *asimetría* de diseño. En la interfaz, un seguimiento como “¿y los exámenes?” tras una pregunta de horario se abstenía erróneamente la primera vez y acertaba al repetirlo. La instrumentación descartó las etapas habituales: la recuperación era determinista y traía el calendario de exámenes correcto, y el *gate* de ámbito exime a los seguimientos anafóricos, de modo que aceptaba la consulta sin evaluarla. El fallo estaba en la *generación*: recibía el historial conversacional *en bruto*, y un turno previo de otro tipo de dato (un horario antes de una pregunta de exámenes) inducía al modelo de 8 B a abstenerse. La causa de fondo era que la recuperación ya resolvía la anáfora de forma determinista, pero la generación seguía dependiendo del historial. La corrección consistió en emplear la *consulta ya resuelta* también en la generación y *no* inyectarle el historial en bruto (Sección 5.8.8): la memoria conversacional se mantiene por el acarreo determinista de entidades, la anáfora se resuelve de extremo a extremo y la abstención errónea desaparece sin perder la capacidad de seguimiento. De nuevo, el diagnóstico, y no el parche apresurado, condujo a la solución.

7.9. Latencia y viabilidad local (H3)

La latencia depende fuertemente del modelo de generación. Con el modelo por defecto, **llama3.1:8b**, la latencia media por consulta (recuperación + generación) es de aproximadamente **1,9 s** y el percentil 95 de **4,6 s**, por debajo del umbral de 5 s del RNF-01 y de la hipótesis H3. **gemma2:9b** es igualmente viable (media 2,4 s, P95 3,5 s). En cambio, **qwen2.5:14b**,

pese a su buena calidad, presenta una latencia *prohibitiva* en local (media 30 s, P95 78 s), muy por encima del límite: su mayor tamaño y su estilo verboso lo hacen inadecuado para uso interactivo en el hardware objetivo. Conviene matizar el efecto del *arranque en frío*: la *primera* consulta de la sesión incluye la carga diferida del modelo de generación, del *reranker* y del modelo de *embeddings*, lo que la dispara a ≈ 13 s con llama y contamina las métricas agregadas. Excluyéndola, la inmensa mayoría de respuestas se resuelven en torno a 1 s y el P95 en régimen “caliente” baja a $\approx 3,8$ s. En un despliegue real este coste se paga una sola vez al iniciar el servicio (los modelos quedan residentes en memoria), por lo que el usuario no lo percibe; la latencia interactiva efectiva es, por tanto, la de régimen estacionario. El resultado confirma que un modelo abierto de tamaño 8–9 B sobre una GPU de consumo es computacionalmente suficiente para un asistente conversacional con latencia aceptable, validando H3, y delimita el tamaño de modelo practicable en este hardware.

7.10. Validación de las hipótesis

- **H1 (calidad):** *cumplida por los tres modelos*. La precisión factual supera el 80 % en todos (llama 87 %, qwen 92 %, gemma 97 %) y la citación verificable es del 100 % en los tres.
- **H2 (reducción de alucinaciones):** *cumplida de forma contundente y consistente*. En los tres modelos la recuperación eleva la precisión factual desde el 3–8 % hasta el 87–97 % y reduce drásticamente la alucinación en consultas sin información (del 64–100 % al 0–18 %).
- **H3 (viabilidad local):** *cumplida por los modelos de 8–9 B* (llama, por defecto: media 1,9 s, P95 4,6 s; gemma: media 2,4 s, P95 3,5 s). El modelo de 14 B (qwen) la incumple (P95 78 s), lo que acota el tamaño de modelo practicable en la GPU objetivo.

En conjunto, llama3.1:8b es el modelo que mejor concilia calidad, prudencia y latencia, y por ello el adoptado por defecto: gemma cumple igualmente las hipótesis y maximiza precisión y velocidad, pero a costa de la prudencia (abstención del 82 %), y qwen maximiza calidad y abstención a un coste de latencia inasumible en local.

7.11. Demostración del funcionamiento del sistema

Más allá de las métricas agregadas, esta sección ilustra el comportamiento de GRAIA en la interfaz conversacional (Capítulo 6) en tres escenarios representativos. Las respuestas se muestran tal como las presenta el sistema (Figura 7.1), con su pie de fuentes verificadas.

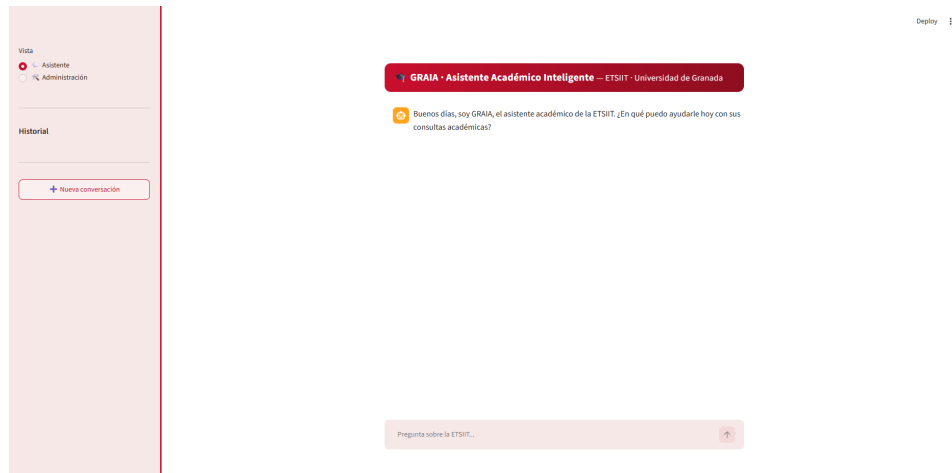


Figura 7.1: Interfaz conversacional de GRAIA: identidad visual y zona de conversación.

7.11.1. Interfaz conversacional

Consulta en ámbito (con cita) y contexto de la conversación. Ante “¿Cuándo son las clases de DI?”, el sistema responde únicamente con el horario de *clase*, sin volcar exámenes ni tutorías, y cita la fuente correcta (Figura 7.2), seguidamente ante “¿Y cuándo es la ordinaria?”, el sistema responde con la fecha y hora del examen ordinario de Derecho Informático, comprobándose así que el modelo está preparado para seguir la conversación y que es inteligente entendiendo el contexto de la pregunta como examen de la convocatoria ordinaria.

 ¿Cuándo son las clases de DI?

 El horario de clase de Derecho Informático (DI) es: Teoría (aula 1.4) Lunes 12:30-14:30 y Martes 11:30-13:30 [1].

Cualquier otra duda, dígamelo sin problema.

Fuentes:

- [1] Horarios Grado en Ingeniería Informática 2025-2026 — <https://etsiit.ugr.es/sites/centros/etsiit/public/inline-files/Horarios%20GII%20%2825-26%29.pdf>

 ¿Y cuando es la ordinaria?

 El horario del examen ordinario de Derecho Informático (DI) en el segundo semestre del cuarto curso, especialidad común a todas las especialidades, es el lunes 8 de junio de 2026, turno de mañana (inicio 9:00).

¿Quiere que mire alguna otra cosa por usted?

Fuentes:

- [1] Calendario de exámenes 2025-2026 —

Pregunta sobre la ETSIIT...



Figura 7.2: Respuesta a una consulta en ámbito y a una consulta de contexto.

Abstención y rechazo. Ante una consulta sin información en el corpus o fuera de ámbito (“¿cuál es la capital de Francia? o ¿Quién crees que va a ganar el mundial?”), el *gate* de ámbito la rechaza *sin* generar y emite una frase predeterminada para rechazar cualquier consulta fuera de ámbito, se simplifica a una única frase sin depender del contexto con la intención demostrar el funcionamiento del rechazo. (Figura 7.3):

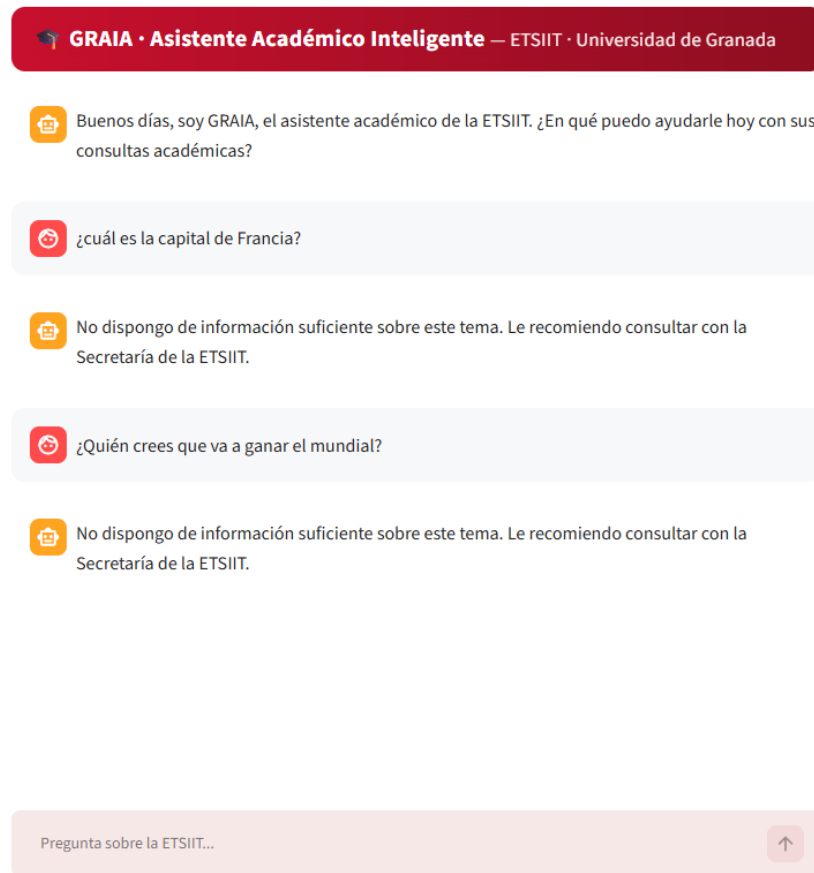


Figura 7.3: El *gate* de ámbito rechaza una consulta fuera de dominio sin generar, con la frase predeterminada de abstención.

Recuperación de un dato tabular verificado. Ante “¿de qué destinos Erasmus es tutora Sylvia Acid Carrillo?”, el sistema responde a partir del documento verificado de tutores de movilidad, citándolo: los destinos incluyen Lyon, Nancy y Toulon, entre otros (Figura 7.4).

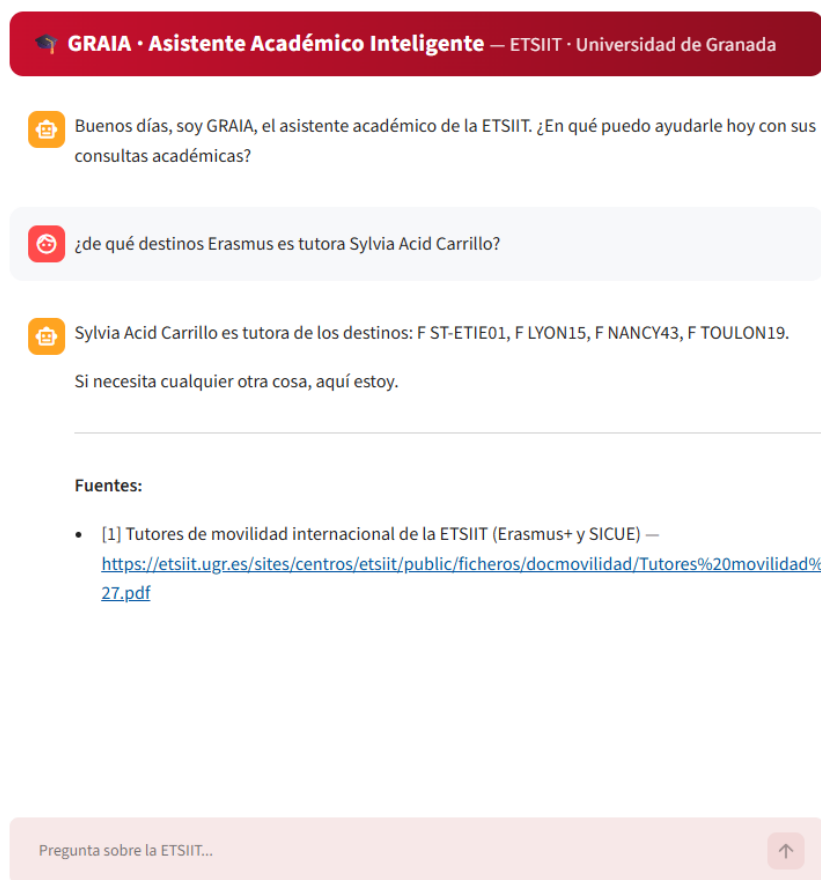


Figura 7.4: Respuesta a partir de un documento verificado de tutores de movilidad, con su cita.

Historial de conversaciones y selección de vista. Se observa, en la zona izquierda de la interfaz conversacional, el historial de todas las conversaciones mantenidas con GRAIA, dando la oportunidad de comenzar una nueva conversación cuando el usuario lo desee. Así mismo, se observa la posibilidad de seleccionar tanto la vista para hablar con el modelo como la vista del administrador. (Figura 7.5).

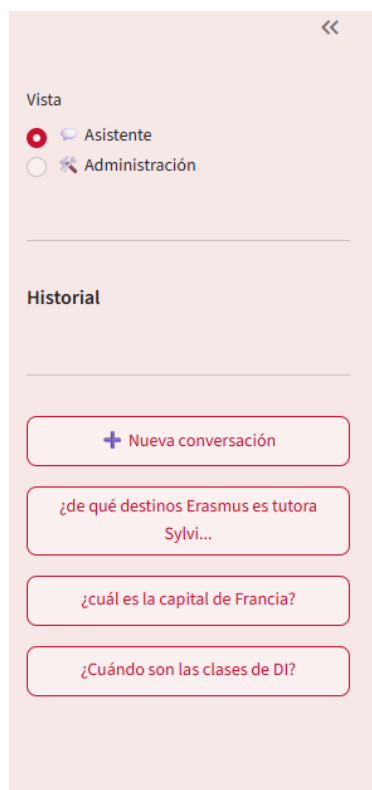


Figura 7.5: Historial de conversaciones y selección de vista de la interfaz visual de GRAIA.

7.11.2. Interfaz de administración

Además de la conversación, GRAIA ofrece un *panel de administración* que materializa las funciones del actor administrador definidas en el Capítulo 4 y otorga autonomía sobre todo el ciclo de vida del corpus sin necesidad de tocar la línea de comandos. La vista general (Figura 7.6) presenta las siete funciones como desplegados independientes; a continuación se detalla cada una con su captura correspondiente.

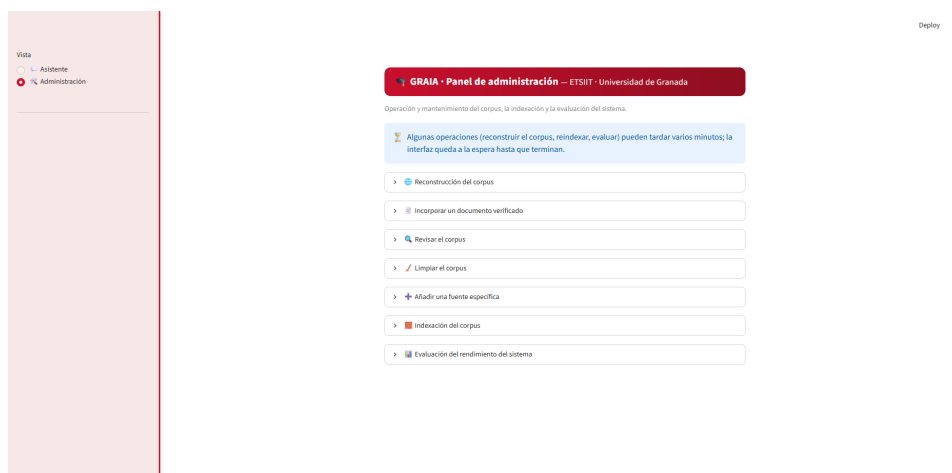


Figura 7.6: Panel de administración de GRAIA con sus siete funciones.

Reconstrucción del corpus. Lanza el rastreo (*crawling*) y la descarga de las fuentes oficiales configuradas, regenerando el corpus desde cero, dando la opción de descargar las fuentes según las categorías seleccionadas en la lista (Figura 7.8) y eligiendo un máximo de páginas por categoría para no saturar el corpus (Figura 7.7).

 **GRAIA · Panel de administración** — ETSIIT · Universidad de Granada

Operación y mantenimiento del corpus, la indexación y la evaluación del sistema.

 Algunas operaciones (reconstruir el corpus, reindexar, evaluar) pueden tardar varios minutos; la interfaz queda a la espera hasta que terminan.

▼  Reconstrucción del corpus

Categorías a procesar 

Todas x 

 ▼

Máximo de páginas (opcional, 0 = sin límite)

0

— +

Reconstruir corpus

>  Incorporar un documento verificado

>  Revisar el corpus

>  Limpiar el corpus

>  Añadir una fuente específica

>  Indexación del corpus

Figura 7.7: Reconstrucción del corpus: rastreo y descarga de las fuentes oficiales.

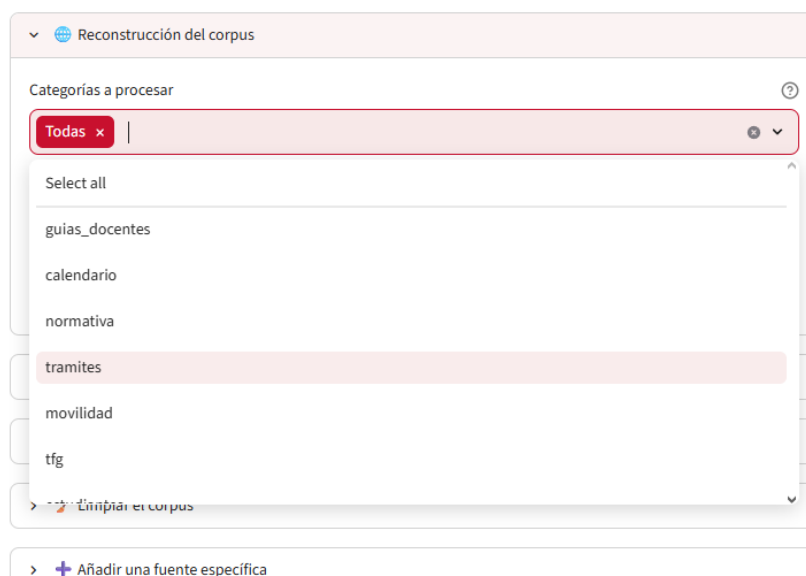


Figura 7.8: Lista de categorías para la reconstrucción del corpus.

Incorporar un documento verificado. Permite subir un documento limpio y verificado a mano que sustituye, conservando su URL, el contenido degradado que la extracción automática no recupera bien (típicamente datos tabulares como horarios o calendarios), tal como se describe en la Sección 5.6.1 (Figura 7.9). Al subir el documento, antes de incorporarlo, se te permite reemplazarlo mediante -.º eliminarlo mediante "x" de la propia subida (Figura 7.10).

> Reconstrucción del corpus

▼ Incorporar un documento verificado

URL oficial (debe estar ya en el corpus, se conserva para las citas)

Título mostrado en las citas

Tipo Categoría (opcional)

horario

Documento verificado (.txt, un registro por línea)

Upload 200MB per file • TXT

Incorporar documento

> Revisar el corpus

> Limpiar el corpus

Figura 7.9: Incorporación de un documento verificado al corpus.

Tipo Categoría (opcional)

horario

Documento verificado (.txt, un registro por línea)

horario...5-2026.txt 59.8KB +

Incorporar documento

> Revisar el corpus

Figura 7.10: Reemplazo o eliminación del documento verificado.

Revisar el corpus. Ofrece un informe resumen del corpus (número de documentos, categorías, tamaños) y una búsqueda por término para inspeccionar qué se ha indexado (Figura 7.11).



> Incorporar un documento verificado

▼ Revisar el corpus

Informe resumen

Buscar en el corpus

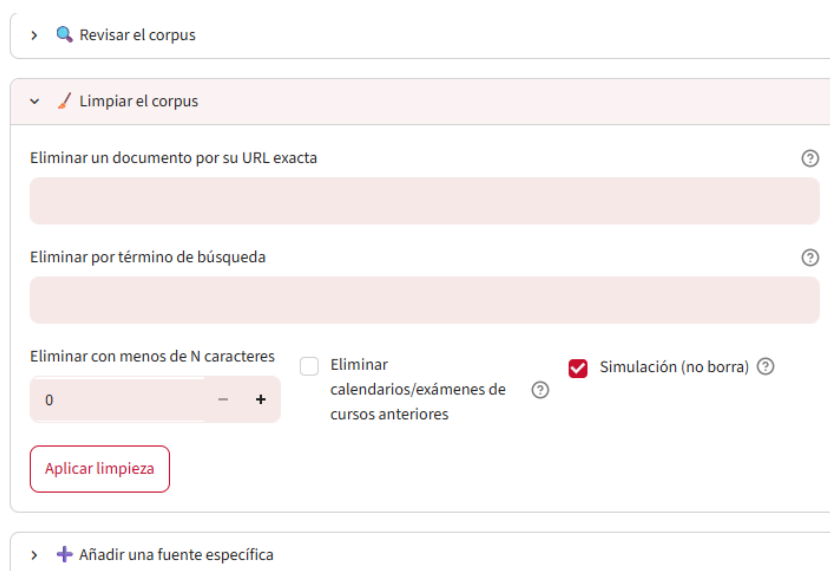
término...

Buscar

> Limpiar el corpus

Figura 7.11: Revisión del corpus: informe resumen y búsqueda por término.

Limpiar el corpus. Depura documentos irrelevantes o duplicados según criterios configurables, manteniendo el corpus libre de ruido (Figura 7.12).



> Revisar el corpus

▼ Limpiar el corpus

Eliminar un documento por su URL exacta ?

Eliminar por término de búsqueda ?

Eliminar con menos de N caracteres

0 - +

☐ Eliminar calendarios/exámenes de cursos anteriores ?

☒ Simulación (no borra) ?

Aplicar limpieza

> + Añadir una fuente específica

Figura 7.12: Limpieza del corpus: depuración de documentos irrelevantes o duplicados.

Añadir una fuente específica. Añade una URL concreta junto con su información al corpus documental dando la opción de incluirla en una categoría. (Figura 7.13).

The screenshot shows a web interface with a sidebar on the left containing three items: 'Limpiar el corpus' (Clean the corpus) with a trash icon, 'Añadir una fuente específica' (Add specific source) with a plus icon, and 'Indexación del corpus' (Index the corpus) with a document icon. The 'Añadir una fuente específica' item is selected and expanded, showing a form with two input fields: 'URL del documento a añadir' (Document URL to add) and 'Categoría' (Category). The 'Categoría' field has 'extra' entered. Below the fields is a red button labeled 'Añadir fuente' (Add source).

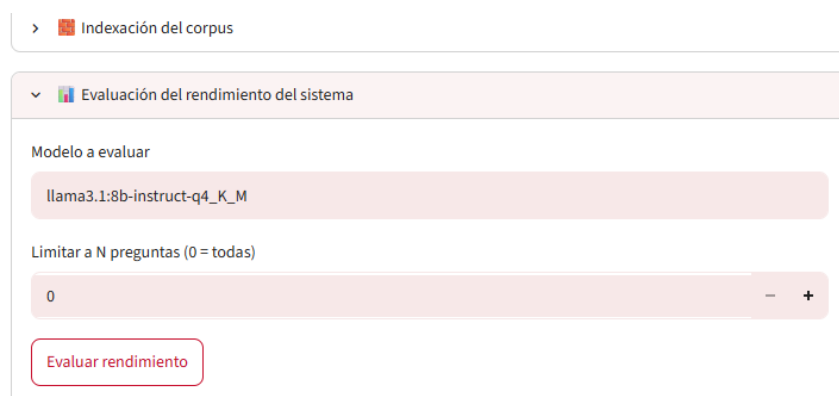
Figura 7.13: Adición de una fuente específica al corpus.

Indexación del corpus. Reconstruye los índices de recuperación (vectorial FAISS y léxico BM25) a partir del corpus actual, dejando el sistema listo para responder (Figura 7.14).

The screenshot shows the same sidebar as Figure 7.13, but now the 'Indexación del corpus' item is selected and expanded. It displays a checkbox labeled 'Simulación (opcional): solo calcular estadísticas de fragmentación' (Simulation (optional): only calculate fragmentation statistics) with a question mark icon. Below the checkbox is a red button labeled 'Reindexar corpus' (Reindex corpus).

Figura 7.14: Indexación: reconstrucción de los índices FAISS y BM25.

Evaluación del rendimiento del sistema. Lanza desde la propia interfaz la batería de métricas descrita en este capítulo, facilitando la verificación periódica de la calidad, el modelo que aparece es el predeterminado, se puede cambiar a otro para su evaluación siempre que dicho modelo esté instalado en la máquina (Figura 7.15).



> Indexación del corpus

▼ Evaluación del rendimiento del sistema

Modelo a evaluar

llama3.1:8b-instruct-q4_K_M

Limitar a N preguntas (0 = todas)

0 - +

Evaluar rendimiento

Figura 7.15: Evaluación del rendimiento: lanzamiento de la batería de métricas.

7.12. Discusión de resultados

Los resultados validan las tres hipótesis y, sobre todo, evidencian que en un dominio cerrado y especializado *la calidad de un asistente no la determina el tamaño del modelo, sino la arquitectura de recuperación y las salvaguardas de calidad* que la rodean: el mismo LLM pasa del 5 % al 87 % de precisión gracias al RAG, y de alucinar en dos de cada tres consultas sin información a abstenerse correctamente casi siempre.

Fortalezas. La separación *recall/ranking*, el *gate* de ámbito basado en el margen del *cross-encoder*, los filtros deterministas de coherencia y la curación del corpus con documentos verificados constituyen un sistema robusto cuya fiabilidad (citación 100 %, abstención 100 %, cero fugas) es especialmente valiosa en un contexto académico donde una respuesta inventada es peor que ninguna.

Limitaciones y amenazas a la validez. Procede ser explícito sobre los límites del estudio. (i) El conjunto de evaluación, aunque variado y construido sobre todo el corpus, es de tamaño moderado (74 preguntas) y autoral, por lo que las cifras deben leerse como evidencia indicativa, no como una medida poblacional; no existe un *benchmark* público comparable para este dominio. (ii) La evaluación de la precisión factual por palabras clave es una aproximación que puede ser conservadora (penaliza respuestas correctas con otra redacción) o, en agregación, indulgente. (iii) Los fallos residuales de *granularidad intra-documento* señalan la principal vía de mejora: un troceo

más fino de las páginas multi-sección y multi-registro, o una atribución por frase (*Evidence Attribution Layer*), que se proponen como trabajo futuro (Capítulo 8). (iv) Para evitar el *sobreajuste*, las mejoras se aceptaron solo cuando no degradaban el conjunto completo, descartándose las que, como la ampliación de k_{listado} , mejoraban un caso a costa de otros.

Capítulo 8

Conclusiones y trabajo futuro

8.1. Conclusiones generales

Este trabajo ha diseñado, implementado y evaluado GRAIA, un asistente conversacional académico para la ETSIIT basado en *Retrieval-Augmented Generation* y ejecutado de forma *íntegramente local* sobre hardware de consumo. Partiendo de un corpus propio de documentación oficial del centro, el sistema integra una cadena completa, ingesta y limpieza del corpus, indexación vectorial y léxica, recuperación híbrida con reordenación neuronal, generación con un LLM abierto y un sistema de citación verificable, gobernada por el principio rector de *fidelidad a las fuentes*: es preferible que el asistente admita su desconocimiento a que invente una respuesta.

La conclusión transversal del trabajo, respaldada por la evaluación del Capítulo 7, es que **en un dominio cerrado y especializado la calidad de un asistente no la determina el tamaño del modelo, sino la arquitectura de recuperación y las salvaguardas de calidad que lo rodean**: el mismo modelo de 8 B pasa de ser prácticamente inservible sin contexto (5 % de acierto factual, alucinación en dos de cada tres consultas sin información) a un asistente fiable con recuperación (87 % de acierto, citación verificable del 100 % y abstención correcta del 100 %).

8.2. Validación de las hipótesis

Las tres hipótesis de investigación (Sección 1.5) quedan *validadas* con la evidencia del Capítulo 7. La **calidad** (H1) se cumple en los tres modelos, con precisión factual por encima del 80 % y citación verificable del 100 %.

La **reducción de alucinaciones** (H2) es contundente y consistente: la recuperación convierte a un LLM que, sin contexto, acierta apenas un 3–8 % de las consultas factuales y alucina masivamente ante la ausencia de datos, en un asistente fiable. Y la **viabilidad local** (H3) se confirma para los modelos de 8–9 B, dentro del umbral de 5 s (P95) en una GPU de consumo, quedando el de 14 B descartado por latencia. Tanto Llama 3.1 8B como Gemma 2 9B cumplen las tres hipótesis; el primero se adopta como modelo por defecto por su mayor prudencia (abstención del 100 % frente al 82 % de gemma), decisiva en un asistente donde abstenerse es preferible a inventar. El detalle cuantitativo y por modelo se recoge en la Sección 7.10.

8.3. Cumplimiento de los objetivos específicos

Más allá de las hipótesis, conviene revisar uno a uno los nueve objetivos específicos planteados en la Sección 1.4.2, indicando cómo se han abordado, con qué medios y en qué parte de la memoria se desarrollan. Las matrices de trazabilidad de los Capítulos 5 y 6 (Cuadros 5.5 y 6.9) recogen esta correspondencia de forma sistemática, a continuación se sintetiza.

- **OE1 (estado del arte).** Cumplido mediante la revisión de los modelos de lenguaje, las arquitecturas RAG, la recuperación vectorial, los chatbots académicos y las métricas de evaluación del Capítulo 3.
- **OE2 (ingesta de documentos heterogéneos).** Abordado con un *crawler* dirigido por categorías (`sources.yaml`), extracción con PyMuPDF, limpieza, deduplicación e incorporación de documentos verificados, diseño en la Sección 5.6 e implementación en la Sección 6.3.
- **OE3 (indexación con *chunking*).** Resuelto con troceo recursivo e indexación a nivel de registro para los datos tabulares, *embeddings* E5 e índice FAISS, diseño en las Secciones 5.7 y 5.7.4, implementación en la Sección 6.4.
- **OE4 (motor de recuperación).** Cubierto por la recuperación híbrida (denso + BM25 + RRF), la reordenación con *cross-encoder*, MMR, los filtros de coherencia y el *gate* de ámbito, diseño en la Sección 5.8, implementación en la Sección 6.5.
- **OE5 (LLM local con contexto).** Logrado con Llama 3.1 8B ejecutado localmente mediante Ollama y un *prompt* de sistema con reglas de fidelidad y citación, diseño en la Sección 5.9, implementación en la Sección 6.6.

- **OE6 (citación trazable).** Materializado con el sistema de citación y su validación y recuperación de fuentes, diseño en la Sección 5.10, implementación en la Sección 6.6.
- **OE7 (interfaz web funcional).** Implementado con la interfaz conversacional en Streamlit y el panel de administración, diseño en la Sección 5.11, implementación en la Sección 6.7.
- **OE8 (dataset de evaluación y métricas).** Cumplido con el conjunto de evaluación propio y las métricas cuantitativas (precisión factual, *recall* de recuperación, citación verificable, abstención y latencia), Secciones 7.3 y 7.5.
- **OE9 (comparativa RAG frente a LLM sin contexto).** Abordado con la comparación del sistema frente al mismo modelo sin recuperación, complementada con la comparación entre tres modelos, Sección 7.6.

Con ello quedan cubiertos los nueve objetivos específicos y, con ellos, el objetivo general (Sección 1.4.1) de diseñar, implementar y evaluar un sistema RAG académico local, preciso, trazable y eficiente.

8.4. Contribuciones del trabajo

Más allá de la construcción del sistema, el trabajo aporta:

- **Una arquitectura RAG local de calidad para dominios homogéneos.** La combinación de recuperación híbrida (denso + BM25 + RRF), reordenación con *cross-encoder* y la separación explícita *recall/ranking* resulta especialmente eficaz en un corpus donde muchos documentos comparten vocabulario.
- **Un *gate* de ámbito de coste casi nulo.** El uso del margen del *cross-encoder*, ya calculado, como detector de consultas fuera de dominio, con un clasificador LLM solo en la banda dudosa, es una solución original y barata al problema del rechazo fuera de ámbito en corpus homogéneos.
- **Salvaguardas deterministas de fidelidad.** Los filtros de coherencia de asignatura y tipo, el control de ruido por categoría, la canonicalización de la abstención y la verificación/recuperación de citas conforman una capa de calidad que compensa de forma fiable las limitaciones de un LLM pequeño.

- **Limpieza de corpus para datos tabulares.** La metodología de documentos verificados (un registro autocontenido por dato) para fuentes que la extracción de PDF degrada.
- **Una evaluación rigurosa y reproducible** con comparación frente a un *baseline* sin recuperación, que cuantifica la contribución del RAG y analiza honestamente los errores.

8.5. Limitaciones identificadas

El trabajo presenta limitaciones que conviene reconocer. La principal de *sistema* es la *granularidad intra-documento*: cuando el documento correcto se recupera pero contiene varias secciones o registros, el modelo puede tomar el equivocado. Persisten asimismo los *límites propios de los modelos de tamaño reducido* (alucinaciones puntuales de un dato presente en el contexto, agregación incompleta). En cuanto a la *evaluación*, el conjunto de 74 preguntas, aunque variado y fundamentado, es de tamaño moderado y autoral, no existe un *benchmark* público para este dominio, y la medición por palabras clave es una aproximación. Finalmente, el sistema opera sobre un corpus *estático*: no detecta cambios en las fuentes oficiales sin una reconstrucción manual.

8.6. Líneas de trabajo futuro

Las limitaciones anteriores y el desarrollo del trabajo abren varias líneas de mejora:

- **Atribución de evidencia por frase** (*Evidence Attribution Layer*): verificar y atribuir *cada afirmación* de la respuesta al fragmento que realmente la soporta, elevando la trazabilidad del nivel de respuesta al de oración [48, 49]. El sistema actual garantiza la trazabilidad a nivel de *respuesta*, la verificación y recuperación de citas (Sección 5.10.4) asegura que toda respuesta se atribuye a fuentes realmente recuperadas, pero no que cada oración individual esté respaldada por su fragmento exacto: una respuesta de varias frases podría arrastrar un matiz no sustentado bajo la misma cita. Esta capa trasladaría *al interior del sistema y en tiempo de ejecución* la misma lógica que la métrica *faithfulness* de RAGAS aplica *offline* (Sección 7.7): descomponer la respuesta en afirmaciones atómicas y contrastar cada una con el contexto. Se reserva como trabajo futuro porque introduce una pasada de verificación adicional, con su coste de latencia, en tensión con el presupuesto de la arquitectura de pasada única (RNF-01), y exige

su propia evaluación; no era necesaria para validar las hipótesis, ya satisfechas con la trazabilidad a nivel de respuesta.

- **Mitigación de la granularidad intra-documento:** troceo más fino de páginas multi-sección y realce del registro pertinente dentro de un mismo documento [40, 39]. Es la respuesta directa al *principal error residual* identificado en la evaluación (Sección 7.8): el documento correcto se recupera, pero el modelo toma la sección o el registro equivocado dentro de él (un plazo distinto del solicitado, o las asignaturas comunes en lugar de las de la mención concreta). Generalizaría a las páginas de *prosa* multi-sección la misma idea que el sistema ya aplica con éxito a los datos tabulares, la indexación a nivel de registro, una unidad recuperable por registro (Sección 5.7.4), de modo que la recuperación y el *reranking* seleccionen exactamente la pieza pedida en vez de entregar un bloque grueso del que el modelo deba aislar la parte relevante. Queda como trabajo futuro porque implica rediseñar la estrategia de *chunking* y añadir una lógica de realce, con su correspondiente ronda de evaluación; la granularidad actual es suficiente para la fiabilidad que validan las hipótesis.
- **Crawl incremental:** un modo de actualización (`--update`) que detecte y reindexe solo los documentos cambiados al inicio de cada curso, evitando la reconstrucción completa del corpus.
- **Arquitectura multi-tenant:** un selector inicial de universidad o escuela con enrutamiento dinámico del corpus, que extienda el sistema a otros centros (generalizando el RNF-11 de portabilidad). Requeriría *particionar el corpus por centro*, ya sea mediante índices independientes por escuela o mediante un único índice con *espacio de nombres* y filtrado por metadatos de pertenencia, de modo que cada consulta se resuelva exclusivamente sobre el corpus de su centro y se evite la contaminación de información entre instituciones.
- **Escalabilidad:** el sistema se ha construido deliberadamente como un prototipo *monousuario* y de corpus acotado, interfaz Streamlit e índice FAISS `IndexFlatIP` de búsqueda exhaustiva, una decisión consciente por simplicidad y para ceñirse al alcance del TFG, que es en sí una acotación de un proyecto de mayor envergadura. La escalabilidad no era necesaria para validar las hipótesis, pero se ha tenido en cuenta qué habría que cambiar para abordarla: (i) sustituir el índice exhaustivo (búsqueda lineal $O(n)$) por uno *aproximado*, HNSW [62] o IVF-PQ [63] de FAISS, para que la latencia de recuperación crezca de forma sublineal con el tamaño del corpus; (ii) desacoplar la interfaz del *backend*, exponiendo el *pipeline* como un servicio (p. ej. una API REST con FastAPI) y reservando Streamlit para el cliente, lo que habilita la con-

currencia multiusuario; (iii) servir el LLM con un motor de inferencia con *batching* de peticiones (p. ej. vLLM [78]) en lugar de la ejecución secuencial de Ollama; y (iv) introducir caché de respuestas frecuentes y paralelizar la fase de indexación. El bajo acoplamiento por capas (RNF-10) facilita justamente este tipo de sustituciones sin reescribir el resto del sistema.

- **Retroalimentación del usuario:** un mecanismo para que el estudiante valore las respuestas (descartado del alcance actual para acotar el proyecto), que permitiría una mejora continua guiada por uso real.
- **Enriquecimiento de las citas:** incorporar el extracto del fragmento y la ruta de sección al objeto de cita para mostrarlos en la interfaz.

8.6.1. Hacia una plataforma académica integral

Las líneas anteriores refinan el sistema *actual*. Más allá de ellas, conviene explicitar que este TFG es una *acotación consciente* de un proyecto de mayor alcance: una plataforma que sitúe a GRAIA como núcleo de la experiencia digital del estudiante en la UGR. Las siguientes direcciones definen esa visión y, aunque exceden con mucho el alcance de un Trabajo Fin de Grado, se apoyan directamente en las bases sentadas aquí.

- **Plataforma unificada de servicios de la UGR (móvil y escritorio).** Una aplicación en la que GRAIA no sea solo un *chat*, sino la puerta de entrada a los servicios de la universidad: desde la propia *app*, acceso al correo institucional, a PRADO, SWAD, Hermes y demás plataformas, mediante *inicio de sesión único* con la cuenta oficial de la UGR y la vinculación de las distintas cuentas correspondientes a las plataformas específicas, con las debidas garantías de seguridad y privacidad. Supone elevar el prototipo monousuario a un producto multiusuario, lo que se apoya directamente en el desacoplamiento del *backend* y la concurrencia descritos en la línea de *escalabilidad*.
- **Accesibilidad y diseño universal: conversación por voz.** Pensando especialmente en las personas con *discapacidad visual*, una opción de conversación por voz *bidireccional*: GRAIA con voz propia (síntesis de voz) y entrada hablada del usuario (reconocimiento de voz). La opción se activaría desde los ajustes y quedaría *persistente* en la cuenta hasta su desactivación, de modo que, al abrir la aplicación, la persona pueda dirigirse directamente a GRAIA por voz *sin depender de un tercero* que reactive el modo en cada uso. Es una aplicación directa de los principios de diseño universal y de accesibilidad.

- **Atención multilingüe para estudiantes internacionales.** Ofrecer al alumnado de movilidad (Erasmus) una conversación en su *lengua nativa* (inglés, francés, alemán...), facilitando la comprensión de cualquier ámbito de la UGR. Esta línea se apoya en una base *ya presente* en el sistema: tanto el modelo de *embeddings* (**multilingual-e5**) [91] como el *reranker* (mmarco multilingüe) son multilingües, por lo que la recuperación ya opera entre idiomas; el esfuerzo se concentraría en la generación y la interfaz en el idioma del usuario. Generaliza el RNF-11 de portabilidad y enlaza con la arquitectura multi-tenant.
- **Tutoría de contenidos con modelos y *hardware* de mayor capacidad.** Con un *hardware* más potente y un modelo de mayor tamaño, levantando la restricción del RNF-12 y la acotación de dominio asumida en este TFG, extender GRAIA más allá de la información académico-administrativa para que, por ejemplo, asista al estudiante con las *dudas de contenido* de cada asignatura. Requeriría ampliar el corpus con materiales docentes y salvaguardas adicionales para *complementar*, y no sustituir, el criterio del profesorado.
- **Personalización a partir de la cuenta vinculada.** Una vez asociada la cuenta oficial, respuestas *personalizadas* según las asignaturas matriculadas, el horario o el expediente del estudiante (p. ej. “¿cuándo es mi próximo examen?”), siempre bajo estrictas garantías de protección de datos personales (RGPD).
- **Proactividad: avisos y recordatorios.** Pasar de un asistente *reactivo* a uno *proactivo*, con notificaciones de plazos, fechas de examen y trámites relevantes, anticipándose a las necesidades del estudiante.
- **Evaluación de usabilidad y experiencia de usuario.** Una valoración formal de la usabilidad, por ejemplo, mediante el cuestionario *System Usability Scale* (SUS) o una evaluación heurística con usuarios reales, se reserva para esta plataforma de mayor envergadura. Este Trabajo Fin de Grado se centra deliberadamente en la calidad y la fiabilidad del sistema RAG, evaluadas con métricas cuantitativas y con RAGAS, y no en la experiencia de uso de una interfaz de producto.

8.7. Uso de la inteligencia artificial en el proyecto

La inteligencia artificial está presente en este trabajo en un doble sentido que conviene explicitar con transparencia.

En primer lugar, como **objeto de estudio**: el núcleo de GRAIA es un modelo de lenguaje que actúa mediante un chat para conversar con el usuario, como comunmente conocemos hoy en día chatgpt, gemini, claude,

grok, etc., y todo el trabajo gira en torno a construir, gobernar y evaluar un sistema basado en IA. El proyecto no se limita, por tanto, a *usar* IA, sino que investiga cómo hacerla *fiable* en un dominio cerrado mediante la recuperación y un conjunto de salvaguardas deterministas.

En segundo lugar, como **herramienta de apoyo** durante el desarrollo. De forma coherente con la realidad actual, en la que los asistentes basados en IA se han incorporado al flujo de trabajo de la ingeniería del software, se ha empleado un asistente de IA (modelos de lenguaje como los analizados en esta memoria, específicamente Claude) como apoyo, mediante la técnica y uso de prompts en tareas concretas: revisión y depuración de código, diagnóstico de errores difíciles de localizar reduciendo en gran medida el tiempo necesario para encontrarlos y corregirlos, contraste de la coherencia interna del documento para no dejar cabos sueltos, que sin darse cuenta, se pueden olvidar u omitir en un TFG de tal magnitud, incorporación de diagramas mediante LaTeX tras el propio diseño del autor para una mayor limpieza visual, y refinamiento de la manera de redactar, posteriormente revisada nuevamente y reescrita por el autor. En todos los casos, su uso se ha regido por tres principios:

- **Supervisión humana:** el autor diseña, dirige, decide, corrige y valida cada resultado.
- **Verificación crítica:** La IA es simplemente una herramienta, puesto que como hemos visto en este TFG existe el término de alucinación y es algo muy común, no se da por válido nada de lo que realice (aunque sea ínfimo) requiriendo la corrección y verificación del propio autor, a sí mismo se contrasta todo con el código, la documentación oficial o la bibliografía, por lo que es la misma exigencia anti-alucinación que el propio sistema impone a su LLM.
- **Autoría y responsabilidad:** el diseño, la dirección, la implementación, la resolución de problemas, la experimentación, el análisis y las decisiones técnicas son todas del autor, que asume la responsabilidad sobre la corrección del trabajo.

Cabe destacar que aunque se haya usado el modelo de Claude como herramienta de apoyo, el mismo no se ha usado dentro del propio sistema (aunque el modelo es muy bueno) debido a (i) su incompatibilidad de ejecución local en el sistema propuesto dependiendo de una API externa, (ii) su licencia siendo un modelo comercial con coste y sin la privacidad que se busca en este TFG y (iii) la imposibilidad de poder personalizar el propio modelo en profundidad.

Por último, es preciso comentar que esta forma de trabajar es especialmente pertinente en un Trabajo Fin de Grado cuyo tema es, justamente,

un sistema conversacional basado en IA: emplear la IA como herramienta de apoyo mediante prompts y su correcto contraste con el propio código y documentación del autor, refleja el mismo equilibrio entre potencia y fiabilidad que se persigue en GRAIA. La IA ha llegado para quedarse y su papel seguirá creciendo, de manera que usarla con transparencia, juicio crítico y responsabilidad es, desde el punto de vista del autor, la actitud adecuada para llevar a cabo su labor en la ingeniería sin menospreciarla y adaptándose a los nuevos cambios del mundo, aumentando así mismo en gran medida la eficiencia para llevar a cabo tareas que, tanto en un proyecto solitario como en una empresa, requerirían más tiempo y coste.

8.8. Valoración personal

En lo personal, este Trabajo Fin de Grado ha sido el proyecto más completo y exigente de la carrera. Integrar en un único sistema funcional la recuperación de información, el procesamiento de lenguaje natural, el uso de diagramas y requisitos, la ingeniería de software por capas, y una evaluación experimental rigurosa me ha obligado a conectar conocimientos que hasta ahora había estudiado por separado, y a aprender otros nuevos sobre la marcha.

De todo el proceso, lo que más valoro es haber pasado de una idea a un sistema real que funciona y responde con fiabilidad sobre la documentación de la propia universidad de mi carrera. En el camino he interiorizado algunas lecciones que me llevo más allá del TFG: que en un dominio cerrado la arquitectura de recuperación y las salvaguardas de calidad importan más que el tamaño del modelo, que una evaluación honesta, que reconoce sus límites, vale más que unas cifras infladas, y que, ante un fallo, diagnosticar la causa y pensar en varias posibles soluciones antes de intentar arreglarlo sin más, ahorra mucho trabajo y evita romper lo que ya funcionaba.

No ha sido nada sencillo hacer este TFG, "dejándome la piel" para hacerlo lo mejor posible, acotando el alcance sin renunciar a la ambición, lidiando con datos tabulares que la extracción degradaba o persiguiendo errores que solo aparecían en la interfaz fueron retos que me hicieron crecer como ingeniero. Terminé el trabajo con la satisfacción de un sistema sólido y defendible y, sobre todo, con la convicción de que esto es solo el cimiento de un posible proyecto de mayor alcance que me gustaría seguir desarrollando con un equipo si se diese el caso, no únicamente yo porque estaríamos hablando de años de trabajo donde la IA habría avanzado tanto que, posiblemente, muchos conocimientos y técnicas usados en este TFG estarían ya obsoletos.

Como cierre, GRAIA demuestra que es posible construir un asistente académico *fiable, trazable y respetuoso con la privacidad*, al no depender de

servicios en la nube, con recursos modestos, y que el camino hacia esa fiabilidad pasa tanto por la ingeniería de la recuperación como por una disciplina de evaluación honesta. Pero, más allá del sistema entregado, este Trabajo Fin de Grado es ante todo un *cimiento*: la base técnica validada, recuperación robusta, salvaguardas deterministas de fidelidad y una arquitectura modular, es el suelo firme sobre el que puede levantarse la plataforma académica integral esbozada en la Sección 8.6.1, llamada a acompañar al estudiante a lo largo de toda su vida universitaria. Lo construido aquí no es un punto final, sino el primer peldaño de un proyecto con vocación de crecer, actualmente con una base pequeña en recursos, pero ambiciosa en propósito.

8.9. Acceso al sistema

Acceso al repositorio github donde se encuentra todo el código del sistema junto al readme que contiene: 1.Visión general, 2.Estructura del repositorio, 3.Requisitos, 4.Instalación, mensaje importante antes del resto de puntos, 5.Guía de ejecución paso a paso(de cero a asistente), 6.Evaluación (Capítulo 7), 7.Pruebas, 8.Scripts de diagnóstico y 9.Referencias.

Repositorio: <https://github.com/sergiomedi/TFG>

Bibliografía

- [1] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, “MTEB: Massive text embedding benchmark,” in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023, pp. 2014–2037. [Online]. Available: <https://arxiv.org/abs/2210.07316>
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [4] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. [Online]. Available: <https://arxiv.org/abs/2202.03629>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [7] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in

- Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734. [Online]. Available: <https://arxiv.org/abs/1406.1078>
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [9] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>
- [10] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024. [Online]. Available: <https://arxiv.org/abs/2104.09864>
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [12] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>
- [13] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Albert: A lite bert for self-supervised learning of language representations,” in *Proceedings of ICLR*, 2020. [Online]. Available: <https://arxiv.org/abs/1909.11942>
- [14] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [15] P. He, X. Liu, J. Gao, and W. Chen, “Deberta: Decoding-enhanced bert with disentangled attention,” in *Proceedings of ICLR*, 2021. [Online]. Available: <https://arxiv.org/abs/2006.03654>
- [16] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Blog*, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

- [17] OpenAI, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [18] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022. [Online]. Available: <https://arxiv.org/abs/2203.02155>
- [19] H. Touvron, T. Lavril, G. Izacard *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [20] A. Dubey, A. Jauhri, A. Pandey *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [21] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [22] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.04088>
- [23] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [24] Google DeepMind, “Gemma 2: Improving open language models at a practical size,” *arXiv preprint arXiv:2408.00118*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.00118>
- [25] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: <https://arxiv.org/abs/1910.10683>
- [26] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” *arXiv preprint arXiv:1910.13461*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.13461>

- [27] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024. [Online]. Available: <https://arxiv.org/abs/2210.11416>
- [28] M. Abdin, S. A. Jacobs, A. A. Amin, J. Aneja, A. Awadallah, H. Awadalla, N. Bach, A. Bahree, A. Bakhtiari, H. Beber *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” *arXiv preprint arXiv:2404.14219*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [29] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [30] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, “Training compute-optimal large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 016–30 030, 2022. [Online]. Available: <https://arxiv.org/abs/2203.15556>
- [31] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.17323>
- [32] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” *Proceedings of Machine Learning and Systems (MLSys)*, 2024. [Online]. Available: <https://arxiv.org/abs/2306.00978>
- [33] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [34] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [35] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp.

- 16 344–16 359, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [36] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” *arXiv preprint arXiv:2305.13245*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.13245>
- [37] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, “Large language models struggle to learn long-tail knowledge,” *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: <https://arxiv.org/abs/2211.08411>
- [38] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller, “Language models as knowledge bases?” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 2463–2473. [Online]. Available: <https://arxiv.org/abs/1909.01066>
- [39] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [Online]. Available: <https://arxiv.org/abs/2307.03172>
- [40] Y. Gao, Y. Xiong, X. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [41] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “Realm: Retrieval-augmented language model pre-training,” *arXiv preprint arXiv:2002.08909*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.08909>
- [42] V. Karpukhin, B. Oğuz, S. Min *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of EMNLP*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.04906>
- [43] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. [Online]. Available: <https://doi.org/10.1561/15000000019>
- [44] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” *arXiv preprint arXiv:2007.01282*, 2020. [Online]. Available: <https://arxiv.org/abs/2007.01282>

- [45] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” *arXiv preprint arXiv:2212.10496*, 2022. [Online]. Available: <https://arxiv.org/abs/2212.10496>
- [46] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” *arXiv preprint arXiv:1901.04085*, 2019. [Online]. Available: <https://arxiv.org/abs/1901.04085>
- [47] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336. [Online]. Available: <https://dl.acm.org/doi/10.1145/290941.291025>
- [48] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, “Measuring attribution in natural language generation models,” *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023. [Online]. Available: <https://arxiv.org/abs/2112.12870>
- [49] L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Y. Zhao, N. Lao, H. Lee, D.-C. Juan, and K. Guu, “RARR: Researching and revising what language models say, using language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 16 477–16 508. [Online]. Available: <https://arxiv.org/abs/2210.08726>
- [50] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.11511>
- [51] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2401.15884*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.15884>
- [52] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity,” *arXiv preprint arXiv:2403.14403*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.14403>
- [53] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.16130>

- [54] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” in *Proceedings of the 1st International Conference on Learning Representations (ICLR) Workshop*, 2013. [Online]. Available: <https://arxiv.org/abs/1301.3781>
- [55] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, “MTEB: Massive text embedding benchmark,” *arXiv preprint arXiv:2210.07316*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.07316>
- [56] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of EMNLP-IJCNLP*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [57] L. Wang, N. Yang, X. Huang *et al.*, “Text embeddings by weakly-supervised contrastive pre-training,” *arXiv preprint arXiv:2212.03533*, 2022. [Online]. Available: <https://arxiv.org/abs/2212.03533>
- [58] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-pack: Packaged resources to advance general chinese embedding,” *arXiv preprint arXiv:2309.07597*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.07597>
- [59] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” *arXiv preprint arXiv:2402.01613*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01613>
- [60] P. Indyk and R. Motwani, “Approximate nearest neighbors: Towards removing the curse of dimensionality,” *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 604–613, 1998.
- [61] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019. [Online]. Available: <https://arxiv.org/abs/1702.08734>
- [62] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: <https://arxiv.org/abs/1603.09320>
- [63] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011. [Online]. Available: <https://doi.org/10.1109/TPAMI.2010.57>

- [64] LangChain, “Text splitters — LangChain documentation,” 2024, documentación oficial de estrategias de fragmentación. [Online]. Available: https://docs.langchain.com/docs/modules/data_connection/document_transformers/
- [65] Universidad de Granada, “Asistente virtual Elvira de la Universidad de Granada,” https://canal.ugr.es/wp-content/uploads/2011/02/07022011_elvira.pdf, 2010, accessed: 2026-05-14.
- [66] —, “Nuevo asistente virtual llamado Alhe, dirigido al estudiantado y futuro estudiantado,” <https://www.ugr.es/universidad/noticias/asistente-virtual-llamado-ahle-dirigidoestudiantado-futuro-estudiantado>, 2024, accessed: 2026-05-14.
- [67] T. Dolci, M. Smeragliuolo, M. L. D. Vedova, and G. Zara, “Unimib assistant: designing a student-friendly RAG-based chatbot for all their needs,” *arXiv preprint arXiv:2411.19554*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.19554>
- [68] P. Nguyen, T. Quan, and D.-V. Nguyen, “URAG: Implementing a unified hybrid RAG for precise answers in university admission chatbots – a case study at HCMUT,” *arXiv preprint arXiv:2501.16276*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.16276>
- [69] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–163. [Online]. Available: <https://arxiv.org/abs/2309.15217>
- [70] N. F. Liu, T. Zhang, and P. Liang, “Evaluating verifiability in generative search engines,” *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 7001–7025, 2023. [Online]. Available: <https://arxiv.org/abs/2304.09848>
- [71] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. [Online]. Available: <https://nlp.stanford.edu/IR-book/>
- [72] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013>
- [73] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “Bertscore: Evaluating text generation with bert,” in *Proceedings of ICLR*, 2020. [Online]. Available: <https://arxiv.org/abs/1904.09675>

- [74] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [75] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.15217>
- [76] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. [Online]. Available: <https://doi.org/10.1177/001316446002000104>
- [77] K. Krippendorff, “Computing Krippendorff’s alpha-reliability,” *Departmental Papers (ASC)*, 2011. [Online]. Available: <https://repository.upenn.edu/asc-papers/43>
- [78] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [79] A. Ronacher, “Flask: A python microframework,” 2010, framework web minimalista para Python. [Online]. Available: <https://flask.palletsprojects.com/>
- [80] Meta Platforms, “React — a javascript library for building user interfaces,” 2013, librería de interfaces de usuario basada en componentes. [Online]. Available: <https://react.dev/>
- [81] IEEE, “IEEE recommended practice for software requirements specifications,” Institute of Electrical and Electronics Engineers, Standard IEEE Std 830-1998, 1998. [Online]. Available: <https://doi.org/10.1109/IEEESTD.1998.88286>
- [82] A. Cockburn, *Writing Effective Use Cases*. Boston, MA: Addison-Wesley, 2001. [Online]. Available: <https://www.pearson.com/en-us/subject-catalog/p/writing-effective-use-cases/P200000009475/9780132702256>
- [83] D. Clegg and R. Barker, *Case Method Fast-Track: A RAD Approach*. Addison-Wesley, 1994, introduces the MoSCoW prioritisation method. [Online]. Available: <https://www.worldcat.org/title/30859582>

- [84] ISO/IEC/IEEE, “ISO/IEC/IEEE international standard – systems and software engineering – life cycle processes – requirements engineering,” IEEE, Standard ISO/IEC/IEEE 29148:2018, 2018. [Online]. Available: <https://doi.org/10.1109/IEEESTD.2018.8559686>
- [85] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994. [Online]. Available: <https://www.pearson.com/en-us/subject-catalog/p/design-patterns-elements-of-reusable-object-oriented-software/P200000009480/9780201633610>
- [86] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester, UK: John Wiley & Sons, 1996. [Online]. Available: <https://www.wiley.com/en-us/Pattern+Oriented+Software+Architecture%2C+Volume+1%2C+A+System+of+Patterns-p-9780471958697>
- [87] Ollama, “Ollama: Get up and running with large language models locally,” 2023, servidor de inferencia local de LLMs en formato GGUF empleado para la generación y la contextualización. [Online]. Available: <https://ollama.com>
- [88] A. Dubey, A. Jauhri, A. Pandey *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [89] Artifex Software, “Pymupdf: Python bindings for the mupdf rendering library,” librería de extracción de texto y tablas y rasterizado de PDFs usada en la ingesta. [Online]. Available: <https://pymupdf.readthedocs.io/>
- [90] L. Richardson, “Beautiful soup: a python library for pulling data out of html and xml files,” parser de HTML utilizado en la ingesta de páginas web del corpus. [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/>
- [91] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 text embeddings: A technical report,” *arXiv preprint arXiv:2402.05672*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.05672>
- [92] Anthropic, “Introducing contextual retrieval,” 2024, técnica de pre-procesado que antepone a cada fragmento un contexto generado por un LLM para mejorar la recuperación. [Online]. Available: <https://www.anthropic.com/news/contextual-retrieval>

- [93] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2009, pp. 758–759. [Online]. Available: <https://doi.org/10.1145/1571941.1572114>
- [94] T. Formal, B. Piwowarski, and S. Clinchant, “SPLADE: Sparse lexical and expansion model for first stage ranking,” in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2021, pp. 2288–2292. [Online]. Available: <https://doi.org/10.1145/3404835.3463098>
- [95] J. Lin, R. Nogueira, and A. Yates, “Pretrained transformers for text ranking: BERT and beyond,” *Synthesis Lectures on Human Language Technologies*, vol. 14, no. 4, pp. 1–325, 2021. [Online]. Available: <https://doi.org/10.2200/S01123ED1V01Y202108HLT053>
- [96] X. Ma, K. Sun, R. Pradeep, and J. Lin, “A replication study of dense passage retriever,” in *Proceedings of the 1st Workshop on Replicability and Reproducibility (ReproNLP)*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.05740>
- [97] Anthropic, “Prompt engineering overview — use xml tags to structure your prompts,” 2024, guía de ingeniería de prompts: delimitación de secciones (contexto, instrucciones, pregunta) mediante etiquetas XML para reducir la ambigüedad. [Online]. Available: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/use-xml-tags>
- [98] R. Smith, “An overview of the Tesseract OCR engine,” in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. IEEE, 2007, pp. 629–633, motor de OCR empleado para extraer texto de los PDFs escaneados durante la ingesta. [Online]. Available: <https://ieeexplore.ieee.org/document/4376991>
- [99] L. Bonifacio, V. Jeronymo, H. Q. Abonizio, I. Campiotti, M. Fadaee, R. Lotufo, and R. Nogueira, “mmarco: A multilingual version of the ms marco passage ranking dataset,” 2021, dataset multilingüe (13 idiomas) sobre el que se entrena el cross-encoder de reranking empleado (mmarco-mMiniLMv2). [Online]. Available: <https://arxiv.org/abs/2108.13897>
- [100] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” *Advances in Neural Information Processing*

Systems (NeurIPS), 2020, arquitectura base, vía destilación, del cross-encoder multilingüe de reranking utilizado. [Online]. Available: <https://arxiv.org/abs/2002.10957>

- [101] Streamlit Inc., “Streamlit documentation: Caching,” <https://docs.streamlit.io/develop/concepts/architecture/caching>, 2024, accessed: 2026-04-18.
- [102] M. Zakaria, “python-json-logger: Json logging for python,” <https://github.com/madzak/python-json-logger>, 2024, accessed: 2026-04-18.
- [103] S. Freeman and N. Pryce, *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley Professional, 2009. [Online]. Available: <https://www.pearson.com/en-us/subject-catalog/p/growing-objectoriented-software-guided-by-tests/P200000008534/9780321503626>

Glosario

Alucinación Fenómeno por el cual un modelo de lenguaje genera información aparentemente plausible pero factualmente incorrecta o carente de soporte en sus fuentes. Constituye uno de los principales problemas de los LLMs en dominios especializados.

Ad hoc Locución latina que significa “para esto” o “para este propósito específico”. En el contexto de este trabajo, un *dataset ad hoc* es un conjunto de datos diseñado expresamente para evaluar el sistema propuesto, en contraposición a los *benchmarks* genéricos preexistentes.

ANN *Approximate Nearest Neighbors*. Familia de algoritmos que permiten buscar los vectores más cercanos en un espacio de alta dimensionalidad de forma aproximada pero eficiente, sacrificando una fracción marginal de precisión a cambio de búsquedas sublineales.

Answer relevancy (Pertinencia de la respuesta) Métrica del *framework* RAGAS que cuantifica cuán pertinente es la respuesta generada respecto a la pregunta original. Se estima generando preguntas a partir de la respuesta y midiendo su similitud con la consulta, por lo que penaliza tanto las respuestas demasiado escuetas como las que añaden información colateral. En GRAIA se emplea, junto con la *faithfulness*, en la evaluación automática del sistema.

API *Application Programming Interface*. Interfaz de programación de aplicaciones que permite la comunicación entre componentes de software.

Auto-regresivo Propiedad de un modelo generativo que produce la salida secuencialmente, prediciendo cada token condicionado a los tokens previamente generados. Los modelos decodificadores (GPT, LLaMA) operan de forma auto-regresiva.

Backoff exponencial Estrategia de reintentos en la que el tiempo de espera entre intentos sucesivos se incrementa exponencialmente (por ejemplo, 1 s, 2 s, 4 s, 8 s). Se emplea en el módulo de ingesta de GRAIA para manejar errores transitorios de red al descargar documentos sin saturar el servidor de origen.

- Batch** Conjunto de elementos procesados simultáneamente en una única operación. En GRAIA, los fragmentos se codifican en *batches* de 32 elementos para aprovechar el paralelismo de la GPU durante la generación de *embeddings*.
- BERTScore** Métrica de evaluación de generación de texto que calcula la similitud semántica token a token entre la respuesta generada y la referencia utilizando *embeddings* contextuales de BERT.
- BFS (*Breadth-First Search*)** Algoritmo de recorrido de grafos que explora todos los nodos vecinos de un nodo antes de pasar a los vecinos de estos, avanzando nivel por nivel. En el contexto del *crawler* de GRAIA, se utiliza BFS para recorrer la estructura de enlaces de la web de la ETSIIT: partiendo de las URLs semilla de cada categoría, se descubren y encolan los enlaces de cada página antes de procesarlos, garantizando una cobertura sistemática y evitando descender demasiado en ramas poco relevantes.
- Bi-encoder** Arquitectura en la que la consulta y el documento se codifican independientemente con modelos separados (o el mismo modelo aplicado por separado), generando *embeddings* comparables mediante similitud coseno. Más eficiente que un *cross-encoder* pero menos preciso en *ranking*.
- BM25 *Best Matching 25***. Función clásica de *ranking* basada en frecuencia de términos y longitud del documento, derivada del modelo probabilístico de Robertson y Z. Walker (2009). Constituye la línea base léxica frente a la que se comparan los métodos de recuperación densa.
- Chunking** Proceso de fragmentación de documentos largos en segmentos más pequeños y manejables para su indexación y recuperación en un sistema RAG.
- Cockburn (plantilla de)** Formato estandarizado para la documentación de casos de uso propuesto por Alistair Cockburn, que estructura cada caso en campos como actor principal, precondiciones, escenario principal, extensiones y postcondiciones. Se adopta en este trabajo por ser el estándar *de facto* en ingeniería del software para describir la interacción usuario-sistema de forma reproducible.
- Conexión residual** Técnica de diseño de redes neuronales profundas, introducida por He et al. (2016), en la que la entrada de cada subcapa se suma directamente a su salida ($\text{output} = \text{subcapa}(x) + x$). Facilita el flujo del gradiente durante el entrenamiento y mitiga el problema de la degradación en redes con muchas capas. Componente fundamental de cada bloque del Transformer.

Codificador (*encoder*) En la arquitectura Transformer, bloque que genera representaciones contextuales bidireccionales de la secuencia de entrada, donde cada token atiende a todos los demás simultáneamente.

Concordancia inter-anotadores Medida estadística del grado de acuerdo entre dos o más evaluadores humanos al etiquetar o puntuar los mismos datos. Se cuantifica mediante coeficientes como el κ de Cohen o el α de Krippendorff.

Corpus Conjunto estructurado de documentos de texto utilizado como fuente de conocimiento de un sistema de procesamiento de lenguaje natural. En este trabajo, el corpus está compuesto por las guías docentes, normativas y demás documentación oficial de la ETSIT y la UGR.

CMS (*Content Management System*) Plataforma de software que permite crear, gestionar y publicar contenido web sin necesidad de programar directamente en HTML. Ejemplos ampliamente extendidos en el ámbito universitario son WordPress, Joomla y Drupal. La web de la ETSIT utiliza Drupal (véase *Drupal*), lo que determina la estructura de URLs y el formato del HTML que el *crawler* debe procesar.

Cross-attention Mecanismo de atención en el que las *queries* provienen de un módulo (e.g., el decodificador) y las *keys/values* de otro (e.g., el codificador), permitiendo que un componente atienda a las representaciones de otro.

Cross-encoder Modelo que recibe como entrada conjunta una consulta y un documento candidato y produce una puntuación escalar de relevancia. A diferencia de los modelos *bi-encoder* (que codifican consulta y documento por separado), captura interacciones finas entre ambos textos, lo que se traduce en mayor precisión de *ranking* a costa de un mayor coste computacional. Se emplea típicamente como segunda etapa en arquitecturas de *reranking*.

Cuantización Técnica de compresión que reduce la precisión numérica de los pesos de un modelo de lenguaje (por ejemplo, de 16 bits a 4 bits) con el objetivo de disminuir su consumo de memoria y acelerar la inferencia, a cambio de una pérdida acotada de calidad.

Dataset Conjunto de datos estructurado, típicamente compuesto por pares de entrada y salida esperada, utilizado para entrenar o evaluar un modelo. En este trabajo, el *dataset* de evaluación contiene preguntas académicas con respuestas de referencia anotadas manualmente.

Decodificador (*decoder*) En la arquitectura Transformer, bloque que genera la salida de forma auto-regresiva utilizando atención causal (cada

token solo atiende a los previos) y, opcionalmente, atención cruzada sobre las representaciones del codificador.

Destilación de conocimiento Técnica de compresión de modelos en la que un modelo pequeño (alumno) se entrena para imitar las distribuciones de salida de un modelo mayor (profesor), logrando retener gran parte del rendimiento con menor coste computacional.

DPR *Dense Passage Retrieval*. Método de recuperación densa propuesto por Karpukhin et al. (2020) que emplea modelos *bi-encoder* basados en BERT para codificar consultas y pasajes como vectores densos, demostrando la superioridad de la recuperación densa frente a métodos léxicos como BM25 en *question answering* de dominio abierto.

Drupal Sistema de gestión de contenidos (CMS) de código abierto, escrito en PHP, ampliamente utilizado por instituciones académicas y gubernamentales. Se caracteriza por una arquitectura modular extensible, un sistema de taxonomías jerárquicas y un esquema de URLs con sufijos de versionado (*_0*, *_1*, ...) que refleja revisiones internas del contenido. La web de la ETSIIT emplea Drupal, lo que condiciona las estrategias de normalización de URLs y filtrado del *crawler* desarrollado en este trabajo.

Embedding Representación vectorial densa de un fragmento de texto en un espacio de alta dimensionalidad que captura relaciones semánticas entre términos y frases.

ETSIIT Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada.

FAISS *Facebook AI Similarity Search*. Biblioteca de búsqueda eficiente de vectores similares en grandes colecciones, desarrollada por Meta AI.

Feed-forward (red) Red neuronal densa de dos capas, con una función de activación no lineal intermedia, aplicada independientemente a cada posición de la secuencia dentro de un bloque Transformer. Expande la dimensionalidad interna (típicamente $\times 4$) y la reduce de nuevo, introduciendo capacidad representativa no lineal que complementa al mecanismo de atención.

Few-shot Modo de evaluación o uso de un LLM en el que se proporcionan unos pocos ejemplos de la tarea en el *prompt*, sin modificar los pesos del modelo. Se contrapone al *zero-shot* (sin ejemplos) y al *fine-tuning* (con entrenamiento).

Faithfulness (Fidelidad) Métrica del *framework* RAGAS que mide la proporción de afirmaciones de una respuesta que están respaldadas por

el contexto recuperado; es una medida directa de la ausencia de alucinación. Un LLM actúa como juez (véase *LLM-as-Judge*), descomponiendo la respuesta en afirmaciones atómicas y verificando cada una contra los fragmentos. Es la métrica RAGAS central en la evaluación de fidelidad de GRAIA.

Fine-tuning Proceso de ajuste adicional de un modelo preentrenado sobre un conjunto de datos específico de un dominio, con el objetivo de especializarlo en una tarea o contexto concreto.

Flash Attention Algoritmo de cómputo de atención propuesto por Dao et al. (2022) que reformula la operación para maximizar el uso de la jerarquía de memoria de la GPU (SRAM vs. HBM), logrando aceleraciones de $2-4\times$ sin alterar el resultado numérico.

GGUF *GPT-Generated Unified Format*. Formato binario de archivo para almacenar modelos de lenguaje cuantizados, diseñado por el proyecto `llama.cpp`. Incluye los pesos, la configuración del modelo y los metadatos del *tokenizer* en un único archivo portable, lo que simplifica la distribución y carga de modelos locales.

GPU *Graphics Processing Unit*. Unidad de procesamiento gráfico, utilizada de forma generalizada para la aceleración de cálculos matriciales en inteligencia artificial.

GQA *Grouped-Query Attention*. Técnica introducida por Ainslie et al. (2023) que comparte claves y valores entre múltiples cabezas de atención, reduciendo el tamaño del KV-Cache y la memoria requerida durante la inferencia. Adoptada por Mistral y LLaMA 3.

GRAIA *Granada Retrieval-Augmented Intelligent Academic-assistant*. Denominación del sistema conversacional desarrollado en este TFG, que combina recuperación de información sobre el corpus de la ETSIT con generación mediante un LLM local.

GRU *Gated Recurrent Unit*. Variante simplificada de LSTM propuesta por Cho et al. (2014) que fusiona las puertas de entrada y olvido en una única puerta de actualización, reduciendo la complejidad del modelo con rendimiento comparable.

HNSW *Hierarchical Navigable Small World*. Algoritmo de búsqueda aproximada de vecinos más cercanos que construye un grafo navegable multicapa, ofreciendo alto *recall* con baja latencia.

HyDE *Hypothetical Document Embeddings*. Técnica de pre-recuperación que genera un documento hipotético a partir de la consulta del usuario y utiliza su *embedding* para buscar en el índice, mitigando la brecha semántica entre consultas cortas y documentos largos.

IEEE 830-1998 Estándar del *Institute of Electrical and Electronics Engineers* que define las prácticas recomendadas para la especificación de requisitos software (*Software Requirements Specification*). Proporciona la estructura de identificador, nombre, descripción, prioridad y métrica seguida en el Capítulo 4 para cada requisito funcional y no funcional.

In-context learning Capacidad de los LLMs de realizar tareas proporcionándoles instrucciones y/o ejemplos directamente en el *prompt*, sin modificar sus pesos. Formalizado por Brown et al. (2020) en GPT-3.

Inferencia Proceso de ejecución de un modelo de lenguaje ya entrenado para generar una respuesta a partir de una entrada dada. Es la fase que se ejecuta sobre la GPU local en GRAIA.

Instruction tuning Técnica de ajuste fino en la que un LLM se entrena sobre un conjunto diverso de tareas formuladas como instrucciones en lenguaje natural, mejorando sustancialmente su capacidad de seguir indicaciones del usuario.

Intent-based Enfoque de chatbot basado en la clasificación de la intención del usuario entre un conjunto predefinido de categorías, devolviendo respuestas preconfiguradas asociadas a cada intención.

JSONL *JSON Lines*. Formato de archivo donde cada línea es un objeto JSON independiente. Se emplea en GRAIA para el registro estructurado de eventos, permitiendo el análisis posterior con herramientas estándar de procesamiento de texto y datos.

Knowledge cutoff Fecha de corte del conocimiento de un LLM: toda información posterior a la fecha en que se recopilaron sus datos de entrenamiento resulta inaccesible para el modelo.

KV-Cache Técnica estándar en la inferencia auto-regresiva que almacena las claves (*keys*) y valores (*values*) calculados para tokens previos, evitando su recómputo en cada paso de generación y reduciendo la complejidad por token.

Layer Normalization Técnica de normalización propuesta por Ba et al. (2016) que reescala las activaciones de cada ejemplo a media cero y varianza unitaria a lo largo de la dimensión de características (en contraste con *Batch Normalization*, que normaliza a lo largo del *batch*). Estabiliza y acelera el entrenamiento de redes profundas y se aplica en cada subcapa del Transformer.

LLM *Large Language Model*. Modelo de lenguaje de gran escala entrenado sobre volúmenes masivos de texto, capaz de comprender y generar lenguaje natural.

LLM-as-Judge Metodología de evaluación en la que un LLM (típicamente GPT-4) actúa como evaluador automático, puntuando respuestas según criterios predefinidos. Múltiples estudios han demostrado alta correlación con las evaluaciones humanas.

LoRA *Low-Rank Adaptation*. Técnica PEFT que congela los pesos originales del modelo e inyecta matrices de bajo rango entrenables en las capas de atención, reduciendo drásticamente los parámetros entrenables.

Lost in the middle Fenómeno observado en LLMs con ventanas de contexto largas por el cual la calidad de la atención a información situada en posiciones intermedias del contexto se degrada, favoreciendo la información al principio y al final.

LSTM *Long Short-Term Memory*. Variante de red recurrente propuesta por Hochreiter y Schmidhuber (1997) que introduce puertas de entrada, olvido y salida para controlar el flujo de información, mitigando el problema de la desaparición del gradiente en secuencias largas.

MLM *Masked Language Modeling*. Objetivo de preentrenamiento introducido por BERT en el que se enmascara aleatoriamente un porcentaje de tokens de la entrada y el modelo debe predecirlos a partir del contexto circundante bidireccional.

MMR *Maximal Marginal Relevance*. Criterio de reordenación de resultados propuesto por Carbonell y Goldstein (1998) que equilibra la relevancia de cada documento respecto a la consulta con su diversidad respecto a los ya seleccionados, mediante un parámetro $\lambda \in [0, 1]$. Se utiliza para mitigar la redundancia entre fragmentos recuperados en sistemas RAG.

MoE *Mixture of Experts*. Arquitectura de red neuronal que emplea múltiples subredes especializadas (expertos) y un mecanismo de enrutamiento que selecciona dinámicamente qué expertos activar para cada entrada, permitiendo escalar el número total de parámetros manteniendo constante el coste computacional por inferencia.

MoSCoW Técnica de priorización de requisitos cuyo acrónimo codifica cuatro niveles: *Must have* (M, imprescindible), *Should have* (S, deseable), *Could have* (C, opcional) y *Won't have* (W, fuera de alcance). Permite acotar el trabajo al tiempo disponible del TFG sin perder trazabilidad de las funcionalidades pospuestas.

NFC *Normalization Form Canonical Composition*. Forma de normalización Unicode que combina secuencias de carácter base + diacrítico en un único *code point* precompuesto cuando existe. Se aplica en la fase de limpieza textual para garantizar que fragmentos visualmente idénticos

tengan una representación binaria única, condición necesaria para la consistencia de los *embeddings*.

NLP *Natural Language Processing*. Procesamiento del lenguaje natural; rama de la inteligencia artificial dedicada al tratamiento computacional del lenguaje humano.

NLU *Natural Language Understanding*. Subárea del NLP centrada en la comprensión del significado del texto, incluyendo tareas como la clasificación de intenciones, el reconocimiento de entidades y la resolución de correferencias.

Ollama Plataforma de código abierto para la ejecución local de modelos de lenguaje de gran escala, utilizada en este trabajo para desplegar el LLM que alimenta a GRAIA.

Overlap Solapamiento entre fragmentos consecutivos durante el proceso de *chunking*. Al repetir un número fijo de *tokens* (típicamente entre 50 y 100) al inicio de cada fragmento, se preserva la continuidad semántica en los límites de corte, evitando que una idea dividida entre dos fragmentos pierda contexto en ambos.

PagedAttention Técnica de gestión de memoria para la atención en LLMs, propuesta por Kwon et al. (2023) en vLLM, que aplica el concepto de paginación del sistema operativo al KV-Cache, permitiendo servir múltiples peticiones concurrentes de forma eficiente.

PEFT *Parameter-Efficient Fine-Tuning*. Familia de técnicas que permiten adaptar un LLM a un dominio específico modificando solo una fracción mínima de sus pesos, reduciendo drásticamente los requisitos de memoria y computación respecto al *fine-tuning* completo.

Perplejidad Métrica intrínseca de calidad de un modelo de lenguaje que mide cuán bien predice una muestra de texto. Valores más bajos indican mejor capacidad predictiva. Se calcula como la exponencial de la entropía cruzada media.

Pipe and Filter Patrón arquitectónico en el que los datos fluyen a través de una secuencia de etapas (*filters*) conectadas por canales (*pipes*). Cada etapa recibe una entrada, la transforma y la pasa a la siguiente. En GRAIA, el *pipeline* de ingesta (descarga → extracción → normalización → segmentación) sigue este patrón.

Pipeline Secuencia encadenada de etapas de procesamiento donde la salida de cada etapa alimenta la entrada de la siguiente. En el contexto RAG, el *pipeline* típico comprende ingesta, indexación, recuperación y generación.

Prompt Instrucción textual proporcionada a un modelo de lenguaje para guiar su respuesta. Incluye tanto las directrices del sistema (*system prompt*) como la consulta del usuario y, en RAG, el contexto recuperado.

Q4_K_M Esquema de cuantización a 4 bits por peso implementado en `llama.cpp`, donde “K” indica el uso de cuantización por bloques con escalas diferenciadas y “M” denota la variante *medium* que equilibra compresión y calidad. Preserva aproximadamente el 97 % de la perplejidad del modelo original, lo que la convierte en el compromiso estándar entre fidelidad y ocupación de VRAM para modelos de 7–8 B de parámetros en GPUs de consumo.

QLoRA *Quantized LoRA*. Técnica que combina la cuantización a 4 bits del modelo base con adaptadores LoRA entrenables, habilitando el ajuste fino de modelos grandes en GPUs con VRAM limitada.

RAG *Retrieval-Augmented Generation*. Arquitectura que combina un módulo de recuperación de información con un modelo generativo, de forma que las respuestas del modelo se condicionan al contexto recuperado de una base de conocimiento.

RAGAS *Retrieval-Augmented Generation Assessment*. Marco de evaluación automática de sistemas RAG que calcula métricas como *faithfulness*, *answer relevancy* o *context precision/recall*.

Re-ranking Etapa de post-recuperación en la que un modelo *cross-encoder* reordena los fragmentos recuperados inicialmente por un *bi-encoder*, mejorando la precisión del *ranking* al capturar interacciones finas entre consulta y documento.

RRF (*Reciprocal Rank Fusion*) Técnica de fusión de rankings que combina las listas de resultados de múltiples sistemas de recuperación (por ejemplo, búsqueda densa y léxica) mediante la fórmula $RRF(d) = \sum_{r \in R} \frac{1}{k+r(d)}$, donde $r(d)$ es la posición del documento d en el ranking r y k es una constante de suavizado (típicamente 60). Produce un ranking unificado que aprovecha las fortalezas complementarias de cada sistema sin requerir calibración de puntuaciones.

Retrieval Etapa de un sistema RAG encargada de localizar y devolver los fragmentos del corpus más relevantes para una consulta dada, habitualmente mediante búsqueda de similitud en el espacio de embeddings.

RGPD Reglamento General de Protección de Datos (Reglamento (UE) 2016/679). Normativa europea que regula el tratamiento de datos personales. En este trabajo fundamenta la decisión de ejecutar el sistema

íntegramente en local, evitando que las consultas de los estudiantes, que pueden contener información académica sensible, sean procesadas por servicios externos.

RLHF *Reinforcement Learning from Human Feedback*. Técnica de alineamiento de LLMs que entrena al modelo para generar respuestas alineadas con las preferencias humanas, combinando aprendizaje supervisado con aprendizaje por refuerzo sobre retroalimentación de evaluadores humanos.

RNN *Recurrent Neural Network*. Arquitectura de red neuronal que procesa secuencias de forma secuencial, manteniendo un estado oculto que se actualiza en cada paso temporal. Precursora de los Transformers, con limitaciones en paralelización y dependencias largas.

RoPE *Rotary Position Embedding*. Codificación posicional rotatoria que inyecta información de posición en las representaciones del Transformer mediante rotaciones en el espacio complejo, ofreciendo mejor generalización a secuencias más largas que las vistas durante el entrenamiento.

ROUGE *Recall-Oriented Understudy for Gisting Evaluation*. Familia de métricas para evaluación automática de resúmenes y generación de texto que mide el solapamiento de n -gramas entre la salida generada y una referencia.

Scaling laws Leyes de escalado. Relaciones empíricas que predicen el rendimiento de un LLM como función del número de parámetros, la cantidad de datos de entrenamiento y el presupuesto computacional, formalizadas por Kaplan et al. (2020).

Scraping Extracción automatizada de información de páginas web mediante programas que analizan su estructura HTML. En GRAIA se emplea para recoger contenido institucional de la ETSIT y la UGR.

Self-attention Mecanismo de atención en el que *queries*, *keys* y *values* provienen de la misma secuencia, permitiendo que cada token atienda simultáneamente a todos los demás tokens de la secuencia para capturar dependencias a cualquier distancia.

Softmax Función matemática que transforma un vector de valores reales en una distribución de probabilidad, donde cada componente está en el rango $(0, 1)$ y la suma es 1. En el mecanismo de atención del Transformer, se aplica sobre los productos escalares para obtener los pesos de atención.

Strategy (patrón) Patrón de diseño de comportamiento que encapsula una familia de algoritmos intercambiables tras una interfaz común,

permitiendo sustituir uno por otro en tiempo de ejecución sin modificar el código cliente. En GRAIA se emplea para abstraer el modelo de *embeddings* y el LLM, de modo que cambiar de modelo requiere únicamente una entrada de configuración.

Streaming Técnica de transmisión incremental en la que los tokens generados por el LLM se envían al cliente conforme se producen, en lugar de esperar a la respuesta completa. Reduce la latencia percibida por el usuario en interfaces conversacionales.

Streamlit Biblioteca de Python para el desarrollo rápido de interfaces web interactivas orientadas a aplicaciones de ciencia de datos y aprendizaje automático.

TF-IDF *Term Frequency–Inverse Document Frequency*. Esquema clásico de ponderación de términos en recuperación de información que combina la frecuencia de un término en un documento (*TF*) con su inversa de la frecuencia en la colección (*IDF*), de modo que un término frecuente en pocos documentos recibe mayor peso. Constituye la base teórica de algoritmos como BM25.

Token Unidad mínima de texto procesada por un modelo de lenguaje. Puede corresponder a una palabra, a un fragmento de palabra o a un signo de puntuación, según el *tokenizador* empleado.

Tokenizer Componente que convierte texto en bruto en una secuencia de *tokens* (unidades numéricas) que el modelo de lenguaje puede procesar. Los *tokenizers* modernos, como BPE (*Byte-Pair Encoding*) o SentencePiece, operan a nivel de subpalabra, lo que les permite manejar vocabulario abierto sin requerir un diccionario fijo.

Transformer Arquitectura de red neuronal introducida en 2017, basada en mecanismos de atención, que constituye la base de la mayoría de los modelos de lenguaje contemporáneos.

Trazabilidad Propiedad de un sistema conversacional por la cual cada respuesta generada puede vincularse explícitamente al fragmento documental del corpus que la sustenta, permitiendo su verificación por el usuario.

UML *Unified Modeling Language*. Lenguaje gráfico estandarizado por el *Object Management Group* para modelar sistemas software. En este trabajo se emplea principalmente el diagrama de casos de uso para representar la interacción entre actores y el sistema.

Ventana de contexto (*context window*). Número máximo de *tokens* que un modelo de lenguaje puede recibir como entrada y procesar simultáneamente en una única inferencia. Determina cuánta información

(consulta del usuario, contexto recuperado, instrucciones del sistema) puede considerarse a la vez para generar una respuesta. Los modelos recientes han ampliado esta ventana desde 512 tokens (BERT) hasta 128K–1M tokens (GPT-4o, Gemini), aunque la calidad de atención a posiciones intermedias puede degradarse (véase *lost in the middle*).

VRAM *Video RAM*. Memoria dedicada de una GPU, factor limitante del tamaño del modelo de lenguaje que es posible ejecutar localmente.

Zero-shot Capacidad de un modelo de lenguaje para realizar una tarea sin haber recibido ningún ejemplo específico de ella durante el entrenamiento ni en el *prompt*. Se contrapone al aprendizaje *few-shot* (con pocos ejemplos) y al *fine-tuning* (con muchos). La evaluación *zero-shot* constituye el modo de operación habitual de GRAIA, donde el LLM responde condicionado únicamente por el contexto recuperado y las instrucciones del *system prompt*.

